Appunti per l'Esame di Stato

Ingegnere Junior — Settore Informazione

Risorsa gratuita e collaborativa per la preparazione all'Esame di Stato.



Autore: Dott. Paolo Pietrelli

Con la collaborazione della community open-source

Appunti per l'Esame di Stato
Ingegnere Junior - Settore Informazione

Dott. Paolo Pietrelli

5 novembre 2026

Prefazione

Questi appunti nascono con l'obiettivo di fornire un supporto gratuito e accessibile a tutti gli studenti che si preparano all'Esame di Stato di Ingegneria dell'Informazione (Sezione B – Ingegnere Junior).

L'idea di base è semplice: **studiare insieme, condividendo**. Come in ogni progetto open-source, la conoscenza diventa più forte quando è aperta e collaborativa. Per questo motivo questo documento è pubblicato liberamente su GitHub e può essere migliorato da chiunque desideri contribuire.

I capitoli trattano in modo sintetico e strutturato le materie più frequenti nelle prove d'esame: sistemi operativi, basi di dati, reti, programmazione, progettazione del software, elettronica e sistemi numerici. Ogni sezione include anche esercizi ed esempi per facilitare la preparazione pratica.

Il documento è organizzato in **due parti**. La **Parte I** raccoglie i *fondamenti* per l'Esame di Stato di Ingegnere Junior. La **Parte II** contiene gli *approfondimenti* per la Sezione A (Ingegnere, laurea magistrale), con un'impostazione orientata a Ingegneria Informatica: temi aperti, prove pratiche di progettazione e relative tracce d'esame. Chi prepara la Sezione B può fermarsi alla Parte I; chi prepara la Sezione A trova nella Parte I le basi su cui la Parte II si appoggia.

Un invito: se noti errori, hai suggerimenti o vuoi aggiungere nuovi contenuti, partecipa! Basta una pull request o una issue sul repository. Così facendo, questi appunti diventeranno non solo un aiuto individuale, ma una risorsa collettiva per tutti i futuri ingegneri.

Dott. Paolo Pietrelli
Forlì, 5 novembre 2026

Collaboratori

Questo documento è stato sviluppato con la collaborazione di:

- Paolo Pietrelli – autore principale; Parte I (Fondamenti)
- Nicolò Venieri – Parte II (Approfondimenti per la Sezione A)

Nota sulla Parte II

La **Parte I** è opera dell'autore originale del progetto. La **Parte II** (Approfondimenti per la Sezione A) è stata aggiunta in un secondo momento da Nicolò Venieri, a partire dalle tracce delle prove passate raccolte nella cartella `TraccePassate`, con l'obiettivo di estendere gli appunti alla Sezione A di Ingegneria Informatica.

Limiti e verifica. Il codice e i calcoli numerici riportati nella Parte II sono stati eseguiti e controllati (tranne gli esempi indicati come pseudocodice). Le parti teoriche, le norme e i valori di riferimento *non* hanno ricevuto la revisione di un docente o di un professionista abilitato: vanno verificati su fonti ufficiali prima di farvi affidamento. Chiunque può segnalare errori con una *issue* o proporre correzioni con una *pull request*.

Indice

Prefazione iii

I Fondamenti: livello Ingegnere Junior 1

1 Sistemi Operativi 3
1.1 Struttura e Organizzazione del Sistema Operativo . . . 3
1.1.1 Componenti Principali . . . 3
1.1.2 Modelli di Sistemi Operativi . . . 4
1.1.3 Processi e Thread . . . 4
1.2 Record di Attivazione (Activation Record / Stack Frame) . . . 5
1.2.1 Contenuto di un Record di Attivazione . . . 5
1.2.2 Ciclo di Vita di un Record di Attivazione . . . 6
1.3 Gestione della Memoria . . . 6
1.3.1 Memoria Virtuale . . . 7
1.3.2 Algoritmi di Allocazione della Memoria (per Blocchi Contigui) . . . 8
1.4 Gestione dei File System . . . 8
1.4.1 Organizzazione Fisica dei Dati su Disco . . . 8
1.4.2 Metodi di Allocazione dei File . . . 9
1.5 Sincronizzazione e Deadlock . . . 10
1.5.1 Problemi di Sincronizzazione . . . 10
1.5.2 Meccanismi di Sincronizzazione . . . 11
1.5.3 Deadlock (Interblocco) . . . 12
1.6 Scheduling della CPU . . . 13
1.6.1 Principali Problematiche dello Scheduling . . . 13
1.6.2 Esempi di Algoritmi di Scheduling . . . 13
Domande e Esercizi . . . 17
Altre Possibili Domande . . . 22

2 Basi di Dati 25
2.1 Modello Concettuale Entità-Relazione (ER) . . . 25
2.1.1 Componenti Base e Rappresentazione Grafica . . . 25
2.1.2 Dettagli degli Attributi e Identificatori . . . 25
2.1.3 Dettagli delle Relazioni e Cardinalità . . . 26
2.1.4 Generalizzazione nel Modello ER . . . 27
2.1.5 Esempi Complessi di Schemi ER . . . 27
2.1.6 Esempi Complessi di Schemi ER . . . 27
2.2 Progettazione Logica: Normalizzazione e Forme Normali . . . 32
2.2.1 Perché la Normalizzazione è Importante . . . 32
2.2.2 Forme Normali: Livelli di Normalizzazione . . . 33

2.2.3 Linguaggio SQL . . . 36
Domande e Esercizi: Basi di Dati . . . 39
Domande d'Esame Principali . . . 39

3 Reti di Calcolatori **45**
3.1 Modello di Comunicazione Client/Server . . . 45
3.1.1 Funzionamento del Modello Client/Server . . . 45
3.1.2 Vantaggi e Svantaggi del Modello Client/Server . . . 45
3.2 Protocollo HTTP . . . 46
3.2.1 Funzionamento e Applicazione in HTTP . . . 46
3.2.2 Connessioni Stateless e Stateful . . . 46
3.2.3 Metodi per Gestire la Persistenza dello Stato in HTTP . . . 47
3.3 Standard ISO/OSI . . . 47
3.3.1 Struttura a Sette Strati del Modello OSI . . . 47
3.4 Livello di Trasporto (TCP vs UDP) . . . 48
3.4.1 TCP (Transmission Control Protocol) . . . 48
3.4.2 UDP (User Datagram Protocol) . . . 48
3.5 Subnetting (Suddivisione di Sottoreti) . . . 49
3.5.1 Obiettivi del Subnetting . . . 49
3.5.2 Indirizzi IP e Maschere di Sottorete . . . 49
3.5.3 Calcolo del Subnetting (Esempio Pratico) . . . 49
Domande e Esercizi: Reti di Calcolatori . . . 51
Domande d'Esame Principali . . . 51
Altre Possibili Domande . . . 52

4 Programmazione Orientata agli Oggetti (e Fondamenti) **57**
4.1 Concetti Base della Programmazione Orientata agli Oggetti (POO) . . . 57
4.2 Strutture Dati Astratte (ADT) e Fondamentali di Programmazione . . . 58
4.2.1 Abstract Data Type (ADT) . . . 58
4.2.2 Passaggio di Parametri nelle Chiamate di Funzione . . . 58
4.2.3 Overloading di Funzioni (o Metodi) . . . 59
4.3 Linguaggi Compilati e Linguaggi Interpretati . . . 60
4.3.1 Linguaggi Compilati . . . 60
4.3.2 Linguaggi Interpretati . . . 60
4.4 Algoritmi e Complessità Computazionale . . . 61
4.4.1 Complessità Temporale . . . 61
4.4.2 Complessità Spaziale . . . 61

5 Progettazione del Software **63**
5.1 Analisi dei Requisiti . . . 63
5.1.1 Tipi di Requisiti . . . 63
5.1.2 Diagrammi di Casi d'Uso UML (Use Case Diagrams) . . . 64
5.2 Progettazione dell'Architettura del Sistema Software . . . 64
5.2.1 Design Patterns (Pattern Architetturali e di Progettazione) . . . 64
5.2.2 Diagrammi UML (Unified Modeling Language) . . . 65

6 Elettronica **75**
6.1 Transistore MOS (Metal-Oxide-Semiconductor Field-Effect Transistor) . . . 75
6.1.1 Struttura del Transistore MOS (nMOS) . . . 75
6.1.2 Principio di Funzionamento (nMOS) . . . 76
6.1.3 Regimi di Funzionamento (per nMOS con $V_{GS} > V_{TH}$) . . . 76

6.1.4 Vantaggi del MOS rispetto al Transistore Bipolare (BJT) . . . 76
6.2 Circuiti CMOS (Complementary MOS) . . . 77
6.2.1 Invertitore CMOS . . . 77
6.2.2 Vantaggi dell'Invertitore CMOS rispetto agli Invertitori con Carico Resistivo 78
6.2.3 Applicazioni Notevoli dell'Invertitore CMOS . . . 78

7 Sistemi Numerici 79
7.1 Rappresentazione per Numeri Interi . . . 79
7.1.1 Complemento a Due . . . 79
7.2 Operazioni Aritmetiche con Complemento a Due . . . 80
7.2.1 Addizione . . . 80
7.2.2 Sottrazione . . . 81
7.2.3 Overflow . . . 81

II Approfondimenti per la Sezione A 83

Introduzione alla Parte II 85

8 Sistemi Distribuiti, Cloud e Big Data 87
8.1 Caratteristiche dei Sistemi Distribuiti . . . 87
8.1.1 Scalabilità: verticale e orizzontale . . . 87
8.1.2 Prestazioni: latenza e throughput . . . 88
8.1.3 Tolleranza ai guasti . . . 88
8.2 Il Teorema CAP . . . 88
8.2.1 Modelli di consistenza . . . 89
8.2.2 ACID e BASE . . . 89
8.3 Database SQL e NoSQL nei Sistemi Distribuiti . . . 89
8.3.1 Database relazionali (SQL) . . . 89
8.3.2 Database NoSQL . . . 89
8.3.3 Confronto e scelta . . . 90
8.3.4 Scaletta per il tema "consistenza SQL/NoSQL e CAP" . . . 90
8.4 Big Data: Elaborazione Batch e Streaming . . . 90
8.4.1 Archiviazione . . . 90
8.4.2 Elaborazione batch e streaming . . . 91
8.5 Architetture di Applicazioni Distribuite . . . 91
8.5.1 Architettura a strati (layered, N-tier) . . . 91
8.5.2 Service-Oriented Architecture (SOA) . . . 91
8.5.3 Microservizi . . . 91
8.5.4 Comunicazione sincrona e asincrona . . . 92
8.5.5 Confronto sintetico . . . 92
8.6 Cloud Computing e Virtualizzazione . . . 92
8.6.1 Virtualizzazione . . . 92
8.6.2 Modelli di servizio e di distribuzione . . . 92
8.6.3 Vantaggi, sfide e best practice . . . 92
8.7 Meccanismi Fondamentali . . . 93
8.7.1 Consenso e elezione del leader . . . 93
8.7.2 Transazioni distribuite . . . 93
8.7.3 Comunicazione tra processi . . . 93
8.7.4 Orologi e ordinamento . . . 93
Tracce Uscite: Sistemi Distribuiti . . . 94

9 Intelligenza Artificiale e Machine Learning **95**
9.1 Ricerca nello Spazio degli Stati . . . 95
9.1.1 Ricerca non informata (cieca) . . . 95
9.1.2 Ricerca informata (euristica) . . . 96
9.1.3 Ricerca locale . . . 96
9.1.4 Confronto: informata e non informata . . . 97
9.2 Machine Learning: Concetti Base . . . 97
9.3 Paradigmi di Apprendimento . . . 97
9.3.1 Apprendimento supervisionato . . . 97
9.3.2 Apprendimento non supervisionato . . . 98
9.3.3 Apprendimento per rinforzo . . . 98
9.3.4 Confronto . . . 98
9.4 Valutazione, Overfitting e Underfitting . . . 99
9.4.1 Suddivisione dei dati . . . 99
9.4.2 Underfitting e overfitting . . . 99
9.4.3 Metriche . . . 99
9.5 Regressione Lineare e Minimi Quadrati . . . 100
9.5.1 Identificazione di sistemi dinamici . . . 100
9.6 Principali Modelli di ML . . . 100
9.6.1 Reti neurali e deep learning . . . 101
9.7 Flusso di un Progetto di ML . . . 101
9.8 Rappresentazione della Conoscenza (cenni) . . . 101
9.9 Esempio: ML in un Sistema Completo . . . 102
Tracce Uscite: Intelligenza Artificiale e ML . . . 103

10 Progettazione di Sistemi e API **105**
10.1 Come Affrontare la Prova Pratica . . . 105
10.1.1 Metodo in sei passi . . . 105
10.1.2 Cosa mostrare . . . 105
10.2 Progettazione di API REST . . . 106
10.2.1 Principi . . . 106
10.2.2 Metodi HTTP . . . 106
10.2.3 Codici di stato . . . 106
10.2.4 Buone pratiche . . . 106
10.2.5 REST, SOAP e gRPC . . . 107
10.3 Architettura del Backend . . . 107
10.3.1 Architettura a livelli . . . 107
10.3.2 Pattern utili . . . 107
10.3.3 Integrare un modulo esterno (per esempio ML) . . . 108
10.4 Concorrenza e Integrità: Evitare le Doppie Prenotazioni . . . 108
10.5 Diagrammi UML per il Flusso Applicativo . . . 109
10.6 Tema Svolto: Cinema Multisala (A3-INF-4, 2026) . . . 109
10.6.1 Parte 1: progettazione della base di dati . . . 109
10.6.2 Parte 2: progettazione del backend . . . 113
10.6.3 Parte 3: modulo di Machine Learning . . . 116
10.7 Checklist e Errori Comuni . . . 118
Tracce Uscite: Progettazione di Sistemi . . . 119

11 Programmazione e Algoritmi **121**
11.1 Perché si Programma ad Oggetti . . . 121

11.1.1 Il problema: la complessità e il cambiamento . . . 121
11.1.2 L'idea di fondo: nascondere ciò che può cambiare . . . 121
11.1.3 Metriche di qualità: coesione e accoppiamento . . . 122
11.1.4 Confronto tra paradigmi . . . 122
11.2 Classi e Oggetti . . . 122
11.2.1 Definizioni . . . 122
11.2.2 Perché una classe e non una struttura con funzioni? . . . 123
11.2.3 Esempio: la classe `Rational` (traccia 2016) . . . 123
11.3 Incapsulamento . . . 124
11.4 Astrazione, Interfacce e Classi Astratte . . . 125
11.5 Ereditarietà . . . 126
11.6 Polimorfismo . . . 127
11.6.1 Definizione . . . 127
11.6.2 Modalità di realizzazione . . . 127
11.6.3 Perché si fa: il problema che risolve . . . 128
11.6.4 Ruolo di interfacce e implementazioni . . . 129
11.6.5 Esempio applicativo: monitoraggio di sensori . . . 129
11.6.6 Limiti e problematiche . . . 131
11.6.7 Scaletta per il tema "polimorfismo" . . . 131
11.7 Principi di Progettazione: SOLID . . . 131
11.8 Chiamate di Funzione, Stack e Passaggio dei Parametri . . . 132
11.8.1 Parametri formali e attuali . . . 132
11.8.2 Che cosa succede in una chiamata (record di attivazione) . . . 132
11.8.3 Modalità di passaggio dei parametri . . . 133
11.9 Ricorsione e Iterazione . . . 134
11.9.1 Attenzione alla ricorsione ridondante: Fibonacci . . . 135
11.10 Complessità degli Algoritmi . . . 136
11.10.1 Notazioni . . . 136
11.10.2 Come calcolarla . . . 136
11.11 Ricerca e Ordinamento . . . 136
11.11.1 Ricerca binaria . . . 136
11.11.2 Algoritmi di ordinamento . . . 137
11.12 Strutture Dati e Grafi . . . 138
11.12.1 Complessità delle operazioni . . . 138
11.12.2 Visite di grafi e cammini minimi . . . 138
11.12.3 Paradigmi algoritmici . . . 139
11.13 Tema Svolto: Corrispondenze tra Due Immagini (2018) . . . 139
11.14 Checklist per un Tema di Informatica . . . 140
Tracce Uscite: Programmazione e Algoritmi . . . 141

12 Sistemi Operativi e Reti: Approfondimenti **143**
12.1 Modalità Utente e Kernel, Chiamate di Sistema, Interrupt . . . 143
12.1.1 Perché due modalità . . . 143
12.1.2 Chiamate di sistema (system call) . . . 143
12.1.3 Interrupt, eccezioni e trap . . . 144
12.2 Processi: Stati, PCB e Cambio di Contesto . . . 144
12.2.1 Stati di un processo . . . 144
12.2.2 Process Control Block (PCB) . . . 144
12.2.3 Cambio di contesto (context switch) . . . 144
12.2.4 Creazione di processi: `fork`, `exec`, `wait` . . . 144

12.2.5 Comunicazione tra processi (IPC) . . . 145
12.3 Thread e Modelli di Multithreading . . . 145
12.3.1 Perché i thread . . . 145
12.3.2 Thread a livello utente e a livello kernel . . . 146
12.3.3 I tre modelli di multithreading . . . 146
12.4 Race Condition: dall'Esempio alla Soluzione . . . 147
12.4.1 Perché `contatore++` non è sicuro . . . 147
12.4.2 Rimedi . . . 148
12.4.3 Problemi classici di sincronizzazione . . . 148
12.5 Sostituzione delle Pagine . . . 148
12.5.1 Costo di un page fault . . . 148
12.5.2 Algoritmi . . . 149
12.5.3 Esempio svolto . . . 149
12.5.4 Thrashing e working set . . . 149
12.5.5 Dimensione della tabella delle pagine . . . 149
12.6 Il Concetto di Protocollo . . . 150
12.6.1 Definizione . . . 150
12.6.2 Perché servono i protocolli . . . 150
12.6.3 Funzioni tipiche di un protocollo . . . 150
12.6.4 Servizio, protocollo e stratificazione . . . 151
12.7 Comunicazione Orientata e Non Orientata alla Connessione . . . 151
12.8 Lo Stack TCP/IP e i Principali Protocolli . . . 151
12.8.1 Applicazione . . . 152
12.8.2 Trasporto . . . 152
12.8.3 Rete e collegamento . . . 152
12.9 TCP in Dettaglio . . . 152
12.9.1 Intestazione e porte . . . 152
12.9.2 Apertura e chiusura . . . 153
12.9.3 Affidabilità . . . 153
12.9.4 Controllo di flusso e di congestione . . . 153
12.10 Indirizzamento, Routing e Servizi di Rete . . . 153
12.10.1 IP, DHCP e NAT . . . 153
12.10.2 Routing e forwarding . . . 154
12.10.3 DNS . . . 154
12.11 Protocolli nei Sistemi Distribuiti . . . 155
12.12 Sicurezza di Rete: TLS e HTTPS . . . 155
12.13 Programmazione con i Socket . . . 155
12.14 Scaletta per il Tema "Protocolli nei Sistemi Distribuiti" . . . 157
Tracce Uscite: Sistemi Operativi e Reti . . . 158

13 Sicurezza Informatica e Crittografia **159**
13.1 Concetti Fondamentali . . . 159
13.1.1 Obiettivi: la triade CIA e oltre . . . 159
13.1.2 Minacce, vulnerabilità, rischio . . . 159
13.1.3 Principi di progettazione sicura . . . 160
13.2 Crittografia Simmetrica . . . 160
13.2.1 Cifrari a blocchi e a flusso . . . 160
13.2.2 Modi di funzionamento . . . 160
13.2.3 Il problema della distribuzione delle chiavi . . . 161
13.3 Crittografia Asimmetrica . . . 161

13.3.1 RSA . . . 161
13.3.2 Scambio di chiavi Diffie–Hellman . . . 162
13.3.3 Crittografia a curve ellittiche (ECC) . . . 162
13.3.4 Confronto e crittografia ibrida . . . 162
13.4 Funzioni Hash, MAC e Firma Digitale . . . 163
13.4.1 Funzioni hash crittografiche . . . 163
13.4.2 MAC e HMAC: integrità e autenticità con chiave condivisa . . . 163
13.4.3 Firma digitale . . . 164
13.5 Infrastruttura a Chiave Pubblica (PKI) e Certificati . . . 164
13.6 Autenticazione e Autorizzazione . . . 164
13.6.1 Fattori di autenticazione . . . 164
13.6.2 Conservare le password . . . 164
13.6.3 Sessioni, token e cookie . . . 165
13.6.4 Autorizzazione . . . 165
13.7 Vulnerabilità delle Applicazioni . . . 166
13.7.1 SQL injection . . . 166
13.7.2 Altre vulnerabilità frequenti . . . 167
13.8 Sicurezza di Rete . . . 167
13.8.1 Difese . . . 167
13.9 Gestione del Rischio, Normativa e Ciclo di Vita . . . 168
13.9.1 Valutare il rischio e modellare le minacce . . . 168
13.9.2 Standard e normativa . . . 168
13.9.3 Sicurezza nel ciclo di sviluppo (DevSecOps) . . . 168
13.10Esempio: Messa in Sicurezza del Sistema di Prenotazione . . . 169
13.11Scaletta per un Tema sulla Sicurezza . . . 170
Collegamenti con le Tracce: Sicurezza . . . 171

14 Basi di Dati Avanzate **173**
14.1 Algebra Relazionale . . . 173
14.1.1 Richiami sul modello relazionale . . . 173
14.1.2 Operatori . . . 174
14.1.3 Equivalenze usate per ottimizzare . . . 174
14.2 Dipendenze Funzionali e Forme Normali . . . 174
14.2.1 Dipendenze funzionali . . . 174
14.2.2 Forme normali, definizioni formali . . . 175
14.2.3 Esempio: 3NF ma non BCNF . . . 176
14.2.4 Denormalizzazione . . . 176
14.3 SQL Avanzato . . . 176
14.3.1 Join esterni e la trappola delle aggregazioni . . . 176
14.3.2 Sottoquery . . . 177
14.3.3 CTE e query ricorsive . . . 178
14.3.4 Funzioni finestra . . . 179
14.3.5 Viste, vincoli e trigger . . . 180
14.4 Transazioni . . . 181
14.4.1 Proprietà ACID, con il perché . . . 181
14.4.2 Gestione . . . 181
14.5 Controllo della Concorrenza . . . 182
14.5.1 Anomalie . . . 182
14.5.2 Livelli di isolamento (SQL standard) . . . 182
14.5.3 Serializzabilità . . . 182

14.5.4 Tecniche . . . 183
14.6 Affidabilità e Ripristino dei Dati . . . 183
14.6.1 Tipi di guasto . . . 183
14.6.2 Log e Write-Ahead Logging . . . 184
14.6.3 Recupero dopo un crash . . . 184
14.6.4 Backup e alta disponibilità . . . 184
14.7 Indici e Progettazione Fisica . . . 184
14.7.1 Perché servono . . . 184
14.7.2 Strutture . . . 184
14.7.3 Tipi di indice . . . 185
14.7.4 Esperimento . . . 185
14.7.5 Costi e quando non indicizzare . . . 186
14.7.6 Indici per il sistema del cinema . . . 186
14.7.7 Altri strumenti di progettazione fisica . . . 186
14.8 Ottimizzazione delle Query . . . 186
14.8.1 Fasi di elaborazione . . . 186
14.8.2 Algoritmi di join . . . 187
14.8.3 Buone pratiche . . . 187
14.9 Accesso ai Dati dalle Applicazioni . . . 187
14.9.1 Il problema N+1 . . . 188
14.10Data Warehouse e Analisi OLAP . . . 188
14.11Scalette di Risposta . . . 189
Tracce Uscite: Basi di Dati Avanzate . . . 190

15 Architettura del Calcolatore **191**
15.1 Il Modello di von Neumann . . . 191
15.1.1 Componenti . . . 191
15.1.2 Il ciclo di istruzione . . . 191
15.1.3 Perché “programma memorizzato” . . . 191
15.1.4 Von Neumann e Harvard . . . 192
15.2 Richiami di Reti Logiche . . . 192
15.3 Instruction Set Architecture (ISA) . . . 192
15.3.1 RISC e CISC . . . 192
15.3.2 RISC-V: registri e formati . . . 193
15.3.3 Modi di indirizzamento . . . 193
15.3.4 Esempio: da C ad assembly . . . 193
15.4 Il Datapath . . . 194
15.4.1 Unità funzionali . . . 194
15.4.2 Segnali di controllo . . . 194
15.4.3 Quali unità usa ciascuna istruzione . . . 194
15.4.4 Datapath a ciclo singolo e perché non basta . . . 194
15.5 La Pipeline . . . 195
15.5.1 L’idea . . . 195
15.5.2 I cinque stadi (RISC classico) . . . 195
15.5.3 Registri di pipeline . . . 195
15.5.4 Diagramma temporale . . . 195
15.5.5 Vantaggi, con i numeri . . . 196
15.6 Hazard: i Problemi della Pipeline . . . 197
15.6.1 Hazard strutturali . . . 197
15.6.2 Hazard sui dati . . . 197

15.6.3 Hazard di controllo . . . 198
15.6.4 Effetto sulle prestazioni: il ciclo di somma . . . 199
15.7 Oltre la Pipeline Semplice . . . 199
15.8 Prestazioni . . . 199
15.8.1 Equazione fondamentale . . . 199
15.8.2 Legge di Amdahl . . . 200
15.8.3 Potenza e consumo . . . 200
15.9 Gerarchia di Memoria e Cache . . . 200
15.9.1 Il problema . . . 200
15.9.2 Il principio di località . . . 201
15.9.3 Organizzazione di una cache . . . 201
15.9.4 Tipi di miss (le "3C") . . . 202
15.9.5 Politiche . . . 202
15.9.6 Tempo medio di accesso . . . 202
15.9.7 Effetto sul codice: percorrere una matrice . . . 202
15.9.8 Cache, multicore e memoria virtuale . . . 203
15.10 Ingresso/Uscita . . . 203
15.11 Microcontrollori . . . 203
15.11.1 Che cos'è . . . 203
15.11.2 Struttura interna . . . 203
15.11.3 Esempio: stazione meteo . . . 204
15.11.4 Esempi di impiego . . . 204
15.11.5 Vincoli tipici . . . 204
15.12 Rappresentazione dei Numeri . . . 205
15.12.1 IEEE 754 (precisione singola, 32 bit) . . . 205
15.12.2 Endianness . . . 205
15.13 Tema Svolto: Datapath RISC e Pipeline (2024) . . . 205
15.14 Scalette di Risposta . . . 206
Tracce Uscite: Architettura del Calcolatore . . . 208

Elenco delle figure

2.1 Esempi di rappresentazione di attributi semplici (es. Codice Fiscale) e attributi composti (es. Data di Nascita) in un Diagramma ER. . . . 26
2.2 Esempi di utilizzo di identificatori esterni, inclusi scenari con entità deboli e concatenazione di identificazione. . . . 27
2.3 Esempio di Generalizzazione nel Diagramma ER: l'entità Persona si generalizza nelle entità Uomo e Donna, mostrando la distinzione e l'ereditarietà delle proprietà. 28
2.4 Diagramma ER che modella le relazioni tra Studenti, Corsi ed Esami, inclusi attributi e cardinalità. . . . 28
2.5 Diagramma ER che modella le relazioni tra Studenti, Corsi ed Esami, inclusi attributi e cardinalità. . . . 29
2.6 Schema ER per l'esercizio 2: Sistema Cinema/Film. . . . 29
2.7 Schema ER per l'esercizio 7: Sistema Campionato di Calcio. . . . 30
2.8 Schema ER per l'esercizio 11: Sistema Gare Ciclistiche. . . . 31
2.9 Diagramma di Venn delle Forme Normali (1NF, 2NF, 3NF, BCNF, 4NF, 5NF), che mostra la loro relazione gerarchica. . . . 33
2.10 Esempio di una tabella non in 1NF e la sua normalizzazione a 1NF, eliminando l'attributo multi-valore. . . . 34
2.11 Esempio di una tabella non in 2NF e la sua normalizzazione a 2NF tramite scomposizione in due relazioni. . . . 34
2.12 Esempio di una tabella "Computer" non in 3NF a causa di dipendenze transitive. 35
2.13 Normalizzazione della tabella "Computer" a 3NF, scomponendo le relazioni per eliminare le dipendenze transitive. . . . 35
2.14 Esempio di una tabella "Impiegato" non in 3NF per dipendenza transitiva. . . . 35
2.15 Esempio di una query SQL che utilizza operatori e funzioni avanzate per filtrare e aggregare i dati. . . . 38
2.16 Diagramma Entità-Relazione per la gestione di corsi, docenti, appelli d'esame e studenti. . . . 40
2.17 Diagramma Entità-Relazione per la gestione delle sessioni dell'Esame di Stato, Commissione e Candidati. . . . 42

5.1 Rappresentazione grafica delle relazioni comuni tra Casi d'Uso in UML (Inclusione, Estensione e Generalizzazione). . . . 66
5.2 Esempio completo di Diagramma dei Casi d'Uso per un sistema di Ristorazione, che illustra le interazioni tra attori (Cameriere, Cliente, Cuoco, Cassiere) e le funzionalità del sistema. . . . 67
5.3 Esempi di rappresentazione base di una classe UML, con attributi e metodi, e come si relaziona al codice. . . . 67
5.4 Esempio di Diagramma delle Classi che illustra l'uso dei modificatori di visibilità (public, private, protected) per attributi e metodi. . . . 68

5.5 Esempio di Diagramma delle Classi che mostra la relazione di ereditarietà tra le classi Persona, Cliente e Potenziale Cliente. 68
5.6 Esempio di Diagramma delle Classi che illustra come le interfacce consentono di simulare l'ereditarietà multipla in linguaggi che non la supportano nativamente. 69
5.7 Confronto visivo tra Aggregazione e Composizione nei Diagrammi delle Classi UML. 69
5.8 Diagramma delle Classi per un sistema di gestione prodotti/catalogo/carrello, che illustra varie classi e le loro relazioni (es. Prodotto, Carrello, DVD, Libro). . 70
5.9 Diagramma delle Classi per un sistema di Chat, che mostra le interazioni tra classi come Utente, Chat e Messaggio, e le loro molteplicità. 71
5.10 Diagramma di Sequenza UML che illustra il flusso di aggiornamento dei dati in un'architettura Model-View-Controller (MVC). 72
5.11 Diagramma di Sequenza UML che illustra il flusso di prestito di un libro in un sistema di gestione bibliotecaria. 73

10.1 Schema ER del cinema multisala. Le cardinalità (min, max) sono scritte accanto all'entità a cui si riferiscono. 110
10.2 Diagramma di sequenza della creazione di una prenotazione. 116

15.1 Datapath a pipeline a cinque stadi: i registri di pipeline separano gli stadi; le linee tratteggiate sono l'unità di forwarding, il rilevamento degli hazard e il salto preso. 196

Listings

1.1 Funzione pow2(x) . . . 21
2.1 Esempio Operatori Logici . . . 37
2.2 Esempio Operatori di Confronto . . . 37
2.3 Esempio Funzioni Stringa . . . 38
2.4 Esempio Funzioni Numeriche . . . 38
2.5 Esempio Funzioni Data/Ora . . . 38
2.6 Query per l'elenco degli iscritti a un appello d'esame specifico . . . 41
2.7 Query per l'elenco dei candidati che hanno superato l'esame in una data specifica 43
4.1 Esempio di Passaggio per Valore . . . 58
4.2 Esempio di Passaggio per Riferimento . . . 59
4.3 Esempio di Function Overloading . . . 59
10.1 Creazione delle tabelle (sintassi in stile PostgreSQL) . . . 111
10.2 Posti ancora disponibili per una proiezione . . . 112
10.3 Esempio: elenco dei posti di una proiezione . . . 114
10.4 Esempio: creazione di una prenotazione . . . 114
10.5 Pseudocodice del servizio di prenotazione (stile Python) . . . 115
11.1 Classe Rational: classe, oggetto, incapsulamento e immutabilita . . . 123
11.2 Incapsulamento: il metodo protegge l'invariante . . . 124
11.3 Dipendere da un'interfaccia: sostituibilita e testabilita . . . 125
11.4 Composizione: una pila non e' una lista . . . 126
11.5 Overriding (run-time) contro overloading (compile-time) . . . 128
11.6 Senza polimorfismo: switch sul tipo . . . 128
11.7 Con polimorfismo: ogni tipo conosce se stesso . . . 128
11.8 Sensori polimorfi: monitor generico . . . 129
11.9 Polimorfismo in Python . . . 130
11.10 Evoluzione dello stack per fact(3) . . . 133
11.11 Valore, riferimento e puntatore in C++ . . . 133
11.12 Python e Java: passaggio per valore del riferimento . . . 134
11.13 Fattoriale: versione iterativa e ricorsiva . . . 134
11.14 Fibonacci: ricorsione ingenua, con memoizzazione, iterativa . . . 135
11.15 Ricerca binaria iterativa e ricorsiva . . . 136
11.16 Merge sort (divide et impera) . . . 137
11.17 BFS e Dijkstra su liste di adiacenza . . . 138
11.18 Mappa di disparita: corrispondenze tra due immagini . . . 139
12.1 fork, exec e wait in C . . . 144
12.2 Race condition e tre rimedi in Java . . . 147
12.3 Server TCP: orientato alla connessione . . . 155
12.4 Client TCP . . . 156
12.5 Client e server UDP: senza connessione . . . 156
13.1 RSA con numeri piccoli (solo didattico) . . . 161

13.2 Diffie-Hellman con numeri piccoli (didattico) . . . 162
13.3 Hash, effetto valanga e HMAC . . . 163
13.4 Salvataggio sicuro di una password (PBKDF2 con sale) . . . 165
13.5 SQL injection e query parametrica . . . 166
13.6 Controllo di autorizzazione contro l'IDOR (pseudocodice) . . . 169
14.1 Chiusura di un insieme di attributi, chiavi candidate e controllo BCNF . . . 175
14.2 Errore classico: LEFT JOIN seguito da SUM . . . 176
14.3 Versione corretta . . . 177
14.4 Anti-join con NOT EXISTS . . . 177
14.5 GROUP BY, HAVING e sottoquery: occupazione sopra il 50 per cento . . . 177
14.6 Sottoquery correlata . . . 178
14.7 Divisione relazionale con doppio NOT EXISTS . . . 178
14.8 La trappola di NOT IN con i valori NULL . . . 178
14.9 CTE ricorsiva: giorni senza proiezioni . . . 179
14.10 Classifica con RANK, DENSE_RANK e ROW_NUMBER . . . 179
14.11 Totale cumulato con una funzione finestra . . . 179
14.12 Vista sull'occupazione delle proiezioni . . . 180
14.13 Trigger di audit sui biglietti (sintassi SQLite) . . . 180
14.14 Transazione: o tutto o niente . . . 181
14.15 Indici semplici e composti; quando NON vengono usati . . . 185
14.16 Paginazione: OFFSET contro keyset . . . 187
14.17 N+1 query contro una sola query con JOIN . . . 188
15.1 Somma di un vettore in assembly RISC-V (eseguita su un simulatore) . . . 193
15.2 Dipendenza RAW . . . 197
15.3 Codice originale: 2 stalli, 13 cicli (con forwarding) . . . 198
15.4 Codice riordinato: 0 stalli, 11 cicli . . . 198
15.5 Stessa somma, due ordini di scansione . . . 202
15.6 Schema in C di un'applicazione embedded (pseudocodice, non eseguito) . . . 204

Parte I

Fondamenti: livello Ingegnere Junior

Capitolo 1

Sistemi Operativi

Un **sistema operativo (SO)** è un software di sistema che gestisce le risorse hardware e software di un computer e fornisce servizi comuni per i programmi del computer e per l'utente. È l'interfaccia tra l'hardware e l'utente/applicazioni. La sua importanza risiede nell'astrazione dell'hardware, nella gestione efficiente delle risorse e nell'esecuzione controllata dei programmi.

1.1 Struttura e Organizzazione del Sistema Operativo

Un sistema operativo è un'entità complessa, ma può essere scomposto in componenti modulari che cooperano per fornire un ambiente funzionale per l'esecuzione dei programmi.

1.1.1 Componenti Principali

I principali componenti di un sistema operativo includono:

- **Kernel**: Il cuore del SO, responsabile della gestione dei processi (creazione, scheduling, terminazione, comunicazione interprocesso), della memoria (allocazione, protezione, gestione della memoria virtuale), dei file system (gestione dei file e delle directory, allocazione dello spazio su disco) e dell'I/O (gestione dei dispositivi di input/output, driver).
- **Gestore dei Processi (Process Management)**: Si occupa della creazione, terminazione, sospensione e ripristino dei processi, e della gestione dei loro stati (pronto, in esecuzione, in attesa).
- **Gestore della Memoria (Memory Management)**: Responsabile dell'allocazione e deallocazione della memoria ai processi, della gestione della memoria virtuale (paginazione, segmentazione) e della protezione della memoria per evitare interferenze tra i processi.
- **File System Management**: Organizza e gestisce i dati su dispositivi di archiviazione, controllando l'accesso e la protezione dei file e delle directory.
- **Gestore I/O (I/O Management)**: Fornisce un'interfaccia standardizzata per interagire con i dispositivi hardware (stampanti, tastiere, dischi) tramite driver specifici.
- **Network Management**: Gestisce le comunicazioni di rete e i protocolli di comunicazione.
- **Security and Protection**: Implementa meccanismi per proteggere le risorse del sistema e i dati degli utenti da accessi non autorizzati o malfunzionamenti.
- **Interfaccia Utente (User Interface)**: Può essere una GUI (Graphical User Interface) con elementi visivi o una CLI (Command Line Interface) basata su testo, permettendo all'utente di interagire con il sistema.

1.1.2 Modelli di Sistemi Operativi

I sistemi operativi possono essere strutturati secondo diversi modelli architetturali:

- **Monolitici**: Tutti i servizi del SO risiedono nello stesso spazio di indirizzamento (kernel space).
  - **Vantaggi**: Alta performance grazie al minimo overhead di comunicazione.
  - **Svantaggi**: Difficili da debuggare, poco flessibili, un crash di un componente può bloccare l'intero sistema.
  - **Esempio**: Linux, Unix (tradizionali).
- **Layered (a Strati)**: Il SO è diviso in strati, ognuno dei quali offre servizi allo strato superiore e utilizza servizi dallo strato inferiore.
  - **Vantaggi**: Modularità, facilità di debug e manutenzione.
  - **Svantaggi**: Performance ridotte a causa dell'overhead di comunicazione tra strati.
  - **Esempio**: THE (Dijkstra).
- **Microkernel**: Solo i servizi essenziali (gestione processi, gestione memoria base, comunicazione interprocesso) risiedono nel kernel (microkernel). Altri servizi (file system, driver, network) sono implementati come processi utente (server).
  - **Vantaggi**: Modularità, robustezza (un crash di un server non blocca il sistema), flessibilità.
  - **Svantaggi**: Performance potenzialmente più basse a causa di più cambi di contesto.
  - **Esempio**: Mach (base per macOS), QNX.
- **Modulari (o Ibridi)**: Un approccio intermedio che combina le migliori caratteristiche dei modelli monolitici e microkernel. Permettono il caricamento dinamico dei moduli kernel (es. driver) senza richiedere un riavvio completo del sistema.
  - **Esempio**: Versioni moderne di Linux, Windows.

1.1.3 Processi e Thread

Nei sistemi operativi moderni, l'esecuzione dei programmi è gestita attraverso due concetti principali: i processi e i thread.

Processo (Processo "Pesante")

Un **processo** è un'istanza di un programma in esecuzione. È un'unità di allocazione delle risorse del sistema operativo e include:

- Il codice del programma.
- I dati (variabili globali, heap).
- Lo stack (per le chiamate di funzione e le variabili locali).
- Il Program Counter (PC) e i registri della CPU.
- Risorse del sistema operativo allocate (file aperti, segnali, memoria, ecc.).

Ogni processo ha il proprio spazio di indirizzamento virtuale separato, il che fornisce isolamento e protezione tra i processi. Se un processo crasha, di solito non influisce sugli altri processi. I processi sono considerati "pesanti" a causa dell'overhead associato alla loro creazione, alla distruzione e al cambio di contesto.

Thread (Processo "Leggero")

Un **thread** (o thread di esecuzione) è un'unità di esecuzione all'interno di un processo. Un singolo processo può contenere più thread. I thread all'interno dello stesso processo condividono lo stesso spazio di indirizzamento virtuale, il codice del programma, i dati globali e le risorse del sistema operativo (file aperti, ecc.). Ogni thread ha il proprio:

- Program Counter (PC).
- Set di registri della CPU.
- Stack separato.

I thread sono considerati "leggeri" perché la loro creazione, distruzione e il cambio di contesto sono molto più veloci ed efficienti rispetto ai processi, dato che non richiedono la creazione di un nuovo spazio di indirizzamento e la copia di risorse.

Punti in Comune e Differenze

- **Punti in Comune**: Sia processi che thread rappresentano unità di esecuzione che possono essere schedulate dalla CPU. Entrambi hanno un Program Counter, un set di registri e uno stack.
- **Differenze Principali**:
  - **Isolamento delle Risorse**: I processi hanno spazi di indirizzamento separati e risorse isolate. I thread all'interno dello stesso processo condividono lo spazio di indirizzamento e le risorse.
  - **Overhead**: I processi hanno un overhead maggiore per creazione, distruzione e cambio di contesto. I thread sono più "leggeri".
  - **Comunicazione**: La comunicazione tra processi (IPC - Inter-Process Communication) è più complessa (richiede meccanismi espliciti come pipe, shared memory, message queues). La comunicazione tra thread (ITC - Inter-Thread Communication) è più semplice e diretta, poiché condividono la memoria.
  - **Robustezza**: Un crash di un thread può compromettere l'intero processo (e tutti i suoi thread). Un crash di un processo non influisce sugli altri processi.

1.2 Record di Attivazione (Activation Record / Stack Frame)

Il **Record di Attivazione** (o *Stack Frame*) è una struttura dati creata sullo stack del programma ogni volta che una funzione (o procedura, o subroutine) viene chiamata. Contiene tutte le informazioni necessarie per la gestione dell'esecuzione di quella specifica chiamata di funzione. È una componente fondamentale del runtime di un linguaggio di programmazione e del supporto fornito dal sistema operativo per l'esecuzione di programmi che utilizzano stack per le chiamate a funzione.

1.2.1 Contenuto di un Record di Attivazione

Un tipico record di attivazione può contenere le seguenti informazioni, sebbene la loro esatta disposizione e i dettagli possano variare a seconda dell'architettura della CPU, del sistema operativo e del compilatore:

- **Parametri Attuali (Actual Parameters)**: I valori degli argomenti passati alla funzione.

- **Indirizzo di Ritorno (Return Address)**: L'indirizzo di memoria della istruzione nel codice chiamante a cui il controllo deve tornare una volta che la funzione corrente ha terminato l'esecuzione.
- **Valore di Ritorno (Return Value)**: Uno spazio per memorizzare il valore restituito dalla funzione (se la funzione restituisce un valore).
- **Variabili Locali (Local Variables)**: Le variabili dichiarate all'interno della funzione e che esistono solo per la durata di quella specifica attivazione.
- **Stato dei Registri Salvati (Saved Register State)**: I valori dei registri della CPU che erano in uso dalla funzione chiamante e che devono essere ripristinati prima del ritorno.
- **Puntatore al Frame Precedente (Control Link / Dynamic Link)**: Un puntatore (indirizzo) al record di attivazione della funzione chiamante, permettendo al sistema di "risalire" lo stack.
- **Puntatore al Contesto Statico (Access Link / Static Link)**: (Solo per linguaggi con scope annidato statico, come Pascal o linguaggi funzionali) Un puntatore al record di attivazione della funzione che definisce lo scope lessicale della funzione corrente, permettendo l'accesso a variabili non locali.

1.2.2 Ciclo di Vita di un Record di Attivazione

Il ciclo di vita di un record di attivazione segue il flusso delle chiamate a funzione e la struttura dello stack:

1. **Creazione (Chiamata di Funzione)**: Quando una funzione viene chiamata, il codice chiamante (o il runtime) crea un nuovo record di attivazione. Questo nuovo record viene "pushed" (inserito) in cima allo stack di esecuzione del processo. Il puntatore dello stack (Stack Pointer) viene aggiornato per puntare a questo nuovo frame.
2. **Esecuzione**: La CPU salta all'indirizzo di inizio del codice della funzione chiamata. La funzione utilizza le informazioni nel suo record di attivazione (parametri, variabili locali, registri) durante la sua esecuzione.
3. **Terminazione (Ritorno da Funzione)**: Quando la funzione termina (raggiunge un 'return' o la fine del suo blocco di codice), il valore di ritorno (se presente) viene posizionato in una posizione designata (spesso un registro). Il record di attivazione corrente viene "popped" (rimosso) dallo stack, liberando lo spazio di memoria che occupava. Il puntatore dello stack viene ripristinato al record di attivazione precedente.
4. **Ripristino del Contesto**: Il controllo del programma viene trasferito all'indirizzo di ritorno memorizzato nel record di attivazione appena rimosso, e i registri salvati vengono ripristinati per continuare l'esecuzione della funzione chiamante.

Questo meccanismo a stack garantisce che le chiamate a funzione e i ritorni avvengano in modo LIFO (Last-In, First-Out), permettendo la corretta gestione delle funzioni annidate e ricorsive.

1.3 Gestione della Memoria

La **gestione della memoria** è una delle funzioni fondamentali del sistema operativo, responsabile di allocare e deallocare la memoria ai processi, di proteggere la memoria per evitare interferenze tra i processi e di fornire un'astrazione della memoria fisica per gli sviluppatori. L'obiettivo principale è massimizzare l'utilizzo della CPU mantenendo molti processi in memoria e fornendo un meccanismo efficiente per accedere ai dati.

1.3.1 Memoria Virtuale

La **memoria virtuale** è una tecnica che permette a un sistema operativo di compensare la carenza di memoria fisica (RAM) usando la memoria secondaria (spazio su disco) per simulare più RAM. Questo permette ai programmi di usare più memoria di quella fisicamente disponibile e facilita il multitasking, isolando lo spazio di indirizzamento di ogni processo. Le tecniche principali per implementare la memoria virtuale sono la paginazione e la segmentazione.

Paginazione (Paging)

La paginazione è una tecnica di gestione della memoria virtuale che suddivide lo spazio di indirizzamento logico di un processo in blocchi di dimensione fissa chiamati **pagine**. La memoria fisica è anch'essa divisa in blocchi della stessa dimensione, chiamati **frame**.

- **Funzionamento**: Quando un processo viene caricato, le sue pagine possono essere caricate in frame non contigui della memoria fisica. La traduzione dell'indirizzo logico (virtuale) in indirizzo fisico avviene tramite una **Page Table** (Tabella delle Pagine). Ogni processo ha la propria Page Table, che mappa le pagine logiche ai frame fisici.
- **Traduzione dell'Indirizzo**: L'indirizzo logico è diviso in due parti: il numero di pagina (page number, p) e l'offset all'interno della pagina (offset, d). La Page Table usa il numero di pagina per trovare il frame corrispondente, e l'offset viene aggiunto al frame address per ottenere l'indirizzo fisico.
- **TLB (Translation Lookaside Buffer)**: Una cache hardware veloce che memorizza le mappature pagina-frame usate più frequentemente per accelerare il processo di traduzione degli indirizzi e compensare il fatto che ogni accesso alla memoria richiede due accessi (uno alla Page Table e uno al dato).
- **Vantaggi**:
  - Elimina la **frammentazione esterna** (non ci sono buchi inutilizzati tra i blocchi allocati).
  - Semplifica l'allocazione della memoria.
- **Svantaggi**:
  - Introduce la **frammentazione interna** (il frame finale potrebbe non essere completamente riempito dalla pagina, lasciando spazio inutilizzato).
  - Overhead della Page Table (può essere grande e richiede memoria).

Segmentazione (Segmentation)

La segmentazione è una tecnica di gestione della memoria che permette di visualizzare la memoria come una collezione di segmenti di dimensione variabile. Ogni segmento corrisponde a un'unità logica del programma (es. codice, dati, stack, subroutine).

- **Funzionamento**: L'indirizzo logico è composto da un numero di segmento e un offset. La **Segment Table** (Tabella dei Segmenti) memorizza l'indirizzo base e la lunghezza di ciascun segmento in memoria fisica. La traduzione avviene verificando che l'offset non superi la lunghezza del segmento e aggiungendo l'offset all'indirizzo base del segmento.
- **Vantaggi**:
  - Riflette la visione logica del programma (moduli, funzioni).
  - Facilita la protezione e la condivisione di segmenti tra processi.
- **Svantaggi**:

  - Soffre di **frammentazione esterna** (spazi liberi tra i segmenti che possono essere troppo piccoli per nuove allocazioni).
  - La gestione delle dimensioni variabili dei segmenti è più complessa.

1.3.2 Algoritmi di Allocazione della Memoria (per Blocchi Contigui)

Quando la memoria fisica viene allocata in blocchi contigui (come nella segmentazione o senza memoria virtuale), il sistema operativo deve decidere quale blocco libero assegnare a una richiesta di memoria.

- **First Fit**:
  - **Descrizione**: Alloca il primo blocco libero che trova che sia abbastanza grande da soddisfare la richiesta.
  - **Vantaggi**: Veloce da implementare, spesso porta a buoni risultati.
  - **Svantaggi**: Può lasciare molti piccoli blocchi liberi all'inizio della lista, o frammentare blocchi grandi.
- **Best Fit**:
  - **Descrizione**: Alloca il blocco libero più piccolo che sia comunque sufficientemente grande a soddisfare la richiesta.
  - **Vantaggi**: Lascia i blocchi liberi più grandi intatti per richieste future, riducendo la frammentazione interna.
  - **Svantaggi**: Richiede una scansione completa della lista dei blocchi liberi (o una ricerca in una lista ordinata), quindi è più lento. Genera molti blocchi liberi molto piccoli.
- **Worst Fit**:
  - **Descrizione**: Alloca il blocco libero più grande disponibile.
  - **Vantaggi**: Si cerca di lasciare un blocco residuo il più grande possibile dopo l'allocazione, teoricamente utile per richieste future.
  - **Svantaggi**: Spesso è l'algoritmo peggiore in termini di frammentazione. Rende la ricerca più lenta e frammenta i blocchi più grandi.

1.4 Gestione dei File System

La **gestione dei file system** è un componente fondamentale del sistema operativo, responsabile di organizzare, archiviare e recuperare i dati su dispositivi di memoria secondaria (come dischi rigidi, SSD). Fornisce un'interfaccia logica per l'utente e le applicazioni per interagire con i dati, astraendo i dettagli di basso livello dell'hardware di archiviazione.

1.4.1 Organizzazione Fisica dei Dati su Disco

La memoria secondaria è tipicamente organizzata in unità di archiviazione fisiche che il file system gestisce.

- **Settori**: La più piccola unità fisica di memorizzazione su un disco.
- **Blocchi (o Cluster)**: L'unità di trasferimento logica più piccola riconosciuta dal file system. Un blocco è composto da uno o più settori contigui. La dimensione del blocco è un compromesso: blocchi grandi riducono l'overhead di I/O ma aumentano la frammentazione

interna; blocchi piccoli aumentano l'overhead di I/O ma riducono la frammentazione interna.
- **Cilindri/Tracce**: Concetti fisici legati ai dischi rotanti (hard disk), dove le tracce sono anelli concentrici e i cilindri sono l'insieme di tracce alla stessa distanza dal centro su tutti i piatti.

Struttura delle Directory

Le directory sono strutture che organizzano i file e altre directory in una gerarchia.

- **Struttura a Singolo Livello**: Tutti i file sono nella stessa directory. Semplice, ma difficile da gestire per molti utenti/file.
- **Struttura a Due Livelli**: Ogni utente ha la propria directory principale, separata dagli altri. Impedisce conflitti di nomi tra utenti, ma non offre organizzazione interna.
- **Struttura Ad Albero (Gerarchica)**: La struttura più comune, con una directory radice e sottodirectory. Offre flessibilità e organizzazione.
- **Grafo Aciclico Diretto (DAG)**: Permette la condivisione di file e directory tra diversi percorsi (tramite link hard o soft/symbolic).
- **Implementazione Interna**: Le directory possono essere implementate come liste di voci (nome file, puntatore a i-node/FAT entry) o tabelle hash per una ricerca più veloce.

1.4.2 Metodi di Allocazione dei File

Il metodo di allocazione determina come i blocchi di un file sono memorizzati e gestiti sul disco, influenzando l'efficienza di accesso e la gestione dello spazio.

Allocazione Contigua

- **Descrizione**: Ogni file è memorizzato come un blocco contiguo di blocchi sul disco. Il file system memorizza solo l'indirizzo del primo blocco e la lunghezza del file.
- **Vantaggi**:
  - Semplice da implementare.
  - Ottime prestazioni per l'accesso sequenziale e diretto (un solo seek per leggere un file intero, accesso diretto immediato a qualsiasi blocco).
- **Svantaggi**:
  - **Frammentazione Esterna**: Con il tempo, lo spazio libero sul disco si frammenta in piccoli buchi non contigui, rendendo difficile trovare blocchi contigui grandi per nuovi file, anche se c'è molto spazio totale disponibile.
  - Difficile prevedere la dimensione finale dei file.
  - Richiede compattazione periodica per recuperare spazio contiguo (operazione costosa).

Allocazione Collegata (Linked Allocation)

- **Descrizione**: Ogni file è una lista collegata di blocchi su disco. Ogni blocco contiene un puntatore al blocco successivo. Il file system memorizza solo l'indirizzo del primo blocco.
- **Vantaggi**:
  - Nessuna frammentazione esterna.
  - Allocazione dinamica e facile espansione dei file.
- **Svantaggi**:

  - Lento per l'accesso diretto/casuale (bisogna seguire la catena di puntatori).
  - Spazio sprecato per i puntatori in ogni blocco (anche se questo è spesso mitigato da una File Allocation Table - FAT).
  - Rischio di perdita dell'intera catena se un puntatore è corrotto.

Allocazione Indicizzata (Indexed Allocation)

- **Descrizione**: Ogni file ha un blocco indice dedicato, che è un array di puntatori ai blocchi di dati effettivi del file. Il file system memorizza solo l'indirizzo del blocco indice.
- **Vantaggi**:
  - Combina i vantaggi dell'allocazione contigua e collegata: supporta sia l'accesso sequenziale che diretto.
  - Nessuna frammentazione esterna.
  - Facile espansione (aggiungendo puntatori al blocco indice o blocchi indice multipli/gerarchici).
- **Svantaggi**:
  - Overhead dello spazio per i blocchi indice.
  - La dimensione del file è limitata dalla dimensione del blocco indice.
- **Implementazione Comune**: **i-nodes (Unix/Linux)**: Ogni file ha un i-node che contiene metadati del file e un array di puntatori ai blocchi di dati, inclusi puntatori a blocchi indiretti per file di grandi dimensioni. **FAT (File Allocation Table - MS-DOS/Windows)**: Una tabella sul disco che memorizza le catene di blocchi per ogni file. Non è esattamente un blocco indice per file singoli, ma un array centrale di puntatori.

1.5 Sincronizzazione e Deadlock

Nei sistemi operativi multi-programmati o multi-thread, dove più processi o thread condividono risorse e dati, è fondamentale garantire la **sincronizzazione** per mantenere la coerenza dei dati e prevenire condizioni di errore come le race condition. La mancanza di sincronizzazione può portare anche a situazioni di **deadlock**.

1.5.1 Problemi di Sincronizzazione

- **Race Condition (Condizione di Corsa)**: Si verifica quando più processi o thread accedono e manipolano dati condivisi concorrentemente, e il risultato finale dell'esecuzione dipende dall'ordine in cui le operazioni dei processi/thread si intersecano. Il risultato non è predicibile.
- **Sezione Critica (Critical Section)**: Una sezione di codice in cui un processo o thread accede a risorse condivise (variabili, file, periferiche). L'obiettivo della sincronizzazione è garantire che, in un dato istante, al massimo un processo/thread sia nella sua sezione critica per quella risorsa.
- **Problema della Sezione Critica (Critical Section Problem)**: Progettare protocolli per garantire che i processi cooperanti accedano alle sezioni critiche in modo sicuro, rispettando le seguenti condizioni:
  - **Mutua Esclusione (Mutual Exclusion)**: Se un processo è nella sua sezione critica, nessun altro processo può entrare nella propria sezione critica.
  - **Progresso (Progress)**: Se nessun processo è nella sezione critica e alcuni processi vogliono entrare, solo quei processi che non sono nella loro sezione remainder possono

partecipare alla decisione di quale processo entrerà nella sezione critica successiva, e questa decisione non deve essere ritardata indefinitamente.
  - **Attesa Limitata (Bounded Waiting)**: Esiste un limite al numero di volte in cui altri processi possono entrare nelle loro sezioni critiche dopo che un processo ha richiesto di entrare nella sua sezione critica e prima che la sua richiesta venga soddisfatta. Questo previene la starvation.

1.5.2 Meccanismi di Sincronizzazione

Diversi strumenti e meccanismi sono stati sviluppati per risolvere il problema della sezione critica:

- **Lock**: Un meccanismo semplice che permette di acquisire e rilasciare un blocco su una risorsa. Prima di entrare nella sezione critica, un processo acquisisce il lock; dopo essere uscito, lo rilascia. Se il lock è già acquisito, il processo deve attendere.
- **Semafori**: Un tipo di variabile intera (generalmente non negativa) a cui si può accedere solo tramite due operazioni atomiche:
  - `wait()` (o `P()`): Decrementa il valore del semaforo. Se il valore diventa negativo, il processo viene bloccato.
  - `signal()` (o `V()`): Incrementa il valore del semaforo. Se ci sono processi bloccati sul semaforo, uno di essi viene sbloccato.
  - **Semafori Binari (Mutex Lock)**: Valori 0 o 1. Usati per la mutua esclusione.
  - **Semafori Contatori**: Valori interi. Usati per gestire l'accesso a un numero finito di risorse.
- **Esempio di Problema Produttore-Consumatore con Semafori**: Un problema classico di sincronizzazione dove un produttore genera dati e li inserisce in un buffer, mentre un consumatore preleva dati dal buffer.

```
// Semafori:
// empty: contatore (inizializzato a N, dimensione buffer) -> numero di
    slot vuoti
// full: contatore (inizializzato a 0) -> numero di slot pieni
// mutex: binario (inizializzato a 1) -> mutua esclusione per accesso al
    buffer

Producer:
  LOOP
    produce item
    CALL wait(empty)
    CALL wait(mutex)
    add item to buffer
    CALL signal(mutex)
    CALL signal(full)
  END LOOP

Consumer:
  LOOP
    CALL wait(full)
    CALL wait(mutex)
    remove item from buffer
    CALL signal(mutex)
```

```
    CALL signal(empty)
    consume item
  END LOOP
```

- **Monitor**: Un costrutto di sincronizzazione di alto livello che incapsula dati condivisi e le procedure che li manipolano. Garantisce che, in ogni momento, al massimo un processo possa essere attivo all'interno del monitor. Utilizza **variabili di condizione** con operazioni 'wait()' e 'signal()' per sospendere e riattivare i processi che devono attendere specifiche condizioni sui dati condivisi.

1.5.3 Deadlock (Interblocco)

Il **deadlock** è una situazione in cui due o più processi sono bloccati indefinitamente, in attesa di una risorsa detenuta da un altro processo bloccato.

Condizioni Necessarie per il Deadlock (Condizioni di Coffman)

Il deadlock può verificarsi se e solo se tutte e quattro le seguenti condizioni sono presenti contemporaneamente:

1. **Mutua Esclusione (Mutual Exclusion)**: Almeno una risorsa deve essere tenuta in modalità non-condivisibile, cioè al massimo un processo può usarla alla volta.
2. **Attesa e Mantenimento (Hold and Wait)**: Un processo deve detenere almeno una risorsa ed essere in attesa di acquisirne altre attualmente detenute da altri processi.
3. **Non-Preemptive (Nessuna Preemption)**: Le risorse non possono essere sottratte a un processo che le detiene; possono essere rilasciate solo volontariamente dal processo che le detiene.
4. **Attesa Circolare (Circular Wait)**: Deve esistere una catena circolare di due o più processi, in cui ogni processo in attesa nella catena sta aspettando una risorsa detenuta dal processo successivo nella catena.

Strategie di Gestione del Deadlock

Per affrontare il deadlock, i sistemi operativi possono adottare diverse strategie:

- **Prevenzione del Deadlock (Deadlock Prevention)**:
  - Obiettivo: Garantire che almeno una delle quattro condizioni necessarie non si verifichi.
  - Come: Negando una o più condizioni (es. non permettere l'attesa e mantenimento, assegnare tutte le risorse all'inizio).
  - Svantaggi: Spesso porta a un basso utilizzo delle risorse e a un throughput ridotto.
- **Evitamento del Deadlock (Deadlock Avoidance)**:
  - Obiettivo: Richiede che il sistema abbia informazioni a priori sulle risorse che un processo richiederà. Il sistema verifica se lo stato corrente è "sicuro" (se esiste una sequenza di esecuzione dei processi che eviterà il deadlock).
  - Algoritmo Esempio: Algoritmo del banchiere.
  - Vantaggi: Meno restrittivo della prevenzione, migliore utilizzo delle risorse.
  - Svantaggi: Richiede conoscenza a priori, può essere computazionalmente costoso.
- **Rilevamento e Ripristino del Deadlock (Deadlock Detection and Recovery)**:

  - Obiettivo: Permettere che il deadlock si verifichi, rilevarlo e poi ripristinare il sistema da esso.
  - Rilevamento: Si usa un algoritmo di rilevamento del ciclo nel grafo di allocazione delle risorse.
  - Ripristino:
    * Terminazione del processo: Terminare uno o più processi coinvolti nel deadlock.
    * Preemption della risorsa: Sottrarre risorse a un processo e assegnarle a un altro.
  - Svantaggi: Comporta un overhead per il rilevamento e la perdita di lavoro per il ripristino.
- **Ignorare il Problema**:
  - Questo è l'approccio più comune in molti sistemi operativi (es. Unix/Linux, Windows), assumendo che il deadlock sia un evento raro e che sia meno costoso lasciarlo gestire all'amministratore (riavvio del sistema) piuttosto che implementare algoritmi complessi.

1.6 Scheduling della CPU

Lo **scheduling della CPU** è l'attività di selezionare quale processo, tra quelli pronti per l'esecuzione, deve essere assegnato alla CPU in un dato momento. È una funzione fondamentale del sistema operativo che ha un impatto cruciale sulle performance complessive del sistema, influenzando parametri come il throughput (numero di processi completati per unità di tempo), il tempo di risposta (tempo tra richiesta e prima risposta), e l'equità nella distribuzione delle risorse della CPU tra i processi.

1.6.1 Principali Problematiche dello Scheduling

Lo scheduling deve affrontare diverse sfide e problematiche per bilanciare l'efficienza e l'equità:

- **Ottimizzazione degli obiettivi**: Bilanciare metriche contrastanti come massimizzare il throughput, minimizzare il tempo di risposta, minimizzare il tempo di attesa e garantire l'equità tra i processi.
- **Contesto Switching (Cambio di Contesto)**: L'overhead di tempo necessario per salvare lo stato di un processo in esecuzione e caricare lo stato del prossimo processo da eseguire. Questo tempo è "sprecato" e non contribuisce all'esecuzione del lavoro utile.
- **Starvation (Inedia)**: Un processo a bassa priorità potrebbe non essere mai eseguito se processi a priorità più alta arrivano continuamente e monopolizzano la CPU.
- **Deadlock**: Sebbene sia una problematica più ampia della gestione della concorrenza, situazioni di deadlock possono emergere in sistemi con scheduling se le risorse non sono gestite correttamente, bloccando indefinitamente i processi.
- **Dipendenza dall'I/O**: Processi che trascorrono molto tempo in attesa di operazioni di I/O (I/O-bound) possono rendere inefficiente lo scheduling se la CPU rimane inattiva mentre attende il completamento di tali operazioni.

1.6.2 Esempi di Algoritmi di Scheduling

Diversi algoritmi sono stati sviluppati per affrontare le problematiche dello scheduling, ognuno con i propri compromessi tra efficienza, equità e complessità di implementazione.

First-Come, First-Served (FCFS)

- **Descrizione**: Non preemptive. I processi vengono eseguiti nell'ordine in cui arrivano nella coda dei processi pronti. Una volta che un processo ottiene la CPU, la tiene fino al completamento o fino a quando non esegue un'operazione di I/O.
- **Vantaggi**: Semplice da implementare e comprendere.
- **Svantaggi**:
  - **"Effetto Convoglio"**: Un processo con un tempo di esecuzione molto lungo può bloccare tutti i processi successivi più brevi, aumentando notevolmente il tempo medio di attesa.
  - Tempo di risposta e throughput possono essere scarsi in scenari sfavorevoli.
- **Esempio**: Consideriamo i processi P1 (burst time 24), P2 (burst time 3), P3 (burst time 3) che arrivano nell'ordine P1, P2, P3.

```
Gantt Chart (FCFS):
| P1 (24) | P2 (3) | P3 (3) |
0        24       27       30
Tempo di attesa:
P1 = 0
P2 = 24
P3 = 27
Tempo medio di attesa = (0 + 24 + 27) / 3 = 17
```

Shortest-Job-First (SJF)

- **Descrizione**: Può essere preemptive (chiamato Shortest-Remaining-Time-First, SRTF) o non preemptive. Assegna la CPU al processo che ha il tempo di esecuzione stimato più breve.
- **Vantaggi**: Ottimale per minimizzare il tempo medio di attesa per un dato insieme di processi.
- **Svantaggi**:
  - **Difficoltà di stima**: È difficile conoscere a priori la durata esatta del "burst time" di un processo (si usano stime basate sulla storia passata).
  - **Starvation**: Processi lunghi potrebbero non essere mai eseguiti se arrivano continuamente processi più brevi.
- **Esempio (Non Preemptive)**: Processi P1 (burst 7), P2 (burst 4), P3 (burst 1), P4 (burst 4). Arrivano quasi contemporaneamente.

```
Gantt Chart (SJF Non-Preemptive):
| P3 (1) | P2 (4) | P4 (4) | P1 (7) |
0        1        5        9        16
Tempo di attesa:
P3 = 0
P2 = 1
P4 = 5
P1 = 9
Tempo medio di attesa = (0 + 1 + 5 + 9) / 4 = 3.75
```

Priority Scheduling

- **Descrizione**: Può essere preemptive o non preemptive. Ad ogni processo viene assegnata una priorità (un numero intero, dove un numero più basso può indicare una priorità più alta, o viceversa). La CPU viene assegnata al processo con la priorità più alta.
- **Vantaggi**: Permette di prioritizzare lavori critici o importanti del sistema.
- **Svantaggi**:
  - **Starvation**: Processi a bassa priorità potrebbero non essere mai eseguiti se processi a priorità più alta arrivano continuamente.
  - **Soluzione per Starvation (Aging)**: La priorità di un processo che aspetta da troppo tempo viene gradualmente aumentata.
- **Esempio**: Processi P1 (burst 10, priority 3), P2 (burst 1, priority 1), P3 (burst 2, priority 4), P4 (burst 1, priority 5), P5 (burst 5, priority 2). (Priorità più basse = priorità più alte).

```
Gantt Chart (Priority Scheduling, Non-Preemptive):
| P2 (1) | P5 (5) | P1 (10) | P3 (2) | P4 (1) |
0        1        6         16       18       19
Tempo di attesa (processi ordinati per arrivo, non priorita'):
P1 = 6
P2 = 0
P3 = 16
P4 = 18
P5 = 1
Tempo medio di attesa = (6 + 0 + 16 + 18 + 1) / 5 = 8.2
```

Round Robin (RR)

- **Descrizione**: Preemptive. Progettato per sistemi time-sharing. Ogni processo ottiene una piccola porzione di tempo di CPU, chiamata "quantum" (o time slice), solitamente da 10 a 100 millisecondi. Se un processo non finisce entro il quantum, viene preempted e messo in coda alla fine della coda dei processi pronti.
- **Vantaggi**:
  - **Equità**: Garantisce che nessun processo aspetti per un tempo eccessivamente lungo.
  - **Buon Tempo di Risposta**: Adatto per processi interattivi, dando l'impressione che tutti i processi siano eseguiti contemporaneamente.
- **Svantaggi**:
  - L'overhead del context switching aumenta se il quantum è troppo piccolo (troppi cambi di contesto).
  - Le performance degradano se il quantum è troppo grande (il RR tende a comportarsi come FCFS).
  - Può essere meno efficiente se i processi hanno durate molto diverse.
- **Esempio**: Processi P1 (burst 24), P2 (burst 3), P3 (burst 3). Quantum = 4.

```
Gantt Chart (Round Robin, Quantum=4):
| P1 (4) | P2 (3) | P3 (3) | P1 (4) | P1 (4) | P1 (4) | P1 (4) | P1 (4) |
0        4        7        10       14       18       22       26
   30
```

```
Tempo di attesa:
P1 = (4+3+3+3+3) = 16 (somma dei tempi non consecutivi)
P2 = 4
P3 = 7
Tempo medio di attesa = (16 + 4 + 7) / 3 = 9
```

Multilevel Queue Scheduling

- **Descrizione**: I processi sono divisi in diverse code, ognuna con il proprio algoritmo di scheduling. Le code possono avere priorità fisse tra di loro, oppure essere gestite con time slices diversi per ogni coda.
- **Esempio**: Coda foreground (processi interattivi) con Round Robin; Coda background (processi batch) con FCFS. I processi non si muovono tra le code.

Multilevel Feedback Queue Scheduling

- **Descrizione**: Permette ai processi di muoversi tra le code in base al loro comportamento. Se un processo usa troppo la CPU, viene spostato in una coda con priorità inferiore o un quantum più grande. Se un processo aspetta molto, può essere spostato in una coda con priorità più alta.
- **Vantaggi**: Altamente configurabile, può approssimare SJF senza conoscere i burst time, può prevenire la starvation (tramite l'aging), e offrire una buona risposta per processi interattivi.
- **Svantaggi**: Molto complesso da implementare e ottimizzare, richiede molti parametri (numero di code, algoritmo per ogni coda, quando spostare i processi tra code).

Domande e Esercizi

Domanda: Struttura e Organizzazione del Sistema Operativo, e Scheduling della CPU

Domanda: Il candidato fornisca una panoramica sulla struttura e organizzazione di un sistema operativo, descrivendo i principali componenti e modelli di sistemi operativi. Si approfondisca, inoltre, il tema dello scheduling della CPU, evidenziando le principali problematiche che questo comporta e illustrando esempi di algoritmi.

Risposta :

Un **sistema operativo (SO)** è un software di sistema che gestisce le risorse hardware e software di un computer, fungendo da interfaccia tra l'hardware e l'utente/applicazioni.

I suoi componenti principali includono:

- **Kernel**: Il cuore del SO, che gestisce processi, memoria, file system e I/O.
- **Gestore dei Processi**: Si occupa della creazione, terminazione e scheduling dei processi.
- **Gestore della Memoria**: Responsabile dell'allocazione, protezione e gestione della memoria virtuale.
- **File System Management**: Organizzazione dei dati su storage secondario.
- **Gestore I/O**: Interazione con i dispositivi hardware.
- **Network Management**: Gestisce le comunicazioni di rete. **Security and Protection**: Protezione delle risorse.

I modelli architettonici dei SO possono essere:

- **Monolitici**
- **Layered** (a strati)
- **Microkernel**
- **Modulari** (Ibridi)

Ciascuno con vantaggi e svantaggi in termini di performance, flessibilità e robustezza.

Lo **scheduling della CPU** è l'attività di selezionare quale processo, tra quelli pronti per l'esecuzione, deve essere assegnato alla CPU in un dato momento. Ha un impatto cruciale sulle performance complessive del sistema. Le principali problematiche dello scheduling includono l'ottimizzazione di obiettivi contrastanti (throughput, tempo di risposta, tempo di attesa, equità), l'overhead del contesto switching, la starvation (processi a bassa priorità che non vengono eseguiti) e la gestione implicita di situazioni di deadlock. Diversi algoritmi di scheduling sono utilizzati:

- **First-Come, First-Served (FCFS)**: Non preemptive; esegue i processi in ordine di arrivo. Semplice, ma soffre l'effetto convoglio.
- **Shortest-Job-First (SJF)**: Può essere preemptive o non preemptive; esegue il processo con il burst time stimato più breve. Ottimale per tempo medio di attesa, ma difficile stimare la durata.
- **Priority Scheduling**: Assegna la CPU al processo con priorità più alta. Permette di prioritizzare lavori critici, ma può causare starvation (risolvibile con l'aging).

- **Round Robin (RR)**: Preemptive; ogni processo ottiene un quantum di tempo. Equo e con buon tempo di risposta, ma con overhead di contesto switching.
- **Multilevel Queue Scheduling**: I processi sono divisi in diverse code, ognuna con il proprio algoritmo.
- **Multilevel Feedback Queue Scheduling**: Permette ai processi di muoversi tra le code per adattarsi al loro comportamento, bilanciando efficienza e fairness.

Domanda: Processi e Thread, Sincronizzazione e Deadlock

Domanda: Il Candidato introduca i concetti di Processo "pesante" e "leggero" (Thread), illustrandone i punti in comune e le differenze. Si illustrino poi brevemente i possibili meccanismi di sincronizzazione tra processi al fine di evitare deadlock, quali ad esempio il meccanismo di lock su risorse, i “semafori", etc. Il Candidato illustri poi i concetti tramite un breve esempio a sua scelta.

Risposta :

Processi ("Pesanti") e Thread ("Leggeri") Nei sistemi operativi moderni, l’esecuzione dei programmi è gestita attraverso i processi e i thread, che rappresentano unità di esecuzione con diverse caratteristiche di gestione delle risorse.

Un **processo** è un’istanza di un programma in esecuzione, che possiede il proprio spazio di indirizzamento virtuale isolato e risorse dedicate come codice, dati, stack, Program Counter (PC) e registri CPU, oltre a file aperti e segnali. I processi sono considerati "pesanti" a causa dell’overhead significativo nella loro creazione, distruzione e nel cambio di contesto. Questo isolamento garantisce che il crash di un processo non influenzi direttamente gli altri.

Un **thread** (o thread di esecuzione) è un’unità di esecuzione più "leggera" all’interno di un processo. Più thread possono esistere all’interno dello stesso processo e condividono lo stesso spazio di indirizzamento virtuale, il codice del programma, i dati globali e le risorse del sistema operativo (es. file aperti). Ogni thread, tuttavia, ha il proprio Program Counter, il proprio set di registri della CPU e uno stack separato. La natura "leggera" dei thread si traduce in una creazione, distruzione e un cambio di contesto molto più rapidi ed efficienti rispetto ai processi.

Punti in Comune:

- Sia i processi che i thread sono unità di esecuzione che possono essere schedulate dalla CPU.
- Entrambi mantengono un Program Counter, un set di registri e uno stack (anche se lo stack è separato per ogni thread).

Differenze Principali:

- **Isolamento delle Risorse**: I processi operano in spazi di indirizzamento separati e con risorse isolate, fornendo una maggiore protezione. I thread all’interno dello stesso processo condividono lo spazio di indirizzamento e le risorse, rendendo la comunicazione più semplice ma con meno isolamento.
- **Overhead**: I processi comportano un overhead maggiore per la gestione del loro ciclo di vita e il cambio di contesto. I thread sono più efficienti in queste operazioni.
- **Comunicazione**: La comunicazione tra processi (IPC) è più complessa e richiede meccanismi specifici. La comunicazione tra thread (ITC) è più diretta grazie alla memoria condivisa.

- **Robustezza**: Un errore critico in un thread può causare il crash dell'intero processo e di tutti i suoi thread. Un processo in crash solitamente non influisce sugli altri processi del sistema.

Meccanismi di Sincronizzazione e Deadlock Nei sistemi multi-programmati o multi-thread, la **sincronizzazione** è cruciale per prevenire **Race Condition** (risultati imprevedibili dovuti all'accesso concorrente a dati condivisi) e garantire la coerenza dei dati. La **sezione critica** è la porzione di codice in cui si accede a risorse condivise, e la mutua esclusione è l'obiettivo primario per garantire che un solo processo/thread vi acceda per volta.

I **meccanismi di sincronizzazione** includono:

- **Lock**: Strumenti che permettono di acquisire un blocco su una risorsa prima di accedervi e rilasciarlo al termine, forzando l'attesa di altri processi.
- **Semafori**: Variabili intere gestite da operazioni atomiche 'wait()' (decrementa e blocca se negativo) e 'signal()' (incrementa e sblocca se ci sono processi in attesa). Esistono semafori binari (mutex) per la mutua esclusione e semafori contatori per risorse multiple. **Esempio (Produttore-Consumatore con Semafori)**: Un problema classico in cui un produttore aggiunge elementi a un buffer e un consumatore li preleva, utilizzando semafori ('empty' per slot vuoti, 'full' per slot pieni, 'mutex' per l'accesso al buffer) per coordinare l'accesso e prevenire la sovrascrittura/lettura di dati non validi.

```
// Semafori: empty (N), full (0), mutex (1)

Producer:
  LOOP
    produce item
    CALL wait(empty)
    CALL wait(mutex)
    add item to buffer
    CALL signal(mutex)
    CALL signal(full)
  END LOOP

Consumer:
  LOOP
    CALL wait(full)
    CALL wait(mutex)
    remove item from buffer
    CALL signal(mutex)
    CALL signal(empty)
    consume item
  END LOOP
```

- **Monitor**: Costrutti di alto livello che incapsulano dati condivisi e le procedure che li manipolano, garantendo l'accesso esclusivo tramite variabili di condizione ('wait()' e 'signal()') per la sospensione/attivazione dei processi.

Il **deadlock** (interblocco) si verifica quando due o più processi sono bloccati indefinitamente, ciascuno in attesa di una risorsa detenuta da un altro processo bloccato. Le **quattro condizioni necessarie** per il deadlock sono:

1. **Mutua Esclusione**: Risorse non condivisibili.
2. **Attesa e Mantenimento**: Un processo detiene risorse e ne attende altre.
3. **Non-Preemptive**: Le risorse non possono essere sottratte forzatamente.
4. **Attesa Circolare**: Esiste una catena circolare di dipendenze tra processi.

Le **strategie di gestione del deadlock** includono:

- **Prevenzione**: Negare una o più delle condizioni necessarie (es. richiedere tutte le risorse all'inizio).
- **Evitamento**: Utilizzare informazioni a priori per decidere se uno stato è "sicuro" prima di allocare risorse (es. Algoritmo del Banchiere).
- **Rilevamento e Ripristino**: Permettere il deadlock, rilevarlo tramite algoritmi di ricerca di cicli e poi ripristinare il sistema (es. terminando processi o preemption di risorse).
- **Ignorare il Problema**: Assumere che il deadlock sia raro e gestirlo manualmente (es. riavvio del sistema).

Domanda: Record di Attivazione

Domanda: Il Candidato illustri il concetto di record di attivazione, descriva cosa esso contenga ed il suo ciclo di vita. Presenti inoltre un esempio di record di attivazione per il caso di una funzione "pow2" che riceve in input un solo parametro "x" e lo ritorna elevato al quadrato.

Risposta :

Concetto di Record di Attivazione Il **Record di Attivazione** (o *Stack Frame*) è una struttura dati fondamentale creata sullo stack di esecuzione di un processo ogni volta che una funzione (o procedura, o subroutine) viene chiamata. La sua funzione principale è contenere tutte le informazioni necessarie per la gestione dell'esecuzione di quella specifica chiamata di funzione, fornendo il contesto per il suo funzionamento e il suo corretto ritorno al punto di chiamata.

Contenuto di un Record di Attivazione Un record di attivazione è una struttura complessa la cui esatta composizione può variare leggermente a seconda dell'architettura della CPU, del sistema operativo e del compilatore, ma tipicamente include:

- **Parametri Attuali**: I valori degli argomenti che vengono passati alla funzione durante la sua chiamata.
- **Indirizzo di Ritorno**: L'indirizzo di memoria dell'istruzione nel codice della funzione chiamante a cui il controllo del programma deve essere trasferito una volta che la funzione corrente ha terminato la sua esecuzione.
- **Valore di Ritorno**: Uno spazio riservato per memorizzare il risultato (se la funzione restituisce un valore) che verrà poi recuperato dalla funzione chiamante.
- **Variabili Locali**: Le variabili dichiarate all'interno del corpo della funzione. Queste variabili hanno scope e durata limitati all'attivazione corrente della funzione.
- **Stato dei Registri Salvati**: I valori dei registri della CPU che erano in uso dalla funzione chiamante e che vengono salvati per essere ripristinati al ritorno, garantendo che lo stato della CPU non sia corrotto dalla funzione chiamata.
- **Puntatore al Frame Precedente (Control Link / Dynamic Link)**: Un puntatore all'indirizzo del record di attivazione della funzione che ha effettuato la chiamata. Questo permette di "risalire" lo stack e ripristinare il contesto della funzione chiamante.

- **Puntatore al Contesto Statico (Access Link / Static Link)**: Un puntatore al record di attivazione della funzione che definisce lo scope lessicale della funzione corrente (rilevante in linguaggi con scope annidato statico, come Pascal, per accedere a variabili non locali).

Ciclo di Vita di un Record di Attivazione Il ciclo di vita di un record di attivazione è strettamente legato alla dinamica delle chiamate e dei ritorni delle funzioni, seguendo una logica LIFO (Last-In, First-Out) tipica delle strutture a stack:

1. **Creazione (Chiamata di Funzione)**: Quando un processo chiama una funzione, un nuovo record di attivazione viene creato (popolato con le informazioni rilevanti) e "pushed" (inserito) in cima allo stack di esecuzione del processo. Il puntatore dello stack viene aggiornato per puntare a questo nuovo frame. Il controllo viene poi passato all'inizio della funzione chiamata.
2. **Esecuzione**: La funzione utilizza i dati e le risorse definite all'interno del suo record di attivazione (parametri, variabili locali) per eseguire le sue operazioni. Durante l'esecuzione, può a sua volta chiamare altre funzioni, che a loro volta creeranno nuovi record di attivazione sul top dello stack.
3. **Terminazione (Ritorno da Funzione)**: Una volta che la funzione completa la sua esecuzione (sia raggiungendo un'istruzione 'return' che la fine del suo blocco di codice), il valore di ritorno (se presente) viene posizionato in un registro designato. Il record di attivazione corrente viene quindi "popped" (rimosso) dalla cima dello stack, liberando lo spazio di memoria che occupava.
4. **Ripristino del Contesto**: Il sistema operativo (o il runtime) utilizza l'indirizzo di ritorno salvato nel record appena rimosso per trasferire il controllo alla funzione chiamante, e ripristina lo stato dei registri della CPU per consentire alla funzione chiamante di riprendere l'esecuzione dal punto in cui era stata interrotta.

Esempio di Record di Attivazione per la funzione "pow2(x)" Consideriamo una semplice funzione 'pow2(x)' che calcola il quadrato del suo input 'x'.

```
FUNCTION pow2(x):
    RETURN x * x
END FUNCTION
```

Listing 1.1: Funzione pow2(x)

Quando la funzione 'pow2(5)' viene chiamata, viene creato il seguente record di attivazione (semplificato) sullo stack:

- **Parametri Attuali**:
  - 'x': 5
- **Indirizzo di Ritorno**: L'indirizzo nel codice della funzione chiamante da cui 'pow2(5)' è stata invocata (es. '0x00A0' nel chiamante).
- **Valore di Ritorno**: Spazio per il risultato (es. 25).
- **Variabili Locali**: Nessuna in questo esempio specifico (o temporanee usate dal compilatore).
- **Stato dei Registri Salvati**: Valori dei registri che devono essere preservati per la funzione chiamante.

- **Puntatore al Frame Precedente**: Indirizzo del record di attivazione della funzione chiamante.

Ciclo di Vita Semplificato per 'pow2(5)':

1. **Chiamata**: La funzione chiamante spinge i parametri e l'indirizzo di ritorno sullo stack. Viene creato e spinto il record di attivazione per 'pow2(5)'.
2. **Esecuzione**: 'pow2' prende il valore di 'x' (5), calcola '5 * 5 = 25'.
3. **Ritorno**: Il valore 25 viene posizionato dove la funzione chiamante lo recupererà. Il record di attivazione di 'pow2' viene rimosso dallo stack.
4. **Ripristino**: Il controllo torna all'indirizzo di ritorno nella funzione chiamante, che continua la sua esecuzione.

Questo illustra come il record di attivazione gestisce il contesto di una singola invocazione di funzione, permettendo l'esecuzione modulare dei programmi.

Altre Possibili Domande

Domanda: Come la Memoria Virtuale Migliora la Gestione della Memoria nei Sistemi Operativi?

Domanda: Come la memoria virtuale migliora la gestione della memoria nei sistemi operativi?

Risposta:

La **memoria virtuale** è una tecnica del sistema operativo che permette di usare lo spazio su disco (memoria secondaria) per simulare una RAM maggiore, consentendo ai programmi di utilizzare più memoria di quella fisica disponibile. Questo facilita il multitasking isolando lo spazio di indirizzamento di ogni processo, aumentandone la protezione e la flessibilità. Le tecniche principali per implementare la memoria virtuale sono la paginazione e la segmentazione.

- **Paginazione**: Divide lo spazio logico in "pagine" di dimensione fissa e la memoria fisica in "frame", mappando le pagine ai frame tramite una Page Table. Questo elimina la frammentazione esterna ma introduce quella interna.
- **Segmentazione**: Vede la memoria come segmenti di dimensione variabile, corrispondenti a unità logiche del programma, facilitando protezione e condivisione ma soffrendo di frammentazione esterna.

Per l'allocazione della memoria (per blocchi contigui), il sistema operativo utilizza algoritmi come **First Fit** (alloca il primo blocco sufficiente), **Best Fit** (alloca il blocco più piccolo sufficiente) e **Worst Fit** (alloca il blocco più grande sufficiente), ognuno con diversi compromessi in termini di velocità e frammentazione.

Domanda: Quali sono i Problemi Principali Legati alla Sincronizzazione dei Processi e Come Vengono Gestiti i Deadlock?

Domanda: Quali sono i problemi principali legati alla sincronizzazione dei processi e come vengono gestiti i deadlock?

Risposta:

Nei sistemi multi-programmati o multi-thread, la **sincronizzazione** è cruciale per evitare "**Race Condition**", dove il risultato delle operazioni su dati condivisi dipende dall'ordine di esecuzione,

portando a risultati imprevedibili. La "**Sezione Critica**" è una porzione di codice in cui un processo accede a risorse condivise, e l'obiettivo è garantire la "**Mutua Esclusione**" (solo un processo alla volta nella sezione critica), il "**Progresso**" (decisione non ritardata indefinitamente) e l'"**Attesa Limitata**" (nessuna starvation). **Meccanismi di sincronizzazione** includono:

- **Lock**: Meccanismi semplici per acquisire e rilasciare un blocco su una risorsa.
- **Semafori**: Variabili intere gestite da operazioni atomiche 'wait()' e 'signal()', usate per mutua esclusione (semafori binari/mutex) o controllo di risorse (semafori contatori).
- **Monitor**: Costrutti di alto livello che incapsulano dati condivisi e procedure, garantendo l'accesso esclusivo tramite variabili di condizione.

Il "**Deadlock**" (interblocco) si verifica quando due o più processi sono bloccati indefinitamente in attesa di risorse detenute da altri processi bloccati. Le **quattro condizioni necessarie** per il deadlock sono:

1. **Mutua Esclusione** (risorse non condivisibili).
2. **Attesa e Mantenimento** (un processo detiene risorse e ne attende altre).
3. **Non-Preemption** (le risorse non possono essere sottratte forzatamente).
4. **Attesa Circolare** (catena circolare di dipendenze).

Le **strategie per gestire il deadlock** includono:

- **Prevenzione**: Negare una o più delle condizioni necessarie (es. richiedere tutte le risorse all'inizio).
- **Evitamento**: Utilizzare informazioni a priori per decidere se uno stato è "sicuro" prima di allocare risorse (es. Algoritmo del Banchiere).
- **Rilevamento e Ripristino**: Permettere il deadlock, rilevarlo tramite algoritmi e poi ripristinare il sistema (es. terminando processi o sottraendo risorse).
- **Ignorare il Problema**: Assumere che il deadlock sia raro e gestirlo manualmente (es. riavvio del sistema).

Domanda: Come Funziona lo Scheduling della CPU e Quali Sono gli Algoritmi Più Comuni?

Domanda: Come funziona lo scheduling della CPU e quali sono gli algoritmi più comuni?

Risposta:

Lo **scheduling della CPU** è il processo di selezione del prossimo processo da assegnare alla CPU tra quelli pronti, con l'obiettivo di ottimizzare throughput, tempo di risposta ed equità. Le problematiche includono l'overhead del context switching, la starvation (processi a bassa priorità mai eseguiti) e la dipendenza dall'I/O. Esempi di algoritmi di scheduling includono:

- **First-Come, First-Served (FCFS)**: Non-preemptive, i processi vengono eseguiti nell'ordine di arrivo. Semplice ma può soffrire dell'"Effetto Convoglio".
- **Shortest-Job-First (SJF)**: Assegna la CPU al processo con il tempo di esecuzione stimato più breve (può essere preemptive come SRTF). Ottimale per minimizzare il tempo medio di attesa ma difficile da stimare e può causare starvation.
- **Priority Scheduling**: Assegna la CPU al processo con priorità più alta. Vantaggioso per lavori critici ma soggetto a starvation, mitigabile con l'Aging.

- **Round Robin (RR)**: Preemptive, ogni processo ottiene un piccolo "quantum" di CPU. Garantisce equità e buon tempo di risposta per processi interattivi, ma l'overhead del context switching aumenta con quantum piccoli.
- **Multilevel Queue Scheduling** e **Multilevel Feedback Queue Scheduling**: Dividono i processi in code con algoritmi e priorità diverse, con l'ultimo che permette ai processi di spostarsi tra le code per prevenire starvation e ottimizzare la risposta.

Capitolo 2

Basi di Dati

Le **Basi di Dati (Database)** sono collezioni organizzate di dati che permettono un'efficiente memorizzazione, recupero e gestione delle informazioni. Sono fondamentali per la maggior parte delle applicazioni software moderne.

2.1 Modello Concettuale Entità-Relazione (ER)

Il **Modello Entità-Relazione (ER)** è uno strumento concettuale di alto livello utilizzato nella fase iniziale della progettazione di database. Permette di rappresentare il mondo reale in termini di "entità" (oggetti o concetti di interesse) e "relazioni" (associazioni tra le entità). L'obiettivo è fornire una rappresentazione intuitiva e facilmente comprensibile della struttura dei dati prima di tradurla in un modello logico.

2.1.1 Componenti Base e Rappresentazione Grafica

I principali componenti del Modello ER sono le entità, gli attributi e le relazioni, ognuno con una specifica rappresentazione grafica che ne facilita la visualizzazione.

- **Entità**: Rappresentano "cose" o "oggetti" del mondo reale su cui si vogliono memorizzare informazioni. Possono essere concrete (es. Persona, Prodotto) o astratte (es. Corso, Ordine). Nel diagramma ER, le entità sono generalmente rappresentate con un rettangolo.
- **Attributi**: Sono le proprietà o caratteristiche che descrivono un'entità o una relazione. Ad esempio, un'entità "Studente" può avere attributi come "Nome", "Cognome", "Matricola". Nel diagramma ER, gli attributi sono rappresentati con un ovale.
- **Relazioni**: Rappresentano associazioni logiche tra due o più entità. Ad esempio, un "Docente" "insegna" a un "Corso". Nel diagramma ER, le relazioni sono rappresentate con un rombo.

2.1.2 Dettagli degli Attributi e Identificatori

Gli attributi possono presentare diverse caratteristiche e alcuni di essi assumono un ruolo speciale come identificatori.

- **Tipi di Attributi**:
  - **Attributi Semplici**: Non possono essere scomposti (es. "Età").

  - **Attributi Composti**: Formati da più attributi (es. "Indirizzo" composto da "Via", "Civico", "Città").
  - **Attributi Mono-valore**: Hanno un singolo valore per istanza (es. "DataNascita").
  - **Attributi Multi-valore**: Possono avere più valori (es. "NumeriDiTelefono").
  - **Attributi Derivati**: Il loro valore può essere calcolato da altri attributi (es. "Età" derivata da "DataNascita").
- **Chiave (Key Attribute)**: Un attributo (o un insieme di attributi) che identifica in modo univoco ogni istanza di un'entità. Viene tipicamente sottolineato nel diagramma ER.

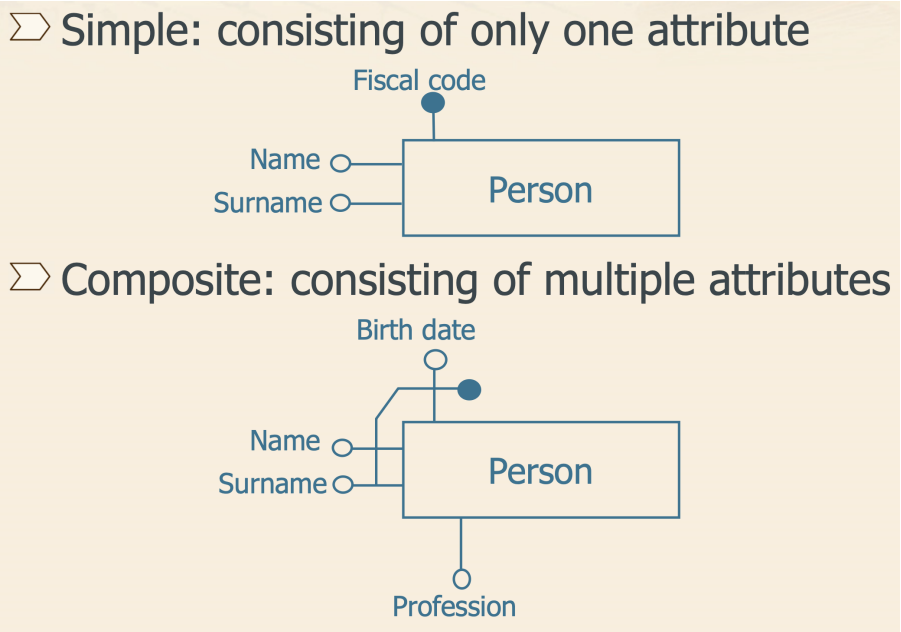


Figura 2.1: Esempi di rappresentazione di attributi semplici (es. Codice Fiscale) e attributi composti (es. Data di Nascita) in un Diagramma ER.

Identificatori

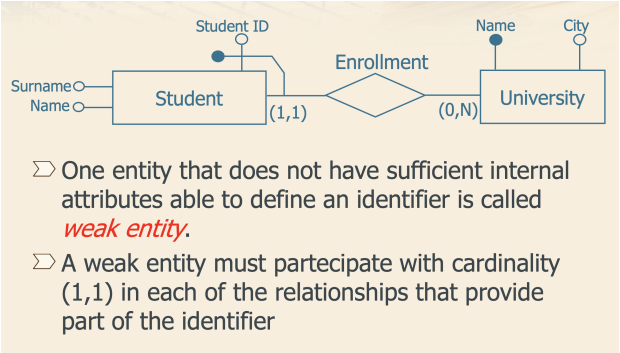
Gli identificatori permettono di distinguere univocamente le istanze di un'entità. Possono essere interni o esterni.

- **Identificatore Interno**: Costituito da uno o più attributi dell'entità stessa (es. Codice Fiscale per Persona).
- **Identificatore Esterno**: Coinvolge attributi dell'entità e la partecipazione a relazioni con altre entità, necessarie per la sua identificazione. È tipico delle entità deboli.

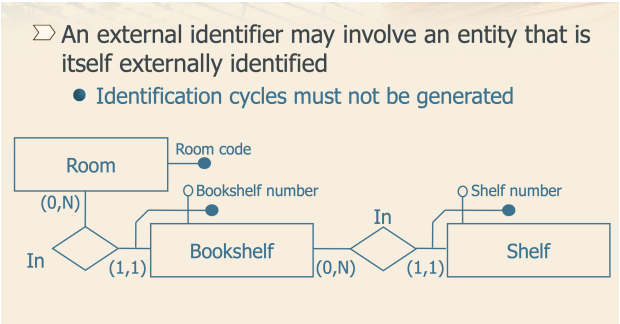
2.1.3 Dettagli delle Relazioni e Cardinalità

Le relazioni specificano le associazioni tra le entità, e le cardinalità definiscono il numero di istanze di un'entità che possono partecipare a una relazione.

- **Cardinalità delle Relazioni**: Definisce il numero minimo e massimo di istanze di un'entità che possono essere associate a un'istanza dell'altra entità nella relazione.
  - **Minima**: Indica se la partecipazione è obbligatoria (1) o opzionale (0).
  - **Massima**: Indica il numero massimo di partecipazioni (1 o N).



(a) Identificatore Esterno per Entità Deboli (Studente, Immatricolazione, Università)



(b) Identificatore Esterno Complesso (Stanza, Scaffale, Ripiano)

Figura 2.2: Esempi di utilizzo di identificatori esterni, inclusi scenari con entità deboli e concatenazione di identificazione.

  - **Notazione (min, max)**: Es. $(0, N)$ per zero a molti, $(1, N)$ per uno a molti.
- **Partecipazione (o Dipendenza)**:
  - **Totale (o Obbligatoria)**: Ogni istanza dell'entità deve partecipare alla relazione (indicata graficamente da una doppia linea di connessione).
  - **Parziale (o Opzionale)**: Un'istanza dell'entità può partecipare o meno alla relazione (indicata graficamente da una singola linea).

2.1.4 Generalizzazione nel Modello ER

La **Generalizzazione** è un costrutto del modello ER che permette di rappresentare una relazione "è un tipo di" (is-a) tra un'entità genitore (generale) e una o più entità figlie (specializzate).

- Ogni istanza di un'entità figlia è anche un'istanza dell'entità genitore.
- Le proprietà (attributi, identificatori, relazioni) dell'entità genitore sono ereditate da tutte le entità figlie.

2.1.5 Esempi Complessi di Schemi ER

Per consolidare la comprensione del Modello ER, è utile analizzare esempi complessi che integrano vari concetti come entità, attributi, relazioni con cardinalità diverse, identificatori, generalizzazioni e entità deboli. Questi schemi rappresentano contesti reali e mostrano come i costrutti ER vengono applicati per modellare domini complessi.

2.1.6 Esempi Complessi di Schemi ER

Per consolidare la comprensione del Modello ER, è utile analizzare esempi complessi che integrano vari concetti come entità, attributi, relazioni con cardinalità diverse, identificatori, generalizzazioni e entità deboli. Questi schemi rappresentano contesti reali e mostrano come i costrutti ER vengono applicati per modellare domini complessi.

Esercizio 2: Sistema Cinema/Film

Descrizione: Si consideri uno schema Entità-Relazione che rappresenta un sistema per la gestione di film, artisti, cinema e città. Il diagramma include entità come Film, Artista, Cinema

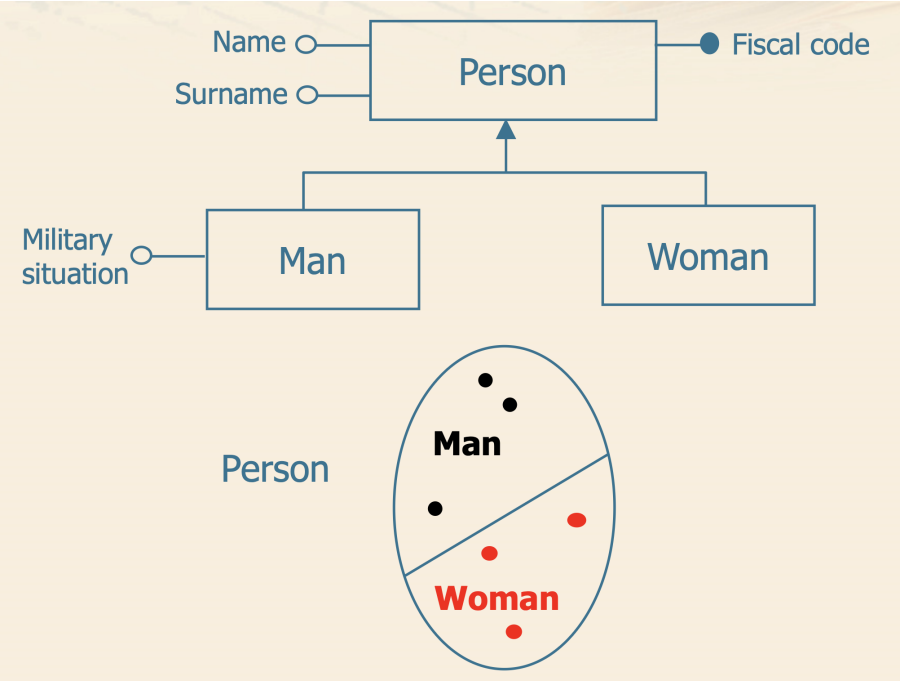


Figura 2.3: Esempio di Generalizzazione nel Diagramma ER: l'entità Persona si generalizza nelle entità Uomo e Donna, mostrando la distinzione e l'ereditarietà delle proprietà.

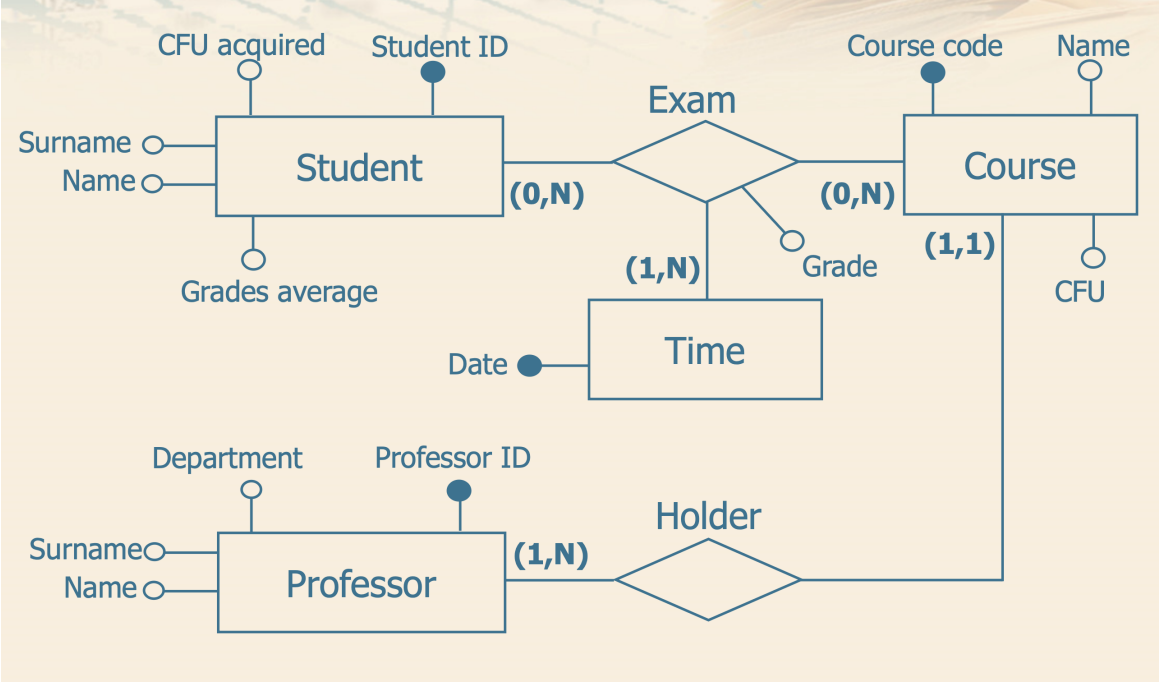


Figura 2.4: Diagramma ER che modella le relazioni tra Studenti, Corsi ed Esami, inclusi attributi e cardinalità.

e Città, con relazioni che specificano la regia, l'interpretazione e la proiezione dei film nei cinema locali.

Schema Relazionale corrispondente:

```
ARTISTI (Codice, Nome, Cognome)
FILM (Titolo, Anno, CasaProduttrice, RegistaArtista) -- RegistaArtista FK a
    ARTISTI
```

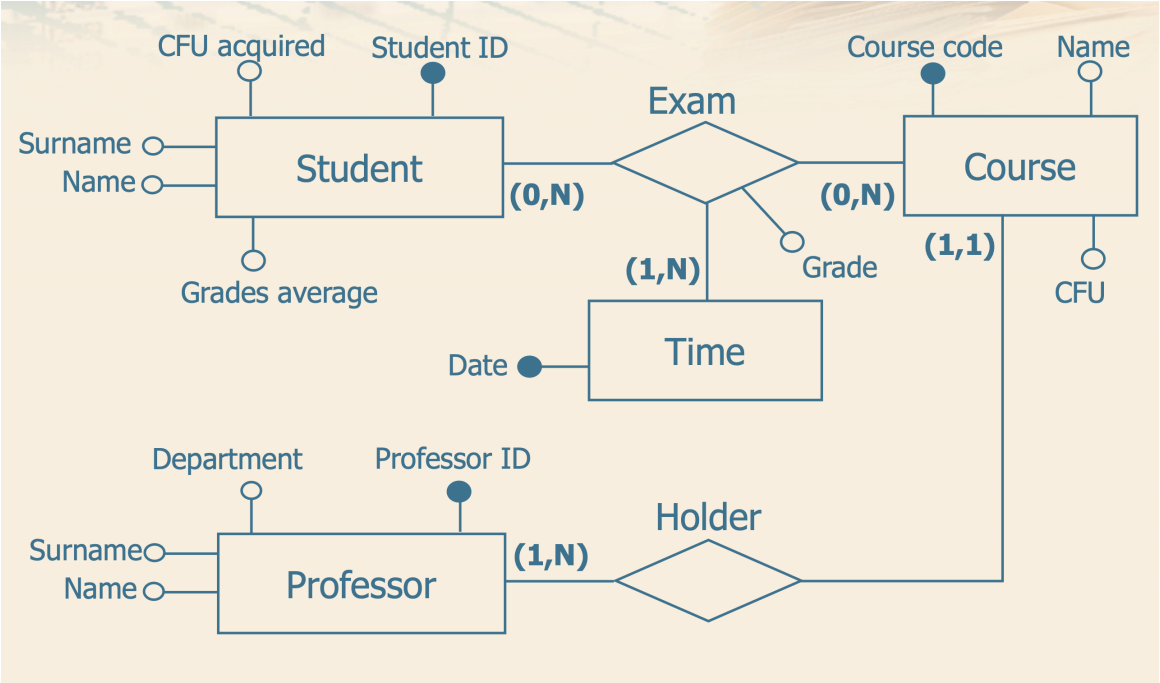


Figura 2.5: Diagramma ER che modella le relazioni tra Studenti, Corsi ed Esami, inclusi attributi e cardinalità.

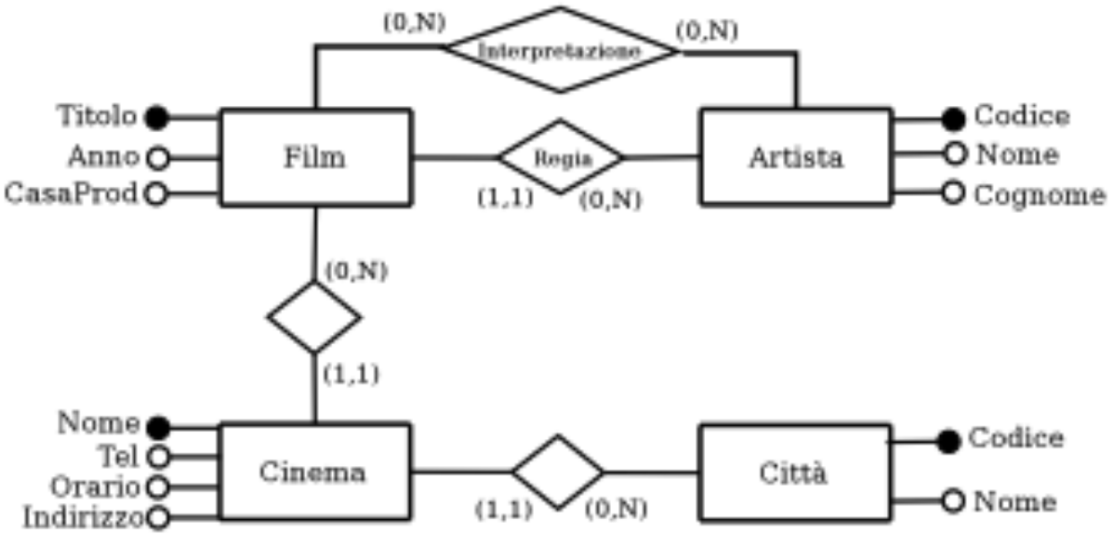


Figura 2.6: Schema ER per l'esercizio 2: Sistema Cinema/Film.

```
INTERPRETAZIONI (FilmTitolo, FilmAnno, ArtistaCodice) -- FK a FILM, FK a
    ARTISTI
CITTA (Codice, Nome)
CINEMA (Nome, Tel, Orario, Indirizzo, CittaCodice) -- CittaCodice FK a CITTA
PROIEZIONI (FilmTitolo, FilmAnno, CinemaNome, OrarioCinema) -- FK a FILM, FK a
     CINEMA
```

Esercizio 7: Sistema Campionato di Calcio

Descrizione: Modellare un sistema per la gestione di un campionato di calcio. Il sistema deve rappresentare squadre, giocatori (con i loro ruoli e dati personali), arbitri, giornate e partite,

inclusi i risultati e le specificità come partite in campo neutro o rinviate.

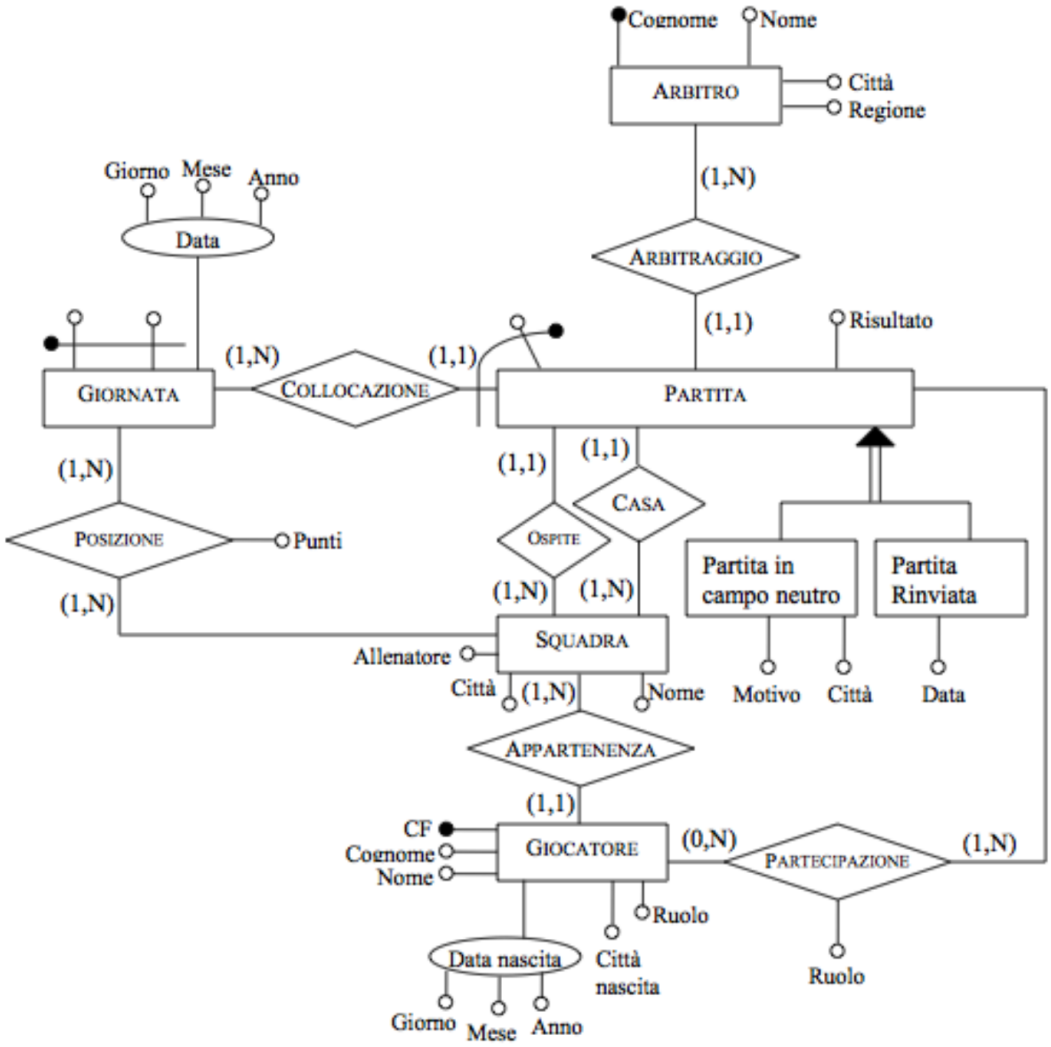


Figura 2.7: Schema ER per l'esercizio 7: Sistema Campionato di Calcio.

Schema Relazionale corrispondente:

```
ARBITRO (Cognome, Nome, Citta, Regione) -- Chiave primaria (Cognome, Nome) o ID_Arbitro generato
GIORNATA (Numero, Serie, Giorno, Mese, Anno) -- Chiave primaria (Numero, Serie)
SQUADRA (Nome, Citta, Allenatore) -- Chiave primaria (Nome)
GIOCATORE (CodiceFiscale, Cognome, Nome, Ruolo, CittaDiNascita) -- Chiave primaria (CodiceFiscale)
PARTITA (NumeroPartita, GiornoGiornata, MeseGiornata, AnnoGiornata, Risultato,
    ArbitroCognome, ArbitroNome, CasaSquadra, OspiteSquadra) -- Chiave primaria (NumeroPartita), FK a GIORNATA, FK a ARBITRO, FK a SQUADRA (due volte)
PARTITA_IN_CAMPO_NEUTRO (PartitaNumero, PartitaGiorno, PartitaMese, PartitaAnno, Motivo, CittaCampoNeutro) -- FK a PARTITA
PARTITA_RINVIATA (PartitaNumero, PartitaGiorno, PartitaMese, PartitaAnno, DataRinvio) -- FK a PARTITA
```

```
POSIZIONE (SquadraNome, GiornoGiornata, MeseGiornata, AnnoGiornata, Punteggio)
    -- Chiave primaria (SquadraNome, GiornoGiornata, MeseGiornata,
    AnnoGiornata), FK a SQUADRA, FK a GIORNATA
APPARTENENZA (GiocatoreCodiceFiscale, SquadraNome, DataInizio, RuoloPrincipale
    , DataFine) -- Chiave primaria (GiocatoreCodiceFiscale, SquadraNome,
    DataInizio), FK a GIOCATORE, FK a SQUADRA
```

Esercizio 11: Sistema Gare Ciclistiche

Descrizione: Un sistema per la gestione di gare ciclistiche, includendo informazioni su ciclisti, squadre, località, competizioni, edizioni e tappe, con dettagli sulle classifiche e gli orari.

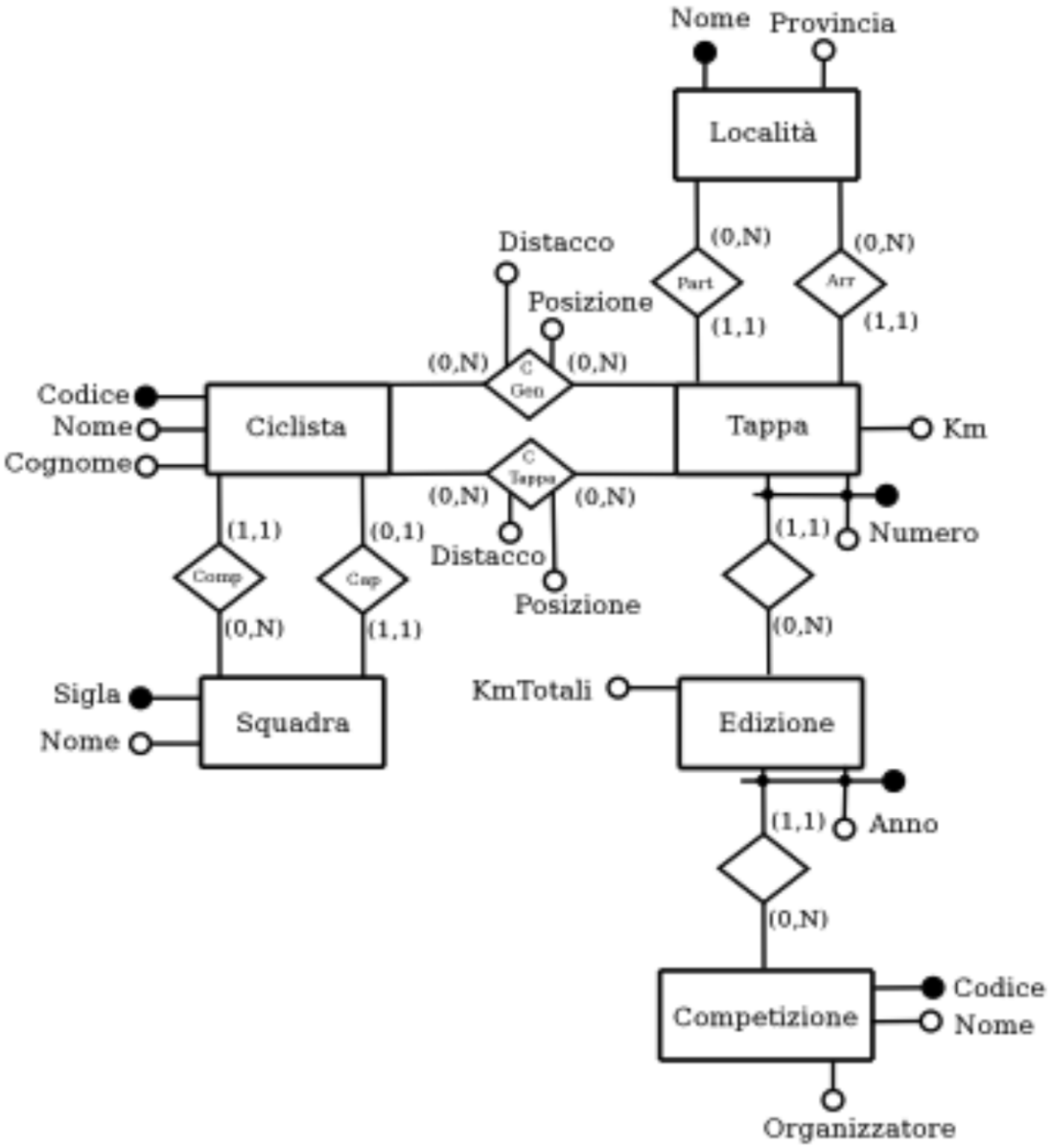


Figura 2.8: Schema ER per l'esercizio 11: Sistema Gare Ciclistiche.

Schema Relazionale corrispondente:

```
SQUADRE (Sigla, Nome, CapitanoCodice) -- CapitanoCodice FK a CICLISTI
CICLISTI (Codice, Cognome, Nome, SquadraSigla) -- SquadraSigla FK a SQUADRE
```

```
LOCALITA (Nome, Provincia) -- Chiave primaria (Nome, Provincia)
COMPETIZIONI (Codice, Nome, Organizzatore) -- Chiave primaria (Codice)
EDIZIONI (AnnoEdizione, CompetizioneCodice, KmTotali) -- Chiave primaria (
    AnnoEdizione, CompetizioneCodice), FK a COMPETIZIONI
TAPPE (NumeroTappa, AnnoEdizione, CompetizioneCodice, LocPartenzaNome,
    LocPartenzaProvincia, LocArrivoNome, LocArrivoProvincia) -- Chiave primaria
     (NumeroTappa, AnnoEdizione, CompetizioneCodice), FK a EDIZIONI, FK a
    LOCALITA (due volte)
CLASSIFICATAPPA (NumeroTappa, AnnoEdizione, CompetizioneCodice, CiclistaCodice
    , Posizione, Distacco) -- FK a TAPPE, FK a CICLISTI
CLASSIFICAGENERALE (NumeroTappa, AnnoEdizione, CompetizioneCodice,
    CiclistaCodice, Posizione, Distacco) -- FK a TAPPE, FK a CICLISTI
```

2.2 Progettazione Logica: Normalizzazione e Forme Normali

La **normalizzazione** è un processo sistematico di organizzazione dei dati in un database relazionale. Il suo scopo è ridurre la ridondanza dei dati, eliminare le anomalie di aggiornamento (inserimento, cancellazione, modifica) e migliorare l'integrità e la coerenza dei dati. La normalizzazione si basa su una serie di regole chiamate "forme normali".

2.2.1 Perché la Normalizzazione è Importante

La normalizzazione è cruciale per diversi motivi:

- **Riduzione della Ridondanza**: Evitare la duplicazione inutile dei dati, che spreca spazio e può portare a incongruenze. La ridondanza può causare spreco di memoria a causa della memorizzazione multipla della stessa informazione.
- **Miglioramento dell'Integrità e Coerenza dei Dati**: Assicurare che i dati siano accurati e consistenti. Si ha incoerenza quando lo stesso campo ha valori diversi in tabelle diverse, fenomeno che può verificarsi quando le tabelle non sono aggiornate o l'aggiornamento non è stato effettuato correttamente.
- **Prevenzione delle Anomalie**:
  - **Anomalia di Inserimento**: Impossibilità di inserire un'informazione a meno che non si inseriscano anche altre informazioni non correlate.
  - **Anomalia di Cancellazione**: La cancellazione di un dato comporta la perdita accidentale di altre informazioni non desiderate.
  - **Anomalia di Aggiornamento**: La modifica di un dato ripetuto richiede l'aggiornamento di più occorrenze, con rischio di inconsistenza se non tutte vengono aggiornate.
- **Efficienza del Database**: Una struttura di database normalizzata consente un accesso più rapido ed efficiente ai dati, migliorando le prestazioni di query e operazioni di recupero.
- **Flessibilità e Manutenibilità**: Rende il database più facile da comprendere, gestire e modificare.

2.2.2 Forme Normali: Livelli di Normalizzazione

Esistono vari livelli di normalizzazione, noti come "forme normali", che certificano la qualità dello schema del database. Ogni forma normale definisce un insieme di regole che lo schema del database deve rispettare per garantire un livello crescente di integrità e coerenza.

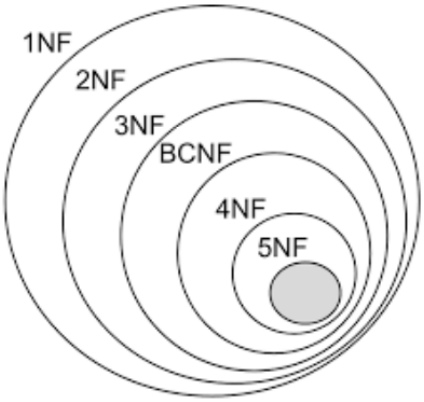


Figura 2.9: Diagramma di Venn delle Forme Normali (1NF, 2NF, 3NF, BCNF, 4NF, 5NF), che mostra la loro relazione gerarchica.

Prima Forma Normale (1NF)

Una relazione è in **Prima Forma Normale (1NF)** se e solo se:

- Non presenta attributi multipli (non ci sono gruppi ripetitivi di attributi all'interno di una singola tabella). Tutti gli attributi sono atomici (indivisibili).
- Esiste una chiave primaria (un insieme di attributi che identifica in modo univoco ogni tupla della relazione).

Esempio di Violazione e Normalizzazione a 1NF: Consideriamo una tabella "Voti" dove un singolo campo contiene più valori, come i voti di uno studente per semestri diversi in una stessa riga. Nella tabella non in 1NF, il campo "Voto" contiene più voti e semestri. Per normalizzare, si scompone la riga in più righe, una per ogni voto/semestre, assicurando che ogni attributo sia atomico.

Seconda Forma Normale (2NF)

Una relazione si dice in **Seconda Forma Normale (2NF)** quando è in 1NF e tutti i campi non-chiave dipendono funzionalmente dall'intera chiave primaria, e non solo da una sua parte. La 2NF elimina la dipendenza parziale degli attributi dalla chiave e riguarda relazioni con chiavi composte (formate da più attributi). **Esempio di Violazione e Normalizzazione a 2NF**: Consideriamo una relazione con chiave composta (es. Codice_atleta, Numero_gara) e un attributo non-chiave (Nome_atleta) che dipende solo da una parte della chiave (Codice_atleta). La soluzione è scomporre la relazione originale in due relazioni, una per la parte della chiave che determina l'attributo (es. Atleti(Codice_atleta, Nome_atleta)) e un'altra per la chiave completa con gli altri attributi (es. Risultati(Codice_atleta, Numero_gara, Tempo)).

non in 1NF

Voti

Matricola	Studente	Materia	Voto
0000-000-01	Pietro	Basi di Dati	1 sem, B ; 2 sem, F
0000-000-02	Pietro	Basi di Dati	1 sem, A ; 2 sem, A
0000-000-03	Sara	Basi di Dati	1 sem, B ; 2 sem, A

in 1NF

Voti

Matricola	Studente	Materia	Semestre	Voto
0000-000-01	Pietro	Basi di Dati	1	B
0000-000-01	Pietro	Basi di Dati	2	F
0000-000-02	Pietro	Basi di Dati	1	A
0000-000-02	Pietro	Basi di Dati	2	A
0000-000-03	Sara	Basi di Dati	1	B
0000-000-03	Sara	Basi di Dati	2	A

Figura 2.10: Esempio di una tabella non in 1NF e la sua normalizzazione a 1NF, eliminando l'attributo multi-valore.

Codice_atleta	Numero_gara	Nome_atleta	tempo
3455	1	Maurizio	36
3455	2	Maurizio	32
3320	1	Luigi	38
3320	2	Luigi	39
3320	3	Luigi	33

la **chiave** è **composta**, in quanto il solo codice non basta per identificare il tempo che è realizzato in gare diverse.
Nome_atleta dipende solo dall'attributo Codice_atleta (***solo da una parte della chiave***)

Codice_atleta	Numero_gara	tempo
3455	1	36
3455	2	32
3320	1	38
3320	2	39
3320	3	33

Codice atleta	Nome atleta
3455	Maurizio
3320	Luigi

la relazione viene "spezzata" in due relazioni per eliminare l'anomalia

Figura 2.11: Esempio di una tabella non in 2NF e la sua normalizzazione a 2NF tramite scomposizione in due relazioni.

Terza Forma Normale (3NF)

Una relazione si dice in **Terza Forma Normale (3NF)** quando è in 2NF e tutti gli attributi non-chiave dipendono dalla chiave primaria soltanto, ossia non esistono attributi che dipendono da altri attributi non-chiave (non esistono dipendenze transitive). **Esempio di Violazione e Normalizzazione a 3NF (Esempio 1: Computer)**: Consideriamo una tabella "Computer" dove attributi come "Velocità" dipendono da "Codice Processore", che a sua volta dipende da "Codice". Questa è una dipendenza transitiva. La soluzione è scomporre la relazione originale in relazioni separate per eliminare le dipendenze transitive. **Esempio 2: Impiegato/Dipartimento**: In una relazione "Impiegato" con attributi (CodImpiegato, Nome, Reparto, CapoReparto), se "CodImpiegato" è la chiave primaria e "CapoReparto" dipende da "Reparto" (che è un attributo non-chiave), si ha una dipendenza transitiva. La soluzione è quella di scomporre la relazione "Impiegato" in due relazioni: una relazione Impiegati(CodImpiegato, Nome, Reparto) e una relazione Dipartimenti(Reparto, CapoReparto), eliminando così la dipendenza transitiva.

Computer

Codice	Descrizione	CodiceMemoria	Dimensione	Codice Processore	Velocità
1	Desktop base	20	512MB	P1	2GHz
2	Desktop medio	21	1GB	P2	3GHz
7	Desktop fascia alta	22	2GB	P3	4GHz
8	Portatile wireless medio	25	1GB	P2	3GHz

- Codice → Descrizione;
- CodiceMemoria → Dimensione;
- CodiceProcessore → Velocità.

Figura 2.12: Esempio di una tabella "Computer" non in 3NF a causa di dipendenze transitive.

Computer

Codice	Descrizione	CodiceMemoria	CodiceProcessore
1	Desktop base	20	P1
2	Desktop medio	21	P2
7	Desktop fascia alta	22	P3
8	Portatile wireless medio	25	P2

Processori

CodiceProcessore	Velocità
P1	2GHz
P2	3GHz
P1	2GHz
P3	1GHz

Memorie

CodiceMemoria	Dimensione
20	512MB
21	1GB
22	2GB
25	1GB

Figura 2.13: Normalizzazione della tabella "Computer" a 3NF, scomponendo le relazioni per eliminare le dipendenze transitive.

CodImpiegato	Nome	Reparto	CapoReparto
1	Verdi	Vendite	Rossi
2	Bianchi	Vendite	Rossi

Figura 2.14: Esempio di una tabella "Impiegato" non in 3NF per dipendenza transitiva.

Oltre la Terza Forma Normale: BCNF, 4NF, 5NF

Esistono forme normali ancora più elevate come la **Forma Normale di Boyce e Codd (BCNF)**, la **Quarta Forma Normale (4NF)** e la **Quinta Forma Normale (5NF)**. Queste

affrontano casi più specifici e complessi di dipendenze (es. dipendenze multi-valore). Tuttavia, in molti casi pratici, la normalizzazione fino alla 3NF è sufficiente per garantire un database efficiente e affidabile.

2.2.3 Linguaggio SQL

Il **Structured Query Language (SQL)** è il linguaggio standard per la gestione dei sistemi di gestione di database relazionali (RDBMS). Permette di definire, manipolare e controllare i dati.

Categorie di Comandi SQL

- **Data Definition Language (DDL)**: Utilizzato per definire e modificare la struttura del database.
  - **CREATE**: Crea database, tabelle, viste, indici, ecc. (es. 'CREATE TABLE Studenti (...)').
  - **ALTER**: Modifica la struttura di oggetti esistenti (es. 'ALTER TABLE Studenti ADD COLUMN Età INT').
  - **DROP**: Cancella oggetti dal database (es. 'DROP TABLE Studenti').
- **Data Manipulation Language (DML)**: Utilizzato per manipolare i dati all'interno delle tabelle.
  - **SELECT**: Recupera dati da una o più tabelle. È la query più usata.
  - **INSERT**: Aggiunge nuove righe a una tabella.
  - **UPDATE**: Modifica righe esistenti in una tabella.
  - **DELETE**: Rimuove righe da una tabella.
- **Data Control Language (DCL)**: Utilizzato per gestire i permessi di accesso ai dati.
  - **GRANT**: Concede privilegi agli utenti.
  - **REVOKE**: Rimuove privilegi dagli utenti.
- **Transaction Control Language (TCL)**: Utilizzato per gestire le transazioni (gruppi di operazioni che devono essere eseguite atomicamente).
  - **COMMIT**: Salva le modifiche di una transazione.
  - **ROLLBACK**: Annulla le modifiche di una transazione.

Elementi Comuni delle Query SQL (SELECT)

La query `SELECT` è la più potente e versatile, permettendo di interrogare il database.

- **SELECT**: Specifica le colonne da recuperare.
  - `SELECT colonna1, colonna2`
  - `SELECT *` (tutte le colonne)
  - `SELECT DISTINCT colonna` (solo valori unici)
- **FROM**: Specifica la tabella (o le tabelle) da cui recuperare i dati.
- **WHERE**: Filtra le righe in base a una condizione specificata.
  - `WHERE condizione` (es. 'WHERE Età > 18')
  - Operatori: '=', '>', '<', '>=', '<=', '<>', 'LIKE' (per pattern matching), 'IN', 'BETWEEN', 'IS NULL'.
- **JOIN**: Combina righe da due o più tabelle basandosi su una colonna correlata.

  - **INNER JOIN**: Restituisce solo le righe che hanno corrispondenze in entrambe le tabelle.
  - **LEFT (OUTER) JOIN**: Restituisce tutte le righe dalla tabella sinistra e le righe corrispondenti dalla tabella destra (con NULL se non ci sono corrispondenze).
  - **RIGHT (OUTER) JOIN**: Simile al LEFT JOIN, ma per la tabella destra.
  - **FULL (OUTER) JOIN**: Restituisce tutte le righe quando c'è una corrispondenza in una delle due tabelle.
- **GROUP BY**: Raggruppa le righe che hanno gli stessi valori in una o più colonne, spesso usato con funzioni di aggregazione.
- **HAVING**: Filtra i gruppi creati da 'GROUP BY' in base a una condizione. Si usa con le funzioni di aggregazione.
- **ORDER BY**: Ordina il set di risultati in base a una o più colonne (ASC per ascendente, DESC per discendente).
- **Funzioni di Aggregazione**: Calcolano un singolo valore da un insieme di valori (es. 'COUNT()', 'SUM()', 'AVG()', 'MAX()', 'MIN()').

Esempi di Operatori e Funzioni SQL Comuni

Oltre agli elementi base delle query `SELECT`, SQL offre un'ampia gamma di operatori e funzioni per manipolare e filtrare i dati in modo più complesso.

- **Operatori Logici**:
  - `AND`: Combina due condizioni, entrambe devono essere vere.
  - `OR`: Combina due condizioni, almeno una deve essere vera.
  - `NOT`: Nega una condizione.

```sql
SELECT FirstName, LastName
FROM Students
WHERE Age > 20 AND City = 'Bologna';
```

Listing 2.1: Esempio Operatori Logici

- **Operatori di Confronto**:
  - `=`: Uguale a.
  - `<>` o `!=`: Diverso da.
  - `<`, `>`, `<=`, `>=`: Minore, maggiore, minore o uguale, maggiore o uguale.
  - `BETWEEN min AND max`: Valore compreso in un intervallo (inclusi gli estremi).
  - `LIKE pattern`: Ricerca stringhe che corrispondono a un pattern (es. `LIKE 'A%'`).
  - `IN (value1, value2, ...)`: Valore presente in una lista di valori.
  - `IS NULL` / `IS NOT NULL`: Verifica se un valore è NULL.

```sql
SELECT ProductName, Price
FROM Products
WHERE Price BETWEEN 10.00 AND 50.00
  AND ProductName LIKE 'Book%';

SELECT OrderID
FROM Orders
```

```sql
WHERE DeliveryDate IS NULL;
```

Listing 2.2: Esempio Operatori di Confronto

- **Funzioni Stringa**:
  - `CONCAT(s1, s2, ...)`: Concatena stringhe.
  - `SUBSTRING(string, start, length)`: Estrae una sottostringa.
  - `LENGTH(string)`: Restituisce la lunghezza di una stringa.
  - `UPPER(string)` / `LOWER(string)`: Converte in maiuscolo/minuscolo.

```sql
SELECT CONCAT(FirstName, ' ', LastName) AS FullName
FROM Users
WHERE LENGTH(FirstName) > 5;

SELECT UPPER(CategoryName)
FROM Categories;
```

Listing 2.3: Esempio Funzioni Stringa

- **Funzioni Numeriche**:
  - `ROUND(number, decimal_places)`: Arrotonda un numero.
  - `ABS(number)`: Valore assoluto.

```sql
SELECT ROUND(UnitPrice * Quantity, 2) AS RoundedTotal
FROM OrderDetails;

SELECT ABS(Balance)
FROM BankAccounts;
```

Listing 2.4: Esempio Funzioni Numeriche

- **Funzioni Data/Ora**:
  - `NOW()`: Data e ora correnti.
  - `CURDATE()`: Data corrente.
  - `DATE_ADD(date, INTERVAL value unit)`: Aggiunge un intervallo a una data.

```sql
SELECT EventName, EventDate
FROM Events
WHERE EventDate > CURDATE();

SELECT DATE_ADD(EventDate, INTERVAL 7 DAY) AS ExpectedEventDate
FROM Events;
```

Listing 2.5: Esempio Funzioni Data/Ora

Figura 2.15: Esempio di una query SQL che utilizza operatori e funzioni avanzate per filtrare e aggregare i dati.

Domande e Esercizi

Domande d'Esame Principali

Domanda: Modello ER, Normalizzazione e SQL (Corsi, Docenti, Appelli)

Domanda: Il candidato descriva il modello concettuale Entità-Relazione (ER) e il concetto di forma normale nella progettazione logica. Produca un diagramma ER che rappresenti il contesto seguente: "Un corso è associato a un docente titolare e può includere più appelli d'esame durante l'anno accademico. Ogni appello d'esame può avere uno o più studenti iscritti." Successivamente il candidato fornisca una progettazione in forma tabellare del modello, normalizzando le relazioni, fornendo infine una query SQL per ottenere l'elenco di tutti gli iscritti a un determinato appello d'esame.

Risposta :

Descrizione del Modello Concettuale Entità-Relazione (ER) Il **Modello Entità-Relazione (ER)** è uno strumento di alto livello per la progettazione concettuale di database. Permette di rappresentare il mondo reale tramite entità (oggetti o concetti di interesse) e relazioni (associazioni tra entità). I suoi componenti principali sono:

- **Entità**: Cose o oggetti reali (es. Studente, Docente). Rappresentate con rettangoli.
- **Attributi**: Proprietà delle entità o relazioni (es. Nome, CodiceFiscale). Rappresentati con ovali. Possono essere semplici, composti, multi-valore, derivati o chiavi (sottolineate).
- **Relazioni**: Associazioni logiche tra entità (es. Insegnare, Iscrizione). Rappresentate con rombi. Hanno cardinalità (min:max) e partecipazione.

Concetto di Forma Normale nella Progettazione Logica La **normalizzazione** è un processo di organizzazione dei dati in un database relazionale per ridurre la ridondanza e migliorare l'integrità dei dati. Si basa sulle forme normali:

- **Prima Forma Normale (1NF)**: Tutti gli attributi sono atomici e ogni record è unico.
- **Seconda Forma Normale (2NF)**: È in 1NF e tutti gli attributi non-chiave dipendono completamente dalla chiave primaria (no dipendenze parziali).
- **Terza Forma Normale (3NF)**: È in 2NF e non contiene dipendenze transitive (nessun attributo non-chiave dipende da un altro attributo non-chiave).

Diagramma ER per il Contesto Specifico Il contesto da modellare è: "Un corso è associato a un docente titolare e può includere più appelli d'esame durante l'anno accademico. Ogni appello d'esame può avere uno o più studenti iscritti."

Spiegazione del Diagramma:

- **Entità**: 'Studente', 'Appello', 'Docente', 'Corso', 'Anno Accademico'.
- **Attributi impliciti/necessari**: Per completezza, assumiamo per 'Studente' (Matricola PK, Nome, Cognome), per 'Appello' (IDAppello PK, Data, Ora, Luogo), per 'Docente' (IDDocente PK, Nome, Cognome), per 'Corso' (IDCorso PK, NomeCorso) e per 'Anno Accademico' (Anno PK).
- **Relazione "Iscrizione" (Studente - Appello)**: È una relazione N:M. Un 'Studente' può iscriversi a zero o più 'Appelli' ('(0,N)' lato Studente). Un 'Appello' deve avere uno o

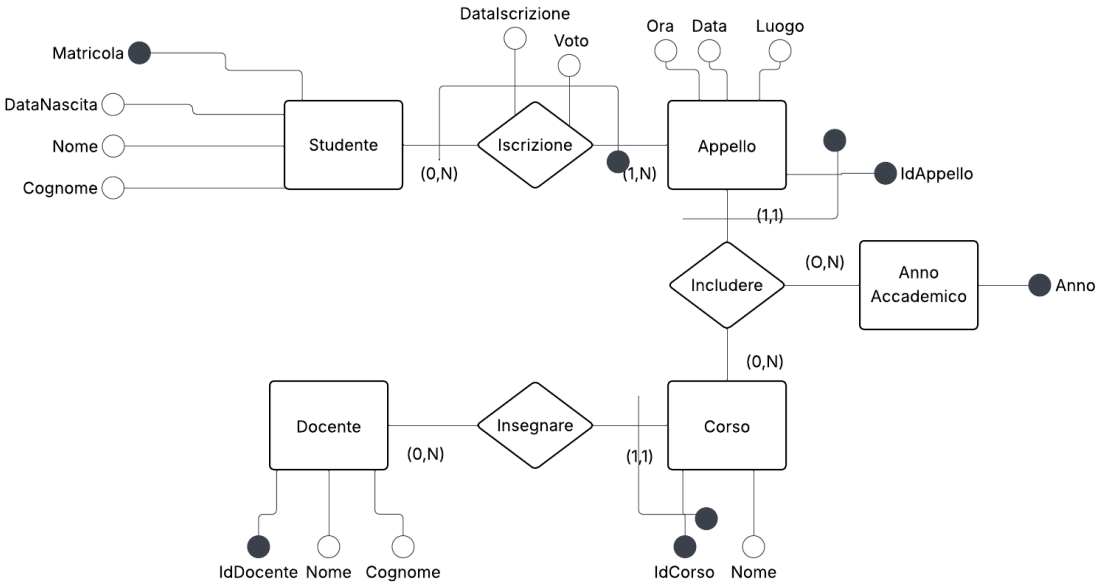


Figura 2.16: Diagramma Entità-Relazione per la gestione di corsi, docenti, appelli d'esame e studenti.

più 'Studenti' iscritti ('(1,N)' lato Appello). Potrebbe avere attributi sulla relazione come 'Voto', 'DataIscrizione'.

- **Relazione "Insegnare" (Docente - Corso)**: È una relazione 1:N. Un 'Docente' può insegnare a zero o più 'Corsi' ('(0,N)' lato Docente). Un 'Corso' è insegnato da uno e un solo 'Docente' ('(1,1)' lato Corso), indicando il docente titolare.
- **Relazione "Includere" (Corso - Appello - Anno Accademico)**: Questa è modellata come una relazione ternaria. Un 'Corso' può avere zero o più 'Appelli' in un dato 'Anno Accademico' ('(0,N)' lato Corso). Un 'Appello' è sempre legato a uno e un solo 'Corso' e a un solo 'Anno Accademico' specifico ('(1,1)' lato Appello). Un 'Anno Accademico' può includere zero o molti 'Appelli' di 'Corsi' ('(0,N)' lato Anno Accademico). Questa modellazione ternaria cattura il vincolo che un appello di un corso si tiene in un determinato anno accademico.

Progettazione in Forma Tabellare (Modello Relazionale Normalizzato) Traducendo lo schema ER in uno schema relazionale e applicando la normalizzazione (fino alla 3NF):

- **DOCENTI** (IDDocente, Nome, Cognome)
- **CORSI** (IDCorso, NomeCorso, IDDocenteFK)
- **ANNI_ACCADEMICI** (Anno)
- **APPELLI** (IDAppello, Data, Ora, Luogo, IDCorsoFK, AnnoAccademicoFK)
- **STUDENTI** (Matricola, Nome, Cognome, DataNascita)
- **ISCRIZIONI** (MatricolaFK, IDAppelloFK, DataIscrizione, Voto)

Motivazione della Normalizzazione:

- Tutte le tabelle sono in 1NF (attributi atomici, PK unica).
- Poiché non ci sono chiavi primarie composte in `DOCENTI`, `CORSI`, `ANNI_ACCADEMICI`, `APPELLI`, `STUDENTI`, queste sono automaticamente in 2NF. Non ci sono dipendenze transitive, quindi sono in 3NF.

- Per `ISCRIZIONI` (chiave composta: `MatricolaFK`, `IDAppelloFK`), gli attributi `DataIscrizione` e `Voto` dipendono da *entrambe* le parti della chiave, quindi è in 2NF. Non ci sono dipendenze transitive, quindi è in 3NF.
- Questa scomposizione riduce la ridondanza (es. nomi di corsi/docenti non ripetuti in `APPELLI`) e previene anomalie di aggiornamento, inserimento e cancellazione.

Query SQL per l'Elenco degli Iscritti a un Determinato Appello d'Esame Supponiamo di voler ottenere l'elenco di tutti gli studenti iscritti all'appello d'esame con 'IDAppello = 'APP2025-001''.

```sql
SELECT
    S.Nome AS StudentFirstName,
    S.Cognome AS StudentLastName,
    S.Matricola AS StudentMatricola,
    A.Data AS ExamDate,
    A.Ora AS ExamTime,
    C.NomeCorso AS CourseName
FROM
    STUDENTI S
JOIN
    ISCRIZIONI I ON S.Matricola = I.MatricolaFK
JOIN
    APPELLI A ON I.IDAppelloFK = A.IDAppello
JOIN
    CORSI C ON A.IDCorsoFK = C.IDCorso
WHERE
    A.IDAppello = 'APP2025-001';
```

Listing 2.6: Query per l'elenco degli iscritti a un appello d'esame specifico

Questa query connette le tabelle 'STUDENTI', 'ISCRIZIONI', 'APPELLI' e 'CORSI' per recuperare i dettagli degli studenti e dell'appello/corso corrispondente, filtrando per un ID di appello specifico.

Domanda: Modello ER, Normalizzazione e SQL (Commissione d'Esame di Stato)

Domanda: Il candidato descriva il modello concettuale Entità-Relazione (ER) e il concetto di forma normale nella progettazione logica. Produca un diagramma ER per una base di dati che tenga conto per ogni sessione dell'esame di stato, i membri effettivi della commissione e i candidati, e per ogni esame memorizzi la data e i nomi dei candidati che lo hanno superato. Successivamente il candidato fornisca una progettazione in forma tabellare del modello, normalizzando le relazioni, fornendo infine una query SQL per ottenere l'elenco di tutti gli candidati che hanno superato l'esame.

Risposta :

Descrizione del Modello Concettuale Entità-Relazione (ER) Il **Modello Entità-Relazione (ER)** è uno strumento di alto livello per la progettazione concettuale di database. Permette di rappresentare il mondo reale tramite entità (oggetti o concetti di interesse) e relazioni (associazioni tra entità). I suoi componenti principali sono:

- **Entità**: Rappresentano "cose" o "oggetti" reali (es. Candidato, Esame di stato, Membro Commissione). Rappresentate con rettangoli.
- **Attributi**: Proprietà delle entità o relazioni (es. Nome, Cognome, Data, Esito). Rappresentati con ovali.
- **Relazioni**: Associazioni logiche tra entità (es. Sostiene, Scrutina). Rappresentate con rombi. Hanno cardinalità (min:max) e partecipazione.

Concetto di Forma Normale nella Progettazione Logica La **normalizzazione** è un processo di organizzazione dei dati in un database relazionale per ridurre la ridondanza e migliorare l'integrità dei dati. Si basa sulle forme normali:

- **Prima Forma Normale (1NF)**: Tutti gli attributi sono atomici e ogni record è unico.
- **Seconda Forma Normale (2NF)**: È in 1NF e tutti gli attributi non-chiave dipendono completamente dalla chiave primaria (no dipendenze parziali).
- **Terza Forma Normale (3NF)**: È in 2NF e non contiene dipendenze transitive (nessun attributo non-chiave dipende da un altro attributo non-chiave).

Diagramma ER per il Contesto Specifico Il contesto da modellare è la gestione delle sessioni d'esame di stato, dei membri della commissione, dei candidati, e il tracciamento dei candidati che hanno superato l'esame.

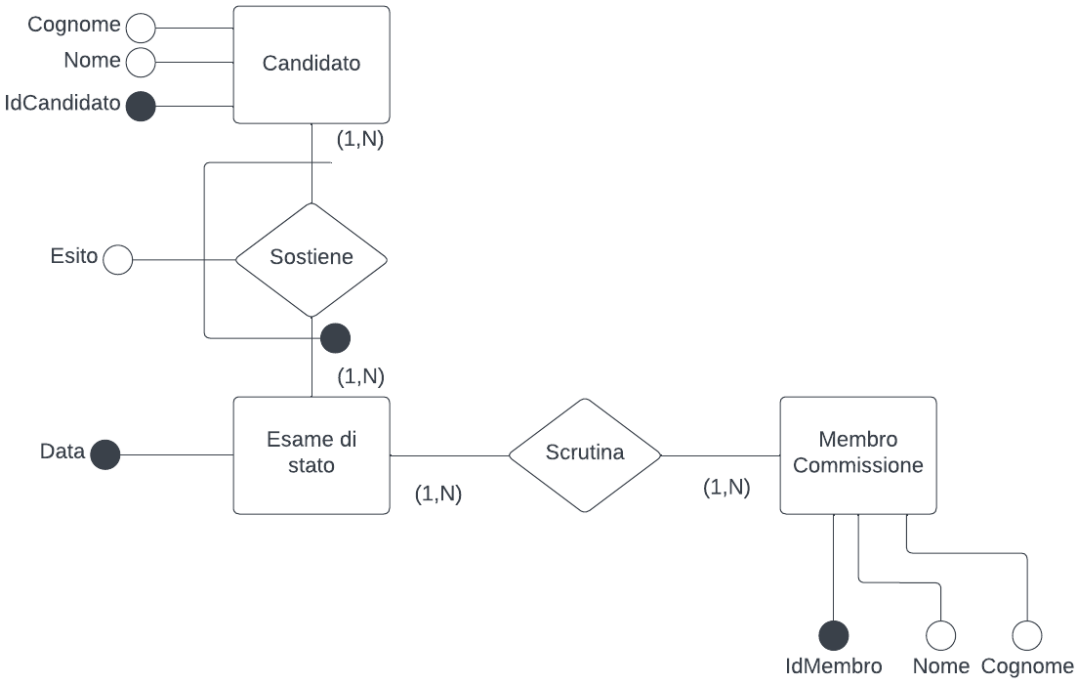


Figura 2.17: Diagramma Entità-Relazione per la gestione delle sessioni dell'Esame di Stato, Commissione e Candidati.

Spiegazione del Diagramma:

- **Entità 'Candidato'**: Rappresenta i partecipanti all'esame. Attributi: `IdCandidato` (PK), `Nome`, `Cognome`.
- **Entità 'Esame di stato'**: Rappresenta le singole sessioni d'esame. Attributi: `Data` (PK).

- **Entità 'Membro Commissione'**: Rappresenta i membri che compongono la commissione d'esame. Attributi: `IdMembro` (PK), `Nome`, `Cognome`.
- **Relazione "Sostiene" (Candidato - Esame di stato)**: Relazione N:M.
  - Cardinalità: Un `Candidato` può sostenere uno o più `Esami di stato` (`(1,N)` lato Candidato). Un `Esame di stato` è sostenuto da uno o più `Candidati` (`(1,N)` lato Esame di stato).
  - Attributo: `Esito` (indica se l'esame è stato superato, non superato, ecc.).
- **Relazione "Scrutina" (Esame di stato - Membro Commissione)**: Relazione N:M.
  - Cardinalità: Un `Esame di stato` è scrutinato da uno o più `Membri Commissione` (`(1,N)` lato Esame di stato). Un `Membro Commissione` può scrutinare uno o più `Esami di stato` (`(1,N)` lato Membro Commissione).

Progettazione in Forma Tabellare (Modello Relazionale Normalizzato) Traducendo lo schema ER in uno schema relazionale e applicando la normalizzazione (fino alla 3NF):

- **CANDIDATI** (IdCandidato, Nome, Cognome)
- **ESAMI_DI_STATO** (Data)
- **MEMBRI_COMMISSIONE** (IdMembro, Nome, Cognome)
- **SOSTIENE** (IdCandidatoFK, DataEsameFK, Esito)
- **SCRUTINA** (DataEsameFK, IdMembroFK)

Motivazione della Normalizzazione:

- Tutte le tabelle sono in 1NF (attributi atomici, chiavi primarie uniche).
- Poiché non ci sono chiavi primarie composte in CANDIDATI, ESAMI_DI_STATO, MEMBRI_COMMISSIONE, queste sono automaticamente in 2NF e 3NF in quanto hanno chiavi primarie singole e non presentano dipendenze transitive.
- Le tabelle SOSTIENE e SCRUTINA sono tabelle di associazione generate da relazioni N:M. I loro attributi (es. Esito per SOSTIENE) dipendono interamente dalla chiave composta (IdCandidatoFK, DataEsameFK per SOSTIENE), garantendo la 2NF. Non presentano dipendenze transitive, quindi sono in 3NF.
- Questa struttura riduce la ridondanza (es. nomi dei candidati non ripetuti nella relazione SOSTIENE) e previene anomalie di aggiornamento, inserimento e cancellazione.

Query SQL per ottenere l'elenco di tutti gli candidati che hanno superato l'esame Supponiamo di voler l'elenco di tutti i candidati che hanno superato un esame in una data specifica (es. il 21 luglio 2025).

```sql
SELECT
    C.Nome AS CandidateFirstName,
    C.Cognome AS CandidateLastName,
    E.Data AS ExamDate
FROM
    CANDIDATI C
JOIN
    SOSTIENE S ON C.IdCandidato = S.IdCandidatoFK
JOIN
    ESAMI_DI_STATO E ON S.DataEsameFK = E.Data
WHERE
```

```
12      S.Esito = 'Superato' AND E.Data = '2025-07-21';
```

Listing 2.7: Query per l'elenco dei candidati che hanno superato l'esame in una data specifica

Questa query unisce le tabelle `CANDIDATI`, `SOSTIENE` e `ESAMI_DI_STATO` per recuperare i nomi dei candidati e la data dell'esame, filtrando per l'esito 'Superato' e la data dell'esame desiderata.

Capitolo 3

Reti di Calcolatori

Le **reti di calcolatori** sono sistemi che permettono a dispositivi interconnessi di scambiare dati e condividere risorse. Sono la base di quasi ogni infrastruttura informatica moderna, dal World Wide Web alle reti aziendali locali.

3.1 Modello di Comunicazione Client/Server

Il **modello Client/Server** è un'architettura di rete distribuita in cui i client (richiedenti servizi) e i server (fornitori di servizi) sono entità separate che comunicano su una rete. È il modello dominante per la maggior parte delle applicazioni web e molte applicazioni enterprise.

3.1.1 Funzionamento del Modello Client/Server

- **Client**: Invia richieste di servizi al server, riceve le risposte e le presenta all'utente. Tipicamente è l'applicazione utente (es. browser web, app mobile).
- **Server**: Ascolta le richieste dei client, le elabora (es. recupera dati, esegue calcoli), e invia le risposte al client. Gestisce le risorse condivise (database, file, stampanti).
- **Comunicazione**: Avviene tramite protocolli di rete (es. TCP/IP) su porte specifiche. Il client avvia la connessione e la richiesta.

3.1.2 Vantaggi e Svantaggi del Modello Client/Server

- **Vantaggi**:
  - **Centralizzazione**: Gestione centralizzata di dati e risorse, facilitando la sicurezza e la manutenzione.
  - **Scalabilità**: Possibilità di scalare il server per gestire più richieste o aggiungere più client alla rete.
  - **Sicurezza**: Controllo più agevole degli accessi e dei permessi sui dati centralizzati.
  - **Manutenzione Facilitata**: Aggiornamenti e backup possono essere eseguiti sul server senza influenzare i client.
- **Svantaggi**:
  - **Single Point of Failure (Punto Singolo di Fallimento)**: Se il server si blocca, tutti i client perdono l'accesso ai servizi.
  - **Collo di Bottiglia del Server**: Un server sovraccarico può rallentare l'intera rete.
  - **Costo**: I server e la loro manutenzione possono essere costosi.

3.2 Protocollo HTTP

L'**Hypertext Transfer Protocol (HTTP)** è il protocollo applicativo fondamentale per il World Wide Web. Opera nel modello client/server ed è stateless.

3.2.1 Funzionamento e Applicazione in HTTP

- Un **client** (tipicamente un browser web) invia una **richiesta HTTP** (es. GET, POST, PUT, DELETE) a un **server web**. La richiesta include URL, metodo, header e, opzionalmente, un body.
- Il **server** riceve la richiesta, la elabora (es. recupera una pagina web, esegue uno script), e invia una **risposta HTTP**. La risposta include uno stato (es. 200 OK, 404 Not Found), header e il body (es. il contenuto HTML della pagina richiesta).
- La comunicazione avviene tipicamente su TCP/IP (porta 80 per HTTP, 443 per HTTPS).

Esempio: Quando un utente digita un URL nel browser, il browser è il client che invia una richiesta HTTP al server. Il server risponde con la pagina HTML e le risorse associate, che il browser poi renderizza.

3.2.2 Connessioni Stateless e Stateful

Stateless Connection (Connessione Senza Stato)

- **Definizione**: Ogni richiesta inviata dal client al server è completamente indipendente e autocontenuta. Il server non mantiene alcuna informazione (stato) sulle richieste precedenti del client. Ogni richiesta include tutte le informazioni necessarie per essere elaborata.
- **Vantaggi**: Scalabilità elevata (il server non deve allocare memoria per lo stato di ogni client), resilienza (se un server si blocca, un altro può prendere il suo posto senza perdere lo stato), semplicità di progettazione lato server.
- **Svantaggi**: Richiede che ogni richiesta contenga potenzialmente informazioni ridondanti (es. credenziali di autenticazione), e può essere meno efficiente per operazioni che richiedono una sequenza di passaggi.
- **HTTP come Stateless**: HTTP è intrinsecamente stateless. Ogni richiesta HTTP è trattata come se fosse la prima e unica richiesta tra il client e il server.

Stateful Connection (Connessione Con Stato)

- **Definizione**: Il server mantiene e ricorda lo stato delle interazioni passate con un client per un certo periodo di tempo. Le richieste successive possono fare riferimento a questo stato. **Vantaggi**: Minore ridondanza di informazioni nelle richieste successive, può semplificare la logica client per sequenze di operazioni complesse.
- **Svantaggi**: Minore scalabilità (il server deve mantenere lo stato per ogni client attivo, consumando risorse), minore resilienza (se il server che detiene lo stato si blocca, la sessione del client viene persa), complessità maggiore.
- **Esempi**: Una connessione TCP (a un livello più basso) è stateful; una sessione di login a un database.

3.2.3 Metodi per Gestire la Persistenza dello Stato in HTTP

Dato che HTTP è stateless, per costruire applicazioni web interattive che richiedono il mantenimento dello stato (es. carrelli della spesa, sessioni utente), sono stati sviluppati diversi meccanismi:

- **Cookies**: Piccoli frammenti di dati che il server invia al browser del client e che il browser memorizza. Ad ogni richiesta successiva verso lo stesso server, il browser invia nuovamente i cookie al server. Usati per ID di sessione, preferenze utente, tracciamento.
- **Session IDs (ID di Sessione)**: Il server crea un ID univoco per ogni sessione utente e lo invia al client (spesso tramite cookie). Il server memorizza i dati della sessione sul proprio lato (nel database o in memoria cache) associati a quell'ID. Usati per mantenere lo stato di login, carrelli della spesa.
- **URL Rewriting (Parametri URL)**: Lo stato viene incorporato direttamente nell'URL come parametri. Usato quando i cookie non sono disponibili.
- **Hidden Form Fields**: Dati nascosti all'interno di moduli HTML che vengono inviati con ogni richiesta POST. Usati per mantenere lo stato tra le pagine di un modulo multi-step.
- **Web Storage (Local Storage, Session Storage)**: API JavaScript che permettono alle applicazioni web di memorizzare dati nel browser del client (Local Storage persistente, Session Storage per la durata della sessione del browser). Usati per memorizzare dati client-side, cache di dati.

3.3 Standard ISO/OSI

Il **modello ISO/OSI (Open Systems Interconnection)** è un modello concettuale che descrive come i sistemi di comunicazione in una rete interagiscono e cooperano. È diviso in sette strati (layer), ciascuno con responsabilità specifiche, che operano sopra lo strato precedente e forniscono servizi a quello successivo.

3.3.1 Struttura a Sette Strati del Modello OSI

1. **Strato Fisico (Physical Layer)**: Gestisce la trasmissione e ricezione di flussi di bit non strutturati e grezzi su un mezzo fisico. Definisce le specifiche elettriche, meccaniche, procedurali e funzionali. (Es. cavi Ethernet, Wi-Fi, connettori).
2. **Strato di Collegamento Dati (Data Link Layer)**: Fornisce la trasmissione di dati da nodo a nodo, rilevando e correggendo potenzialmente gli errori che possono verificarsi a livello fisico. Gestisce l'indirizzamento MAC e il controllo del flusso. (Es. Ethernet, PPP).
3. **Strato di Rete (Network Layer)**: Gestisce l'instradamento dei pacchetti attraverso la rete (routing). È responsabile dell'indirizzamento logico (IP) e della selezione del percorso migliore. (Es. IP, ICMP).
4. **Strato di Trasporto (Transport Layer)**: Fornisce la comunicazione end-to-end tra processi su host diversi. Assicura la consegna affidabile dei dati, il controllo di flusso e il controllo della congestione. (Es. TCP, UDP).
5. **Strato di Sessione (Session Layer)**: Stabilisce, gestisce e termina le sessioni di comunicazione tra applicazioni. Gestisce la sincronizzazione e il dialogo.
6. **Strato di Presentazione (Presentation Layer)**: Si occupa della sintassi e della semantica dei dati scambiati. Traduce i dati tra il formato dell'applicazione e il formato di rete, e gestisce la crittografia/decrittografia e la compressione.

7. **Strato di Applicazione (Application Layer)**: Fornisce servizi di rete direttamente alle applicazioni dell'utente finale. (Es. HTTP, FTP, SMTP, DNS).

3.4 Livello di Trasporto (TCP vs UDP)

Il **Livello di Trasporto** è il quarto strato del modello OSI e fornisce servizi di comunicazione end-to-end tra applicazioni in esecuzione su host diversi. I due protocolli principali a questo livello sono TCP e UDP.

3.4.1 TCP (Transmission Control Protocol)

TCP è un protocollo orientato alla connessione, affidabile e con controllo di flusso e congestione.

- **Orientato alla Connessione**: Stabilisce una connessione (handshake a tre vie) prima di iniziare la trasmissione dei dati e la termina esplicitamente.
- **Affidabile**: Garantisce la consegna dei dati, senza perdite o duplicazioni, e nell'ordine corretto.
  - **Numerazione e Riconoscimenti (ACK)**: Ogni segmento inviato è numerato e il mittente si aspetta un riconoscimento (ACK) dal destinatario. Se un ACK non arriva entro un certo tempo, il segmento viene ritrasmesso.
  - **Checksum**: Utilizza un checksum per rilevare errori nei dati.
- **Controllo di Flusso**: Impedisce a un mittente veloce di sovraccaricare un destinatario lento. Il destinatario comunica al mittente quanto spazio buffer è disponibile (finestra di ricezione).
- **Controllo di Congestione**: Evita che un mittente invii troppi dati in una rete congestionata, riducendo la velocità di trasmissione se rileva congestione (es. tramite perdite di pacchetti o ritardi).
- **Segmentazione e Riasssemblaggio**: Spezza i dati dell'applicazione in segmenti più piccoli per la trasmissione e li riassembla alla destinazione.
- **Applicazioni Tipiche**: Web Browse (HTTP), trasferimento file (FTP), email (SMTP), connessioni sicure (SSH).

3.4.2 UDP (User Datagram Protocol)

UDP è un protocollo senza connessione, inaffidabile e che non implementa controllo di flusso o congestione.

- **Senza Connessione**: Non stabilisce una connessione preliminare. Ogni datagramma viene inviato indipendentemente.
- **Inaffidabile (Best-Effort)**: Non garantisce la consegna dei dati, l'ordine di arrivo, né che non ci siano duplicazioni. Non ci sono ACK né ritrasmissioni automatiche.
- **Nessun Controllo di Flusso/Congestione**: Trasmette i dati alla massima velocità possibile senza preoccuparsi della capacità del destinatario o della rete.
- **Overhead Minimo**: Ha un header molto piccolo, il che lo rende molto efficiente in termini di overhead.
- **Applicazioni Tipiche**: Streaming multimediale (audio/video), VoIP, DNS, giochi online, dove la velocità è più importante dell'affidabilità perfetta (piccole perdite possono essere accettabili).

3.5 Subnetting (Suddivisione di Sottoreti)

Il **subnetting** è il processo di divisione di una singola rete IP (Internet Protocol) più grande in sottoreti (subnets) più piccole e gestibili. Questa pratica è fondamentale per migliorare l'efficienza della rete, la sicurezza e la gestione degli indirizzi IP. Ogni sottorete è una rete IP indipendente e logicamente separata.

3.5.1 Obiettivi del Subnetting

- **Efficienza nell'uso degli indirizzi IP**: Invece di allocare un'intera classe di indirizzi a una singola organizzazione, il subnetting permette di suddividere lo spazio IP, utilizzando gli indirizzi in modo più parsimonioso.
- **Riduzione del traffico di rete**: Il traffico di broadcast è confinato alla propria sottorete, riducendo il carico sui router e migliorando le prestazioni complessive della rete.
- **Miglioramento della Sicurezza**: È più facile implementare politiche di sicurezza e isolare segmenti della rete.
- **Facilitazione della Gestione della Rete**: Le reti più piccole sono più facili da gestire, diagnosticare e risolvere i problemi.

3.5.2 Indirizzi IP e Maschere di Sottorete

Un indirizzo IP (IPv4) è un numero a 32 bit, diviso in quattro ottetti (separati da punti). È composto da due parti:

- **Indirizzo di Rete (Network Address)**: Identifica la rete o sottorete a cui appartiene un dispositivo. Tutti i dispositivi sulla stessa sottorete hanno lo stesso indirizzo di rete.
- **Indirizzo Host (Host Address)**: Identifica un dispositivo specifico all'interno di quella rete o sottorete.

La **maschera di sottorete (subnet mask)** è un numero a 32 bit che aiuta a distinguere la parte di rete dalla parte host di un indirizzo IP. È composta da una serie di '1' (bit di rete) seguiti da una serie di '0' (bit di host).

Notazione CIDR (Classless Inter-Domain Routing)

La notazione **CIDR** è un modo conciso per rappresentare indirizzi IP e subnet mask. Consiste nell'indirizzo IP seguito da uno slash ('/') e un numero intero che indica il numero di bit che compongono l'indirizzo di rete (la lunghezza della maschera di sottorete).

- **Esempio**: '192.168.1.0/24' indica che i primi 24 bit sono dedicati alla rete, e i rimanenti 8 bit agli host.
- Il numero dopo lo slash è la **lunghezza del prefisso di rete**.

3.5.3 Calcolo del Subnetting (Esempio Pratico)

Consideriamo una rete di Classe C (es. '192.168.1.0/24') e vogliamo suddividerla per ospitare diverse sottoreti per dipartimenti diversi, ognuno con un numero specifico di host.

Scenario: Una rete '192.168.1.0/24' deve essere divisa in 4 sottoreti per ospitare almeno 50 host per sottorete.

Passo 1: Determinare il numero di bit necessari per le sottoreti. Abbiamo bisogno di 4 sottoreti. Il numero di bit (n) necessari per creare S sottoreti è dato da $2^n \geq S$. $2^n \geq 4 \Rightarrow n = 2$ bit.

Passo 2: Determinare il numero di bit per gli host. La maschera originale è /24, quindi abbiamo 8 bit dedicati agli host ($32 - 24 = 8$). Usando 2 bit per le sottoreti, rimangono $8 - 2 = 6$ bit per gli host in ogni sottorete. Il numero massimo di host per sottorete sarà $2^H - 2$, dove H è il numero di bit per gli host. $2^6 - 2 = 64 - 2 = 62$ host. Questo soddisfa il requisito di almeno 50 host per sottorete.

Passo 3: Calcolare la nuova maschera di sottorete. La nuova maschera avrà i bit di rete originali (24) più i bit presi in prestito per le sottoreti (2). Nuova lunghezza prefisso = $24 + 2 = 26$. La nuova maschera di sottorete sarà '/26'. In forma decimale: '255.255.255.192' (perché i 2 bit accesi nell'ultimo ottetto sono $128 + 64 = 192$).

Passo 4: Elencare le sottoreti valide. Le sottoreti iniziano a intervalli del "blocco size" determinato dal numero di bit presi in prestito. La dimensione del blocco per l'ultimo ottetto è $256 - 192 = 64$.

- **Sottorete 1**:
  - Indirizzo di Rete: '192.168.1.0/26'
  - Primo Indirizzo Host Valido: '192.168.1.1'
  - Ultimo Indirizzo Host Valido: '192.168.1.62'
  - Indirizzo di Broadcast: '192.168.1.63'
- **Sottorete 2**:
  - Indirizzo di Rete: '192.168.1.64/26'
  - Primo Indirizzo Host Valido: '192.168.1.65'
  - Ultimo Indirizzo Host Valido: '192.168.1.126'
  - Indirizzo di Broadcast: '192.168.1.127'
- **Sottorete 3**:
  - Indirizzo di Rete: '192.168.1.128/26'
  - Primo Indirizzo Host Valido: '192.168.1.129'
  - Ultimo Indirizzo Host Valido: '192.168.1.190'
  - Indirizzo di Broadcast: '192.168.1.191'
- **Sottorete 4**:
  - Indirizzo di Rete: '192.168.1.192/26'
  - Primo Indirizzo Host Valido: '192.168.1.193'
  - Ultimo Indirizzo Host Valido: '192.168.1.254'
  - Indirizzo di Broadcast: '192.168.1.255'

Questo esempio mostra come una singola rete può essere efficientemente suddivisa per soddisfare esigenze specifiche di un'organizzazione.

Domande e Esercizi

Domande d'Esame Principali

Domanda: Standard ISO/OSI e Livello di Trasporto

Domanda: Il candidato dia una panoramica della standard architetturale per reti di calcolatori interoperabili ISO/OSI, approfondendo in particolare il livello di trasporto motivando e spiegando le scelte alla base di alcune tecniche di consegna affidabile del dato.

Risposta :

Panoramica sullo Standard Architetturale ISO/OSI Il **modello ISO/OSI (Open Systems Interconnection)** è un modello concettuale a sette strati (layer) che descrive come i sistemi di comunicazione in una rete interagiscono e cooperano. Ogni strato ha responsabilità specifiche, opera sopra lo strato precedente e fornisce servizi a quello successivo, facilitando l'interoperabilità tra sistemi eterogenei. La struttura a sette strati è la seguente:

1. **Strato Fisico (Physical Layer)**: Gestisce la trasmissione e ricezione di flussi di bit non strutturati su un mezzo fisico (es. cavi Ethernet, Wi-Fi). Definisce specifiche elettriche e meccaniche.
2. **Strato di Collegamento Dati (Data Link Layer)**: Fornisce la trasmissione di dati da nodo a nodo, rilevando e potenzialmente correggendo errori. Gestisce l'indirizzamento MAC e il controllo del flusso (es. Ethernet, PPP).
3. **Strato di Rete (Network Layer)**: Gestisce l'instradamento dei pacchetti attraverso la rete (routing) ed è responsabile dell'indirizzamento logico (IP) e della selezione del percorso migliore (es. IP, ICMP).
4. **Strato di Trasporto (Transport Layer)**: Fornisce la comunicazione end-to-end tra processi su host diversi. Assicura la consegna affidabile dei dati, il controllo di flusso e il controllo della congestione (es. TCP, UDP).
5. **Strato di Sessione (Session Layer)**: Stabilisce, gestisce e termina le sessioni di comunicazione tra applicazioni, gestendo sincronizzazione e dialogo.
6. **Strato di Presentazione (Presentation Layer)**: Si occupa della sintassi e della semantica dei dati scambiati, traducendo i dati e gestendo crittografia/decrittografia e compressione.
7. **Strato di Applicazione (Application Layer)**: Fornisce servizi di rete direttamente alle applicazioni dell'utente finale (es. HTTP, FTP, SMTP, DNS).

Approfondimento sul Livello di Trasporto e Tecniche di Consegna Affidabile del Dato Il Livello di Trasporto è cruciale per stabilire una comunicazione logica tra applicazioni che risiedono su host diversi. I due protocolli principali a questo livello sono TCP (Transmission Control Protocol) e UDP (User Datagram Protocol), con TCP che si distingue per la sua capacità di fornire una consegna affidabile del dato.

TCP (Transmission Control Protocol): TCP è un protocollo orientato alla connessione, affidabile e con controllo di flusso e congestione. È la scelta preferita per applicazioni che richiedono garanzie di consegna dei dati. Le **scelte alla base della consegna affidabile del dato** in TCP includono:

- **Numerazione dei Segmenti e Riconoscimenti (ACK - Acknowledgement)**: Ogni segmento di dati inviato da TCP è numerato. Il mittente si aspetta un ACK dal destinatario

per ogni segmento ricevuto. Se un ACK non viene ricevuto entro un determinato timeout, il segmento viene considerato perso e ritrasmesso. Questo meccanismo garantisce che tutti i dati arrivino a destinazione e nell'ordine corretto, gestendo perdite e duplicazioni.

- **Retrasmissione Selettiva o Go-Back-N**: Se vengono rilevate perdite (es. tramite timeout o ACK duplicati), TCP può ritrasmettere solo i segmenti persi (selettiva) o ritrasmettere dal primo segmento non riconosciuto (Go-Back-N), assicurando la completezza dei dati.
- **Checksum**: TCP calcola un checksum per ogni segmento e lo include nell'header. Il destinatario ricalcola il checksum e lo confronta: se non corrispondono, il segmento viene scartato, garantendo l'integrità dei dati.
- **Controllo di Flusso (Flow Control)**: Impedisce che un mittente veloce sovraccarichi il buffer di un destinatario lento. Il destinatario comunica al mittente la dimensione della sua "finestra di ricezione" (receive window), cioè la quantità di spazio buffer disponibile. Il mittente non invierà più dati di quanto il destinatario possa gestire, prevenendo la perdita di dati dovuta a buffer pieni.
- **Controllo di Congestione (Congestion Control)**: Evita che un mittente invii troppi dati in una rete congestionata, il che potrebbe portare a un collasso della rete. TCP rileva la congestione (es. tramite perdite di pacchetti o ritardi) e riduce dinamicamente la sua velocità di trasmissione, rallentando fino a quando la congestione non diminuisce. Implementa algoritmi come Slow Start, Congestion Avoidance, Fast Retransmit, e Fast Recovery.

UDP (User Datagram Protocol): In contrasto, UDP è un protocollo senza connessione e inaffidabile ("best-effort"). Non garantisce la consegna, l'ordine o l'assenza di duplicazioni. Ha un overhead minimo ed è utilizzato per applicazioni dove la velocità è più critica dell'affidabilità perfetta (es. streaming multimediale, VoIP, DNS, giochi online), e dove l'applicazione stessa può gestire eventuali perdite o ritrasmissioni.

In sintesi, il Livello di Trasporto offre una scelta tra un servizio affidabile e robusto (TCP) e uno veloce e a basso overhead (UDP), permettendo alle applicazioni di scegliere il protocollo più adatto alle proprie esigenze.

Altre Possibili Domande

Domanda: Modello Client/Server e Applicazione in HTTP

Domanda: Come funziona il Modello Client/Server, quali sono i suoi vantaggi e svantaggi, e come si applica nel contesto HTTP?

Risposta:

Il **modello Client/Server** è un'architettura di rete distribuita in cui i client (richiedenti servizi) e i server (fornitori di servizi) sono entità separate che comunicano su una rete.

Funzionamento : Il Client invia richieste di servizi al Server, che le elabora e invia risposte. La comunicazione avviene tramite protocolli di rete (es. TCP/IP) su porte specifiche.

Vantaggi :

- **Centralizzazione**: Gestione centralizzata di dati e risorse (sicurezza, manutenzione).
- **Scalabilità**: Possibilità di scalare il server per gestire più richieste.

- **Manutenzione Facilitata**: Aggiornamenti sul server senza influenzare i client.

Svantaggi :

- **Single Point of Failure**: Se il server si blocca, tutti i client perdono l'accesso.
- **Collo di Bottiglia**: Un server sovraccarico rallenta l'intera rete.
- **Costo**: Server e manutenzione possono essere costosi.

Applicazione in HTTP : L'**Hypertext Transfer Protocol (HTTP)** è il protocollo applicativo fondamentale per il World Wide Web, che opera nel modello client/server ed è intrinsecamente stateless. Il client (browser) invia richieste HTTP (GET, POST) a un server web, che risponde con lo stato e il contenuto richiesto (es. pagina HTML).

Domanda: Connessioni Stateless e Stateful in HTTP

Domanda: Qual è la differenza tra connessioni stateless e stateful nel contesto HTTP, e quali metodi sono utilizzati per gestire la persistenza dello stato in HTTP?

Risposta:

Connessioni Stateless e Stateful

- **Stateless Connection (Senza Stato)**: Ogni richiesta dal client al server è indipendente e autocontenuta; il server non ricorda le interazioni passate.
  - **Vantaggi**: Alta scalabilità, resilienza, semplicità lato server.
  - **Svantaggi**: Richiede informazioni ridondanti per ogni richiesta.
  - **HTTP**: È intrinsecamente stateless.
- **Stateful Connection (Con Stato)**: Il server mantiene e ricorda lo stato delle interazioni passate con un client per un periodo.
  - **Vantaggi**: Meno ridondanza, semplifica logica per sequenze complesse.
  - **Svantaggi**: Minore scalabilità, minore resilienza, maggiore complessità.

Metodi per Gestire la Persistenza dello Stato in HTTP (per simulare Stateful) : Dato che HTTP è stateless, si utilizzano meccanismi come:

- **Cookies**: Piccoli dati inviati dal server al browser, che li memorizza e li invia con richieste successive (per ID sessione, preferenze utente).
- **Session IDs**: Il server crea un ID univoco e lo invia al client (spesso tramite cookie); il server memorizza i dati della sessione.
- **URL Rewriting**: Lo stato incorporato direttamente nell'URL come parametri.
- **Hidden Form Fields**: Dati nascosti in moduli HTML inviati con richieste POST.
- **Web Storage** (Local Storage, Session Storage): API JavaScript per memorizzare dati nel browser del client.

Domanda: Struttura a Sette Strati del Modello ISO/OSI

Domanda: Qual è la struttura a sette strati del Modello ISO/OSI e quali sono le responsabilità di ciascun strato?

Risposta:

Il **modello ISO/OSI (Open Systems Interconnection)** è un modello concettuale che descrive come i sistemi di comunicazione in una rete interagiscono. È diviso in sette strati (layer), ciascuno con responsabilità specifiche:

1. **Strato Fisico (Physical Layer)**: Gestisce la trasmissione di bit grezzi su un mezzo fisico (es. cavi Ethernet).
2. **Strato di Collegamento Dati (Data Link Layer)**: Fornisce la trasmissione dati da nodo a nodo, rilevando errori e gestendo l'indirizzamento MAC (es. Ethernet).
3. **Strato di Rete (Network Layer)**: Gestisce l'instradamento dei pacchetti (routing) e l'indirizzamento logico (IP).
4. **Strato di Trasporto (Transport Layer)**: Fornisce la comunicazione end-to-end tra processi, assicurando la consegna affidabile dei dati, controllo di flusso e congestione (es. TCP, UDP).
5. **Strato di Sessione (Session Layer)**: Stabilisce, gestisce e termina le sessioni di comunicazione tra applicazioni.
6. **Strato di Presentazione (Presentation Layer)**: Si occupa della sintassi e semantica dei dati, inclusa crittografia/decrittografia e compressione.
7. **Strato di Applicazione (Application Layer)**: Fornisce servizi di rete direttamente alle applicazioni dell'utente finale (es. HTTP, FTP).

Domanda: Confronto tra TCP e UDP

Domanda: Confronta il protocollo TCP con il protocollo UDP, evidenziando le loro caratteristiche principali, vantaggi, svantaggi e applicazioni tipiche.

Risposta:

Il **Livello di Trasporto** del modello OSI gestisce la comunicazione end-to-end tra processi. I protocolli principali sono TCP e UDP.

TCP (Transmission Control Protocol) :

- **Orientato alla Connessione**: Stabilisce una connessione (handshake a tre vie).
- **Affidabile**: Garantisce la consegna ordinata e senza perdite (numerazione, ACK, ritrasmissioni).
- **Controllo di Flusso**: Previene il sovraccarico del destinatario (finestra di ricezione).
- **Controllo di Congestione**: Riduce la velocità in reti congestionate.
- **Applicazioni Tipiche**: Web (HTTP), trasferimento file (FTP), email (SMTP), SSH.

UDP (User Datagram Protocol) :

- **Senza Connessione**: Invia datagrammi indipendenti senza handshake.
- **Inaffidabile (Best-Effort)**: Non garantisce consegna, ordine o assenza di duplicazioni.
- **Nessun Controllo**: Non implementa controllo di flusso o congestione.
- **Overhead Minimo**: Header piccolo, molto efficiente.
- **Applicazioni Tipiche**: Streaming multimediale (VoIP, video), DNS, giochi online (dove la velocità è prioritaria).

Domanda: Concetto e Calcolo del Subnetting

Domanda: Spiega il concetto di subnetting, i suoi obiettivi e illustra con un esempio pratico come si calcolano le sottoreti per una data rete IP.

Risposta:

Il **subnetting** è il processo di divisione di una rete IP più grande in sottoreti più piccole e gestibili, per migliorare efficienza, sicurezza e gestione degli indirizzi IP.

Obiettivi :

- **Efficienza nell'uso degli indirizzi IP**: Sfruttare meglio lo spazio IP.
- **Riduzione del traffico di rete**: Confinare il traffico di broadcast.
- **Miglioramento della Sicurezza**: Isolare segmenti di rete.
- **Facilitazione della Gestione**: Reti più piccole più facili da gestire.

Un indirizzo IP (IPv4) ha 32 bit (parte di rete + parte host). La **maschera di sottorete** (subnet mask) separa queste due parti. La notazione **CIDR** (`IP/prefisso`) indica la lunghezza del prefisso di rete.

Calcolo del Subnetting (Esempio Pratico) : **Scenario**: Una rete `192.168.1.0/24` deve essere divisa in 4 sottoreti per ospitare almeno 50 host per sottorete.

1. **Bit per sottoreti**: $2^n \geq 4 \Rightarrow n = 2$ bit.
2. **Bit per host**: Maschera originale /24 (8 bit host). Usando 2 bit per sottoreti, rimangono $8 - 2 = 6$ bit per host. Max host per sottorete: $2^6 - 2 = 62$ host (sufficiente).
3. **Nuova maschera**: $24 + 2 = 26$. Maschera `/26` (`255.255.255.192`).
4. **Sottoreti valide** (intervalli di 64 nell'ultimo ottetto):
   - Sottorete 1: `192.168.1.0/26` (Host: .1 a .62, Broadcast: .63)
   - Sottorete 2: `192.168.1.64/26` (Host: .65 a .126, Broadcast: .127)
   - Sottorete 3: `192.168.1.128/26` (Host: .129 a .190, Broadcast: .191)
   - Sottorete 4: `192.168.1.192/26` (Host: .193 a .254, Broadcast: .255)

Capitolo 4

Programmazione Orientata agli Oggetti (e Fondamenti)

La **Programmazione Orientata agli Oggetti (OOP)** è un paradigma di programmazione basato sul concetto di "oggetti", che possono contenere dati e codice. È uno dei paradigmi più diffusi per lo sviluppo di software moderno.

4.1 Concetti Base della Programmazione Orientata agli Oggetti (POO)

La POO si fonda su alcuni pilastri fondamentali che ne definiscono la struttura e il funzionamento:

- **Classe**: Una blueprint o un modello per creare oggetti. Definisce le proprietà (attributi/campi) e i comportamenti (metodi/funzioni) che gli oggetti di quel tipo avranno. Non è un'entità fisica, ma una definizione logica.
- **Oggetto**: Un'istanza di una classe. È un'entità concreta che ha uno stato (valori specifici degli attributi) e un comportamento (i metodi che può eseguire).
- **Incapsulamento (Encapsulation)**: Il principio di raggruppare i dati (attributi) e le funzioni (metodi) che operano su quei dati all'interno di un'unica unità (la classe). Protegge i dati interni dall'accesso diretto esterno, permettendone la manipolazione solo tramite metodi pubblici della classe. Questo migliora la sicurezza e la manutenibilità del codice.
- **Ereditarietà (Inheritance)**: Un meccanismo che permette a una classe (sottoclasse o classe derivata) di ereditare proprietà e comportamenti da un'altra classe (superclasse o classe base). Promuove il riutilizzo del codice e la creazione di gerarchie di classi che riflettono relazioni "è un tipo di".
- **Polimorfismo (Polymorphism)**: Il concetto che un oggetto possa assumere molte forme. In OOP, si riferisce alla capacità di oggetti di classi diverse di rispondere allo stesso messaggio (chiamata di metodo) in modi diversi, o alla capacità di un'interfaccia di riferirsi a oggetti di diverse classi che la implementano. Si manifesta tramite:
  - **Overriding**: Una sottoclasse fornisce un'implementazione specifica di un metodo già definito nella sua superclasse.
  - **Overloading**: Definire più metodi con lo stesso nome all'interno della stessa classe, ma con liste di parametri diverse (numero, tipo o ordine).

4.2 Strutture Dati Astratte (ADT) e Fondamentali di Programmazione

4.2.1 Abstract Data Type (ADT)

Un **Abstract Data Type (ADT)** è una definizione matematica di una struttura dati, che specifica un insieme di dati e un insieme di operazioni che possono essere eseguite su quei dati. È "astratto" perché si concentra sul "cosa" la struttura dati fa (il suo comportamento) piuttosto che sul "come" lo fa (la sua implementazione interna). L'ADT separa l'interfaccia (pubblica) dall'implementazione (privata).

- **Caratteristiche**:
  - **Astrazione dei Dati**: Nasconde i dettagli di rappresentazione interna dei dati.
  - **Astrazione Funzionale**: Nasconde i dettagli di implementazione delle operazioni.
  - **Insieme di Operazioni**: Definisce un insieme ben preciso di funzioni o metodi che possono essere applicati ai dati.
- **Esempi Comuni di ADT**:
  - **ADT List (Lista)**: Un insieme ordinato di elementi. Operazioni tipiche: inserimento (add), rimozione (remove), accesso a un elemento per indice (get), dimensione (size), verifica se vuota (isEmpty). L'implementazione può essere con array, liste concatenate, ecc.
  - **ADT Stack (Pila)**: Una collezione di elementi che segue il principio LIFO (Last-In, First-Out). Operazioni tipiche: push (aggiungere un elemento in cima), pop (rimuovere l'elemento in cima), peek (vedere l'elemento in cima senza rimuoverlo), isEmpty.
  - **ADT Queue (Coda)**: Una collezione di elementi che segue il principio FIFO (First-In, First-Out). Operazioni tipiche: enqueue (aggiungere un elemento in coda), dequeue (rimuovere l'elemento in testa), peek, isEmpty.

4.2.2 Passaggio di Parametri nelle Chiamate di Funzione

Quando si chiama una funzione (o routine, o metodo), i valori o i riferimenti alle variabili vengono passati come argomenti. Esistono due meccanismi principali per il passaggio dei parametri:

Passaggio per Valore (Call by Value)

- **Descrizione**: Viene passata una **copia del valore** dell'argomento alla funzione. La funzione opera su questa copia locale.
- **Effetto**: Qualsiasi modifica apportata alla copia del parametro all'interno della funzione **non influisce** sulla variabile originale passata dal chiamante.
- **Quando usato**: Per tipi di dati primitivi (interi, booleani, caratteri) nella maggior parte dei linguaggi, e per oggetti complessi quando si desidera che la funzione non alteri l'originale.

```
FUNCTION ModificaValore(numero: Integer):
    numero = numero + 10
    PRINT "Dentro la funzione: ", numero
END FUNCTION

```

```
DECLARE myNumber: Integer = 5
CALL ModificaValore(myNumber)
PRINT "Dopo la funzione: ", myNumber
// Output atteso: "Dopo la funzione: 5" (myNumber non e' cambiato)
```

Listing 4.1: Esempio di Passaggio per Valore

Passaggio per Riferimento (Call by Reference / Call by Address)

- **Descrizione**: Viene passato l'**indirizzo di memoria** della variabile originale alla funzione. La funzione accede e opera direttamente sulla variabile originale tramite questo indirizzo.
- **Effetto**: Qualsiasi modifica apportata al parametro all'interno della funzione **influenza direttamente** la variabile originale passata dal chiamante.
- **Quando usato**: Spesso per oggetti complessi (in linguaggi come Java, Python, C# gli oggetti sono tipicamente passati per riferimento implicito, anche se tecnicamente è un "passaggio per valore del riferimento"), o esplicitamente in linguaggi come C++ (usando '&' o puntatori).
- **Differenze Chiave**: La differenza fondamentale è se la funzione lavora su una copia (per valore) o direttamente sull'originale (per riferimento). Il passaggio per riferimento è più efficiente per oggetti grandi in quanto evita la copia, ma richiede maggiore attenzione per evitare effetti collaterali indesiderati.

```
FUNCTION ModificaArray(arrayRef: Array of Integer):
    arrayRef[0] = arrayRef[0] + 10
    PRINT "Dentro la funzione: ", arrayRef[0]
END FUNCTION

DECLARE myArray: Array of Integer = [1, 2, 3]
CALL ModificaArray(myArray)
PRINT "Dopo la funzione: ", myArray[0]
// Output atteso: "Dopo la funzione: 11" (myArray[0] e' cambiato)
```

Listing 4.2: Esempio di Passaggio per Riferimento

4.2.3 Overloading di Funzioni (o Metodi)

L'**Overloading di funzioni** (o metodi, nel contesto OOP) è la capacità di definire più funzioni o metodi con lo **stesso nome** all'interno dello stesso scope (solitamente la stessa classe), ma che si distinguono per avere **liste di parametri diverse**. La lista dei parametri può differire per:

- Il **numero** di parametri.
- Il **tipo** dei parametri.
- L'**ordine** dei parametri.

Il compilatore (o l'interprete) determina quale versione del metodo chiamare basandosi sul numero e tipo degli argomenti forniti durante la chiamata. **Esempio**:

```
FUNCTION Add(a: Integer, b: Integer):
    RETURN a + b
END FUNCTION
```

```
4
5  FUNCTION Add(a: Double, b: Double):
6      RETURN a + b
7  END FUNCTION
8
9  FUNCTION Add(a: Integer, b: Integer, c: Integer):
10     RETURN a + b + c
11 END FUNCTION
```

Listing 4.3: Esempio di Function Overloading

4.3 Linguaggi Compilati e Linguaggi Interpretati

La distinzione tra linguaggi compilati e interpretati riguarda il modo in cui il codice sorgente viene eseguito dal computer.

4.3.1 Linguaggi Compilati

- **Descrizione**: Il codice sorgente viene tradotto (compilato) in codice macchina (o bytecode per la JVM) una sola volta da un programma chiamato "compilatore" prima dell'esecuzione. Il file eseguibile risultante può essere eseguito direttamente dal sistema operativo.
- **Processo**: Codice Sorgente → Compilatore → Codice Macchina Eseguibile.
- **Vantaggi**:
  - **Prestazioni Elevate**: Il codice macchina è ottimizzato per l'hardware specifico, risultando in esecuzioni molto veloci.
  - **Esecuzione Indipendente**: Una volta compilato, l'eseguibile non richiede il compilatore per essere eseguito.
  - **Rilevamento Errori Precoce**: La maggior parte degli errori di sintassi e alcuni errori logici vengono rilevati in fase di compilazione.
- **Svantaggi**:
  - **Tempo di Compilazione**: Richiede un passaggio aggiuntivo di compilazione prima dell'esecuzione.
  - **Dipendenza dalla Piattaforma**: L'eseguibile compilato è specifico per una particolare architettura hardware e sistema operativo (a meno di VM come Java).
- **Esempi**: C, C++, Java (compila in bytecode che viene interpretato/JIT compilato dalla JVM), Go, Rust.

4.3.2 Linguaggi Interpretati

- **Descrizione**: Il codice sorgente viene eseguito riga per riga da un programma chiamato "interprete" al momento dell'esecuzione, senza una fase di compilazione preventiva in codice macchina.
- **Processo**: Codice Sorgente → Interprete → Esecuzione.
- **Vantaggi**:
  - **Flessibilità e Rapidità di Sviluppo**: Non c'è un passo di compilazione, quindi le modifiche possono essere testate immediatamente.

  - **Indipendenza dalla Piattaforma**: Lo stesso codice sorgente può essere eseguito su qualsiasi piattaforma che abbia un interprete installato.
- **Svantaggi**:
  - **Prestazioni Generalmente Inferiori**: L'interprete analizza e traduce il codice in tempo reale, il che è più lento dell'esecuzione di codice macchina nativo.
  - **Rilevamento Errori Tardo**: Gli errori (anche di sintassi) vengono spesso rilevati solo a runtime, quando l'interprete tenta di eseguire la riga problematica.
- **Esempi**: Python, JavaScript, Ruby, PHP.

4.4 Algoritmi e Complessità Computazionale

L'**analisi della complessità computazionale** è lo studio delle risorse richieste da un algoritmo per risolvere un problema. Le risorse principali sono il tempo di esecuzione e lo spazio di memoria.

4.4.1 Complessità Temporale

Misura il tempo che un algoritmo impiega per completare la sua esecuzione, in funzione della dimensione dell'input. Viene espressa utilizzando la **notazione O-grande (Big O notation)** ($O(n)$), che descrive il tasso di crescita superiore del tempo di esecuzione dell'algoritmo al crescere della dimensione dell'input.

- **O(1) - Tempo Costante**: Il tempo di esecuzione non dipende dalla dimensione dell'input.
- **O(log n) - Tempo Logaritmico**: Il tempo di esecuzione cresce logaritmicamente con la dimensione dell'input (es. ricerca binaria).
- **O(n) - Tempo Lineare**: Il tempo di esecuzione cresce linearmente con la dimensione dell'input (es. scansione di un array).
- **O(n log n) - Tempo Lineare-Logaritmico**: Tempo di esecuzione per algoritmi di ordinamento efficienti (es. Merge Sort, Quick Sort).
- **O(n^2) - Tempo Quadratico**: Il tempo di esecuzione è proporzionale al quadrato della dimensione dell'input (es. algoritmi di ordinamento semplici come Bubble Sort, nested loops).
- **O(2^n) - Tempo Esponenziale**: Il tempo di esecuzione cresce esponenzialmente con la dimensione dell'input (es. problemi NP-completi non ottimizzati, ricerca esaustiva).

4.4.2 Complessità Spaziale

Misura la quantità di memoria ausiliaria (oltre all'input stesso) che un algoritmo richiede per completare la sua esecuzione, in funzione della dimensione dell'input. Anche questa è espressa con la notazione O-grande.

- **O(1) - Spazio Costante**: L'algoritmo utilizza una quantità fissa di memoria, indipendentemente dalla dimensione dell'input.
- **O(n) - Spazio Lineare**: La memoria richiesta cresce linearmente con la dimensione dell'input (es. memorizzare una copia dell'input).
- **O(n^2) - Spazio Quadratico**: La memoria richiesta cresce quadraticamente con la dimensione dell'input (es. memorizzare una matrice $N \times N$).

Capitolo 5

Progettazione del Software

La **Progettazione del Software** è il processo di definizione dell'architettura, dei componenti, delle interfacce e di altri attributi di un sistema o di un componente. È una fase cruciale nel ciclo di vita dello sviluppo del software, che traduce i requisiti in un piano dettagliato per la costruzione del sistema.

5.1 Analisi dei Requisiti

L'**analisi dei requisiti** è il processo di definizione, documentazione e mantenimento dei requisiti software. È la fase iniziale di qualsiasi progetto software e mira a comprendere le esigenze degli stakeholder per il sistema da costruire.

5.1.1 Tipi di Requisiti

I requisiti possono essere classificati in due categorie principali:

- **Requisiti Funzionali**: Descrivono ciò che il sistema *deve fare*. Definiscono le funzioni, i servizi e i comportamenti specifici che il sistema deve fornire agli utenti.
  - **Esempi**: "Il sistema deve consentire la registrazione di nuovi utenti.", "Il sistema deve calcolare la somma delle vendite giornaliere.", "Il sistema deve permettere la visualizzazione del calendario delle partite."
- **Requisiti Non Funzionali (o di Qualità)**: Descrivono come il sistema *deve funzionare*. Definiscono le caratteristiche di qualità, i vincoli e gli attributi del sistema, piuttosto che le sue funzionalità specifiche. Spesso influenzano l'architettura e l'implementazione.
  - **Esempi**:
    - **Prestazioni**: "Il sistema deve rispondere a una query entro 2 secondi."
    - **Scalabilità**: "Il sistema deve supportare 1000 utenti concorrenti."
    - **Sicurezza**: "Il sistema deve autenticare gli utenti tramite username e password con criptazione."
    - **Usabilità**: "L'interfaccia utente deve essere intuitiva e di facile apprendimento."
    - **Affidabilità**: "Il sistema deve essere disponibile il 99.9% del tempo."
    - **Manutenibilità**: "Il codice deve essere modulare e ben documentato."
    - **Portabilità**: "Il sistema deve funzionare su Windows e Linux."

5.1.2 Diagrammi di Casi d'Uso UML (Use Case Diagrams)

I **Diagrammi di Casi d'Uso** sono uno strumento UML (Unified Modeling Language) utilizzato nell'analisi dei requisiti funzionali. Descrivono le interazioni tra gli utenti (attori) e il sistema, rappresentando le diverse funzionalità che il sistema offre dal punto di vista dell'utente.

- **Componenti Principali**:
  - **Attore (Actor)**: Rappresenta un ruolo esterno che interagisce con il sistema (persona, altro sistema, dispositivo). Disegnato come un omino stilizzato.
  - **Caso d'Uso (Use Case)**: Rappresenta una funzionalità specifica del sistema, un servizio che il sistema fornisce all'attore. Disegnato come un ovale.
  - **Confine del Sistema (System Boundary)**: Un rettangolo che racchiude i casi d'uso, distinguendo ciò che è all'interno del sistema da ciò che è esterno.
  - **Relazioni**:
    - **Associazione**: L'interazione tra un attore e un caso d'uso (linea semplice).
    - **Include**: Un caso d'uso include la funzionalità di un altro caso d'uso (freccia tratteggiata da caso d'uso includente a caso d'uso incluso, con etichetta '«include»').
    - **Extend**: Un caso d'uso estende il comportamento di un altro caso d'uso in circostanze specifiche (freccia tratteggiata da caso d'uso estendente a caso d'uso esteso, con etichetta '«extend»').
    - **Generalizzazione**: Una relazione di ereditarietà tra attori o casi d'uso (freccia con punta triangolare).
- **Scopo**: Fornire una visione ad alto livello dei requisiti funzionali, facilitare la comunicazione tra stakeholder e sviluppatori, e servire da base per la progettazione successiva.

5.2 Progettazione dell'Architettura del Sistema Software

La **progettazione dell'architettura del software** definisce la struttura di alto livello di un sistema software, delineando come i suoi componenti interagiscono e sono organizzati. Include la scelta di pattern architetturali e di design.

5.2.1 Design Patterns (Pattern Architetturali e di Progettazione)

I **Design Patterns** sono soluzioni riutilizzabili a problemi comuni che si presentano nella progettazione del software. Non sono soluzioni pronte all'uso, ma modelli da adattare al contesto specifico.

- **Vantaggi**:
  - **Riutilizzo di Soluzioni Comprovate**: Utilizzano approcci che hanno dimostrato di funzionare.
  - **Vocabolario Condiviso**: Facilitano la comunicazione tra gli sviluppatori.
  - **Miglioramento della Qualità del Codice**: Portano a codice più manutenibile, scalabile e flessibile.
- **Esempi Comuni**:
  - **MVC (Model-View-Controller)**: Un pattern architetturale che separa l'applicazione in tre componenti interconnessi per gestire meglio l'interfaccia utente.
    - **Model**: Gestisce i dati e la logica di business.

    - **View**: Si occupa della presentazione dei dati all'utente.
    - **Controller**: Gestisce l'input dell'utente e coordina Model e View.
  - **Singleton**: Garantisce che una classe abbia una sola istanza e fornisce un punto di accesso globale a essa. Utile per risorse uniche come gestori di configurazione o log.
  - **Factory Method**: Definisce un'interfaccia per creare un oggetto, ma lascia alle sottoclassi la decisione di quale classe istanziare. Permette di creare oggetti senza specificare la classe esatta che verrà creata.
  - **Observer**: Definisce una dipendenza uno-a-molti tra oggetti in modo che quando un oggetto cambia stato, tutti i suoi dipendenti vengono notificati e aggiornati automaticamente. Utile per eventi e notifiche.
  - **Strategy**: Definisce una famiglia di algoritmi, incapsula ciascuno di essi e li rende intercambiabili. Permette all'algoritmo di variare indipendentemente dai client che lo usano.

5.2.2 Diagrammi UML (Unified Modeling Language)

Il **Unified Modeling Language (UML)** è un linguaggio di modellazione standardizzato, ampiamente utilizzato nell'ingegneria del software per specificare, visualizzare, costruire e documentare gli artefatti di un sistema software. Offre una notazione grafica per rappresentare vari aspetti di un sistema, dalla sua struttura statica al suo comportamento dinamico.

Diagrammi dei Casi d'Uso (Use Case Diagrams)

I **Diagrammi dei Casi d'Uso** sono diagrammi comportamentali UML utilizzati nella fase di analisi dei requisiti. Descrivono le interazioni tra gli utenti (attori) e il sistema, rappresentando le diverse funzionalità che il sistema offre dal punto di vista dell'utente. Si concentrano sul "cosa" il sistema fa per i suoi attori, piuttosto che sul "come" lo fa.

- **Scopo**: Identificare e documentare i requisiti funzionali del sistema. Forniscono una visione ad alto livello delle funzionalità, facilitano la comunicazione tra stakeholder e sviluppatori.
- **Componenti Principali**:
  - **Attore (Actor)**: Un ruolo esterno (persona, altro sistema, dispositivo hardware) che interagisce con il sistema. Rappresentato da un omino stilizzato.
  - **Caso d'Uso (Use Case)**: Una funzionalità specifica del sistema, un servizio che il sistema fornisce all'attore. Rappresentato da un ovale.
  - **Confine del Sistema (System Boundary)**: Un rettangolo che racchiude i casi d'uso, distinguendo ciò che è all'interno del sistema da ciò che è esterno.
  - **Relazioni**:
    - **Associazione**: L'interazione tra un attore e un caso d'uso (linea semplice).
    - **Include (‘«include»‘)**: Indica che un caso d'uso include la funzionalità di un altro caso d'uso. La freccia è tratteggiata, va dal caso d'uso includente a quello incluso. Usato per riutilizzare comportamenti comuni.
    - **Extend (‘«extend»‘)**: Indica che un caso d'uso estende il comportamento di un altro caso d'uso in circostanze specifiche. La freccia è tratteggiata, va dal caso d'uso estendente a quello esteso. Usato per funzionalità opzionali o eccezionali.
    - **Generalizzazione**: Una relazione di ereditarietà tra attori o casi d'uso (freccia con punta triangolare non piena).

Esempio: Relazioni tra Casi d'Uso La comprensione delle relazioni 'Include', 'Extend' e 'Generalizzazione' è cruciale per modellare accuratamente il comportamento del sistema e i requisiti opzionali o riutilizzabili.

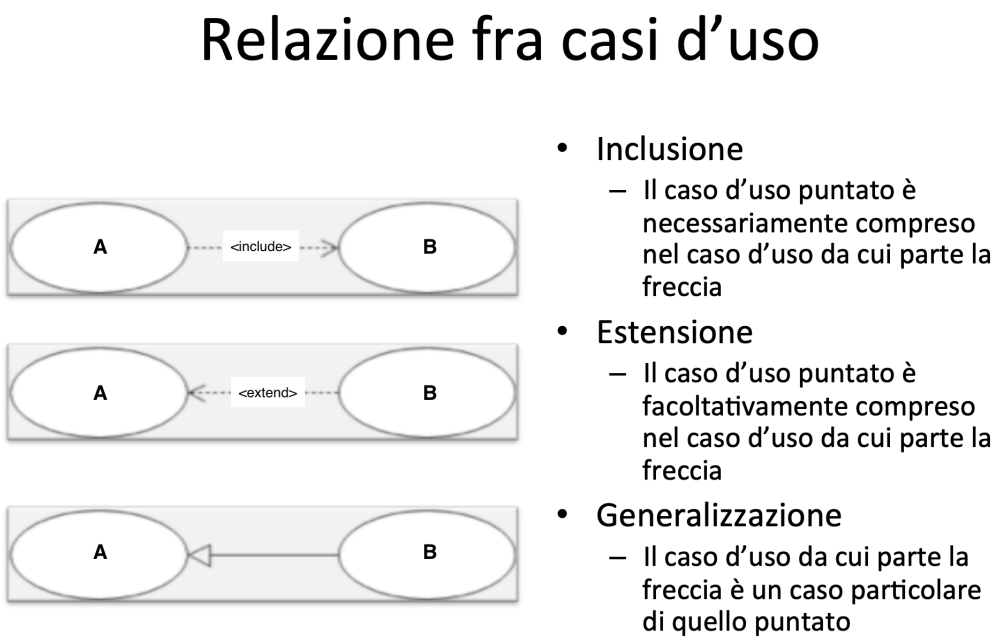


Figura 5.1: Rappresentazione grafica delle relazioni comuni tra Casi d'Uso in UML (Inclusione, Estensione e Generalizzazione).

Esempio Completo: Sistema di Ristorazione Un esempio pratico aiuta a consolidare la comprensione di come i casi d'uso e le loro relazioni si applichino in un contesto reale. Il diagramma seguente mostra le funzionalità di un sistema di gestione per un ristorante, illustrando le interazioni tra clienti, personale e il sistema stesso.

Diagrammi delle Classi (Class Diagrams)

I **Diagrammi delle Classi** sono diagrammi strutturali UML che mostrano la struttura statica di un sistema, le classi, i loro attributi (dati), i loro metodi (operazioni) e le relazioni tra le classi. Sono fondamentali per la progettazione orientata agli oggetti e per modellare il design logico del database.

- **Scopo**: Modellare il design logico del sistema, la struttura del codice, le relazioni tra le classi e le dipendenze. Utile per visualizzare l'architettura del software a livello di classi.
- **Componenti Principali**:
  - **Classe**: Rappresentata da un rettangolo diviso in tre sezioni: nome della classe, attributi (con visibilità e tipo), e metodi (con visibilità, parametri e tipo di ritorno).
  - **Visibilità (Visibility)**: Indica l'accessibilità degli attributi e metodi, rappresentata da simboli specifici (`+`, `-`, `\#`, `\textasciitilde`).
  - **Relazioni**: Specificano le associazioni tra le classi.

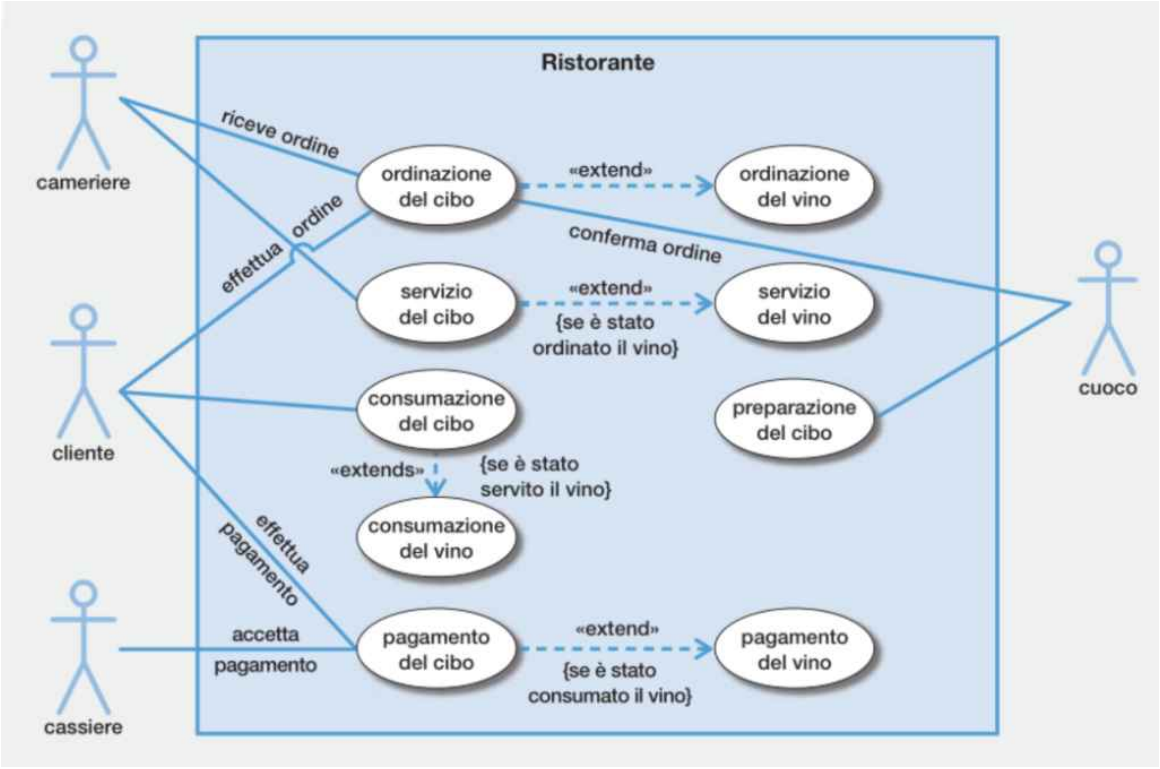
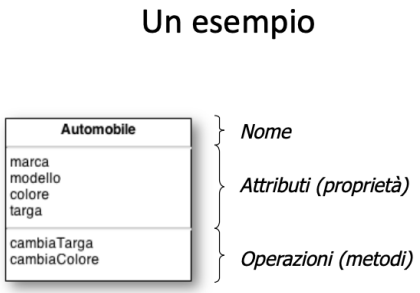
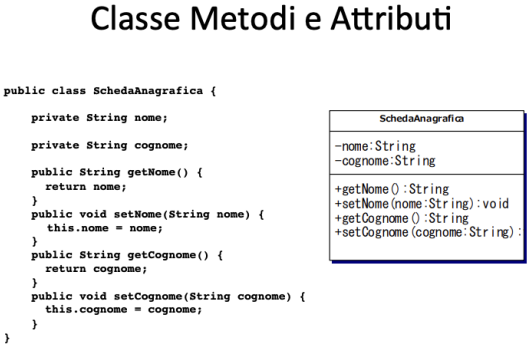


Figura 5.2: Esempio completo di Diagramma dei Casi d'Uso per un sistema di Ristorazione, che illustra le interazioni tra attori (Cameriere, Cliente, Cuoco, Cassiere) e le funzionalità del sistema.

Struttura Base e Attributi Un diagramma delle classi inizia con la rappresentazione della singola classe, evidenziando il suo nome, i suoi attributi (le proprietà) e le sue operazioni (i metodi).



(a) Struttura di una Classe (Automobile)



(b) Classe con Metodi e Attributi (Scheda Anagrafica)

Figura 5.3: Esempi di rappresentazione base di una classe UML, con attributi e metodi, e come si relaziona al codice.

L'uso di modificatori di visibilità definisce l'accessibilità di questi attributi e metodi, come mostrato nell'esempio dei Clienti e Fornitori.

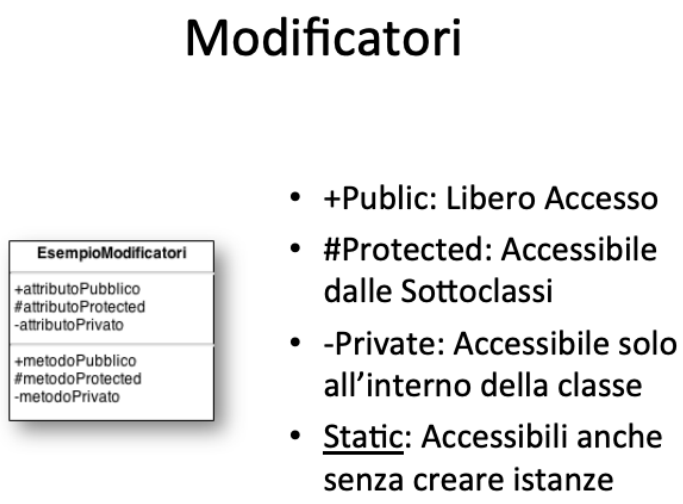


Figura 5.4: Esempio di Diagramma delle Classi che illustra l'uso dei modificatori di visibilità (public, private, protected) per attributi e metodi.

Relazioni di Generalizzazione (Ereditarietà) La generalizzazione indica che una classe (sottoclasse) eredita proprietà e comportamenti da un'altra classe (superclasse). Questo promuove il riutilizzo del codice. In contesti come Java, dove l'ereditarietà multipla non è ammessa (a

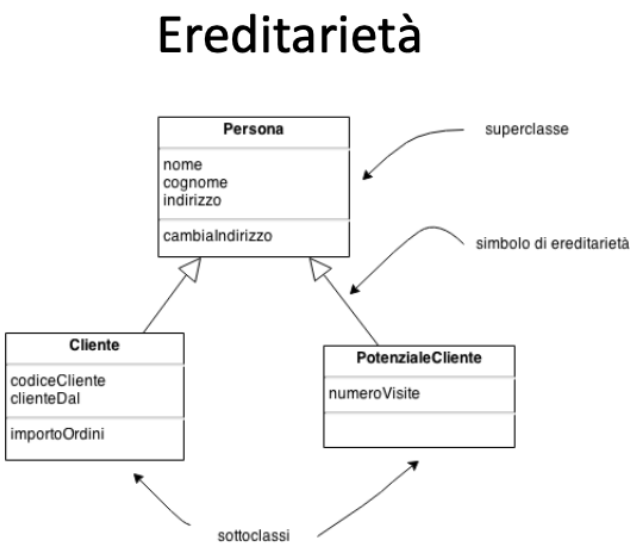


Figura 5.5: Esempio di Diagramma delle Classi che mostra la relazione di ereditarietà tra le classi Persona, Cliente e Potenziale Cliente.

differenza di C++), le interfacce vengono utilizzate per ovviare a questa limitazione, permettendo a una classe di implementare più interfacce.

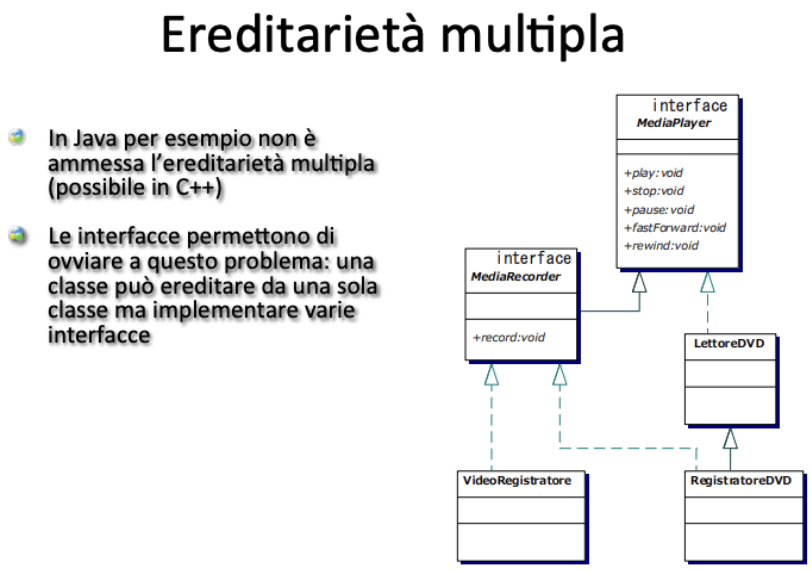
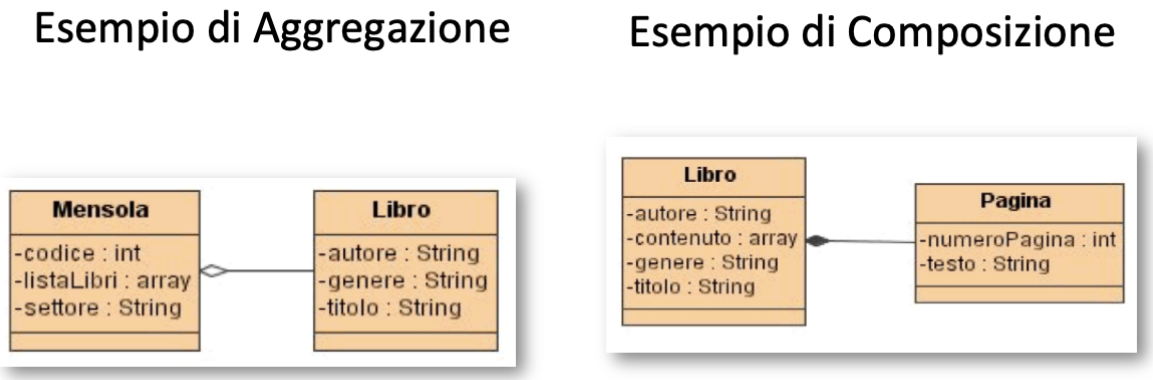


Figura 5.6: Esempio di Diagramma delle Classi che illustra come le interfacce consentono di simulare l'ereditarietà multipla in linguaggi che non la supportano nativamente.

Relazioni Strutturali: Aggregazione e Composizione Aggregazione e composizione sono forme specifiche di associazione che esprimono relazioni "parte di" tra classi, distinguendosi per la dipendenza esistenziale della parte rispetto al tutto.

- **Aggregazione**: Rappresenta una relazione uno a molti in cui l'oggetto "parte" può esistere indipendentemente dall'oggetto "tutto" (es. un libro può esistere senza una mensola specifica). Viene rappresentata con una freccia con la punta a diamante vuota all'estremità del "tutto".
- **Composizione**: Una forma più forte di aggregazione, che implica una esclusività. La "parte" non può esistere da sola senza il "tutto", e la distruzione del "tutto" comporta la distruzione delle "parti" (es. una pagina non può esistere senza il suo libro). Il diamante si disegna pieno.



(a) Esempio di Aggregazione (Mensola e Libro) (b) Esempio di Composizione (Libro e Pagina)

Figura 5.7: Confronto visivo tra Aggregazione e Composizione nei Diagrammi delle Classi UML.

Esempi Complessi di Diagrammi delle Classi Per illustrare l'applicazione di queste relazioni in contesti più ampi, consideriamo sistemi con diverse classi interconnesse, come un sistema e-commerce o un'applicazione di chat. Questi esempi mostrano come le classi interagiscono per formare un sistema coerente.

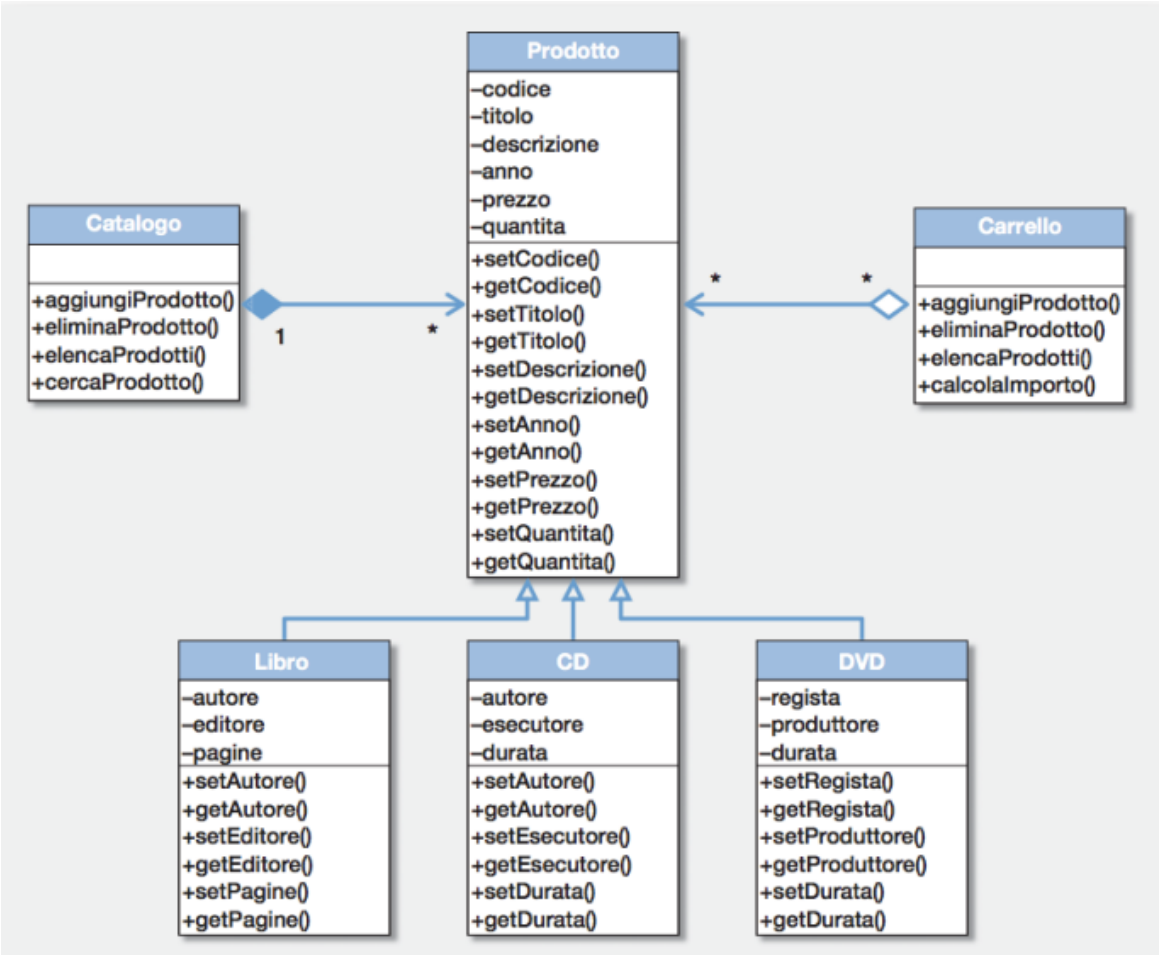


Figura 5.8: Diagramma delle Classi per un sistema di gestione prodotti/catalogo/carrello, che illustra varie classi e le loro relazioni (es. Prodotto, Carrello, DVD, Libro).

Diagrammi di Sequenza (Sequence Diagrams)

Un **Diagramma di Sequenza**, o in inglese *Sequence Diagram*, è un diagramma appartenente alla famiglia del linguaggio **UML (Unified Modeling Language)**, che viene utilizzato principalmente per rappresentare le interazioni tra gli oggetti, rispettando l'ordine sequenziale in cui avvengono questi scambi di messaggi. Sono utili non solo agli sviluppatori per modellare il comportamento di un'applicazione, ma anche ai dirigenti di un'azienda, in quanto possono riprodurre il comportamento dei vari elementi del sistema che costituiscono il business, mostrando come interagiscono tra loro. Lo staff tecnico di un'organizzazione potrebbe avvalersi di questi diagrammi per documentare un comportamento. Ad esempio, durante la fase di progettazione, architetti e sviluppatori possono utilizzare questo schema per aggiungere o eliminare le interazioni tra le componenti del sistema.

È possibile utilizzare i diagrammi di sequenza a diversi livelli durante il processo di sviluppo:

- Nella fase di **analisi**, possono aiutare ad identificare le classi di cui un sistema ha bisogno e ciò che gli oggetti di classe fanno nelle interazioni.

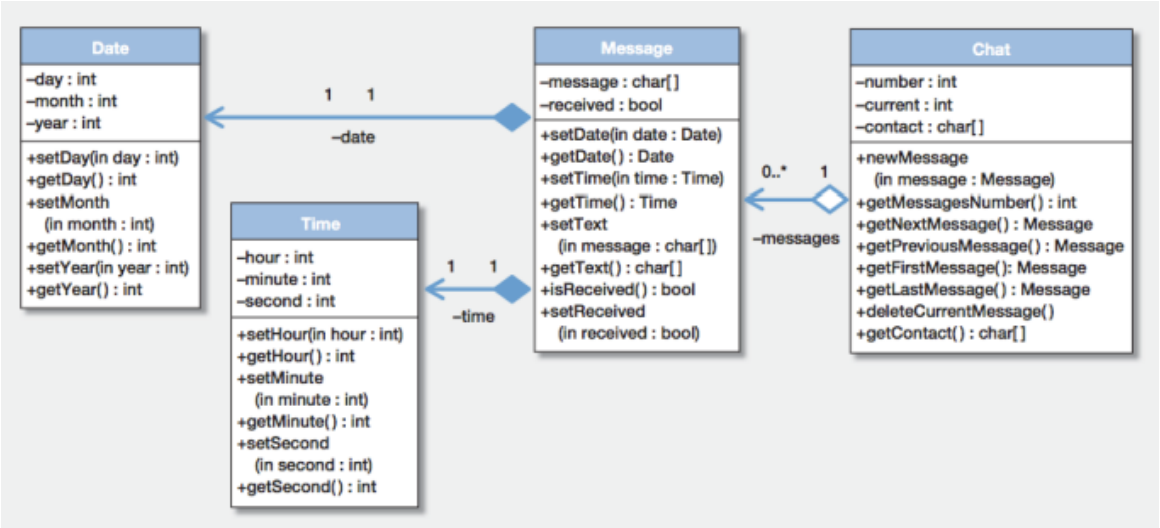


Figura 5.9: Diagramma delle Classi per un sistema di Chat, che mostra le interazioni tra classi come Utente, Chat e Messaggio, e le loro molteplicità.

- Nella fase di **progettazione**, spiegano come il sistema funziona per compiere le interazioni.
- Durante la costruzione di un'**architettura di un sistema**, è possibile utilizzarli per mostrare il funzionamento dei modelli di progettazione ed i meccanismi che il sistema utilizza.

Uno dei principali usi del diagramma è quello di raffinare, in uno o più diagrammi, i requisiti espressi nei diagrammi dei casi d'uso UML, che vengono usati per la descrizione delle funzioni o servizi offerti da un sistema, così come sono percepiti e utilizzati dagli attori che interagiscono col sistema stesso. Lo scopo principale è quello di definire sequenze di eventi che portano a qualche risultato desiderato. L'attenzione si concentra sull'ordine in cui si verificano i messaggi piuttosto che sul loro contenuto. Il diagramma trasmette queste informazioni lungo le dimensioni orizzontali e verticali:

- In **verticale**, dall'alto verso il basso, è evidenziata la sequenza temporale dei messaggi secondo l'ordine in cui si verificano.
- In **orizzontale**, da sinistra a destra, le istanze degli oggetti a cui sono inviati i messaggi.

Componenti Principali dei Diagrammi di Sequenza

- **Lifeline (Linea di Vita)**: Rappresenta la partecipazione di un oggetto o attore all'interazione, mostrata come una linea verticale tratteggiata. In cima alla lifeline c'è un rettangolo (per gli oggetti) o un omino (per gli attori).
- **Attore**: Utente o sistema esterno che avvia o partecipa all'interazione.
- **Messaggio**: Una comunicazione o chiamata di metodo tra lifeline, rappresentata da una freccia orizzontale. Possono essere:
  - **Sincroni**: Freccia piena, indica una chiamata bloccante in attesa di risposta.
  - **Asincroni**: Freccia con punta aperta, indica una chiamata non bloccante.
  - **Risposta**: Freccia tratteggiata, indica il ritorno di un valore o la fine di una chiamata sincrona.
- **Barra di Attivazione (Activation Bar/Execution Occurrence)**: Un rettangolo sottile posizionato verticalmente sulla lifeline, che indica il periodo di tempo durante il quale un oggetto è attivo e sta eseguendo un'operazione o aspettando una risposta.

- **Frammenti Combinati (Combined Fragments)**: Costrutti per mostrare strutture di controllo logico:
  - **alt (alternative)**: Per mostrare blocchi if-else.
  - **opt (optional)**: Per mostrare un blocco if.
  - **loop (loop)**: Per mostrare iterazioni. **par (parallel)**: Per mostrare esecuzioni parallele.

Esempio 1: Flusso di Aggiornamento Dati in MVC Questo esempio illustra un tipico flusso di aggiornamento dei dati in un'architettura Model-View-Controller (MVC), mostrando le interazioni tra l'utente, la View, il Controller e il Model. È un esempio comune per descrivere come le modifiche da parte dell'utente si propagano attraverso i livelli di un'applicazione.

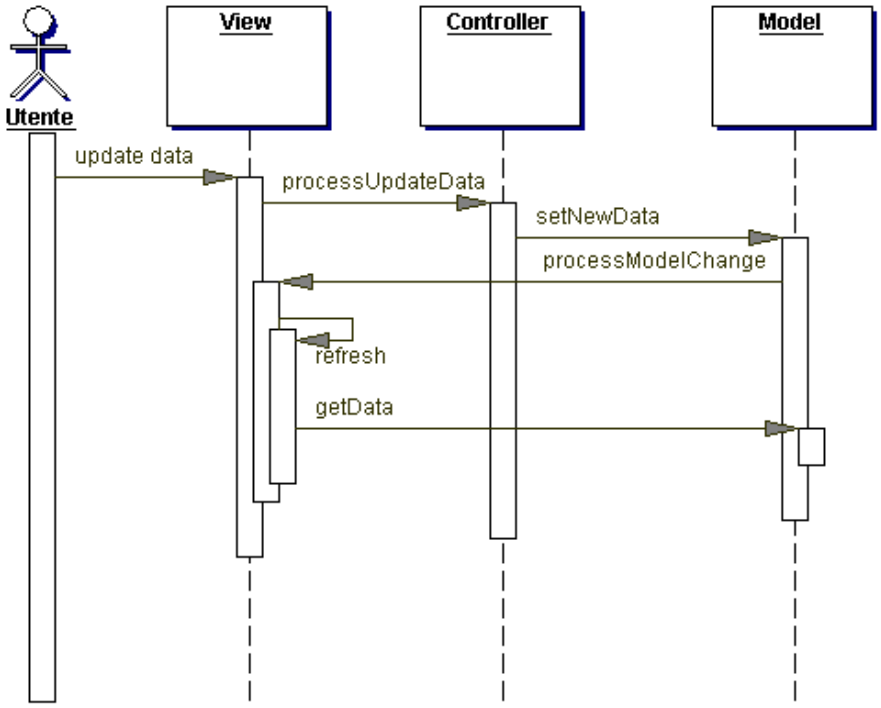


Figura 5.10: Diagramma di Sequenza UML che illustra il flusso di aggiornamento dei dati in un'architettura Model-View-Controller (MVC).

Esempio 2: Flusso di Prestito in un Sistema Biblioteca Questo diagramma mostra un processo più orientato al dominio di business, come la gestione del prestito di un libro in un sistema di biblioteca. Evidenzia l'interazione tra l'utente, un gestore dei prestiti, l'entità libro, e i registri dei prestiti totali, illustrando come i sistemi gestiscono gli stati e le verifiche per completare un'operazione.

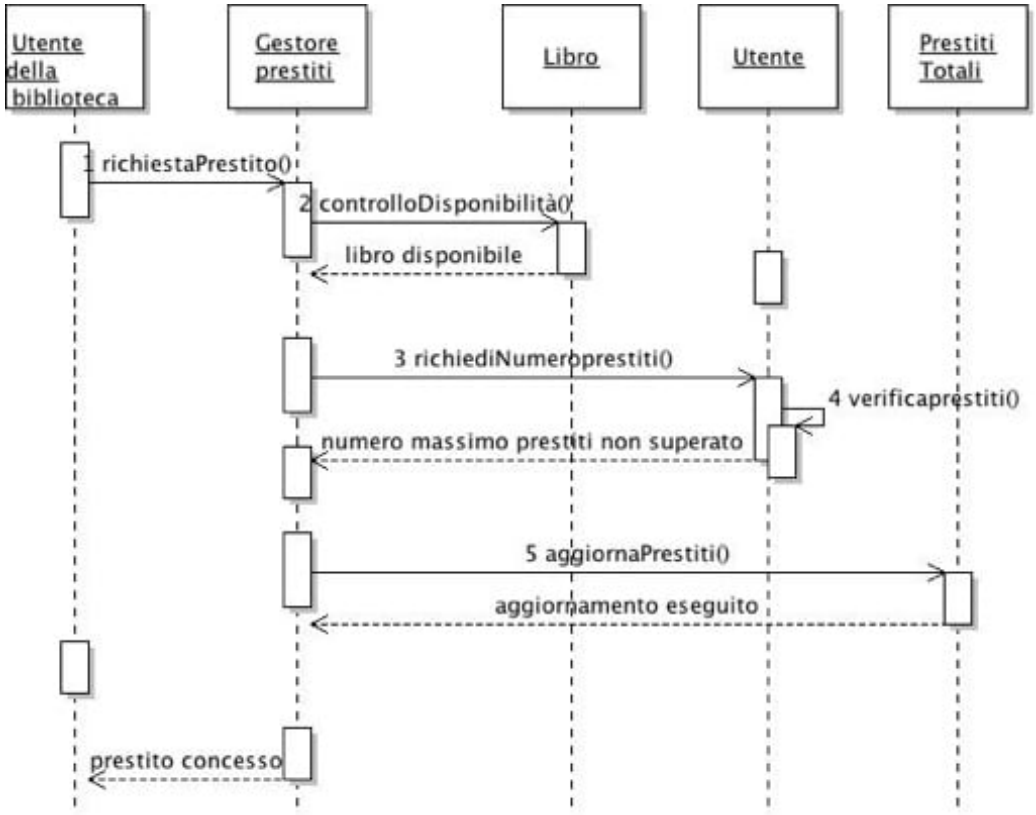


Figura 5.11: Diagramma di Sequenza UML che illustra il flusso di prestito di un libro in un sistema di gestione bibliotecaria.

Capitolo 6

Elettronica

L'**Elettronica** è la branca dell'ingegneria e della fisica che si occupa del controllo del flusso di elettroni, tipicamente attraverso dispositivi semiconduttori, per costruire circuiti e sistemi che elaborano informazioni o controllano energia. È alla base di tutti i dispositivi digitali e di molteplici sistemi analogici moderni.

6.1 Transistore MOS (Metal-Oxide-Semiconductor Field-Effect Transistor)

Il **transistore MOS** è il dispositivo fondamentale della microelettronica moderna e la spina dorsale di microprocessori, memorie e altri circuiti integrati. È un transistore a effetto di campo (FET), dove la corrente tra due terminali (Source e Drain) è controllata da un campo elettrico generato da una tensione applicata a un terzo terminale (Gate).

6.1.1 Struttura del Transistore MOS (nMOS)

Per comprendere il funzionamento, si consideri la realizzazione più comune: l'nMOS (n-channel Metal-Oxide-Semiconductor).

- **Substrato (Bulk/Body)**: Generalmente di tipo P per un nMOS. È il materiale semiconduttore di base su cui viene costruito il dispositivo.
- **Source (S) e Drain (D)**: Due regioni altamente drogate (N+ per nMOS) impiantate nel substrato P. Sono simmetriche e collegate ai terminali esterni. Il Source è tipicamente la sorgente di portatori di carica, il Drain è dove fluiscono.
- **Canale**: La regione del substrato (P) tra Source e Drain. È qui che si formerà il canale di conduzione per il flusso di corrente.
- **Ossido di Gate (SiO_2)**: Uno strato isolante molto sottile (diossido di silicio) deposto sopra il canale. Agisce come dielettrico, isolando elettricamente il Gate dal canale.
- **Gate (G)**: Uno strato conduttivo (metallo o polisilicio) deposto sopra l'ossido di gate. È il terminale di controllo attraverso il quale si applica la tensione per creare il campo elettrico.
- **Terminali**: Gate (G), Source (S), Drain (D), Bulk/Substrate (B). Spesso il Bulk è collegato al Source o a una tensione fissa (es. massa per nMOS, V_{DD} per pMOS).

6.1.2 Principio di Funzionamento (nMOS)

Il funzionamento del transistore MOS dipende dalla tensione applicata tra Gate e Source (V_{GS}) e tra Drain e Source (V_{DS}). Si basa sulla modulazione della conduttività del canale tramite un campo elettrico.

- **Interdizione (Cut-off Region)**:
  - **Condizione**: $V_{GS} < V_{TH}$ (Tensione di Soglia).
  - **Descrizione**: Non c'è un campo elettrico sufficiente per attrarre elettroni al canale. Non si forma un canale di conduzione tra Source e Drain. La corrente I_{DS} è praticamente nulla (solo una piccola corrente di leakage). Il transistore agisce come un interruttore aperto.
- **Conduzione (quando $V_{GS} > V_{TH}$)**:
  - Quando una tensione positiva V_{GS} sufficiente (maggiore di V_{TH}) viene applicata al Gate, il campo elettrico attrae gli elettroni (portatori minoritari nel substrato P) verso la superficie del semiconduttore, sotto l'ossido.
  - Si forma uno strato sottile di elettroni, creando un **canale di conduzione** a bassa resistenza tra Source e Drain.
  - Se si applica una tensione $V_{DS} > 0$, una corrente I_{DS} fluirà attraverso questo canale.

6.1.3 Regimi di Funzionamento (per nMOS con $V_{GS} > V_{TH}$)

Una volta che il transistore è in conduzione, il suo comportamento (e la corrente I_{DS}) dipende dalla relazione tra V_{GS} e V_{DS}.

- **Regione di Triodo (o Lineare/Ohmica)**:
  - **Condizioni**: $V_{GS} > V_{TH}$ e $V_{DS} < (V_{GS} - V_{TH})$.
  - **Descrizione**: Il canale è completamente formato e la sua profondità è relativamente uniforme. Il transistore si comporta come una resistenza controllata in tensione, e la corrente I_{DS} è approssimativamente lineare rispetto a V_{DS}.
  - **Applicazione**: Usata come interruttore chiuso (ON) in circuiti digitali (con bassa resistenza) o come resistenza variabile in circuiti analogici.
- **Regione di Saturazione**:
  - **Condizioni**: $V_{GS} > V_{TH}$ e $V_{DS} \geq (V_{GS} - V_{TH})$.
  - **Descrizione**: Aumentando V_{DS}, la tensione canale-gate verso il lato del Drain si riduce. Quando V_{DS} raggiunge $V_{GS} - V_{TH}$, il canale si "pizzica" (pinch-off) vicino al Drain. Oltre questo punto, ulteriori aumenti di V_{DS} non aumentano significativamente la corrente I_{DS}, che diventa quasi costante.
  - **Applicazione**: Questa è la regione di funzionamento preferita per gli amplificatori (circuiti analogici) in quanto si comporta come una sorgente di corrente controllata in tensione, e per gli stati ON (logico '1') in circuiti digitali per un'elevata impedenza di uscita.

6.1.4 Vantaggi del MOS rispetto al Transistore Bipolare (BJT)

Il transistore MOS ha soppiantato in gran parte il BJT nella microelettronica digitale e in molte applicazioni analogiche grazie a diversi vantaggi chiave:

- **Alta Impedenza di Ingresso**: Il gate del MOS è isolato dall'ossido, il che lo rende quasi idealmente un circuito aperto con una corrente di gate praticamente nulla ($I_G \approx 0$). Questo riduce il carico sui circuiti precedenti e semplifica il design degli stadi di ingresso. Il BJT, invece, è controllato in corrente (corrente di base) e presenta un'impedenza di ingresso inferiore.
- **Minore Dissipazione di Potenza Statica (in CMOS)**: Nelle configurazioni Complementary MOS (CMOS), quando il circuito è in stato stabile (non commuta), uno dei transistori è sempre spento, eliminando un percorso di corrente diretto tra alimentazione e massa. Questo porta a una dissipazione di potenza statica estremamente bassa, fondamentale per dispositivi a batteria e circuiti ad alta integrazione. I BJT, anche quando "spenti", possono avere correnti di leakage maggiori e configurazioni logiche basate su BJT tendono a dissipare più potenza.
- **Scalabilità e Densità di Integrazione**: I MOS possono essere miniaturizzati molto più facilmente rispetto ai BJT, permettendo la realizzazione di circuiti integrati con miliardi di transistori su un singolo chip. La loro struttura planare si presta bene alla fabbricazione su larga scala.
- **Costo di Fabbricazione**: Il processo di fabbricazione MOS è generalmente più semplice e meno costoso rispetto a quello BJT, che richiede più passaggi di drogaggio e diffusione.
- **Immunità al Rumore (in CMOS)**: I circuiti CMOS hanno buoni margini di rumore, rendendoli robusti contro le fluttuazioni di tensione indesiderate.

Nonostante ciò, i BJT mantengono vantaggi in alcune applicazioni specifiche (es. alta frequenza, alta potenza) grazie a una maggiore transconduttanza e velocità in particolari contesti.

6.2 Circuiti CMOS (Complementary MOS)

La tecnologia **CMOS** è il design più diffuso per la realizzazione di circuiti integrati digitali, caratterizzata dall'uso complementare di transistori nMOS e pMOS.

6.2.1 Invertitore CMOS

L'**invertitore CMOS** è la porta logica fondamentale in tecnologia CMOS, che realizza la funzione NOT (negazione logica). Produce un'uscita opposta all'ingresso.

- **Realizzazione**: È composto da due transistori MOS collegati in serie tra V_{DD} e GND, con i gate collegati insieme a formare l'ingresso (IN) e i drain collegati a formare l'uscita (OUT).
  - Un **pMOS** (pull-up network) è collegato tra V_{DD} e OUT. Il pMOS conduce quando il suo gate è basso.
  - Un **nMOS** (pull-down network) è collegato tra OUT e GND. L'nMOS conduce quando il suo gate è alto.
- **Funzionamento**:
  - **Ingresso Alto (Logico '1', $V_{IN} = V_{DD}$)**: L'nMOS è ON (conduce), creando un percorso a bassa resistenza tra OUT e GND. Il pMOS è OFF (interdetto). L'uscita V_{OUT} è tirata a GND (Logico '0').
  - **Ingresso Basso (Logico '0', $V_{IN} = GND$)**: Il pMOS è ON (conduce), creando un percorso a bassa resistenza tra OUT e V_{DD}. L'nMOS è OFF (interdetto). L'uscita V_{OUT} è tirata a V_{DD} (Logico '1').

6.2.2 Vantaggi dell'Invertitore CMOS rispetto agli Invertitori con Carico Resistivo

Gli invertitori CMOS offrono vantaggi significativi rispetto alle implementazioni più vecchie che usavano un transistore (BJT o MOS) con una resistenza di carico:

- **Minore Dissipazione di Potenza Statica (Vantaggio Primario)**: Nello stato stabile (ingresso alto o basso), uno dei due transistori (nMOS o pMOS) è sempre spento. Non c'è un percorso di corrente diretto da V_{DD} a GND. La corrente statica è limitata a correnti di leakage quasi nulle, risultando in un consumo di potenza estremamente basso a riposo. Gli invertitori con carico resistivo, invece, dissipano continuamente potenza quando l'uscita è nello stato "basso", poiché la corrente fluisce attraverso il resistore di carico e il transistore acceso.
- **Migliori Margini di Rumore**: Le curve di trasferimento tensione-tensione degli invertitori CMOS sono quasi ideali, con una transizione molto ripida tra i due stati logici. Questo fornisce ampi margini di rumore, rendendo i circuiti più robusti alle fluttuazioni di tensione indesiderate.
- **Prestazioni Simmetriche (Potenziale)**: Con un corretto dimensionamento dei transistori (rapporto W/L), i tempi di salita (charge time) e di discesa (discharge time) dell'uscita possono essere resi quasi simmetrici. Negli invertitori con resistore, il tempo di salita (carica del condensatore di carico attraverso il resistore) è tipicamente più lento del tempo di discesa.
- **Densità di Integrazione**: I transistori MOS occupano meno area rispetto ai resistori su chip, consentendo una maggiore densità di circuiti.
- **Ampio Range di Tensione Operativa**: I circuiti CMOS possono operare su un'ampia gamma di tensioni di alimentazione mantenendo buone prestazioni.

6.2.3 Applicazioni Notevoli dell'Invertitore CMOS

L'invertitore CMOS è un blocco costruttivo fondamentale per una vasta gamma di applicazioni:

- **Porta Logica Fondamentale**: È il componente di base da cui vengono costruite tutte le altre porte logiche digitali (NAND, NOR, XOR, latch, flip-flop) in tecnologia CMOS.
- **Buffer e Driver**: Utilizzando cascate di invertitori, si possono creare buffer (per aumentare la capacità di pilotaggio di un segnale) o driver per linee lunghe o carichi capacitivi elevati (es. clock network nei microprocessori).
- **Celle di Memoria**: Due invertitori CMOS collegati in back-to-back formano un latch, che è la cella di memoria fondamentale nelle SRAM (Static RAM), capace di memorizzare un bit.
- **Oscillatori ad Anello (Ring Oscillators)**: Una catena di un numero dispari di invertitori, con l'uscita dell'ultimo collegata all'ingresso del primo, crea un oscillatore che produce un'onda quadra. Usati per testare le prestazioni di velocità e per generare segnali di clock.
- **Livelli di Tensione**: Possono essere usati per convertire livelli di tensione logici tra diversi standard o blocchi di circuiti.

Capitolo 7

Sistemi Numerici

I **sistemi numerici** sono metodi per rappresentare i numeri utilizzando simboli specifici e regole ben definite. In informatica, la rappresentazione binaria è fondamentale, ma è cruciale anche capire come i numeri negativi vengono gestiti, in particolare tramite il complemento a due.

7.1 Rappresentazione per Numeri Interi

I numeri interi possono essere rappresentati in diversi modi all'interno di un sistema digitale. Le rappresentazioni più comuni per i numeri con segno includono segno e modulo, complemento a uno, e complemento a due. Il complemento a due è la rappresentazione più utilizzata nei sistemi digitali moderni per la sua efficienza nelle operazioni aritmetiche.

7.1.1 Complemento a Due

La rappresentazione in **complemento a due** è il metodo più diffuso per rappresentare numeri interi con segno nei sistemi digitali. Offre il vantaggio di semplificare le operazioni di somma e sottrazione, poiché la sottrazione può essere implementata come una somma con il complemento a due del sottraendo, eliminando la necessità di circuiti dedicati alla sottrazione.

Principio di Funzionamento

- Un numero positivo in complemento a due è rappresentato esattamente come nella notazione binaria pura (senza segno), con il bit più significativo (MSB, most significant byte) pari a 0.
- Un numero negativo in complemento a due è ottenuto complementando (invertendo) tutti i bit del suo valore assoluto (passando da 0 a 1 e viceversa) e poi sommando 1 al risultato. Il bit più significativo (MSB) sarà sempre 1 per i numeri negativi.
- Il bit più significativo (MSB) indica il segno: 0 per i positivi, 1 per i negativi.
- Il range di valori rappresentabile con N bit in complemento a due va da -2^{N-1} a $2^{N-1}-1$. Ad esempio, con 8 bit, si possono rappresentare numeri da -128 a 127.

Derivazione della Rappresentazione in Complemento a Due

Per convertire un numero decimale negativo in complemento a due (con N bit):

1. Prendi il valore assoluto del numero decimale (positivo).
2. Converti il valore assoluto in binario su N bit.

3. Inverti tutti i bit (complemento a uno, 0 diventa 1, 1 diventa 0).
4. Somma 1 al risultato binario.

Per convertire un numero binario in complemento a due a decimale:

- Se il MSB è 0: il numero è positivo. Convertilo come un normale binario senza segno.
- Se il MSB è 1: il numero è negativo.
  1. Inverti tutti i bit del numero binario.
  2. Somma 1 al risultato binario.
  3. Converti questo risultato binario in decimale e anteponi il segno meno.

Esempio: Rappresentazione in Binario a 16 bit del numero decimale -15

Per rappresentare il numero decimale -15 in complemento a due su 16 bit:

1. **Valore assoluto**: $|-15| = 15$.
2. **15 in binario su 16 bit**: $0000\,0000\,0000\,1111_2$.
3. **Inverti tutti i bit (complemento a uno)**: $1111\,1111\,1111\,0000_2$.
4. **Somma 1**: $1111\,1111\,1111\,0000_2 + 1_2 = 1111\,1111\,1111\,0001_2$.

Quindi, la rappresentazione in complemento a due di -15 su 16 bit è $1111\,1111\,1111\,0001_2$.

7.2 Operazioni Aritmetiche con Complemento a Due

Il vantaggio principale del complemento a due è che le operazioni di addizione e sottrazione possono essere eseguite utilizzando lo stesso circuito per l'addizione.

7.2.1 Addizione

L'addizione di due numeri (positivi o negativi) in complemento a due viene eseguita come una normale addizione binaria. Qualsiasi bit di riporto che esce dal bit più significativo (MSB) viene semplicemente ignorato.

- **Esempio**: $5 + (-2)$ su 4 bit ($N = 4$, range da -8 a 7)
  - $5_{10} = 0101_2$
  - -2_{10}:
    - $2_{10} = 0010_2$
    - Complemento a uno: 1101_2
    - Somma 1: $1101_2 + 1_2 = 1110_2$ (che è -2_{10} in complemento a due)
  - **Somma**:

```
  0101  (5)
+ 1110  (-2)
-----
 10011
```

    Il bit di riporto (il primo 1 a sinistra) viene ignorato. Il risultato è $0011_2 = 3_{10}$, che è corretto.

7.2.2 Sottrazione

La sottrazione $A - B$ è implementata come la somma $A + (-B)$. Per calcolare $-B$, si determina il complemento a due di B.

- **Esempio**: $5 - 2$ su 4 bit ($N = 4$, range da -8 a 7)
  - $5_{10} = 0101_2$
  - $-2_{10} = 1110_2$ (già calcolato sopra)
  - **Sottrazione come somma**: $5 + (-2)$

```
  0101  (5)
+ 1110  (-2)
-----
 10011
```

Ignorando il riporto, il risultato è $0011_2 = 3_{10}$, che è corretto.

7.2.3 Overflow

L'**overflow** si verifica quando il risultato di un'operazione aritmetica supera il range di valori rappresentabili con il numero di bit a disposizione.

- **Rilevamento**: Si verifica un overflow se:
  - Si sommano due numeri positivi e il risultato è negativo.
  - Si sommano due numeri negativi e il risultato è positivo.
  - Un modo più formale per rilevarlo è controllare se il riporto nel bit del segno (MSB) è diverso dal riporto fuori dal bit del segno.

Parte II

Approfondimenti per la Sezione A

Introduzione alla Parte II

La Parte I raccoglie i fondamenti richiesti per l'Esame di Stato di **Ingegnere Junior** (Sezione B). Questa seconda parte estende gli appunti al livello della **Sezione A**, cioè dell'Esame di Stato di **Ingegnere** per chi si laurea nel corso magistrale, con un'impostazione orientata a **Ingegneria Informatica**.

Perché una seconda parte

Le prove della Sezione A sono diverse da quelle della Sezione B:

- i temi sono **aperti**: si sviluppa in modo discorsivo un argomento, con definizioni, schemi, esempi di codice e motivazioni delle scelte;
- alcune sessioni prevedono una **prova pratica di progettazione** (per esempio, nel 2026, un sistema di prenotazione per un cinema multisala con database, backend, API, diagrammi UML e un modulo di Machine Learning, da svolgere punto per punto);
- gli argomenti sono più avanzati: sistemi distribuiti, intelligenza artificiale, progettazione di sistemi, sicurezza, architettura del calcolatore.

I contenuti sono stati scelti analizzando le **tracce delle prove passate** (dal 2016 al 2026) raccolte nella cartella `TraccePassate` del repository.

Come sono organizzati i capitoli

Ogni capitolo della Parte II ha la stessa struttura:

1. la **teoria** necessaria, con attenzione al *perché* delle scelte (non solo al *che cosa*);
2. **esempi svolti**, in particolare codice, query SQL e calcoli: quelli riportati sono stati **eseguiti e verificati** con appositi strumenti di prova (tranne quelli indicati come pseudocodice), e dove un risultato è indicato è quello effettivamente ottenuto;
3. le **scalette di risposta** per i temi tipici;
4. una sezione finale **Tracce Uscite** che collega ogni traccia reale alle sezioni in cui trovare la risposta.

Che cosa copre e che cosa no

Per un candidato di Ingegneria Informatica sono più rilevanti i temi di Informatica; nelle tracce analizzate ogni prova ne offre da **uno a tre**. La Parte II si concentra su quelli.

Area delle tracce	**Copertura**	**Dove**
Informatica	**Ampia**	Sistemi distribuiti, IA e ML, progettazione di sistemi e API, programmazione e algoritmi, sistemi operativi e reti, sicurezza, basi di dati
Elettronica (parte digitale)	Parziale	Architettura del calcolatore: datapath RISC, pipeline, cache, microcontrollori, potenza
Elettronica (parte analogica)	**Non trattata**	Diodo, amplificatori, filtri, convertitori, ADC: solo cenni nel capitolo *Elettronica* (MOS, CMOS)
Automazione, Telecomunicazioni, Biomedica, Gestionale	**Non trattate**	Le tracce di Automazione e Telecomunicazioni sono frequenti ma richiedono capitoli dedicati; solo l'identificazione dei sistemi e il machine learning sono trattati (capitolo *IA e ML*)

Le aree non trattate sono una **scelta di perimetro**, non una dimenticanza: chi sceglie di prepararle deve integrare con altre fonti.

Avvertenze

- L'organizzazione, i programmi e le prove **cambiano da ateneo a ateneo e da sessione a sessione**: consultare sempre il bando dell'università in cui si sostiene l'esame.
- Le parti teoriche, le norme e i valori di riferimento (per esempio standard di sicurezza, regolamenti, valori tipici di prestazioni) sono stati redatti a partire dalla conoscenza dei redattori e **non hanno ricevuto la revisione di un docente**: vanno **verificati su fonti ufficiali** prima di fare affidamento su di essi.
- Gli esempi di codice sono didattici: **non sono pronti per la produzione**.
- Si invita a segnalare errori o imprecisioni con una *issue* o una *pull request* sul repository.

Capitolo 8

Sistemi Distribuiti, Cloud e Big Data

Un **sistema distribuito** è un insieme di computer autonomi, collegati da una rete, che appare all'utente come un unico sistema coerente. Questo capitolo è il più ricorrente nelle tracce della Sezione A (Settore Informazione, tema di Informatica) e va studiato per *scrivere un tema*: ogni sezione termina con una scaletta di risposta.

8.1 Caratteristiche dei Sistemi Distribuiti

Le proprietà che un sistema distribuito deve garantire, con le tecniche che le realizzano:

Caratteristica	**Significato**	**Tecniche**
Scalabilità	Mantenere le prestazioni al crescere di utenti e dati	Scale-out (più nodi), load balancing, partizionamento (sharding), caching
Tolleranza ai guasti	Continuare a funzionare se un nodo o un collegamento si guasta	Replicazione, ridondanza, failover, timeout e retry, checkpoint
Disponibilità	Frazione di tempo in cui il servizio risponde	Replicazione, cluster attivo–attivo, health check
Trasparenza	Nascondere la distribuzione (accesso, locazione, replicazione, guasto)	Naming service, middleware, RPC
Apertura	Interfacce standard e documentate	API REST, protocolli standard
Consistenza	Le repliche mostrano gli stessi dati	Protocolli di consenso, transazioni distribuite

8.1.1 Scalabilità: verticale e orizzontale

- **Scale-up (verticale)**: macchina più potente (CPU, RAM). Semplice, ma ha un limite fisico ed è un singolo punto di guasto.
- **Scale-out (orizzontale)**: più macchine in parallelo dietro un **load balancer**. Richiede che il servizio sia (il più possibile) **stateless**. È l'approccio tipico del cloud.
- **Sharding**: i dati sono divisi in partizioni su nodi diversi (per intervallo o per hash della chiave).
- **Replicazione**: gli stessi dati sono copiati su più nodi, per disponibilità e per distribuire le letture.

8.1.2 Prestazioni: latenza e throughput

- **Latenza**: tempo di risposta di una singola richiesta.
- **Throughput**: richieste servite per unità di tempo.
- Le due misure sono in tensione: aggiungere replicazione sincrona o controlli di consistenza *aumenta* la latenza; batching e pipelining aumentano il throughput ma anche la latenza media.
- La rete non è affidabile: i messaggi possono essere persi, duplicati, ritardati o arrivare fuori ordine, e non esiste un orologio globale. Per questo servono timeout, ritrasmissioni e operazioni **idempotenti**.

8.1.3 Tolleranza ai guasti

- **Guasti**: crash di un nodo, partizione di rete, messaggi persi, guasti bizantini (nodi che si comportano in modo arbitrario o malevolo).
- **Rilevamento**: heartbeat e timeout.
- **Recupero**: failover su una replica, rielezione del leader, riesecuzione da log o checkpoint.
- **Pattern**: retry con *exponential backoff*, *circuit breaker*, *bulkhead*.

8.2 Il Teorema CAP

Il **teorema CAP** (Brewer) afferma che un sistema distribuito che mantiene dati replicati può garantire contemporaneamente *al più due* delle tre proprietà:

- **C – Consistency**: ogni lettura restituisce la scrittura più recente (vista unica e aggiornata dei dati, *linearizzabilità*).
- **A – Availability**: ogni richiesta ricevuta da un nodo non guasto riceve una risposta.
- **P – Partition tolerance**: il sistema continua a funzionare anche se la rete si divide e alcuni nodi non riescono a comunicare.

In una rete reale le partizioni *possono sempre verificarsi*, quindi P non è opzionale. La scelta vera è, **durante una partizione**, tra C e A:

Scelta	Comportamento in partizione	Esempi
CP	Si rifiutano (o si bloccano) le operazioni sui nodi che non vedono il quorum: i dati restano coerenti, ma il servizio è indisponibile per una parte degli utenti	MongoDB (default), HBase, Redis cluster, ZooKeeper, database relazionali replicati in modo sincrono
AP	Tutti i nodi rispondono, ma possono restituire dati vecchi; le divergenze vengono riconciliate dopo	Cassandra, DynamoDB, CouchDB, Riak
CA	Possibile solo senza partizioni, cioè in un sistema non distribuito (database su un singolo nodo)	RDBMS su un nodo

Estensione PACELC. Anche *senza* partizione c'è un compromesso tra **latenza** e **consistenza**: replicare in modo sincrono è coerente ma lento. PACELC: *se Partition, scegli A o C; Else, scegli Latency o Consistency.*

8.2.1 Modelli di consistenza

- **Consistenza forte**: dopo una scrittura, ogni lettura vede il nuovo valore. Costosa in latenza.
- **Consistenza eventuale**: in assenza di nuove scritture, le repliche convergono allo stesso valore *col tempo*. Alta disponibilità e bassa latenza.
- **Altri**: consistenza causale, read-your-writes, monotonic reads.
- **Quorum**: con N repliche, scrivendo su W e leggendo da R nodi, se $R + W > N$ ogni lettura incontra almeno una replica aggiornata.

8.2.2 ACID e BASE

	ACID (database relazionali)	**BASE (molti NoSQL)**
Significato	*Atomicity, Consistency, Isolation, Durability*	*Basically Available, Soft state, Eventual consistency*
Priorità	Correttezza dei dati	Disponibilità e scalabilità
Transazioni	Sì, su più righe e tabelle	Spesso limitate a un singolo documento o chiave

8.3 Database SQL e NoSQL nei Sistemi Distribuiti

8.3.1 Database relazionali (SQL)

Dati strutturati in tabelle con schema fisso, join, vincoli di integrità e transazioni ACID. Scalano bene *in verticale*; la scalabilità orizzontale è possibile (replicazione master–slave per le letture, sharding) ma complessa, perché join e transazioni distribuite sono costosi (per esempio con il protocollo **two-phase commit**, che blocca le risorse e riduce la disponibilità).

8.3.2 Database NoSQL

Modello	**Descrizione**	**Uso tipico (esempio)**
Chiave–valore	Coppie chiave → valore opaco; accesso molto veloce	Cache, sessioni, carrelli (Redis, DynamoDB)
Documenti	Documenti JSON/BSON con schema flessibile	Cataloghi prodotti, profili utente (MongoDB, CouchDB)
Colonnari (wide-column)	Dati organizzati per famiglie di colonne, ottimi per scritture massive e scansioni	Log, serie temporali, IoT (Cassandra, HBase)
Grafi	Nodi e archi con proprietà; interrogazioni sulle relazioni	Social network, raccomandazioni, frodi (Neo4j)

8.3.3 Confronto e scelta

	SQL	**NoSQL**
Schema	Rigido, definito in anticipo	Flessibile (schema-on-read)
Scalabilità	Principalmente verticale	Orizzontale nativa (sharding, replicazione)
Consistenza	Forte (ACID)	Spesso eventuale (BASE), regolabile
Interrogazioni	SQL standard, join, aggregazioni complesse	API specifiche, join limitati o assenti
CAP (tendenza)	CA/CP	Prevalentemente AP (alcuni CP)

Quando preferire SQL. Applicazioni in cui la correttezza è critica e i dati sono fortemente relazionali: sistemi bancari e contabili, ordini, magazzino (un doppio addebito o una vendita doppia dello stesso articolo non sono accettabili).

Quando preferire NoSQL. Grandi volumi di dati eterogenei, picchi di traffico e bassa latenza, a costo di consistenza eventuale: feed di un social network, catalogo di un e-commerce, telemetria IoT, log. Un like che compare dopo qualche secondo è accettabile.

Persistenza poliglotta. Spesso un'azienda usa entrambi: SQL per ordini e pagamenti, NoSQL (documenti o chiave–valore) per catalogo, sessioni e cache.

8.3.4 Scaletta per il tema "consistenza SQL/NoSQL e CAP"

1. Definire il problema: dati replicati su più nodi, rete inaffidabile.
2. Enunciare CAP e spiegare perché P è obbligatoria; scegliere tra C e A.
3. Descrivere ACID e BASE; consistenza forte vs eventuale; quorum.
4. Confrontare SQL (CA/CP) e NoSQL (AP), con una tabella.
5. Esempi concreti: banca → SQL/CP; social feed e carrello → NoSQL/AP.
6. Chiudere con persistenza poliglotta e con PACELC.

8.4 Big Data: Elaborazione Batch e Streaming

I **Big Data** si caratterizzano per le "V": **Volume**, **Velocità**, **Varietà**, a cui si aggiungono Veridicità e Valore. Sono dati troppo grandi o troppo veloci per un singolo database tradizionale.

8.4.1 Archiviazione

- **File system distribuito** (HDFS): file divisi in blocchi replicati su più nodi.
- **Data lake**: dati grezzi di qualsiasi formato (schema-on-read), a basso costo.
- **Data warehouse**: dati puliti e strutturati per analisi (OLAP), schema-on-write.
- **OLTP vs OLAP**: OLTP gestisce molte piccole transazioni (ordini); OLAP gestisce poche interrogazioni analitiche pesanti su grandi dati storici.

8.4.2 Elaborazione batch e streaming

	Batch	**Streaming**
Dati	Insiemi finiti e completi	Flussi continui di eventi
Latenza	Minuti–ore	Millisecondi–secondi
Modello	MapReduce, Spark	Kafka, Flink, Spark Streaming
Pro	Alto throughput, risultati accurati e riproducibili	Reazione in tempo reale
Contro	Dati non freschi	Più complesso (ordine degli eventi, finestre, stato)
Usi	Report, addestramento di modelli, ETL notturni	Rilevamento frodi, monitoraggio, raccomandazioni live

MapReduce. Modello di calcolo parallelo: la fase **Map** trasforma i dati in coppie chiave–valore elaborate in parallelo; la fase **Reduce** aggrega i valori con la stessa chiave. Il framework gestisce distribuzione, parallelismo e guasti.

Architetture Lambda e Kappa. *Lambda*: un livello batch (accurato) più un livello speed (veloce) uniti in un livello di servizio. *Kappa*: un solo percorso streaming, con possibilità di rielaborare lo storico.

8.5 Architetture di Applicazioni Distribuite

8.5.1 Architettura a strati (layered, N-tier)

Presentazione → logica applicativa → accesso ai dati → database, con ogni strato che dipende solo da quello sottostante.

- **Pro**: semplice, separazione delle responsabilità, facile da capire e testare.
- **Contro**: spesso monolitica; per scalare si replica l'intero blocco; un cambiamento richiede di rilasciare tutto.

8.5.2 Service-Oriented Architecture (SOA)

Servizi riutilizzabili, a grana grossa, con interfacce standard (SOAP/WSDL) che comunicano tramite un **Enterprise Service Bus**. Pensata per integrare sistemi aziendali eterogenei. Il bus centrale può diventare un collo di bottiglia.

8.5.3 Microservizi

L'applicazione è divisa in piccoli servizi indipendenti, ciascuno con la propria responsabilità di business e *spesso con il proprio database*, che comunicano tramite API leggere (REST/gRPC) o messaggi.

- **Pro**: scalabilità selettiva (si replica solo il servizio sotto carico), isolamento dei guasti, rilasci indipendenti, libertà tecnologica, team autonomi.
- **Contro**: complessità operativa, latenza di rete tra servizi, consistenza dei dati distribuita (niente transazioni ACID globali: si usano **saga** ed eventual consistency), debug e osservabilità più difficili.

- **Componenti tipici**: API gateway, service discovery, load balancer, message broker, tracing distribuito, circuit breaker.

8.5.4 Comunicazione sincrona e asincrona

- **Sincrona** (richiesta/risposta: REST, gRPC): semplice, ma accoppia i servizi nel tempo; se uno è lento, rallenta l'altro.
- **Asincrona** (code e topic: RabbitMQ, Kafka): disaccoppia produttore e consumatore, assorbe i picchi, migliora la resilienza; la consistenza diventa eventuale.
- **Event-driven**: i servizi reagiscono a eventi pubblicati su un broker (publish/subscribe).

8.5.5 Confronto sintetico

	Layered/monolite	**SOA**	**Microservizi**
Scalabilità	Replica dell'intero blocco	Per servizio, ma bus centrale	Fine, per servizio
Tolleranza ai guasti	Un guasto può fermare tutto	Dipende dal bus	Isolamento dei guasti
Complessità	Bassa	Media	Alta (operativa)
Rilasci	Unico e grande	Per servizio	Indipendenti e frequenti

8.6 Cloud Computing e Virtualizzazione

8.6.1 Virtualizzazione

L'**hypervisor** permette di eseguire più macchine virtuali (VM) isolate sullo stesso hardware fisico.

- **Tipo 1 (bare-metal)**: gira direttamente sull'hardware (ESXi, KVM, Xen).
- **Tipo 2 (hosted)**: gira sopra un sistema operativo (VirtualBox, VMware Workstation).
- **Container** (Docker): condividono il kernel dell'host e isolano i processi (namespace e cgroup). Sono più leggeri e veloci da avviare delle VM, ma l'isolamento è più debole.
- **Orchestrazione** (Kubernetes): distribuisce i container sui nodi, li riavvia in caso di guasto, scala in base al carico (autoscaling), gestisce rete e aggiornamenti progressivi (rolling update).

8.6.2 Modelli di servizio e di distribuzione

- **IaaS** (infrastruttura: VM, storage, rete), **PaaS** (piattaforma per eseguire applicazioni), **SaaS** (software finito), e **FaaS**/serverless (funzioni a evento).
- **Pubblico**, **privato**, **ibrido**, multi-cloud.

8.6.3 Vantaggi, sfide e best practice

- **Vantaggi**: elasticità (risorse a richiesta), pagamento a consumo, alta disponibilità tramite regioni e zone, rapidità di rilascio.
- **Sfide**: vendor lock-in, costi non controllati, sicurezza e conformità (GDPR, localizzazione dei dati), latenza, dipendenza dalla rete.

- **Best practice**: progettare per il guasto (*design for failure*), servizi stateless, infrastruttura come codice, replicazione su più zone, backup e disaster recovery, monitoraggio e osservabilità, CI/CD.

8.7 Meccanismi Fondamentali

8.7.1 Consenso e elezione del leader

Per mettere d'accordo i nodi su un valore (per esempio chi è il leader o quale transazione è confermata) si usano protocolli di consenso come **Paxos** e **Raft**. Raft elegge un leader che replica un log di comandi; un comando è confermato quando la maggioranza dei nodi (quorum) lo ha scritto. Con N nodi tollera $\lfloor (N-1)/2 \rfloor$ guasti.

8.7.2 Transazioni distribuite

- **Two-phase commit (2PC)**: un coordinatore chiede a tutti i partecipanti di preparare la transazione (fase 1) e poi di confermare o annullare (fase 2). Garantisce atomicità, ma è bloccante se il coordinatore si guasta.
- **Saga**: sequenza di transazioni locali, ciascuna con un'azione compensativa in caso di errore. Tipica dei microservizi.

8.7.3 Comunicazione tra processi

- **RPC** (chiamata di procedura remota): nasconde la rete dietro una chiamata di funzione. Semantiche: at-least-once, at-most-once, exactly-once (quest'ultima difficile).
- **REST**: risorse identificate da URL, verbi HTTP (GET, POST, PUT, DELETE), stateless, rappresentazioni in JSON.
- **Message queue**: consegna asincrona, bufferizzata e affidabile.

8.7.4 Orologi e ordinamento

Non esiste un tempo globale: si usano **orologi logici** di Lamport e **vector clock** per stabilire l'ordine causale degli eventi.

Tracce Uscite

Prima sessione 2024 – Informatica #1

Traccia: consistenza del dato in un sistema distribuito; soluzioni tipiche di database SQL e NoSQL con esempi e casi in cui un'azienda preferirebbe l'uno o l'altro; teorema CAP e il suo effetto sulla scelta.

Dove trovare la risposta: sezioni *Il Teorema CAP* e *Database SQL e NoSQL*; scaletta a fine sezione.

Prima sessione 2024 – Informatica #2

Traccia: principali caratteristiche di un sistema distribuito e tecniche che le supportano (scalabilità, tolleranza ai guasti...), con esempi pratici e influenza sul design e sulle prestazioni.

Dove trovare la risposta: sezione *Caratteristiche dei Sistemi Distribuiti*, con la tabella caratteristica → tecnica.

Seconda sessione 2024 – Informatica #2

Traccia: metodologie e modelli architetturali per applicazioni distribuite enterprise (microservizi, SOA, N-tier); impatto su scalabilità, tolleranza ai guasti, affidabilità; latenza, throughput, uso delle risorse; ruolo di cloud e virtualizzazione, con vantaggi, sfide e best practice.

Dove trovare la risposta: sezioni *Architetture di Applicazioni Distribuite* e *Cloud Computing e Virtualizzazione*.

Seconda sessione 2024 – Informatica #3

Traccia: database per Big Data; modelli relazionale e non relazionale; scalabilità, flessibilità, velocità; elaborazione batch e streaming; impatto del teorema CAP; esempi e casi d'uso.

Dove trovare la risposta: sezioni *Database SQL e NoSQL*, *Big Data* e *Il Teorema CAP*.

Prima sessione 2026 – A2-INF-4

Traccia: concetto di protocollo nei sistemi distribuiti, ruolo nell'interoperabilità, affidabilità e sincronizzazione; funzioni di un protocollo di comunicazione; confronto tra modelli.

Dove trovare la risposta: sezione *Meccanismi Fondamentali*, e capitolo *Reti di Calcolatori* per ISO/OSI e TCP/IP.

Prima sessione 2026 – A3-INF-4 (tema di progetto)

Traccia: sistema per un cinema multisala con interfaccia Web, backend applicativo, base di dati e modulo di Machine Learning; progetto del backend, delle API (metodi HTTP, endpoint, dati scambiati) e del flusso di prenotazione in UML.

Dove trovare la risposta: sezioni *Architetture di Applicazioni Distribuite* e *Meccanismi Fondamentali* (REST); capitoli *Basi di Dati* e *Progettazione del Software* per ER e UML.

Capitolo 9

Intelligenza Artificiale e Machine Learning

L'**intelligenza artificiale (IA)** studia sistemi capaci di compiere compiti che richiedono intelligenza: ragionare, pianificare, apprendere, percepire. Il **machine learning (ML)** ne è il ramo in cui il comportamento non è programmato esplicitamente, ma *appreso dai dati*. Il capitolo copre due filoni usciti agli Esami di Stato: la **ricerca nello spazio degli stati** (2024) e i **paradigmi di apprendimento** (2024), più l'identificazione e l'overfitting (2024, tema di Automazione) e un'applicazione di ML nel tema di progetto del 2026.

9.1 Ricerca nello Spazio degli Stati

Molti problemi (percorsi, puzzle, pianificazione) si formalizzano così:

- **Stato iniziale**, **azioni** possibili in ogni stato, **modello di transizione** (stato risultante), **test di obiettivo**, **costo di cammino**.
- Una **soluzione** è una sequenza di azioni dallo stato iniziale a uno stato obiettivo; è **ottima** se ha costo minimo.
- Si costruisce un **albero (o grafo) di ricerca**; la *frontiera* contiene i nodi generati e non ancora espansi. Nella ricerca su grafo si tiene una lista dei nodi già visitati per evitare cicli.

Parametri con cui si valutano gli algoritmi, con b = fattore di ramificazione, d = profondità della soluzione meno profonda, m = profondità massima:

- **Completezza**: trova sempre una soluzione se esiste?
- **Ottimalità**: trova la soluzione di costo minimo?
- **Complessità temporale** e **spaziale**: nodi generati e nodi tenuti in memoria.

9.1.1 Ricerca non informata (cieca)

Usa solo la definizione del problema, senza sapere "quanto è vicino" l'obiettivo.

Algoritmo	**Idea**	**Completo**	**Ottimo**	**Tempo / Spazio**
Ampiezza (BFS)	Espande per livelli; frontiera FIFO	Sì	Sì (costi uguali)	$O(b^d)$, $O(b^d)$
Costo uniforme	Espande il nodo con costo di cammino g minore	Sì	Sì	$\approx O(b^{1+C^*/\varepsilon})$
Profondità (DFS)	Va in fondo a un ramo; frontiera LIFO	No (cicli, rami infiniti)	No	$O(b^m)$ / $O(bm)$
Profondità limitata	DFS con limite ℓ	No	No	$O(b^\ell)$ / $O(b\ell)$
Approfondimento iterativo (IDS)	DFS con limite crescente	Sì	Sì (costi uguali)	$O(b^d)$ / $O(bd)$
Bidirezionale	Due ricerche, da start e da goal	Sì	Sì	$O(b^{d/2})$

9.1.2 Ricerca informata (euristica)

Usa una **funzione euristica** $h(n)$ che stima il costo dal nodo n all'obiettivo, per espandere prima i nodi più promettenti.

- **Greedy best-first**: espande il nodo con $h(n)$ minore. Veloce, ma né ottimo né completo (può finire in vicoli ciechi).
- **A***: espande il nodo con $f(n) = g(n) + h(n)$ minore, dove $g(n)$ è il costo già speso. È **completo** e **ottimo** se h è *ammissibile*.
- **Euristica ammissibile**: non sovrastima mai il costo reale, $h(n) \leq h^*(n)$. Per A* su grafo serve la **consistenza** (monotonia): $h(n) \leq c(n, n') + h(n')$.
- **Euristica dominante**: tra $h_1 \leq h_2$ ammissibili, h_2 espande meno nodi.
- **Come costruire euristiche**: rilassando il problema (per esempio la distanza di Manhattan nel puzzle dell'8, la distanza in linea d'aria per i percorsi stradali).
- **IDA***: A* con approfondimento iterativo, per ridurre la memoria.

Problema di A*: nel caso peggiore la memoria è esponenziale, perché mantiene tutta la frontiera.

9.1.3 Ricerca locale

Non tiene traccia del cammino, ma solo dello stato corrente: utile quando conta lo stato finale (ottimizzazione, scheduling, N-regine).

- **Hill climbing**: passa al vicino migliore. Rischio di massimi locali, plateau e creste.
- **Simulated annealing**: accetta talvolta mosse peggiorative con probabilità decrescente ("temperatura").
- **Algoritmi genetici**: popolazione di soluzioni, selezione, crossover, mutazione.

9.1.4 Confronto: informata e non informata

	Non informata	**Informata**
Conoscenza	Solo la definizione del problema	Euristica $h(n)$ specifica del dominio
Efficienza	Esplora molti nodi inutili (esplosione combinatoria)	Esplora molto meno, se h è buona
Ottimalità	BFS/UCS/IDS sì	A* sì (con h ammissibile); greedy no
Memoria	DFS/IDS molto contenuta; BFS/UCS esponenziale	A* può essere esponenziale
Costo di progetto	Nessuno	Serve una buona euristica
Esempi	Esplorare un labirinto senza mappa; verifica esaustiva di configurazioni	Navigatori (percorsi), puzzle, pianificazione di robot, giochi

Scaletta per il tema "ricerca informata e non informata".

1. Formulare il problema (stati, azioni, costo) e i criteri di valutazione (completezza, ottimalità, tempo, spazio).
2. Non informata: BFS, UCS, DFS, IDS, con complessità e limiti.
3. Informata: euristica, greedy, A*, ammissibilità e consistenza.
4. Tabella di confronto su efficienza, memoria e ottimalità.
5. Esempi pratici per ciascuna classe (labirinto/verifica esaustiva contro navigatore/puzzle) e cenno alla ricerca locale.

9.2 Machine Learning: Concetti Base

Un algoritmo di ML apprende da un insieme di **esempi** (*training set*) una funzione f che generalizza a dati nuovi.

- **Feature** (caratteristiche in ingresso x), **etichetta/target** (y), **modello** con **parametri** θ da apprendere, **funzione di costo** (loss) da minimizzare.
- **Iperparametri**: scelti prima dell'addestramento (profondità di un albero, k dei k-NN, tasso di apprendimento).
- **Obiettivo**: non adattarsi ai dati visti, ma *generalizzare*.

9.3 Paradigmi di Apprendimento

9.3.1 Apprendimento supervisionato

Si dispone di dati **etichettati** (x_i, y_i) e si apprende la relazione $x \to y$ minimizzando l'errore di predizione.

- **Classificazione**: y è una categoria (spam/non spam, diagnosi). Algoritmi: regressione logistica, k-NN, alberi di decisione, random forest, SVM, reti neurali.
- **Regressione**: y è un valore continuo (prezzo di una casa, consumo energetico). Algoritmi: regressione lineare, alberi di regressione, reti neurali.
- **Esempio**: classificare le email come spam usando migliaia di email già etichettate.
- **Vantaggi**: obiettivo ben definito, prestazioni misurabili con precisione.

- **Limiti**: serve una grande quantità di dati etichettati (costosi, spesso etichettati a mano); le etichette possono contenere errori o pregiudizi; non scopre strutture nuove.

9.3.2 Apprendimento non supervisionato

Dati **non etichettati**: si cerca una struttura nascosta.

- **Clustering**: raggruppa esempi simili (k-means, clustering gerarchico, DBSCAN).
- **Riduzione della dimensionalità**: PCA, autoencoder.
- **Regole di associazione** e **rilevamento di anomalie**.
- **Esempio**: segmentare i clienti di un negozio in gruppi con abitudini di acquisto simili, senza sapere a priori quali gruppi esistano.
- **Vantaggi**: usa dati abbondanti e a basso costo; può scoprire strutture inattese.
- **Limiti**: risultati difficili da valutare e da interpretare (non c'è una "risposta giusta"); sensibile alla scelta della metrica di distanza e del numero di gruppi.

9.3.3 Apprendimento per rinforzo

Un **agente** interagisce con un **ambiente**: in ogni stato s sceglie un'azione a, riceve una **ricompensa** r e passa in un nuovo stato. Obiettivo: apprendere una **politica** $\pi(s)$ che massimizzi la ricompensa cumulata (con fattore di sconto γ).

- **Formalismo**: processo decisionale di Markov (MDP); funzione valore $V(s)$ e funzione azione-valore $Q(s, a)$.
- **Q-learning**: $Q(s, a) \leftarrow Q(s, a) + \alpha\,[\, r + \gamma \max_{a'} Q(s', a') - Q(s, a)\,]$.
- **Dilemma esplorazione/sfruttamento**: provare azioni nuove contro scegliere quella migliore nota (ε-greedy).
- **Esempio**: un robot che impara a camminare, un programma che gioca a scacchi o Go, il controllo di un impianto di climatizzazione.
- **Vantaggi**: non servono dati etichettati, ottimizza decisioni sequenziali a lungo termine.
- **Limiti**: richiede moltissime interazioni (spesso in simulazione), ricompensa difficile da definire, addestramento instabile, problemi di sicurezza durante l'esplorazione.

9.3.4 Confronto

	Supervisionato	**Non supervisionato**	**Per rinforzo**
Dati	Etichettati (x, y)	Solo x, non etichettati	Esperienze (s, a, r, s') da un ambiente
Segnale di apprendimento	Errore rispetto alla risposta corretta	Nessuno esplicito (struttura dei dati)	Ricompensa, spesso ritardata
Obiettivo	Predire y	Scoprire struttura	Massimizzare la ricompensa cumulata
Problemi tipici	Classificazione, regressione	Clustering, riduzione dimensionale	Controllo, giochi, robotica
Esempio	Diagnosi da immagini	Segmentazione clienti	Guida autonoma, gioco

Esistono anche forme intermedie: **semi-supervisionato** (pochi dati etichettati e molti no), **auto-supervisionato** (le etichette si ricavano dai dati stessi, come nei modelli linguistici).

Scegliere il paradigma. Etichette disponibili e obiettivo di predizione chiaro → supervisionato. Solo dati grezzi e voglia di esplorarli → non supervisionato. Decisioni sequenziali con feedback, senza una soluzione corretta nota → rinforzo.

9.4 Valutazione, Overfitting e Underfitting

9.4.1 Suddivisione dei dati

- **Training set**: usato per apprendere i parametri.
- **Validation set**: usato per scegliere iperparametri e confrontare modelli.
- **Test set**: usato *una sola volta* alla fine per stimare le prestazioni su dati nuovi. Non deve influenzare in alcun modo l'addestramento (nessun *data leakage*).
- **Cross-validation** a k fold: i dati sono divisi in k parti, ognuna usata a turno come validazione; utile con pochi dati. Tipici: 60–20–20 oppure 70–15–15.

9.4.2 Underfitting e overfitting

	Underfitting	**Overfitting**
Problema	Modello troppo semplice	Modello troppo complesso, memorizza anche il rumore
Errore di training	Alto	Molto basso
Errore su validation/test	Alto	Alto (molto più alto del training)
Bias / varianza	Alto bias	Alta varianza
Rimedi	Modello più ricco, più feature, meno regolarizzazione	Più dati, modello più semplice, **regolarizzazione** (L_1, L_2), early stopping, dropout, data augmentation, cross-validation

Il **compromesso bias–varianza**: all'aumentare della complessità il bias diminuisce e la varianza cresce; la complessità ottima minimizza l'errore sul validation set. La **capacità di generalizzazione** è la prestazione su dati mai visti.

9.4.3 Metriche

- **Classificazione**: matrice di confusione (TP, FP, TN, FN);

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2\,\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

  Con classi sbilanciate l'accuracy inganna: in una diagnosi con l'1% di malati, dire sempre "sano" ha il 99% di accuracy ma recall nulla.
- **Regressione**: errore quadratico medio MSE $= \frac{1}{n}\sum_i (y_i - \hat{y}_i)^2$, RMSE, MAE, R^2.

9.5 Regressione Lineare e Minimi Quadrati

Modello lineare nei parametri: $\hat{y} = \theta^\top x$, ovvero in forma matriciale $\hat{y} = \Phi\theta$, dove Φ è la matrice dei regressori (una riga per esempio).

Metodo dei minimi quadrati. Si cerca θ che minimizza il costo

$$J(\theta) = \sum_{i=1}^{N} \big(y_i - \theta^\top x_i\big)^2 = \|y - \Phi\theta\|^2.$$

Imponendo il gradiente nullo si ottengono le **equazioni normali**

$$\Phi^\top \Phi\, \hat{\theta} = \Phi^\top y \quad \Longrightarrow \quad \hat{\theta} = (\Phi^\top \Phi)^{-1} \Phi^\top y.$$

- **Interpretazione geometrica**: $\Phi\hat{\theta}$ è la **proiezione ortogonale** del vettore y sul sottospazio generato dalle colonne di Φ; il residuo $y - \Phi\hat{\theta}$ è ortogonale a tale sottospazio.
- **Identificabilità**: la soluzione è unica se e solo se $\Phi^\top\Phi$ è invertibile, cioè le colonne di Φ sono linearmente indipendenti. Se non lo sono (regressori ridondanti) esistono infinite soluzioni e si ricorre alla pseudo-inversa o alla regolarizzazione.
- **Persistenza dell'eccitazione** (identificazione di sistemi dinamici): per stimare in modo univoco i parametri di un sistema l'ingresso deve essere *sufficientemente ricco* (per esempio rumore bianco o somma di sinusoidi di frequenze diverse); un ingresso costante o troppo povero non rende $\Phi^\top\Phi$ invertibile.
- **Regolarizzazione (ridge)**: $\hat{\theta} = (\Phi^\top\Phi + \lambda I)^{-1}\Phi^\top y$; penalizza i parametri grandi e riduce l'overfitting.
- **Minimi quadrati ricorsivi (RLS)**: aggiornano la stima a ogni nuovo dato, senza ricalcolare tutto; usati nei sistemi che variano nel tempo.
- **Alternativa iterativa**: discesa del gradiente $\theta \leftarrow \theta - \eta\, \nabla J(\theta)$.

9.5.1 Identificazione di sistemi dinamici

Dato un sistema, si stima un modello dai dati ingresso–uscita.

- **Modelli parametrici lineari**: ARX, $y(t) = -a_1 y(t-1) - \cdots - a_n y(t-n) + b_1 u(t-1) + \cdots + b_m u(t-m) + e(t)$, lineare nei parametri $\Rightarrow$ si risolve con i minimi quadrati. Altri: ARMAX, Output-Error, Box–Jenkins.
- **Predizione a un passo** (usa le uscite misurate passate) e **simulazione** (usa solo l'ingresso e le uscite stimate): la simulazione è un test più severo del modello.
- **Scelta dell'ordine**: troppo basso $\rightarrow$ underfitting, troppo alto $\rightarrow$ overfitting; si usa la validazione e criteri come AIC/BIC.
- **Validazione**: analisi dei residui (devono essere bianchi e incorrelati con l'ingresso), prestazioni su dati non usati per la stima.
- **Modelli non lineari basati su ML**: reti neurali, NARX e LSTM quando la dinamica non è lineare.

9.6 Principali Modelli di ML

- **k-nearest neighbors**: classifica un punto in base alla maggioranza dei suoi k vicini. Semplice, nessun addestramento, ma lento in predizione e sensibile alla scala delle feature.

- **Regressione logistica**: modello lineare con sigmoide $\sigma(z) = 1/(1+e^{-z})$ per la probabilità di classe.
- **Alberi di decisione e random forest**: l'albero divide lo spazio con test sulle feature (criterio: guadagno di informazione/Gini); interpretabile ma incline all'overfitting. Una *random forest* media molti alberi (*bagging*) e riduce la varianza.
- **SVM**: separa le classi con l'iperpiano a margine massimo; con i *kernel* gestisce casi non lineari.
- **k-means**: assegna i punti al centroide più vicino e ricalcola i centroidi, iterando fino a convergenza; richiede di fissare k.
- **Naive Bayes**: classificatore probabilistico basato sul teorema di Bayes con ipotesi di indipendenza tra le feature.

9.6.1 Reti neurali e deep learning

- **Neurone artificiale**: $a = \phi\Big(\sum_i w_i x_i + b\Big)$, con funzione di attivazione ϕ (ReLU, sigmoide, tanh).
- **Rete feed-forward (MLP)**: strati di neuroni; il deep learning usa molti strati.
- **Addestramento**: si minimizza la loss con la **discesa del gradiente** (SGD, Adam); i gradienti sono calcolati con la **retropropagazione** (*backpropagation*), che applica la regola della catena.
- **Architetture**: **CNN** (immagini: filtri convoluzionali che condividono i pesi), **RNN/LSTM** (sequenze), **Transformer** (attenzione; base dei modelli linguistici).
- **Tecniche anti-overfitting**: dropout, regolarizzazione dei pesi, early stopping, data augmentation, batch normalization.

9.7 Flusso di un Progetto di ML

1. **Definizione del problema** e della metrica di successo.
2. **Raccolta e pulizia dei dati**: valori mancanti, outlier, duplicati, bilanciamento delle classi.
3. **Feature engineering**: normalizzazione/standardizzazione, codifica delle variabili categoriche, selezione delle feature.
4. **Suddivisione** in training, validation e test.
5. **Scelta e addestramento del modello**, con ottimizzazione degli iperparametri.
6. **Valutazione** sul test set.
7. **Deployment** e **monitoraggio**: il modello si degrada se la distribuzione dei dati cambia (*data drift*); servono riaddestramento periodico (MLOps).

Aspetti etici e di affidabilità. Pregiudizi (*bias*) nei dati, interpretabilità (spiegabilità) delle decisioni, privacy e conformità (GDPR), robustezza contro attacchi avversari.

9.8 Rappresentazione della Conoscenza (cenni)

- **Logica proposizionale e del primo ordine**: rappresentano fatti e regole; l'inferenza è basata su risoluzione o concatenazione in avanti/indietro.
- **Sistemi basati su regole** (sistemi esperti), **ontologie** e **grafi di conoscenza**.

- **Ragionamento probabilistico**: reti bayesiane, che codificano dipendenze condizionali tra variabili casuali.
- **Giochi a due avversari**: algoritmo **minimax** e potatura **alfa–beta**.

9.9 Esempio: ML in un Sistema Completo

Nel tema di progetto del 2026 (cinema multisala) un **modulo di Machine Learning** supporta la pianificazione delle proiezioni, prevedendo le presenze.

- **Problema**: regressione supervisionata. Ingresso: film, genere, giorno e orario, stagione, prezzi, presenze storiche. Uscita: biglietti venduti attesi.
- **Dati**: estratti dalla base di dati delle prenotazioni; pulizia e codifica di giorno e genere.
- **Modello**: partire da una baseline (regressione lineare o random forest), valutare con MAE/RMSE su un test set *successivo nel tempo* (split temporale, per evitare leakage).
- **Integrazione**: il modello è esposto da un servizio con un'API (per esempio `POST /predizioni`) chiamata dal backend; riaddestramento periodico con i nuovi dati e monitoraggio dell'errore.
- **Alternativa non supervisionata**: segmentare gli spettatori per abitudini (clustering) per campagne mirate.

Tracce Uscite

Prima sessione 2024 – Informatica #3

Traccia: tecniche di ricerca informata e non informata; vantaggi e svantaggi in termini di efficienza computazionale, memoria, ottimalità; esempi di problemi pratici risolvibili con una classe di tecniche rispetto all'altra.

Dove trovare la risposta: sezione *Ricerca nello Spazio degli Stati*, con tabelle e scaletta di risposta.

Seconda sessione 2024 – Informatica #1

Traccia: apprendimento supervisionato, non supervisionato e per rinforzo; principio e funzionamento; confronto su necessità di dati etichettati, uso dei dati, ambiti di applicazione; esempio concreto per ciascuno; limiti e casi in cui un approccio è più vantaggioso.

Dove trovare la risposta: sezione *Paradigmi di Apprendimento*, con confronto in tabella e paragrafo *Scegliere il paradigma*.

Prima sessione 2024 – Automazione #1

Traccia: modellazione e analisi di sistemi dinamici; regressione lineare per identificazione e apprendimento automatico; underfitting e overfitting; scelta di training e validation set; metodo dei minimi quadrati con interpretazione geometrica, identificabilità e persistenza dell'eccitazione.

Dove trovare la risposta: sezioni *Valutazione, Overfitting e Underfitting* e *Regressione Lineare e Minimi Quadrati*.

Seconda sessione 2024 – Automazione #2

Traccia: identificazione di sistemi dinamici con modelli parametrici lineari; predizione e simulazione; test di accuratezza e scelta dell'ordine; minimi quadrati e ricorsivi; identificazione non lineare con machine learning; generalizzazione, overfitting, underfitting, training e validation set.

Dove trovare la risposta: sottosezione *Identificazione di sistemi dinamici* e sezioni su overfitting e minimi quadrati.

Prima sessione 2026 – A3-INF-4 (tema di progetto)

Traccia: nel sistema del cinema multisala è previsto un modulo di Machine Learning a supporto della pianificazione delle proiezioni.

Dove trovare la risposta: sezione *Esempio: ML in un Sistema Completo*.

Capitolo 10

Progettazione di Sistemi Software e API: dalla Traccia alla Soluzione

Nella Sezione A, oltre ai temi discorsivi, il Settore Informazione prevede una **prova pratica di progettazione**: viene descritto un sistema reale (per esempio un cinema multisala) e si chiede di progettarne database, backend, API, diagrammi UML e un eventuale modulo di Machine Learning, *rispondendo a tutti i punti*. Questo capitolo raccoglie gli strumenti e, nell'ultima parte, svolge per intero la traccia **A3-INF-4 della prima sessione 2026**.

10.1 Come Affrontare la Prova Pratica

10.1.1 Metodo in sei passi

1. **Leggere tutta la traccia** e sottolineare nomi (candidati entità), verbi (operazioni e API), vincoli espliciti ("il sistema deve impedire…") e componenti già dati (per esempio "l'interfaccia Web si assume già realizzata").
2. **Dichiarare le assunzioni** in poche righe: dove la traccia è ambigua, scegliere e scrivere la scelta. Le assunzioni ragionevoli e dichiarate sono valutate positivamente.
3. **Rispondere nell'ordine dei punti numerati**, con la stessa numerazione della traccia.
4. **Un diagramma o un codice per ogni punto che lo richiede**: se si chiede "diagramma ER", "SQL" o "UML", serve proprio quello, non una descrizione.
5. **Motivare le scelte** ("scelgo X perché…, alternativa Y scartata perché…").
6. **Rileggere i vincoli**: ogni vincolo della traccia deve avere un meccanismo che lo fa rispettare (vedi l'impossibilità di prenotare due volte lo stesso posto) e va indicato esplicitamente.

10.1.2 Cosa mostrare

- Un **modello dei dati** coerente: chiavi, cardinalità, vincoli di integrità.
- **Separazione delle responsabilità**: livelli, componenti, interfacce.
- **Gestione degli errori e della concorrenza**: cosa succede se due utenti agiscono insieme o una richiesta fallisce a metà.
- **Sicurezza di base**: autenticazione, autorizzazione, validazione dell'input, HTTPS.
- **Qualità non funzionali**: scalabilità, affidabilità, manutenibilità (vedi capitolo *Sistemi Distribuiti*).

10.2 Progettazione di API REST

REST (*Representational State Transfer*) è uno stile architetturale per servizi su HTTP. Un servizio che lo rispetta si dice *RESTful*.

10.2.1 Principi

- **Risorse**: ogni concetto del dominio (film, proiezione, prenotazione) è una risorsa identificata da un **URL**. Gli URL contengono *nomi*, non verbi: `/prenotazioni/17`, non `/creaPrenotazione`.
- **Interfaccia uniforme**: le operazioni sono i metodi HTTP standard.
- **Stateless**: ogni richiesta contiene tutto ciò che serve (compresa l'autenticazione, tipicamente un token). Il server non conserva lo stato di sessione, quindi si può replicare facilmente dietro un load balancer.
- **Rappresentazioni**: la stessa risorsa può avere formati diversi; oggi quasi sempre **JSON**.
- **Cacheabilità** delle risposte `GET` e **sistema a livelli** (gateway, proxy, load balancer trasparenti al client).

10.2.2 Metodi HTTP

Metodo	Uso	Sicuro	Idempotente	Successo tipico
GET	Legge una risorsa o una collezione	Sì	Sì	200
POST	Crea una risorsa o avvia un'azione	No	No	201 + `Location`
PUT	Sostituisce interamente una risorsa	No	Sì	200 / 204
PATCH	Modifica parziale	No	Non garantito	200 / 204
DELETE	Elimina (o annulla) una risorsa	No	Sì	204

Sicuro = non modifica lo stato sul server. *Idempotente* = ripetere la richiesta n volte ha lo stesso effetto di una sola volta.

10.2.3 Codici di stato

- **2xx successo**: 200 OK, 201 Created, 204 No Content.
- **3xx**: 301/302 redirect, 304 Not Modified (cache).
- **4xx errore del client**: 400 Bad Request (richiesta malformata), 401 Unauthorized (non autenticato), 403 Forbidden (autenticato ma non autorizzato), 404 Not Found, **409 Conflict** (conflitto con lo stato della risorsa: per esempio posto già occupato), 422 Unprocessable Entity (sintassi corretta ma dati semanticamente non validi), 429 Too Many Requests.
- **5xx errore del server**: 500 Internal Server Error, 503 Service Unavailable.

10.2.4 Buone pratiche

- **Naming**: sostantivi al plurale, gerarchie per le relazioni di appartenenza (`/proiezioni/42/posti`).
- **Filtri, ordinamento, paginazione**: tramite query string (`?data=2026-11-10&film=7&page=2&size=20`).
- **Versionamento**: `/api/v1/...`, per poter evolvere senza rompere i client.
- **Formato d'errore uniforme**: per esempio `{"errore":"POSTO_OCCUPATO","dettaglio":"..."}`.

- **Idempotenza delle creazioni**: l'header `Idempotency-Key` permette di ripetere un `POST` (per esempio dopo un timeout) senza creare duplicati.
- **Sicurezza**: HTTPS, autenticazione con token (OAuth 2 / JWT), controllo dei permessi per ruolo (utente, gestore), validazione e sanificazione dell'input, rate limiting. L'identità dell'utente si ricava dal token, *non* dal corpo della richiesta.
- **Documentazione**: specifica **OpenAPI** (Swagger).

10.2.5 REST, SOAP e gRPC

	REST	**SOAP**	**gRPC**
Formato	JSON (o altro) su HTTP	XML su HTTP	Protocol Buffers su HTTP/2
Stile	Risorse e metodi HTTP	Operazioni (RPC) con contratto WSDL	Chiamate remote tipizzate
Pro	Semplice, diffuso, cacheabile	Standard enterprise, transazioni e sicurezza avanzate	Molto efficiente, streaming
Uso tipico	API pubbliche, Web e mobile	Integrazione legacy aziendale	Comunicazione tra microservizi

10.3 Architettura del Backend

10.3.1 Architettura a livelli

La struttura più comune e facile da giustificare è a **tre livelli logici** dentro il backend:

- **Controller / API layer**: riceve le richieste HTTP, valida il formato, autentica e autorizza, traduce JSON in oggetti e risponde con il codice di stato corretto. *Non* contiene logica di business.
- **Service / livello di dominio**: implementa le regole di business ("una prenotazione contiene almeno un posto", "non si prenota una proiezione passata") e coordina le transazioni.
- **Repository / accesso ai dati**: unico punto che parla con il database (SQL o ORM).

Accanto a questi: **client verso servizi esterni** (per esempio il servizio di ML), **autenticazione**, **logging e monitoraggio**, **gestione della configurazione**.

10.3.2 Pattern utili

- **Repository**: incapsula l'accesso ai dati dietro un'interfaccia; si può sostituire il database nei test.
- **DTO** (*Data Transfer Object*): oggetti usati solo per lo scambio dati con i client, separati dal modello di dominio.
- **Dependency Injection**: i componenti ricevono le dipendenze dall'esterno; semplifica test e sostituzione.
- **Facade / Gateway**: un unico punto di ingresso verso sottosistemi esterni (per esempio il modulo di ML).
- **MVC**: Model–View–Controller, per separare dati, presentazione e controllo.

10.3.3 Integrare un modulo esterno (per esempio ML)

- Il modulo di ML è un **servizio separato** con la propria API (`POST /predizioni`); il backend lo chiama tramite un client dedicato. Vantaggi: tecnologie diverse (Python per ML), scalabilità e rilascio indipendenti, il guasto del servizio ML non blocca le prenotazioni.
- L'**addestramento** (offline, periodico) è separato dalla **predizione** (online o in batch); i dati sono letti da una copia o da una vista del database, per non appesantire il database transazionale.
- Le chiamate al servizio vanno protette con **timeout** e un **valore di ripiego** (*fallback*).

10.4 Concorrenza e Integrità: Evitare le Doppie Prenotazioni

Quando due utenti prenotano lo stesso posto quasi contemporaneamente si verifica una **race condition**. Un controllo "leggo i posti liberi, poi inserisco" non basta: tra la lettura e la scrittura l'altro utente può inserire lo stesso posto (problema *check-then-act*).

Tecnica	Come funziona	Note
Vincolo di unicità (chiave primaria o `UNIQUE`) sul biglietto	Il database rifiuta il secondo inserimento dello stesso (proiezione, posto)	**Soluzione di base, la più robusta**: la garanzia sta nel database, vale anche con più istanze del backend
Transazione con isolamento adeguato	Inserimento della prenotazione e dei biglietti in un'unica transazione ACID: o tutto o niente	Evita prenotazioni parziali; da sola non basta se manca il vincolo
Lock pessimistico (`SELECT ... FOR UPDATE`)	Blocca le righe dei posti finché la transazione non termina	Corretto, ma riduce la concorrenza e può causare deadlock se i posti non sono sempre bloccati nello stesso ordine
Lock ottimistico (colonna `versione`)	Si aggiorna solo se la versione non è cambiata; altrimenti si riprova	Adatto quando i conflitti sono rari
Blocco temporaneo (*hold*) con scadenza	Il posto è riservato per alcuni minuti durante il pagamento; poi torna libero	Migliora l'esperienza utente; richiede un job di pulizia o TTL (per esempio cache con scadenza)

Ordine di risposta consigliato. (1) vincolo di unicità nel database come garanzia finale; (2) transazione per atomicità; (3) il backend traduce la violazione del vincolo in risposta **409 Conflict**; (4) opzionale: hold temporaneo e verifica preliminare dei posti per un messaggio d'errore chiaro; (5) lock pessimistico solo se serve.

Livelli di isolamento.

- `READ COMMITTED`: evita le letture sporche.
- `REPEATABLE READ` e `SERIALIZABLE`: evitano anche anomalie come le *non-repeatable read* e i *phantom*.
- Con un vincolo di unicità basta in genere `READ COMMITTED`.

10.5 Diagrammi UML per il Flusso Applicativo

- **Diagramma di sequenza**: mostra *chi chiama chi e in quale ordine* nel tempo. È il diagramma giusto per "il flusso di creazione di una prenotazione, dalla richiesta fino alla registrazione nel database". Le **linee di vita** sono verticali, le frecce piene sono chiamate sincrone, quelle tratteggiate sono risposte; i frammenti `alt` (alternativa), `opt` (opzionale) e `loop` descrivono rami e cicli.
- **Diagramma delle attività**: mostra il flusso di controllo con decisioni e parallelismo; adatto a descrivere una procedura di business.
- **Diagramma dei componenti** e **di deployment**: mostrano i blocchi del sistema (Web, backend, database, servizio ML) e dove vengono eseguiti.
- **Diagramma delle classi**: struttura statica del dominio.

Per gli aspetti generali di UML si veda il capitolo *Progettazione del Software*.

10.6 Tema Svolto: Cinema Multisala (A3-INF-4, 2026)

Sintesi della traccia. Un cinema multisala vuole un sistema per consultare la programmazione, prenotare posti e pianificare le proiezioni future. Componenti: interfaccia Web (già realizzata), backend applicativo, base di dati relazionale, modulo di Machine Learning. Un film ha più proiezioni (sala, data e ora, prezzo); una sala ha posti identificati da fila e numero; gli utenti sono già registrati, scelgono una proiezione e prenotano uno o più posti; **lo stesso posto non può essere prenotato due volte per la stessa proiezione**. Il modulo di ML stima la domanda delle future proiezioni. La prova richiede di svolgere *tutti* i 9 punti, divisi in tre parti.

Assunzioni dichiarate. (i) Gli utenti sono già registrati e autenticati con un token. (ii) Il pagamento è fuori dall'ambito: una prenotazione si considera confermata alla creazione. (iii) Un posto appartiene a una sola sala e ogni proiezione si svolge in una sola sala. (iv) Il prezzo è unico per proiezione. (v) Il gestore è un ruolo distinto dall'utente.

10.6.1 Parte 1: progettazione della base di dati

Punto 1 – Modello concettuale (ER)

Entità e attributi principali

- **FILM**(id_film, titolo, durata_min, genere, anno).
- **SALA**(id_sala, nome).
- **POSTO**(fila, numero) – **entità debole** di SALA: identificata da (id_sala, fila, numero), perché "fila C, numero 7" esiste in ogni sala.
- **PROIEZIONE**(id_proiezione, data_ora, prezzo).
- **UTENTE**(id_utente, nome, cognome, email unica).
- **PRENOTAZIONE**(id_prenotazione, creata_il, stato).

Associazioni e cardinalità

- FILM $(0, N)$ – *è programmato* – $(1, 1)$ PROIEZIONE: un film può avere molte proiezioni (o nessuna); ogni proiezione è di un solo film.
- SALA $(0, N)$ – *si tiene in* – $(1, 1)$ PROIEZIONE.
- SALA $(1, N)$ – *contiene* – $(1, 1)$ POSTO (identificazione esterna).

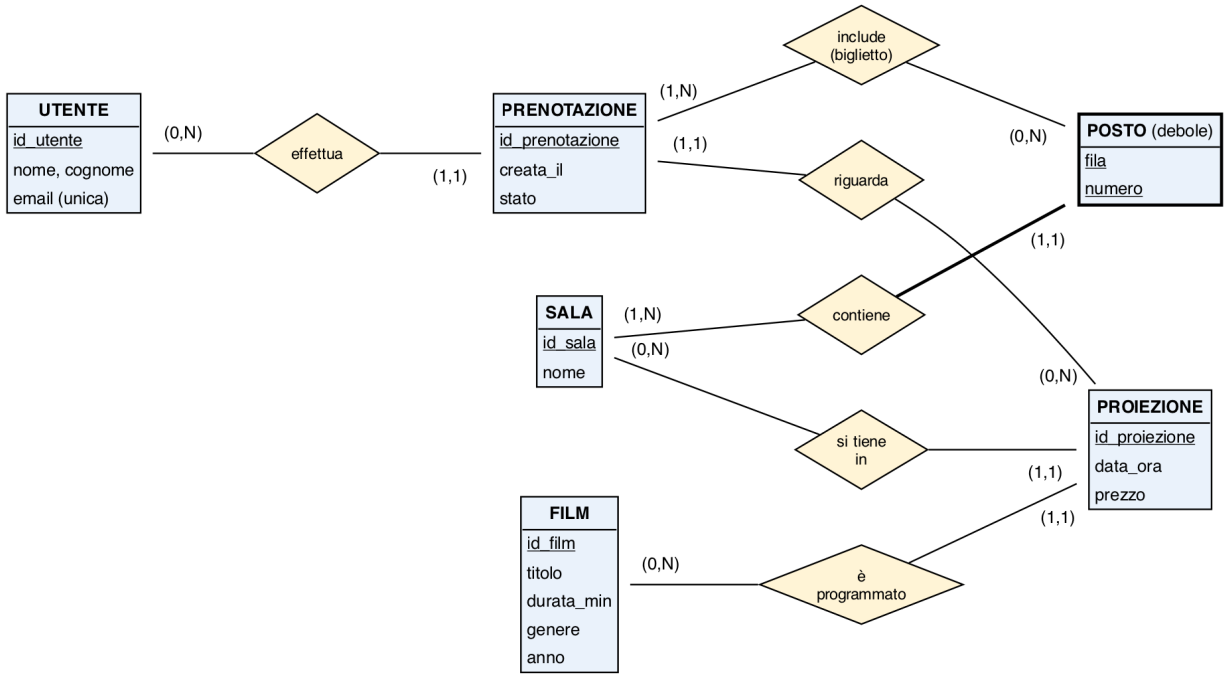


Figura 10.1: Schema ER del cinema multisala. Le cardinalità (min, max) sono scritte accanto all'entità a cui si riferiscono.

- UTENTE $(0, N)$ – *effettua* – $(1, 1)$ PRENOTAZIONE.
- PRENOTAZIONE $(1, 1)$ – *riguarda* – $(0, N)$ PROIEZIONE.
- PRENOTAZIONE $(1, N)$ – *include* (con la proiezione: *biglietto*) – $(0, N)$ POSTO: una prenotazione contiene uno o più posti. Il biglietto lega prenotazione, proiezione e posto.

Vincoli significativi (non esprimibili nel solo diagramma)

1. **Un posto non può essere prenotato due volte per la stessa proiezione**: la coppia (proiezione, posto) compare al più in un biglietto.
2. Il posto prenotato deve appartenere alla **sala in cui si tiene la proiezione**.
3. I posti di una prenotazione si riferiscono alla *stessa* proiezione della prenotazione.
4. Due proiezioni nella stessa sala non possono avere la stessa data e ora (e, volendo, non si devono sovrapporre).
5. Il prezzo non è negativo; l'email dell'utente è unica.

Punto 2 – Modello relazionale

Le chiavi primarie sono sottolineate, le chiavi esterne indicate con $\rightarrow$.

- FILM(<u>id_film</u>, titolo, durata_min, genere, anno)
- SALA(<u>id_sala</u>, nome)
- POSTO(<u>id_sala</u>$\rightarrow$SALA, <u>fila</u>, <u>numero</u>)
- PROIEZIONE(<u>id_proiezione</u>, id_film$\rightarrow$FILM, id_sala$\rightarrow$SALA, data_ora, prezzo)
  Chiavi candidate: (id_sala, data_ora) e (id_proiezione, id_sala).
- UTENTE(<u>id_utente</u>, nome, cognome, email)

- PRENOTAZIONE(id_prenotazione, id_utente→UTENTE, id_proiezione→PROIEZIONE, creata_il, stato)
  Chiave candidata: (id_prenotazione, id_proiezione).
- BIGLIETTO(id_proiezione, fila, numero, id_prenotazione, id_sala)
  Chiavi esterne:
  (id_prenotazione, id_proiezione)→PRENOTAZIONE;
  (id_proiezione, id_sala)→PROIEZIONE;
  (id_sala, fila, numero)→POSTO.

Come i vincoli sono garantiti

- Il vincolo 1 è garantito dalla **chiave primaria** (id_proiezione, fila, numero) di BIGLIETTO: lo stesso posto non può comparire due volte per la stessa proiezione, *qualunque sia il numero di istanze del backend*.
- I vincoli 2 e 3 sono garantiti dalle **chiavi esterne composte**: il biglietto referenzia la coppia (id_proiezione, id_sala) di PROIEZIONE e il posto (id_sala, fila, numero) di POSTO, quindi il posto deve appartenere alla sala della proiezione; referenzia la coppia (id_prenotazione, id_proiezione) di PRENOTAZIONE.
- Il vincolo 4 con `UNIQUE (id_sala, data_ora)`; il 5 con `CHECK` e `UNIQUE`.

Punto 3 – SQL

```sql
CREATE TABLE film (
  id_film     INTEGER PRIMARY KEY,
  titolo      VARCHAR(200) NOT NULL,
  durata_min  INTEGER NOT NULL CHECK (durata_min > 0),
  genere      VARCHAR(50),
  anno        INTEGER
);

CREATE TABLE sala (
  id_sala  INTEGER PRIMARY KEY,
  nome     VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE posto (
  id_sala  INTEGER NOT NULL REFERENCES sala(id_sala),
  fila     CHAR(1) NOT NULL,
  numero   INTEGER NOT NULL CHECK (numero > 0),
  PRIMARY KEY (id_sala, fila, numero)
);

CREATE TABLE proiezione (
  id_proiezione  INTEGER PRIMARY KEY,
  id_film        INTEGER NOT NULL REFERENCES film(id_film),
  id_sala        INTEGER NOT NULL REFERENCES sala(id_sala),
  data_ora       TIMESTAMP NOT NULL,
  prezzo         DECIMAL(6,2) NOT NULL CHECK (prezzo >= 0),
  UNIQUE (id_sala, data_ora),
  UNIQUE (id_proiezione, id_sala)
);

```

```sql
CREATE TABLE utente (
  id_utente  INTEGER PRIMARY KEY,
  nome       VARCHAR(50) NOT NULL,
  cognome    VARCHAR(50) NOT NULL,
  email      VARCHAR(120) NOT NULL UNIQUE
);

CREATE TABLE prenotazione (
  id_prenotazione  INTEGER PRIMARY KEY,
  id_utente        INTEGER NOT NULL REFERENCES utente(id_utente),
  id_proiezione    INTEGER NOT NULL REFERENCES proiezione(id_proiezione),
  creata_il        TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  stato            VARCHAR(12) NOT NULL DEFAULT 'CONFERMATA'
                   CHECK (stato IN ('CONFERMATA', 'ANNULLATA')),
  UNIQUE (id_prenotazione, id_proiezione)
);

CREATE TABLE biglietto (
  id_proiezione    INTEGER NOT NULL,
  fila             CHAR(1) NOT NULL,
  numero           INTEGER NOT NULL,
  id_prenotazione  INTEGER NOT NULL,
  id_sala          INTEGER NOT NULL,
  -- un posto non puo' essere venduto due volte per la stessa proiezione
  PRIMARY KEY (id_proiezione, fila, numero),
  FOREIGN KEY (id_prenotazione, id_proiezione)
      REFERENCES prenotazione(id_prenotazione, id_proiezione),
  -- la proiezione si tiene in quella sala...
  FOREIGN KEY (id_proiezione, id_sala)
      REFERENCES proiezione(id_proiezione, id_sala),
  -- ...e il posto appartiene a quella sala
  FOREIGN KEY (id_sala, fila, numero)
      REFERENCES posto(id_sala, fila, numero)
);
```

Listing 10.1: Creazione delle tabelle (sintassi in stile PostgreSQL)

```sql
SELECT p.fila, p.numero
FROM proiezione pr
JOIN posto p ON p.id_sala = pr.id_sala
WHERE pr.id_proiezione = :id_proiezione
  AND NOT EXISTS (
    SELECT 1
    FROM biglietto b
    WHERE b.id_proiezione = pr.id_proiezione
      AND b.fila   = p.fila
      AND b.numero = p.numero
  )
ORDER BY p.fila, p.numero;
```

Listing 10.2: Posti ancora disponibili per una proiezione

La query prende i posti della sala in cui si tiene la proiezione e scarta quelli che compaiono tra i biglietti della stessa proiezione (`NOT EXISTS`, *anti-join*). Un'alternativa equivalente usa `LEFT JOIN ... WHERE b.fila IS NULL`. Con la chiave primaria di BIGLIETTO, la ricerca per proiezione usa già l'indice.

Nota. L'annullamento di una prenotazione deve **liberare i posti**: si eliminano i suoi biglietti (eventualmente copiandoli prima in una tabella di storico) e si pone lo stato a `ANNULLATA`.

10.6.2 Parte 2: progettazione del backend

Punto 4 – Componenti logici e interazioni

Componente	Responsabilità	Interazioni
API / Controller	Espone gli endpoint REST, valida l'input, autentica (token) e autorizza per ruolo, traduce errori in codici HTTP	Riceve le richieste dall'interfaccia Web; chiama i servizi
Servizio Programmazione	Elenca film e proiezioni, filtri per data e film	Legge dal database tramite repository
Servizio Prenotazioni	Calcola i posti disponibili, crea e annulla prenotazioni con le regole di business e la transazione	Usa il repository; **unico punto che scrive prenotazioni e biglietti**
Repository / DAO	Accesso al database relazionale (SQL), mappatura tra tabelle e oggetti	Database
Servizio Previsioni (client ML)	Prepara le richieste per il modulo di ML, chiama il suo servizio, applica timeout e valori di ripiego, espone i risultati al gestore	Modulo di Machine Learning; database per i dati storici
Autenticazione e autorizzazione	Verifica il token e il ruolo (utente o gestore)	Usato dal controller
Logging e monitoraggio	Registra richieste ed errori, metriche	Trasversale

Flusso di una richiesta: interfaccia Web → Controller → Servizio → Repository → Database, e risposta lungo il percorso inverso. Il modulo ML è un servizio separato: il backend lo chiama *solo* per le funzioni del gestore, *mai* nel percorso critico della prenotazione, così un guasto del modulo ML non blocca i clienti.

Punto 5 – API

Metodo	Endpoint	Descrizione	Esiti
GET	`/api/v1/film`	Elenco film (filtri: data, genere)	200
GET	`/api/v1/proiezioni?data=&film=`	Programmazione: proiezioni filtrate per data e film, con sala, orario, prezzo	200, 400
GET	`/api/v1/proiezioni/{id}`	Dettaglio di una proiezione	200, 404
GET	`/api/v1/proiezioni/{id}/posti`	Posti con stato *libero/occupato*	200, 404
POST	`/api/v1/proiezioni/{id}/prenotazioni`	Crea una prenotazione per uno o più posti	201, 400, 401, 404, **409**, 422
GET	`/api/v1/prenotazioni/{id}`	Dettaglio di una prenotazione dell'utente	200, 403, 404
DELETE	`/api/v1/prenotazioni/{id}`	Annulla e libera i posti	204, 403, 404
GET	`/api/v1/gestore/previsioni?data=&...`	Previsioni di domanda (ruolo gestore)	200, 403, 503

```
GET /api/v1/proiezioni/42/posti

200 OK
{
  "id_proiezione": 42,
  "sala": "Sala 3",
  "prezzo": 9.50,
  "posti": [
    { "fila": "A", "numero": 1, "stato": "libero" },
    { "fila": "A", "numero": 2, "stato": "occupato" }
  ]
}
```

Listing 10.3: Esempio: elenco dei posti di una proiezione

```
POST /api/v1/proiezioni/42/prenotazioni
Authorization: Bearer <token>
Idempotency-Key: 3f2a-...
{ "posti": [ { "fila": "C", "numero": 7 }, { "fila": "C", "numero": 8 } ] }

201 Created
Location: /api/v1/prenotazioni/1017
{ "id_prenotazione": 1017, "id_proiezione": 42, "totale": 19.00,
  "posti": [ { "fila": "C", "numero": 7 }, { "fila": "C", "numero": 8 } ] }

409 Conflict   (posto gia' occupato)
{ "errore": "POSTO_OCCUPATO", "posti": [ { "fila": "C", "numero": 7 } ] }
```

Listing 10.4: Esempio: creazione di una prenotazione

Scelte progettuali. L'identità dell'utente deriva dal *token*, non dal corpo della richiesta. L'`Idempotency-Key` evita prenotazioni doppie se il client ripete la richiesta dopo un timeout. Le

risposte `GET` sui posti non sono cacheabili a lungo, perché cambiano di continuo (e la disponibilità letta non è una garanzia: l'unica garanzia è l'inserimento).

```
def crea_prenotazione(db, id_utente, id_proiezione, posti):
    if not posti:
        raise RichiestaNonValida("almeno un posto")
    with db.transazione():                          # BEGIN ... COMMIT/ROLLBACK
        pr = db.una_riga("SELECT id_sala, data_ora FROM proiezione "
                         "WHERE id_proiezione = %s", id_proiezione)
        if pr is None:
            raise NonTrovato()                      # -> 404
        if pr.data_ora < adesso():
            raise RichiestaNonValida("proiezione passata")   # -> 422
        id_pren = db.scalare(
            "INSERT INTO prenotazione (id_utente, id_proiezione) "
            "VALUES (%s, %s) RETURNING id_prenotazione",
            id_utente, id_proiezione)
        try:
            for fila, numero in sorted(posti):     # ordine fisso: niente
                deadlock
                db.esegui(
                    "INSERT INTO biglietto "
                    "(id_proiezione, fila, numero, id_prenotazione, id_sala) "
                    "VALUES (%s, %s, %s, %s, %s)",
                    id_proiezione, fila, numero, id_pren, pr.id_sala)
        except ViolazioneUnicita:                  # chiave primaria del
            biglietto
            raise PostoOccupato()                  # rollback automatico ->
                409
        except ViolazioneChiaveEsterna:            # posto inesistente nella
            sala
            raise RichiestaNonValida("posto non valido")     # -> 422
    return id_pren
```

Listing 10.5: Pseudocodice del servizio di prenotazione (stile Python)

Punto 6 – Diagramma di sequenza e prevenzione dei doppioni

Flusso

1. L'interfaccia Web invia `POST /proiezioni/42/prenotazioni` con il token e i posti scelti.
2. Il controller valida il formato e il token e chiama il servizio.
3. Il servizio **apre una transazione**, inserisce la prenotazione e inserisce un biglietto per ogni posto.
4. Se tutti gli inserimenti riescono, la transazione fa **COMMIT** e il controller risponde **201 Created**.
5. Se un inserimento viola la chiave primaria (posto già venduto per quella proiezione), la transazione fa **ROLLBACK** (nessuna prenotazione parziale) e il controller risponde **409 Conflict**.

Operazioni che impediscono la doppia assegnazione

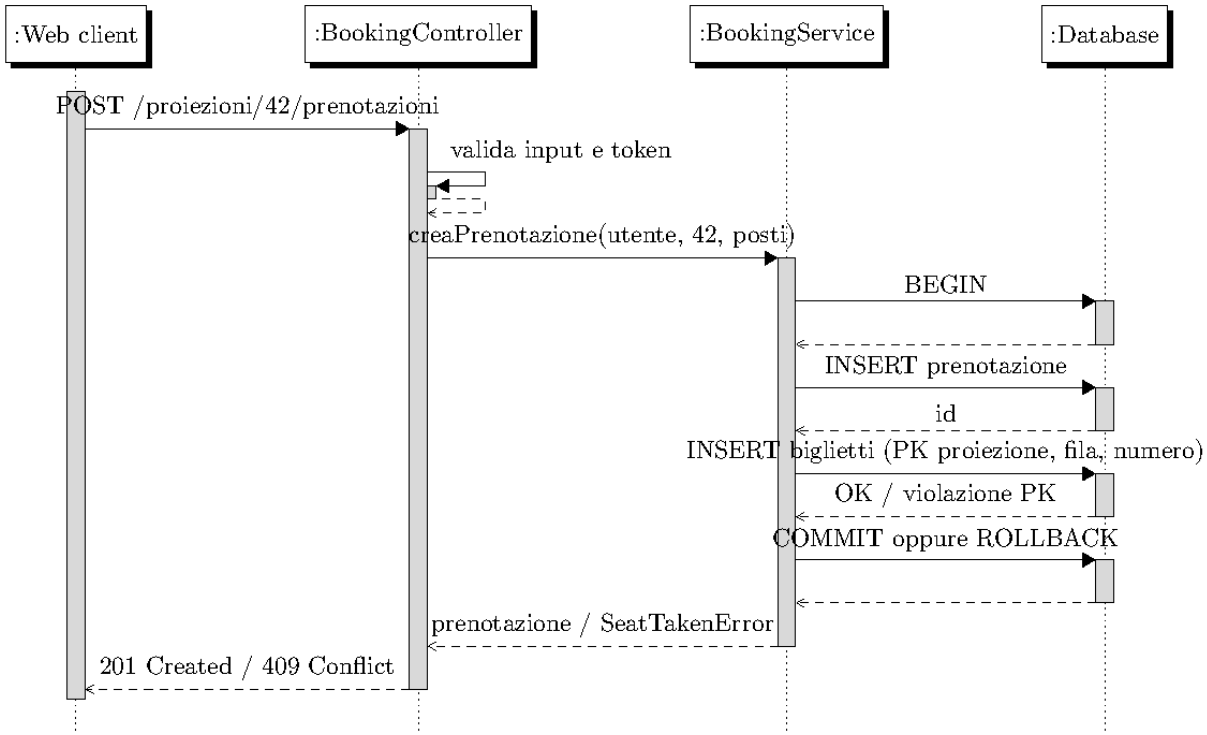


Figura 10.2: Diagramma di sequenza della creazione di una prenotazione.

- **Vincolo di unicità** (chiave primaria del biglietto): è la garanzia vera, perché è valutata dal database in modo atomico anche con più richieste concorrenti. Se due utenti prenotano lo stesso posto nello stesso istante, uno solo vince.
- **Transazione**: prenotazione e biglietti sono scritti insieme o per nulla.
- **Ordine fisso** dei posti negli inserimenti, per evitare deadlock tra richieste che toccano insiemi di posti sovrapposti.
- **Alternativa**: lock pessimistico (`SELECT ... FOR UPDATE`) sui posti, oppure *hold* temporaneo con scadenza durante il pagamento. Non sostituiscono il vincolo ma migliorano l'esperienza utente.
- **Controllo preliminare** (lettura dei posti liberi) solo come cortesia per l'interfaccia: *non* è una garanzia.

10.6.3 Parte 3: modulo di Machine Learning

Punto 7 – Progetto della soluzione

- **Problema**: **regressione supervisionata**. Si hanno dati storici con l'esito reale per ogni proiezione.
- **Variabile da prevedere** (*target*): il **numero di biglietti venduti** per proiezione, oppure il **tasso di occupazione** = biglietti venduti / capienza della sala (più confrontabile tra sale diverse).
- **Variabili di ingresso (feature)**
  - *Film*: genere, durata, anno di uscita, settimane dall'uscita, regista o cast o saga, giudizi o popolarità, vendite delle proiezioni precedenti dello stesso film.
  - *Tempo*: giorno della settimana, fascia oraria, festivo o prefestivo, vacanze, stagione, mese; eventuale meteo.
  - *Proiezione*: prezzo, sala e capienza, formato (3D, lingua originale).

  - *Contesto*: numero di proiezioni in concorrenza nello stesso orario, promozioni.
- **Modello**: **gradient boosting su alberi** (o *random forest*) come scelta principale: gestisce feature miste (numeriche e categoriche), relazioni non lineari e interazioni (giorno × ora × genere) con pochi dati e poca preparazione; è robusto e abbastanza interpretabile (importanza delle feature). Come **baseline** si usano la media storica per lo stesso giorno, ora e genere oppure una **regressione lineare**; il modello complesso va adottato solo se batte la baseline. Con molti dati si possono valutare reti neurali o modelli per serie temporali.

Punto 8 – Addestramento e valutazione

1. **Preparazione dei dati**: estrazione dal database (proiezioni e biglietti), **pulizia** (valori mancanti, errori), codifica delle categoriche, costruzione delle feature.
2. **Suddivisione dei dati *temporale*** (non casuale): addestramento sul periodo più vecchio, validazione sul periodo successivo (scelta degli iperparametri), test sull'ultimo periodo (per esempio 70%/15%/15% in ordine cronologico). Una suddivisione casuale farebbe "vedere il futuro" al modello (*data leakage*) e darebbe una stima ottimistica. In alternativa, **validazione a finestre scorrevoli** (*time series cross-validation*).
3. **Addestramento**: ottimizzazione degli iperparametri sul set di validazione, **regolarizzazione** e early stopping per evitare l'overfitting.
4. **Metriche**: **MAE** (errore medio in biglietti, facile da interpretare), **RMSE** (penalizza gli errori grandi), **WAPE/MAPE** (errore relativo; MAPE è instabile con valori prossimi a zero); confronto con la baseline. Si controlla anche l'errore per sottogruppi (sala, giorno, film nuovi).
5. **Test finale** *una sola volta* sul test set, e **monitoraggio** dopo il deployment con riaddestramento periodico.

Punto 9 – Uso delle predizioni e limiti

Come usarle per il gestore

- **Assegnazione delle sale**: abbinare la domanda stimata alla **capienza**; film con alta domanda previsto → sala grande; domanda bassa → sala piccola, per riempire le sale e non sprecare risorse.
- **Numero e orari delle proiezioni**: se la domanda prevista in una fascia supera la capienza, aggiungere una proiezione o spostarla in una sala maggiore; ridurre le proiezioni nelle fasce con bassa domanda.
- **Prezzi e promozioni**: prezzi più bassi per fasce deboli, promozioni mirate.
- **Uso come supporto**: si mostrano al gestore la stima e un **intervallo di incertezza**; si può formulare l'assegnazione come problema di **ottimizzazione vincolata** (sale disponibili, orari, durate dei film) con le previsioni come input.

Limiti

- **Dati censurati**: i biglietti venduti non possono superare la capienza; se una sala si riempie, la domanda reale era maggiore. Il modello sottostima la domanda dei successi.
- **Circolo vizioso** (*feedback loop*): se il modello assegna una sala piccola, le vendite saranno limitate e il modello "imparerà" che la domanda è bassa.
- **Film nuovi** (*cold start*) senza storico; eventi imprevedibili (successi inattesi, maltempo, scioperi); **cambiamenti** nelle abitudini (*drift*).

- **Incertezza**: le stime sono previsioni, non garanzie; l'errore va comunicato.
- **Decisione finale al gestore**, che conosce informazioni non nei dati; attenzione a equità e privacy dei dati degli utenti (GDPR: dati aggregati e anonimizzati).

10.7 Checklist e Errori Comuni

- **Rispondere a tutti i punti** e nell'ordine; usare la stessa numerazione della traccia.
- **Non scrivere solo prosa** dove si chiede un diagramma, un SQL o una tabella.
- **Dichiarare le assunzioni** e usarle in modo coerente.
- **Ogni vincolo della traccia deve avere un meccanismo**: qui, il vincolo di unicità.
- Nel modello relazionale: **chiavi primarie ed esterne** indicate e coerenti con lo schema ER.
- **Non mettere logica di business nel controller** né SQL nei controller.
- Per le API: **nomi (non verbi)** negli URL, metodi e codici di stato corretti, 409 per i conflitti.
- Per il ML: **suddivisione temporale**, baseline, metrica adeguata, limiti dichiarati.
- **Controllare il tempo**: schemi semplici e corretti valgono più di schemi complicati e incompleti.

Tracce Uscite

Prima sessione 2026 – A3-INF-4 (prova pratica di progettazione)

Traccia: cinema multisala con interfaccia Web, backend, base di dati relazionale e modulo di Machine Learning. Nove punti in tre parti: (1) ER, (2) modello relazionale, (3) SQL e query dei posti disponibili; (4) componenti del backend, (5) API REST, (6) diagramma UML di sequenza per la prenotazione con prevenzione dei doppioni; (7) progetto del modello di previsione, (8) addestramento e valutazione, (9) uso delle predizioni e limiti.

Dove trovare la risposta: sezione *Tema Svolto: Cinema Multisala*, che risponde a ciascun punto con la stessa numerazione.

Prima sessione 2018 – Informatica (seconda prova)

Traccia: progettare un algoritmo che confronta due immagini A e B di dimensione $W \times H$ per trovare le corrispondenze tra pixel (ricerca del pixel più simile in B nell'intervallo $[x - d]$, $d \in [0, D]$), restituendo una nuova immagine con lo spostamento d minimo; indicare la complessità in tempo e memoria.

Dove trovare la risposta: richiede pseudocodice con tre cicli annidati; vedi il capitolo *Programmazione Orientata agli Oggetti (e Fondamenti)*, sezione sulla complessità. Complessità: tempo $O(W \cdot H \cdot D)$, memoria $O(W \cdot H)$ per l'immagine di uscita.

Capitolo 11

Programmazione ad Oggetti e Algoritmi: Teoria, Codice e Motivazioni

Nelle tracce della Sezione A il tema di informatica chiede quasi sempre due cose insieme: **spiegare un concetto** (classe, polimorfismo, chiamata a funzione, complessità) e **mostrarlo in codice**. Una buona risposta non si limita a definire, ma spiega *perché* si programma in quel modo: quale problema risolve e che cosa succederebbe senza. Questo capitolo completa il capitolo *Programmazione Orientata agli Oggetti (e Fondamenti)* e ne approfondisce gli argomenti usciti negli esami: classe e oggetto (2016), ricorsione e iterazione (2018), algoritmi e complessità (2018), stack e passaggio dei parametri (2025), polimorfismo (2026).

Gli esempi sono in **Java** (tipizzazione statica, utile per spiegare classi e interfacce), **Python** (algoritmi) e **C++** (riferimenti e puntatori). Nell'esame il linguaggio è a scelta, o si può usare pseudocodice.

11.1 Perché si Programma ad Oggetti

11.1.1 Il problema: la complessità e il cambiamento

Un programma piccolo si può scrivere come una lunga sequenza di istruzioni. Un sistema reale (centinaia di migliaia di righe, decine di sviluppatori, anni di vita) pone invece tre problemi:

1. **Complessità**: nessuno riesce a tenere a mente l'intero programma. Serve poter ragionare su una *parte* alla volta, senza conoscere i dettagli del resto.
2. **Cambiamento**: i requisiti cambiano di continuo (nuovo metodo di pagamento, nuovo tipo di sensore, nuovo database). Il costo di una modifica dipende da *quanto codice è toccato* da quella modifica.
3. **Lavoro in gruppo e riuso**: più persone devono lavorare in parallelo senza ostacolarsi, e riusare parti già scritte e testate.

11.1.2 L'idea di fondo: nascondere ciò che può cambiare

Il principio guida (D. Parnas, *information hiding*) è: **scomporre il sistema in moduli in modo che ogni decisione di progetto che potrebbe cambiare sia nascosta dentro un solo modulo**. Gli altri moduli conoscono solo l'*interfaccia* stabile. Così un cambiamento resta locale.

La programmazione a oggetti applica questa idea in modo sistematico: l'unità di modularità è la **classe**, che riunisce *dati* e *operazioni* sui dati, espone un'interfaccia pubblica e nasconde l'implementazione.

11.1.3 Metriche di qualità: coesione e accoppiamento

- **Coesione alta**: tutto ciò che sta in una classe ha uno scopo comune (una classe `Rational` contiene solo ciò che riguarda i numeri razionali).
- **Accoppiamento basso**: le classi dipendono il meno possibile dai dettagli interni delle altre. Se A usa solo l'interfaccia di B, cambiare B non rompe A.

Tutti gli strumenti dell'OOP (incapsulamento, interfacce, polimorfismo, composizione) servono a ottenere **alta coesione e basso accoppiamento**.

11.1.4 Confronto tra paradigmi

	Procedurale	**Orientato agli oggetti**	**Funzionale**
Unità di base	Funzione e dati separati	Oggetto = dati + metodi	Funzione pura
Come si gestisce la complessità	Scomposizione in funzioni	Scomposizione in classi e responsabilità	Composizione di funzioni, dati immutabili
Stato	Variabili condivise	Incapsulato negli oggetti	Evitato (immutabilità)
Punto di forza	Semplicità, efficienza	Sistemi grandi che evolvono, modelli del dominio	Parallelismo, correttezza, trasformazione di dati
Esempi	C	Java, C++, Python	Haskell, funzionale in Python e Java

Nessun paradigma è migliore in assoluto: per un piccolo script o una pipeline di dati l'OOP può essere eccessiva; per un sistema grande e in evoluzione offre una struttura che resiste ai cambiamenti. Molti linguaggi moderni li combinano.

11.2 Classi e Oggetti

11.2.1 Definizioni

- **Classe**: descrizione di un insieme di oggetti con la stessa struttura e lo stesso comportamento. Definisce gli **attributi** (campi) e i **metodi**. È un *tipo*. Esiste nel codice sorgente.
- **Oggetto**: *istanza* di una classe, creata a tempo di esecuzione con `new` (o chiamando il costruttore). Ha un'**identità**, uno **stato** (i valori degli attributi) e un **comportamento** (i metodi). Esiste in memoria.
- **Differenza**: la classe è lo *stampino*, gli oggetti sono i *biscotti*. Una classe, molti oggetti; ciascuno ha il proprio stato e condivide il codice dei metodi.
- **Costruttore**: metodo speciale che inizializza l'oggetto e deve lasciarlo in uno stato *valido*.
- **Riferimento**: una variabile di tipo classe non contiene l'oggetto, ma un *riferimento* a esso nello **heap**; due variabili possono riferirsi allo stesso oggetto (*aliasing*).
- **Membri statici**: appartengono alla classe, non alla singola istanza (contatori, costanti, metodi di utilità).
- **Identità e uguaglianza**: `==` confronta i riferimenti (stesso oggetto?), `equals` confronta lo stato (stesso valore?). Se si ridefinisce `equals` bisogna ridefinire anche `hashCode`.

11.2.2 Perché una classe e non una struttura con funzioni?

Con una semplice *struct* (record di dati) e funzioni separate, **qualunque** parte del programma può modificare i campi e rompere le regole del dominio (un conto con saldo negativo, una frazione con denominatore zero). Per sapere se i dati sono sempre validi bisognerebbe controllare tutto il codice.

Con una classe:

- i dati sono **privati**: si modificano solo tramite i metodi della classe;
- le regole (**invarianti**) sono controllate *in un solo punto* (il costruttore e i metodi che modificano lo stato);
- se cambia la rappresentazione interna (per esempio una frazione ridotta ai minimi termini invece di no) l'esterno non se ne accorge.

Questa è la risposta alla domanda "perché si fa così": una classe è un *custode degli invarianti* del proprio stato.

11.2.3 Esempio: la classe `Rational` (traccia 2016)

Traccia: classe `Rational` con numeratore e denominatore interi e metodo `mult` (moltiplicazione per un intero) che ritorna un *nuovo* oggetto `Rational`.

```
1  public final class Rational {
2      private final int num;   // privati: nessuno li modifica dall'esterno
3      private final int den;
4
5      public Rational(int num, int den) {
6          if (den == 0)                          // invariante: denominatore != 0
7              throw new IllegalArgumentException("denominatore nullo");
8          if (den < 0) { num = -num; den = -den; }   // segno sempre sul
               numeratore
9          int g = gcd(Math.abs(num), den);    // forma canonica: ridotta ai
               minimi termini
10         this.num = num / g;
11         this.den = den / g;
12     }
13
14     private static int gcd(int a, int b) {  // algoritmo di Euclide
15         return b == 0 ? a : gcd(b, a % b);
16     }
17
18     public Rational mult(int k) {           // ritorna un NUOVO oggetto
19         return new Rational(num * k, den);
20     }
21
22     public Rational mult(Rational r) {      // overloading: stesso nome, altri
           parametri
23         return new Rational(num * r.num, den * r.den);
24     }
25
26     public int getNum() { return num; }
27     public int getDen() { return den; }
```

```
28
29      @Override public String toString() { return num + "/" + den; }
30
31      @Override public boolean equals(Object o) {
32          if (!(o instanceof Rational)) return false;
33          Rational r = (Rational) o;
34          return num == r.num && den == r.den;    // vale perche' e' in forma
                canonica
35      }
36      @Override public int hashCode() { return 31 * num + den; }
37  }
38
39  // Uso:
40  //   Rational a = new Rational(3, 4);
41  //   Rational b = a.mult(2);        // 3/2: a resta 3/4
```

Listing 11.1: Classe Rational: classe, oggetto, incapsulamento e immutabilita

Perché questa progettazione

- **Campi `private final`** ⇒ la classe è **immutabile**: dopo la creazione l'oggetto non cambia mai. Un oggetto immutabile si può condividere senza rischi (niente effetti collaterali, niente problemi di concorrenza) ed è facile da ragionare.
- **`mult` ritorna un nuovo oggetto** (come chiede la traccia): è la conseguenza dell'immutabilità; è lo stesso modello di `String` in Java.
- **Costruttore che valida e normalizza**: l'invariante ("$den \neq 0$, ridotta ai minimi termini, segno sul numeratore") è vera per *ogni* oggetto creato, in un unico punto. Così `equals` si riduce a confrontare i due campi.
- **Overloading di `mult`**: si usa la stessa operazione concettuale con argomenti diversi.

11.3 Incapsulamento

L'**incapsulamento** (o *information hiding*) consiste nel **nascondere lo stato interno** e consentirne l'accesso solo tramite metodi che ne garantiscono la coerenza.

```
1   public class ContoCorrente {
2       private long saldoCentesimi;                // stato nascosto
3
4       public ContoCorrente(long saldoIniziale) {
5           if (saldoIniziale < 0) throw new IllegalArgumentException();
6           this.saldoCentesimi = saldoIniziale;
7       }
8       public void preleva(long importo) {         // unico modo per ridurre il
            saldo
9           if (importo <= 0 || importo > saldoCentesimi)
10              throw new IllegalStateException("operazione non valida");
11          saldoCentesimi -= importo;
12      }
13      public long getSaldo() { return saldoCentesimi; }
14  }
```

Listing 11.2: Incapsulamento: il metodo protegge l'invariante

Perché

- Se `saldoCentesimi` fosse pubblico, ogni riga di codice potrebbe scrivere un valore negativo: l'invariante "saldo ≥ 0" sarebbe verificabile solo ispezionando *tutto* il programma.
- La rappresentazione è libera di cambiare (da `long` a `BigDecimal`) senza toccare i chiamanti.
- **Attenzione**: un campo privato con `getX()` e `setX()` per ogni campo *non* è vero incapsulamento: espone comunque tutto. Meglio metodi che esprimono operazioni del dominio (`preleva`, `versa`) rispetto a `setSaldo`. Principio *Tell, don't ask*: si chiede all'oggetto di fare, non gli si chiedono i dati per decidere al posto suo.
- **Modificatori di accesso**: `private` (solo la classe), `protected` (classe e sottoclassi), `public`. La regola è esporre il minimo necessario.

11.4 Astrazione, Interfacce e Classi Astratte

- **Astrazione**: rappresentare solo gli aspetti rilevanti, ignorando i dettagli. Un'**interfaccia** è il *contratto* di un tipo: cosa si può chiedere, non come viene fatto.
- **Interfaccia**: elenco di metodi (firme) senza stato; una classe la `implements`, anche più di una.
- **Classe astratta**: classe parzialmente implementata che non si può istanziare; contiene anche stato e codice comune e metodi astratti da completare nelle sottoclassi.

	Interfaccia	**Classe astratta**
Stato (campi)	No (solo costanti)	Sì
Codice implementato	Solo metodi di default	Sì, anche costruttori
Ereditarietà multipla	Una classe può implementare più interfacce	Una sola superclasse
Si usa quando	Si definisce un *ruolo* o capacità ("leggibile", "ordinabile") indipendente dalla gerarchia	Si condivide codice e stato fra classi strettamente imparentate ("è un tipo di")

Perché "programmare verso l'interfaccia, non verso l'implementazione". Se il codice dipende da un tipo concreto (`PostgresDatabase`), cambiare il database significa riscrivere tutti gli utilizzatori. Se dipende da un'interfaccia (`Repository`), si può sostituire l'implementazione, anche solo per i test, senza modificare il resto. È il **principio di inversione delle dipendenze**.

```
interface PrenotazioneRepository {
    void salva(String idPrenotazione);
    boolean esiste(String idPrenotazione);
}

class PostgresRepository implements PrenotazioneRepository {
    public void salva(String id) { /* INSERT INTO ... via JDBC */ }
    public boolean esiste(String id) { /* SELECT ... */ return false; }
}

class InMemoryRepository implements PrenotazioneRepository {   // per i test
    private final java.util.Set<String> dati = new java.util.HashSet<>();
    public void salva(String id) { dati.add(id); }
    public boolean esiste(String id) { return dati.contains(id); }
```

```
}

class ServizioPrenotazioni {
    private final PrenotazioneRepository repo;          // dipende dall'
        interfaccia
    ServizioPrenotazioni(PrenotazioneRepository repo) { // iniezione della
        dipendenza
        this.repo = repo;
    }
    void prenota(String id) {
        if (repo.esiste(id)) throw new IllegalStateException("gia' prenotata")
            ;
        repo.salva(id);
    }
}
```

Listing 11.3: Dipendere da un'interfaccia: sostituibilita e testabilita

Il servizio funziona con `PostgresRepository` in produzione e con `InMemoryRepository` nei test, *senza modificare una riga*. È la ragione per cui nel progetto del backend (capitolo *Progettazione di Sistemi e API*) il livello di accesso ai dati è un'interfaccia *Repository*.

11.5 Ereditarietà

L'**ereditarietà** permette a una **sottoclasse** di acquisire attributi e metodi della **superclasse**, estendendoli o ridefinendoli. Modella la relazione "*è un*".

Perché si usa

- **Riuso**: il codice comune sta in un solo posto (una correzione vale per tutte le sottoclassi).
- **Sottotipazione**: un oggetto della sottoclasse può essere usato dove ci si aspetta la superclasse; è la base del polimorfismo.

Rischi e quando evitarla

- **Accoppiamento forte**: la sottoclasse dipende dai dettagli della superclasse (problema della *fragile base class*: una modifica innocua alla base rompe le derivate).
- **Principio di sostituzione di Liskov** (LSP): una sottoclasse deve poter sostituire la superclasse *senza cambiare il comportamento atteso*. L'esempio classico è `Quadrato extends Rettangolo`: se `setLarghezza` nel quadrato cambia anche l'altezza, il codice che assume "cambio la larghezza e l'altezza resta uguale" si rompe. Una sottoclasse che non è davvero un sottotipo è un errore di progetto.
- **Gerarchie profonde** difficili da capire; **ereditarietà multipla** (problema del rombo).

Composizione al posto dell'ereditarietà (*"favor composition over inheritance"*). Relazione "*ha un*": l'oggetto *contiene* un altro oggetto e gli inoltra le chiamate.

```
// SBAGLIATO: Pila extends ArrayList<Integer>
//   espone anche add(indice, x), remove(indice)... che violano la regola LIFO
    .
```

```
3
4  class Pila {
5      private final java.util.ArrayDeque<Integer> elementi = new java.util.
           ArrayDeque<>();
6      public void push(int x) { elementi.addFirst(x); }
7      public int pop()        { return elementi.removeFirst(); }
8      public boolean vuota()  { return elementi.isEmpty(); }
9  }
```

Listing 11.4: Composizione: una pila non e' una lista

La pila *usa* un contenitore ma ne nasconde l'interfaccia: espone solo le operazioni dell'ADT Stack. L'implementazione è sostituibile senza che il chiamante se ne accorga.

11.6 Polimorfismo

Traccia 2026 (A1-INF-4): illustrare il polimorfismo, principi, finalità e vantaggi (modularità, estensibilità, riusabilità); le modalità di realizzazione e il ruolo tra interfacce e implementazioni; limiti e problematiche; un esempio applicativo nell'Ingegneria dell'Informazione, con motivazione delle scelte.

11.6.1 Definizione

Polimorfismo ("molte forme"): la capacità di **un'unica interfaccia** di riferirsi a oggetti di tipi diversi, ciascuno dei quali risponde alla stessa chiamata **a modo proprio**. Il codice chiamante scrive `x.leggi()` senza sapere *quale* classe concreta sia `x`: la scelta del metodo da eseguire avviene **a tempo di esecuzione** in base al tipo reale dell'oggetto.

11.6.2 Modalità di realizzazione

Tipo	Come funziona	Quando viene scelto il metodo
Di sottotipo (inclusione, *overriding*)	Una sottoclasse ridefinisce un metodo della superclasse o implementa un'interfaccia; un riferimento al tipo base punta a un oggetto derivato	**Run-time** (binding dinamico, *dynamic dispatch*)
Ad hoc (*overloading*)	Più metodi con lo stesso nome e parametri diversi	**Compile-time** (in base ai tipi *dichiarati* degli argomenti)
Parametrico (*generics*, *template*)	Lo stesso codice funziona per qualunque tipo: `List<T>`, `Comparable<T>`	Compile-time
Duck typing (linguaggi dinamici)	"Se cammina e fa qua come un'anatra...": conta che l'oggetto abbia il metodo richiesto, non il suo tipo dichiarato	Run-time

Come funziona il binding dinamico. Ogni classe con metodi ridefinibili ha una **tabella dei metodi virtuali** (*vtable*); ogni oggetto contiene un puntatore alla vtable della sua classe. Una chiamata polimorfica significa: seguire il puntatore, leggere la voce del metodo, saltare. Il costo è una *indirezione* in più.

```java
class Animale { String verso() { return "..."; } }
class Cane extends Animale { @Override String verso() { return "bau"; } }

class Demo {
    static String descrivi(Animale a) { return "animale"; }
    static String descrivi(Cane c)    { return "cane"; }

    public static void main(String[] args) {
        Animale a = new Cane();       // tipo dichiarato Animale, tipo reale Cane
        System.out.println(a.verso());      // "bau": overriding, scelto a RUN-TIME
        System.out.println(descrivi(a));    // "animale": overloading, scelto in base
                                            // al tipo DICHIARATO (compile-time)
    }
}
```

Listing 11.5: Overriding (run-time) contro overloading (compile-time)

11.6.3 Perché si fa: il problema che risolve

Senza polimorfismo, per gestire tipi diversi il codice usa **blocchi `if`/`switch` sul tipo**. Ogni nuovo tipo obbliga a modificare *tutti* quei blocchi, sparsi in tutto il programma.

```java
enum Tipo { CERCHIO, RETTANGOLO }

class Forma { Tipo tipo; double a, b; }      // a = raggio oppure base, b = altezza

static double area(Forma f) {
    switch (f.tipo) {
        case CERCHIO:     return Math.PI * f.a * f.a;
        case RETTANGOLO:  return f.a * f.b;
        default: throw new IllegalArgumentException();
    }
}
static double perimetro(Forma f) { /* un altro switch identico nella struttura */ return 0; }
// Aggiungere TRIANGOLO: bisogna modificare area, perimetro e ogni altra funzione
// con lo switch. Se se ne dimentica una: errore a run-time, non a compile-time.
```

Listing 11.6: Senza polimorfismo: switch sul tipo

```java
interface Forma {
    double area();
    double perimetro();
}
```

```
class Cerchio implements Forma {
    private final double r;
    Cerchio(double r) { this.r = r; }
    public double area()      { return Math.PI * r * r; }
    public double perimetro() { return 2 * Math.PI * r; }
}
class Rettangolo implements Forma {
    private final double b, h;
    Rettangolo(double b, double h) { this.b = b; this.h = h; }
    public double area()      { return b * h; }
    public double perimetro() { return 2 * (b + h); }
}

static double areaTotale(java.util.List<Forma> forme) {
    double s = 0;
    for (Forma f : forme) s += f.area();     // nessun if sul tipo
    return s;
}
// Aggiungere Triangolo: si scrive UNA nuova classe. areaTotale non cambia.
```

Listing 11.7: Con polimorfismo: ogni tipo conosce se stesso

Vantaggi

- **Estensibilità** (*principio aperto/chiuso*): il codice è *aperto all'estensione* (nuove classi) ma *chiuso alla modifica* (il codice esistente non si tocca). Aggiungere un tipo non rischia di rompere ciò che funzionava.
- **Modularità**: la logica specifica di ogni tipo sta *nella sua classe*; il chiamante è generico.
- **Riusabilità**: algoritmi scritti sull'interfaccia (ordinare, stampare, monitorare) funzionano con tutti i tipi presenti e futuri.
- **Testabilità**: si sostituiscono le implementazioni reali con *finte* (mock, stub); vedi il Repository in memoria.
- **Minore accoppiamento**: il chiamante conosce solo il contratto.

11.6.4 Ruolo di interfacce e implementazioni

L'interfaccia fissa il **contratto** (che cosa, con quali pre-condizioni e post-condizioni); le implementazioni concrete decidono il **come**. Il chiamante dipende solo dal contratto. Il polimorfismo è il meccanismo che, a tempo di esecuzione, collega la chiamata astratta all'implementazione giusta. Un'interfaccia ben progettata è **piccola e coesa** (*interface segregation*).

11.6.5 Esempio applicativo: monitoraggio di sensori

Un sistema embedded o IoT legge molti tipi di sensore; ciascuno ha una modalità di acquisizione e una conversione diversa. Il modulo di monitoraggio deve funzionare con sensori noti e futuri.

```
import java.util.List;

interface Sensore {
    String nome();
    double leggi();          // lettura gia' convertita in unita' fisica
```

```
    String unita();
}

class SensoreTemperatura implements Sensore {
    private final int adcRaw;                       // valore grezzo del
        convertitore
    SensoreTemperatura(int adcRaw) { this.adcRaw = adcRaw; }
    public String nome()  { return "Temperatura"; }
    public double leggi() { return adcRaw * 0.1 - 40.0; }   // conversione
        specifica
    public String unita() { return "C"; }
}

class SensoreUmidita implements Sensore {
    private final double frazione;                  // 0..1
    SensoreUmidita(double frazione) { this.frazione = frazione; }
    public String nome()  { return "Umidita"; }
    public double leggi() { return frazione * 100.0; }
    public String unita() { return "%"; }
}

class Monitor {                                     // conosce solo l'
    interfaccia Sensore
    static void stampa(List<Sensore> sensori) {
        for (Sensore s : sensori)
            System.out.println(s.nome() + ": " + s.leggi() + " " + s.unita());
    }
}
// Nuovo sensore (pressione, CO2...): basta una nuova classe che implementa
    Sensore.
// Monitor non si modifica e non si ricompila.
```

Listing 11.8: Sensori polimorfi: monitor generico

Scelte progettuali motivate

- **Interfaccia** e non classe base: i sensori non condividono codice, solo un *ruolo*; un'interfaccia non impone una gerarchia e una classe può implementarne più d'una.
- **Conversione dentro ogni classe**: il dettaglio che varia (curva di calibrazione) è nascosto; `Monitor` resta stabile.
- **Immutabilità** dei dati di lettura per semplificare l'uso concorrente.

Lo stesso esempio in Python (*duck typing*)

```
class SensoreTemperatura:
    def __init__(self, adc_raw): self.adc_raw = adc_raw
    def nome(self):  return "Temperatura"
    def leggi(self): return self.adc_raw * 0.1 - 40.0
    def unita(self): return "C"

class SensoreUmidita:
    def __init__(self, frazione): self.frazione = frazione
    def nome(self):  return "Umidita"
    def leggi(self): return self.frazione * 100.0
```

```
11      def unita(self): return "%"
12
13  def stampa(sensori):
14      for s in sensori:                  # basta che l'oggetto abbia questi
            metodi
15          print(f"{s.nome()}: {s.leggi()} {s.unita()}")
```

Listing 11.9: Polimorfismo in Python

In Python non serve dichiarare l'interfaccia: il contratto è *implicito* (duck typing), con il rischio che l'errore emerga solo a run-time; Python offre le classi astratte (`abc`) e i `Protocol` per renderlo esplicito.

11.6.6 Limiti e problematiche

- **Costo a run-time**: indirezione della vtable e impossibilità di alcune ottimizzazioni (*inlining*); trascurabile nella maggior parte dei casi, rilevante nei cicli critici (sistemi in tempo reale, calcolo numerico).
- **Comprensibilità e debug**: leggendo `s.leggi()` non si sa quale codice verrà eseguito; serve conoscere tutte le implementazioni.
- **Violazione del contratto**: una sottoclasse che non rispetta il comportamento atteso rompe il codice che usa il tipo base (violazione di Liskov), con errori difficili da trovare.
- **Esplosione di classi** se si abusa di gerarchie per piccole differenze (spesso meglio un parametro o la composizione).
- **Problema dell'espressione**: con l'OOP aggiungere un *nuovo tipo* è facile, ma aggiungere una *nuova operazione* a tutti i tipi richiede di modificare tutte le classi; con `switch` (o con tipi di dati algebrici) è il contrario. Il pattern **Visitor** è una risposta.
- **Downcast e `instanceof`**: se per usare un oggetto bisogna controllare il tipo concreto, il polimorfismo è stato progettato male.
- **Ereditarietà multipla** (problema del rombo) e fragilità della classe base.
- **Sicurezza**: i linguaggi dinamici rinviano molti errori di tipo a run-time.

11.6.7 Scaletta per il tema "polimorfismo"

1. Definizione e perché serve: gestire tipi diversi senza `if`/`switch`.
2. Principi: interfaccia, sottotipazione, binding dinamico (vtable).
3. Modalità: sottotipo (overriding), ad hoc (overloading), parametrico (generics), duck typing.
4. Finalità e vantaggi: modularità, estensibilità (aperto/chiuso), riusabilità, testabilità.
5. Ruolo di interfaccia e implementazioni (contratto, Liskov, dependency inversion).
6. Limiti: costo, comprensibilità, Liskov, esplosione di classi, problema dell'espressione.
7. Esempio: codice con interfaccia e almeno due implementazioni, e motivazione delle scelte.

11.7 Principi di Progettazione: SOLID

Cinque principi che riassumono il *perché* dell'OOP:

1. **S – Single Responsibility**: una classe ha *un solo motivo per cambiare*. Alta coesione.

2. **O – Open/Closed**: aperte all'estensione, chiuse alla modifica (polimorfismo).
3. **L – Liskov Substitution**: le sottoclassi devono poter sostituire le superclassi.
4. **I – Interface Segregation**: molte interfacce piccole, meglio di una enorme.
5. **D – Dependency Inversion**: dipendere da astrazioni, non da implementazioni concrete.

Altri principi: **DRY** (non ripetersi), **KISS** (semplicità), **YAGNI** (non costruire ciò che non serve ora), **legge di Demetra** (parlare solo con i vicini diretti). Per i *design pattern* (Strategy, Observer, Factory, Repository...) si veda il capitolo *Progettazione del Software*: sono applicazioni di questi principi.

11.8 Chiamate di Funzione, Stack e Passaggio dei Parametri

Traccia 2025 (E05): descrivere il processo di chiamata a funzione; ruolo dello stack nell'allocazione dei dati; distinzione tra parametri formali e attuali; conseguenze delle diverse modalità di passaggio dei parametri; esempi di codice.

11.8.1 Parametri formali e attuali

- **Parametri formali**: i nomi (con tipo) dichiarati nella *definizione* della funzione: `int somma(int a, int b)`. Sono variabili locali della funzione.
- **Parametri attuali** (argomenti): le espressioni passate nella *chiamata*: `somma(x, 3 + 4)`.
- La chiamata associa a ogni formale il corrispondente attuale; *come* avviene l'associazione è la **modalità di passaggio**.

11.8.2 Che cosa succede in una chiamata (record di attivazione)

Lo **stack** (pila di chiamata) contiene un **record di attivazione** (*stack frame*) per ogni funzione in esecuzione (vedi anche il capitolo *Sistemi Operativi*).

1. Il chiamante **valuta gli argomenti** e li mette sullo stack (o in registri, a seconda della convenzione di chiamata).
2. Si salva l'**indirizzo di ritorno** (l'istruzione successiva alla chiamata).
3. Si crea il **frame** della funzione: parametri formali, variabili locali, indirizzo di ritorno, puntatore al frame precedente.
4. Il controllo passa alla funzione, che accede alle proprie variabili tramite il frame.
5. Al `return`, il valore di ritorno è consegnato (registro o stack), il frame viene **rimosso** (si ripristina lo stack pointer) e l'esecuzione riprende all'indirizzo di ritorno.

Perché uno stack: le chiamate seguono una disciplina LIFO (l'ultima funzione chiamata è la prima a terminare), quindi allocare e liberare i frame è *estremamente veloce* (basta spostare un puntatore) e consente la **ricorsione** (ogni invocazione ha i propri dati) e il **rientro** (*reentrancy*, thread diversi). Il limite è la dimensione finita dello stack.

	Stack	**Heap**
Contiene	Frame delle chiamate: locali, parametri, indirizzi di ritorno	Oggetti e dati allocati dinamicamente (`new`, `malloc`)
Allocazione	Automatica, LIFO, molto veloce	Esplicita o gestita dal *garbage collector*, più lenta
Durata	Fino al ritorno della funzione	Fino a deallocazione o raccolta
Dimensione	Piccola e limitata (*stack overflow*)	Grande (limitata dalla memoria)
Errori tipici	Ricorsione infinita; array locali troppo grandi	Memory leak, puntatori pendenti (C/C++)

Esempio di stack con la ricorsione. Per `fact(3)` lo stack cresce fino a quattro frame ($n = 3, 2, 1$ più la chiamata iniziale) e poi si svuota in ordine inverso:

```
chiamate:                ritorni:
 fact(1)  -> n=1          fact(1) ritorna 1
 fact(2)  -> n=2          fact(2) ritorna 2 * 1 = 2
 fact(3)  -> n=3          fact(3) ritorna 3 * 2 = 6
 main     -> ...
```

Listing 11.10: Evoluzione dello stack per fact(3)

11.8.3 Modalità di passaggio dei parametri

Modalità	**Cosa riceve la funzione**	**Modifiche visibili al chiamante?**	**Linguaggi**
Per valore	Una **copia** del valore	No	C, Java (primitivi), Python, C++ (default)
Per riferimento	Un **alias** della variabile originale	Sì	C++ (`&`), C# (`ref`)
Per indirizzo (puntatore)	La copia di un **indirizzo**; si accede all'originale con `*p`	Sì (tramite il puntatore)	C, C++
Per valore del riferimento (condivisione)	Una copia del **riferimento** all'oggetto	L'oggetto *sì* (se mutabile); la variabile del chiamante *no*	Java (oggetti), Python
Per nome / per risultato	Casi storici o particolari (Algol, Ada `out`)	–	–

```cpp
#include <iostream>

void scambia_valore(int a, int b)      { int t = a; a = b; b = t; }   // copie
    : inutile
void scambia_rif(int& a, int& b)       { int t = a; a = b; b = t; }   // alias
    : funziona
void scambia_ptr(int* a, int* b)       { int t = *a; *a = *b; *b = t; }

int main() {
    int x = 1, y = 2;
    scambia_valore(x, y);  std::cout << x << " " << y << "\n";   // 1 2
    scambia_rif(x, y);     std::cout << x << " " << y << "\n";   // 2 1
    scambia_ptr(&x, &y);   std::cout << x << " " << y << "\n";   // 1 2
```

```
}
```

Listing 11.11: Valore, riferimento e puntatore in C++

```python
def aggiungi(lista):
    lista.append(99)          # modifica l'OGGETTO condiviso -> visibile fuori

def riassegna(lista):
    lista = [0]               # cambia solo la variabile LOCALE -> invisibile
        fuori

def incrementa(n):
    n += 1                    # int e' immutabile: crea un nuovo int locale

a = [1, 2, 3]
aggiungi(a);   print(a)       # [1, 2, 3, 99]
riassegna(a);  print(a)       # [1, 2, 3, 99]
k = 5
incrementa(k); print(k)       # 5
```

Listing 11.12: Python e Java: passaggio per valore del riferimento

Conseguenze e motivazioni

- **Per valore**: sicuro (la funzione non può alterare l'originale), ma costoso per dati grandi perché copia.
- **Per riferimento/puntatore**: efficiente (nessuna copia) e permette di **restituire più risultati**; ma introduce **effetti collaterali** e **aliasing**, e in C/C++ i rischi di puntatori nulli o pendenti. Si usa `const T&` per passare dati grandi senza copiarli e senza poterli modificare.
- **Oggetti mutabili** passati per "valore del riferimento": la funzione può modificare l'oggetto del chiamante; per evitare sorprese si usano oggetti **immutabili** o **copie difensive**.
- **Stack overflow**: ogni chiamata consuma uno stack frame; una ricorsione troppo profonda (o infinita) lo esaurisce.

11.9 Ricorsione e Iterazione

Traccia 2018: implementare `fact(x)` in versione iterativa e ricorsiva, discutendo le differenze.

```python
def fact_iter(x):
    if x < 0:
        raise ValueError("x deve essere >= 0")
    r = 1
    for i in range(2, x + 1):      # un ciclo, una sola variabile accumulatore
        r *= i
    return r

def fact_rec(x):
    if x < 0:
        raise ValueError("x deve essere >= 0")
```

```
    if x <= 1:                      # caso base: ferma la ricorsione
        return 1
    return x * fact_rec(x - 1)      # passo ricorsivo su un problema piu'
        piccolo
```

Listing 11.13: Fattoriale: versione iterativa e ricorsiva

	Iterativa	**Ricorsiva**
Idea	Ripete un blocco con un ciclo e variabili che si aggiornano	La funzione richiama se stessa su un sottoproblema
Tempo	$O(n)$	$O(n)$
Spazio	$O(1)$	$O(n)$: un frame per ogni chiamata
Rischio	Errori sugli indici e sullo stato	**Stack overflow** (limite di Python $\approx$ 1000 chiamate; `fact_rec(5000)` fallisce), caso base mancante
Leggibilità	Chiara per problemi lineari	Più vicina alla definizione matematica; ottima per strutture ricorsive

Quando preferire la ricorsione. Quando il problema è *definito in modo ricorsivo* o i dati sono ricorsivi: **alberi**, grafi, espressioni, **divide et impera** (merge sort, ricerca binaria), backtracking (labirinti, N-regine). In questi casi l'iterativa richiederebbe uno stack esplicito.

Quando preferire l'iterazione. Per cicli semplici, per ridurre memoria e rischio di overflow, e nei sistemi con stack limitato (embedded).

Ricorsione in coda. Se la chiamata ricorsiva è l'*ultima* operazione, un compilatore può riusare lo stesso frame (*tail call optimization*) e rendere la ricorsione efficiente quanto un ciclo; Java e Python *non* lo fanno.

Ogni ricorsione si può trasformare in iterazione (con uno stack esplicito), e viceversa.

11.9.1 Attenzione alla ricorsione ridondante: Fibonacci

```
from functools import lru_cache

def fib_naive(n):                   # O(2^n): ricalcola gli stessi valori
    return n if n < 2 else fib_naive(n - 1) + fib_naive(n - 2)

@lru_cache(maxsize=None)
def fib_memo(n):                    # O(n): ogni valore calcolato una sola
    volta
    return n if n < 2 else fib_memo(n - 1) + fib_memo(n - 2)

def fib_iter(n):                    # O(n) tempo, O(1) spazio
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

Listing 11.14: Fibonacci: ricorsione ingenua, con memoizzazione, iterativa

Il caso ingenuo ricalcola ripetutamente gli stessi sottoproblemi: `fib_naive(40)` richiede oltre cento milioni di chiamate. È l'idea alla base della **programmazione dinamica** (memoizzazione dei sottoproblemi sovrapposti).

11.10 Complessità degli Algoritmi

11.10.1 Notazioni

- $O(f(n))$: **limite superiore** asintotico (al più così). $\Omega(f(n))$: limite inferiore. $\Theta(f(n))$: limite stretto (sia superiore sia inferiore).
- Si analizza il **caso peggiore**, il **caso migliore** e il **caso medio**; di solito si riporta il caso peggiore.
- Si ignorano **costanti** e **termini di ordine inferiore**: $3n^2 + 10n + 7 = O(n^2)$.

11.10.2 Come calcolarla

- **Cicli**: un ciclo che scorre n elementi costa $O(n)$; due cicli annidati $O(n^2)$; tre cicli indipendenti annidati di lunghezze W, H, D costano $O(W \cdot H \cdot D)$.
- **Sequenza**: i costi si sommano e domina il maggiore.
- **Ricorsione**: si scrive la **relazione di ricorrenza**. Per il divide et impera $T(n) = a\,T(n/b) + f(n)$ vale il **teorema dell'esperto** (*master theorem*), con $c = \log_b a$:
  - se $f(n) = O(n^{c-\varepsilon})$: $T(n) = \Theta(n^c)$;
  - se $f(n) = \Theta(n^c)$: $T(n) = \Theta(n^c \log n)$;
  - se $f(n) = \Omega(n^{c+\varepsilon})$ (con regolarità): $T(n) = \Theta(f(n))$.

  Esempi: **ricerca binaria** $T(n) = T(n/2) + O(1) \Rightarrow \Theta(\log n)$; **merge sort** $T(n) = 2T(n/2) + O(n) \Rightarrow \Theta(n \log n)$.

Classe	Esempio	$n = 10^6$ (ordine di grandezza)
$O(1)$	Accesso a un array, a una tabella hash (caso medio)	immediato
$O(\log n)$	Ricerca binaria	≈ 20 operazioni
$O(n)$	Scansione lineare	10^6
$O(n \log n)$	Merge sort, heap sort	$\approx 2 \cdot 10^7$
$O(n^2)$	Insertion sort, doppio ciclo	10^{12} (ore)
$O(2^n)$	Fibonacci ingenuo, sottoinsiemi	intrattabile

11.11 Ricerca e Ordinamento

11.11.1 Ricerca binaria

Su un array *ordinato*: si confronta con l'elemento centrale e si scarta metà dell'intervallo. $O(\log n)$ tempo; $O(1)$ spazio (iterativa) o $O(\log n)$ (ricorsiva, per lo stack). Richiede accesso diretto agli indici e dati ordinati.

```
def ricerca_binaria(a, x):              # a ordinato crescente
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == x:   return mid
        elif a[mid] < x:  lo = mid + 1
```

```
        else:             hi = mid - 1
    return -1                          # non trovato

def ricerca_binaria_rec(a, x, lo=0, hi=None):
    if hi is None: hi = len(a) - 1
    if lo > hi: return -1
    mid = (lo + hi) // 2
    if a[mid] == x:  return mid
    if a[mid] < x:   return ricerca_binaria_rec(a, x, mid + 1, hi)
    return ricerca_binaria_rec(a, x, lo, mid - 1)
```

Listing 11.15: Ricerca binaria iterativa e ricorsiva

11.11.2 Algoritmi di ordinamento

Algoritmo	Migliore	Medio	Peggiore	Spazio	Stabile
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sì
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sì
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sì
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Sì

Un ordinamento è **stabile** se mantiene l'ordine relativo degli elementi uguali. Ogni ordinamento *per confronti* richiede $\Omega(n \log n)$ confronti nel caso peggiore; il *counting sort* lo supera sfruttando il fatto che le chiavi siano interi in un intervallo limitato.

```
def merge_sort(a):
    if len(a) <= 1:                    # caso base
        return a
    mid = len(a) // 2
    sx, dx = merge_sort(a[:mid]), merge_sort(a[mid:])   # divide
    out, i, j = [], 0, 0
    while i < len(sx) and j < len(dx):                  # fonde due liste
        ordinate
        if sx[i] <= dx[j]:             # <= mantiene la stabilita'
            out.append(sx[i]); i += 1
        else:
            out.append(dx[j]); j += 1
    return out + sx[i:] + dx[j:]
```

Listing 11.16: Merge sort (divide et impera)

11.12 Strutture Dati e Grafi

11.12.1 Complessità delle operazioni

Struttura	Accesso/ricerca	Inserimento	Cancellazione	Note
Array	$O(1)$ per indice; $O(n)$ per valore	$O(n)$	$O(n)$	Memoria contigua
Lista concatenata	$O(n)$	$O(1)$ in testa	$O(1)$ se si ha il nodo	Nessun accesso diretto
Pila / coda	$O(1)$ in cima / in testa	$O(1)$	$O(1)$	LIFO / FIFO
Tabella hash	$O(1)$ medio, $O(n)$ peggiore	$O(1)$ medio	$O(1)$ medio	Non ordinata
Albero binario di ricerca (bilanciato)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$ se degenere
Heap (min/max)	$O(1)$ per il minimo	$O(\log n)$	$O(\log n)$	Coda con priorità

Perché tante strutture: ognuna rende veloci alcune operazioni a scapito di altre; la scelta dipende da *quali operazioni sono più frequenti*. Nasconderne l'implementazione dietro un ADT (interfaccia `Mappa`, `Pila`) permette di cambiarla senza modificare il resto.

11.12.2 Visite di grafi e cammini minimi

Un grafo $G = (V, E)$ è rappresentato con **liste di adiacenza** (spazio $O(V + E)$, adatto ai grafi sparsi) o **matrice di adiacenza** (spazio $O(V^2)$, test di un arco in $O(1)$).

- **BFS** (ampiezza): usa una coda; trova i cammini con meno archi; $O(V + E)$.
- **DFS** (profondità): usa lo stack (o la ricorsione); rileva cicli, componenti connesse, ordinamento topologico; $O(V + E)$.
- **Dijkstra**: cammini minimi da una sorgente con pesi non negativi; con un heap $O((V + E) \log V)$. È la *ricerca a costo uniforme* del capitolo di Intelligenza Artificiale.

```python
from collections import deque
import heapq

def bfs(grafo, s):                          # grafo: dict nodo -> lista di vicini
    dist, coda = {s: 0}, deque([s])
    while coda:
        u = coda.popleft()
        for v in grafo[u]:
            if v not in dist:
                dist[v] = dist[u] + 1
                coda.append(v)
    return dist

def dijkstra(grafo, s):                     # grafo: dict nodo -> lista di (
    vicino, peso)
    dist, heap = {s: 0}, [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist.get(u, float("inf")):
```

```
            continue                        # voce obsoleta
        for v, w in grafo[u]:
            nd = d + w
            if nd < dist.get(v, float("inf")):
                dist[v] = nd
                heapq.heappush(heap, (nd, v))
    return dist
```

Listing 11.17: BFS e Dijkstra su liste di adiacenza

11.12.3 Paradigmi algoritmici

- **Divide et impera**: divide il problema in sottoproblemi indipendenti, risolve e ricombina (merge sort, ricerca binaria).
- **Programmazione dinamica**: sottoproblemi *sovrapposti* risolti una sola volta (Fibonacci, zaino, cammini minimi).
- **Greedy**: a ogni passo sceglie la scelta localmente migliore (Dijkstra, codici di Huffman); corretto solo se il problema ha la proprietà della scelta greedy.
- **Backtracking**: prova le scelte e torna indietro se non funzionano (N-regine, sudoku).
- **Classi P e NP**: P = risolvibili in tempo polinomiale; NP = soluzioni verificabili in tempo polinomiale; i problemi **NP-completi** (SAT, commesso viaggiatore) non hanno algoritmi polinomiali noti e si affrontano con euristiche e approssimazioni.

11.13 Tema Svolto: Corrispondenze tra Due Immagini (2018)

Traccia: date due immagini A e B di dimensione $W \times H$ (pixel con valori 0–255), per ogni pixel $A[x][y]$ cercare in B il pixel più simile nella posizione $[x-d][y]$ con $d \in [0, D]$. Due pixel sono tanto più simili quanto più bassa è la differenza in valore assoluto; se $x - d$ esce dall'immagine, la differenza è infinita. L'output è un'immagine C in cui ogni pixel contiene il valore d che dà la differenza minore. Indicare la complessità in tempo e memoria.

(È il calcolo della *mappa di disparità* in visione stereoscopica.)

```
def corrispondenze(A, B, D):
    """A, B: matrici H x W (liste di liste); ritorna C con lo spostamento d
       ottimo."""
    H, W = len(A), len(A[0])
    C = [[0] * W for _ in range(H)]
    for y in range(H):                          # per ogni riga
        for x in range(W):                      # per ogni pixel di A
            migliore = float("inf")
            d_ottimo = 0
            for d in range(D + 1):              # per ogni candidato
                if x - d < 0:                   # fuori immagine: differenza
                    infinita
                    break                       # (e lo e' anche per ogni d
                        maggiore)
                diff = abs(A[y][x] - B[y][x - d])
```

```
13                if diff < migliore:              # < : a parita' vince il d
                      piu' piccolo
14                    migliore, d_ottimo = diff, d
15            C[y][x] = d_ottimo
16    return C
```

Listing 11.18: Mappa di disparita: corrispondenze tra due immagini

Spiegazione e scelte

- Tre cicli annidati: righe (H), colonne (W), candidati ($D+1$). Per ogni pixel si tiene il minimo corrente.
- Il `break` quando $x-d<0$ è corretto perché per d maggiori anche $x-d$ è negativo (differenza infinita per tutti); evita confronti inutili ma non cambia la complessità asintotica.
- In caso di parità si sceglie il d minore (la condizione è $<$, non $\leq$): va dichiarato come assunzione.

Complessità

- **Tempo**: $O(W \cdot H \cdot D)$, perché per ognuno dei $W \cdot H$ pixel si valutano al più $D+1$ candidati con costo costante.
- **Memoria**: $O(W \cdot H)$ per l'immagine di uscita C (oltre alle due immagini di ingresso, che sono già date); la memoria *ausiliaria* è $O(1)$ oltre all'output (solo `migliore`, `d_ottimo` e gli indici).

Possibili miglioramenti (da citare)

- Confrontare **finestre** di pixel (blocchi $k \times k$) invece di singoli pixel, per ridurre il rumore: il costo diventa $O(W \cdot H \cdot D \cdot k^2)$; con somme cumulate (*integral image*) si torna a $O(W \cdot H \cdot D)$.
- **Parallelizzazione**: ogni pixel è indipendente $\Rightarrow$ si può calcolare su GPU o con thread.

11.14 Checklist per un Tema di Informatica

- **Definire**, poi spiegare *perché* serve (problema risolto, che cosa accadrebbe senza).
- **Un esempio in codice** piccolo ma completo e corretto; commentare le scelte.
- Ogni scelta di progetto con la sua **motivazione** (incapsulamento $\rightarrow$ invarianti; interfaccia $\rightarrow$ sostituibilità; immutabilità $\rightarrow$ sicurezza).
- **Vantaggi e limiti**: mostrare che si conoscono i compromessi.
- **Complessità** dichiarata in tempo e spazio, con la motivazione.
- Dichiarare le **assunzioni** (parità, valori non validi, input vuoto) e gestire i **casi limite**.

Tracce Uscite

Prima sessione 2016 – Informatica

Traccia: concetto di classe, di oggetto e differenze nella programmazione ad oggetti; esempio in pseudocodice o in un linguaggio a scelta di una classe `Rational` (numeratore e denominatore interi) con funzione `mult` (moltiplicazione per un intero) che ritorna un nuovo oggetto `Rational`.

Dove trovare la risposta: sezione *Classi e Oggetti*, con la classe `Rational` completa e le motivazioni (immutabilità, invarianti).

Prima sessione 2018 – Informatica (prima prova)

Traccia: implementare `fact(x)`, una versione iterativa e una ricorsiva, discutendo le differenze tra i due approcci.

Dove trovare la risposta: sezione *Ricorsione e Iterazione*.

Prima sessione 2018 – Informatica (seconda prova)

Traccia: algoritmo di corrispondenza tra due immagini $W \times H$, con ricerca dello spostamento $d \in [0, D]$; complessità in tempo e memoria.

Dove trovare la risposta: sezione *Tema Svolto: Corrispondenze tra Due Immagini*.

Seconda sessione 2018 – Informatica

Traccia: modello ISO/OSI con ogni livello e confronto con TCP/IP.

Dove trovare la risposta: capitolo *Reti di Calcolatori*.

Prima sessione 2024 – Informatica #3

Traccia: ricerca informata e non informata, con efficienza, memoria e ottimalità.

Dove trovare la risposta: capitolo *Intelligenza Artificiale e Machine Learning*; per BFS, DFS e Dijkstra vedi la sezione *Strutture Dati e Grafi*.

Prima sessione 2025 (Bologna) – Ing. Informatica E05

Traccia: processo di chiamata a funzione; ruolo dello stack nell'allocazione dei dati; distinzione semantica tra parametri formali e attuali; conseguenze delle diverse modalità di passaggio dei parametri; esempi di codice in un linguaggio a piacere o pseudocodice.

Dove trovare la risposta: sezione *Chiamate di Funzione, Stack e Passaggio dei Parametri*.

Prima sessione 2026 – A1-INF-4

Traccia: polimorfismo nella programmazione orientata agli oggetti: principi, finalità e vantaggi (modularità, estensibilità, riusabilità); modalità di realizzazione e ruolo nell'interazione tra interfacce e implementazioni; limiti e problematiche; esempio applicativo nell'Ingegneria dell'Informazione con motivazione delle scelte progettuali.

Dove trovare la risposta: sezione *Polimorfismo*, con la scaletta di risposta e l'esempio dei sensori.

Bologna (sessione non indicata nel file) – Ing. Informatica E04

Traccia: concetti di processo e thread in un sistema operativo moderno; differenze; modelli di multithreading (many-to-one, one-to-one, many-to-many); problematiche di concorrenza come le race condition.

Dove trovare la risposta: capitolo *Sistemi Operativi*, sezione *Processi e Thread*; per le race condition e la sincronizzazione vedi anche la sezione *Concorrenza e Integrità* del capitolo *Progettazione di Sistemi e API*.

Capitolo 12

Sistemi Operativi e Reti: Approfondimenti per la Sezione A

Questo capitolo completa i capitoli *Sistemi Operativi* e *Reti di Calcolatori* con gli argomenti usciti nella Sezione A o tipici delle prove di Ingegneria Informatica: processi, thread e concorrenza con esempi pratici, sostituzione delle pagine, il concetto di protocollo, lo stack TCP/IP nel dettaglio, routing, DNS e programmazione con i socket.

Parte I – Sistemi Operativi

12.1 Modalità Utente e Kernel, Chiamate di Sistema, Interrupt

12.1.1 Perché due modalità

Se ogni programma potesse eseguire qualsiasi istruzione e leggere qualsiasi indirizzo, un'applicazione difettosa o ostile potrebbe corrompere gli altri processi o il sistema stesso. La CPU distingue quindi due modalità (un bit nel registro di stato):

- **Modalità utente**: i programmi applicativi; istruzioni privilegiate (accesso diretto all'hardware, gestione della memoria, disabilitare gli interrupt) *vietate*.
- **Modalità kernel** (supervisore): il nucleo del sistema operativo; accesso completo.

Il **perché** è la *protezione*: l'hardware impone l'isolamento, il sistema operativo ne è il garante.

12.1.2 Chiamate di sistema (system call)

Un programma utente chiede un servizio al kernel (aprire un file, creare un processo, inviare dati in rete) con una **system call**:

1. il programma chiama una funzione di libreria (per esempio `read`);
2. la libreria esegue un'istruzione di **trap** che commuta la CPU in modalità kernel e salta a un indirizzo *fissato dal sistema operativo*;
3. il kernel verifica i parametri e il permesso, esegue il servizio;
4. ritorno in modalità utente e ripresa del programma.

Il programma non può saltare a un punto qualsiasi del kernel: entra solo dalle porte previste.

12.1.3 Interrupt, eccezioni e trap

- **Interrupt**: segnale *asincrono* da un dispositivo (disco, rete, timer). La CPU salva lo stato, esegue la routine di servizio (*ISR*) e riprende.
- **Eccezione**: evento *sincrono* causato dall'istruzione (divisione per zero, accesso non valido, **page fault**).
- **Trap**: eccezione *voluta* (system call).
- **Timer interrupt**: permette al sistema operativo di riprendere il controllo della CPU, e quindi la **prelazione** (*preemption*) nello scheduling.

12.2 Processi: Stati, PCB e Cambio di Contesto

12.2.1 Stati di un processo

- **Nuovo** → **Pronto** (*ready*: attende la CPU) ↔ **In esecuzione** → **In attesa** (*waiting*: attende un evento, come un'operazione di I/O) → **Pronto** → … → **Terminato**.
- Transizioni: *dispatch* (pronto → esecuzione), *prelazione* per scadenza del quanto (esecuzione → pronto), *richiesta di I/O* (esecuzione → attesa), *completamento dell'I/O* (attesa → pronto).

12.2.2 Process Control Block (PCB)

Struttura dati del kernel che descrive il processo: identificatore (PID), stato, **program counter** e registri, informazioni di scheduling (priorità), informazioni sulla memoria (tabella delle pagine), file aperti, informazioni di contabilità.

12.2.3 Cambio di contesto (context switch)

Passaggio della CPU da un processo a un altro: si **salva lo stato** (registri, PC) nel PCB del processo uscente e si **carica** quello dell'entrante.

- È **puro overhead**: durante il cambio non si esegue lavoro utile. Costo tipico di qualche microsecondo, aumentato dalla perdita di contenuto di cache e TLB.
- Per i **thread dello stesso processo** è più economico: non cambia lo spazio di indirizzamento.
- È il **compromesso dello scheduling**: un quanto piccolo dà reattività, ma più cambi di contesto e meno lavoro utile.

12.2.4 Creazione di processi: `fork`, `exec`, `wait`

In Unix un processo nasce *duplicando* il genitore (`fork`); il figlio può poi *sostituire* il proprio programma con un altro (`exec`); il genitore attende con `wait`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

```

```c
int main(void) {
    int x = 10;
    pid_t pid = fork();              // crea una copia del processo
    if (pid < 0) {                   // errore
        perror("fork");
        exit(1);
    } else if (pid == 0) {           // codice del FIGLIO (fork ritorna 0)
        x += 5;                      // modifica la SUA copia di x
        printf("figlio: x = %d\n", x);
        fflush(stdout);              // svuota il buffer: exec lo scarterebbe
        execlp("echo", "echo", "il figlio ora esegue un altro programma", (
            char *)NULL);
        perror("exec");              // si arriva qui solo se exec fallisce
        exit(1);
    } else {                         // codice del PADRE (fork ritorna il pid
        del figlio)
        wait(NULL);                  // attende la terminazione del figlio
        printf("padre: x = %d\n", x);   // x e' ancora 10: spazi di
            indirizzamento separati
    }
    return 0;
}
```

Listing 12.1: fork, exec e wait in C

Dopo `fork` i due processi hanno **spazi di indirizzamento separati**: la modifica di `x` nel figlio non è visibile al padre. È l'isolamento tra processi.

12.2.5 Comunicazione tra processi (IPC)

Poiché i processi sono isolati servono meccanismi espliciti:

Meccanismo	Descrizione	Note
Memoria condivisa	Un'area di memoria mappata in più processi	Molto veloce (nessuna copia, nessun intervento del kernel dopo la creazione); *richiede sincronizzazione* esplicita
Scambio di messaggi (message passing)	Il kernel recapita messaggi: `send`/`receive`	Più lento (copie e system call), ma più semplice e sicuro, funziona anche tra macchine diverse
Pipe / FIFO	Canale unidirezionale di byte	Semplice; pipe tra processi imparentati, FIFO con nome
Code di messaggi	Messaggi strutturati in coda	Disaccoppia mittente e destinatario
Socket	Comunicazione bidirezionale locale o di rete	Base delle applicazioni client/server
Segnali	Notifica asincrona minima (`SIGTERM`, `SIGINT`)	Trasporta pochissima informazione

12.3 Thread e Modelli di Multithreading

12.3.1 Perché i thread

- **Reattività**: l'interfaccia grafica continua a rispondere mentre un altro thread lavora.

- **Condivisione delle risorse**: i thread di un processo condividono codice, dati e file aperti; la comunicazione è una semplice lettura di memoria.
- **Economia**: creare un thread e passare da uno all'altro costa molto meno che con i processi.
- **Parallelismo** sui sistemi multicore: thread diversi su core diversi.
- Il prezzo è la **perdita di isolamento**: un thread difettoso può danneggiare tutto il processo, e la memoria condivisa introduce le *race condition*.

12.3.2 Thread a livello utente e a livello kernel

- **Thread utente**: gestiti da una libreria nello spazio utente; il kernel *non sa* che esistono. Creazione e cambio rapidissimi, ma una chiamata bloccante di un thread blocca tutto il processo e non usano più core.
- **Thread kernel**: creati e schedulati direttamente dal sistema operativo; il kernel può eseguirli in parallelo su più core e un thread bloccato non ferma gli altri; costo maggiore (system call).

12.3.3 I tre modelli di multithreading

Il modello descrive come i thread utente sono *mappati* sui thread kernel.

	Many-to-one	**One-to-one**	**Many-to-many**
Mappatura	Molti thread utente su *un* thread kernel	Ogni thread utente ha un thread kernel	m thread utente su $n \leq m$ thread kernel
Parallelismo multicore	**No** (un solo thread kernel)	**Sì**	Sì (fino a n)
Chiamata bloccante	Blocca *tutti* i thread del processo	Blocca solo quel thread	Blocca solo quel thread; il sistema può riassegnare
Costo di creazione	Molto basso	Più alto (un thread kernel per thread)	Basso: i thread utente sono economici
Numero di thread	Molto elevato	Limitato dal kernel (risorse)	Molto elevato
Complessità	Semplice	Semplice	**Alta** (scheduler a due livelli)
Esempi	Vecchie librerie (per esempio i *green thread* delle prime JVM)	**Linux, Windows, macOS**, Java (thread della piattaforma)	Go (*goroutine*), Java 21 (*virtual thread*), Erlang

Perché esistono tutti e tre

- **Many-to-one** è economico, ma non sfrutta i multicore e un'operazione bloccante ferma tutto: oggi è raro.
- **One-to-one** è il modello più diffuso per la sua semplicità e perché sfrutta il parallelismo; il limite è che *ogni thread costa un thread kernel* (memoria dello stack, risorse del kernel), quindi non si possono avere milioni di thread.
- **Many-to-many** cerca il meglio dei due: moltissimi thread leggeri multiplati su pochi thread kernel (adatto a server con milioni di connessioni), a costo di uno scheduler più complesso.

Thread pool. Creare un thread per ogni richiesta è costoso e illimitato (rischio di esaurire le risorse). Un *pool* crea in anticipo un numero fisso di thread che prendono le richieste da una coda: riuso dei thread, limite al carico e prevedibilità.

Nota su Python. L'implementazione di riferimento (CPython) ha un **GIL** (*Global Interpreter Lock*): un solo thread alla volta esegue bytecode Python. I thread aiutano per l'I/O, ma non per i calcoli intensivi, per i quali si usano più *processi*.

12.4 Race Condition: dall'Esempio alla Soluzione

Una **race condition** si ha quando il risultato dipende dall'*ordine temporale* di accessi concorrenti a dati condivisi, di cui almeno uno in scrittura.

12.4.1 Perché `contatore++` non è sicuro

L'istruzione `contatore++` non è un'operazione sola, ma tre: **leggere** il valore, **sommare** 1, **scrivere** il risultato. Se due thread si alternano in mezzo, un aggiornamento viene perso (*lost update*):

Passo	Thread A	Thread B	contatore
1	legge 5		5
2		legge 5	5
3	calcola 6	calcola 6	5
4	scrive 6		6
5		scrive 6	**6** (atteso: 7)

```java
import java.util.concurrent.atomic.AtomicInteger;

public class Race {
    static int contatore = 0;                       // condiviso, NON protetto
    static int sicuro = 0;
    static final Object lock = new Object();
    static AtomicInteger atomico = new AtomicInteger();

    public static void main(String[] args) throws Exception {
        Runnable r = () -> {
            for (int i = 0; i < 1_000_000; i++) {
                contatore++;                        // lettura, somma, scrittura: 3 passi
                synchronized (lock) { sicuro++; }   // sezione critica (mutua esclusione)
                atomico.incrementAndGet();          // operazione atomica (CAS)
            }
        };
        Thread t1 = new Thread(r), t2 = new Thread(r);
        t1.start(); t2.start();
        t1.join();  t2.join();
        System.out.println(contatore + " " + sicuro + " " + atomico.get());
    }
}
```

```
// Risultato tipico (cambia ad ogni esecuzione):  1999973  2000000  2000000
```

Listing 12.2: Race condition e tre rimedi in Java

Il contatore non protetto restituisce un valore **inferiore e diverso a ogni esecuzione**: è questo il segno delle race condition. Sono bug *non deterministici*: possono sparire con il debugger o con un carico diverso.

12.4.2 Rimedi

- **Mutua esclusione** (lock, `synchronized`, mutex): una sola esecuzione per volta nella sezione critica. Semplice, ma serializza il lavoro (*contesa*) e introduce il rischio di **deadlock**.
- **Operazioni atomiche** (*compare-and-swap*): l'hardware garantisce l'esecuzione indivisibile; senza blocchi, ottime per contatori e flag.
- **Immutabilità**: dati che non cambiano non possono avere race condition (vedi `Rational` nel capitolo sulla programmazione).
- **Confinamento**: ogni dato appartiene a un solo thread; si evita la condivisione.
- **Scambio di messaggi** tra thread o processi, senza stato condiviso (modello degli *attori*, code).
- **Strutture dati concorrenti** della libreria (`ConcurrentHashMap`, `BlockingQueue`).

Attenzione: proteggere tutto con un unico lock risolve le race ma azzera il parallelismo; il lavoro sta nel proteggere *solo* ciò che serve, per il tempo minimo, e acquisire più lock **sempre nello stesso ordine** per evitare deadlock.

12.4.3 Problemi classici di sincronizzazione

Problema	Descrizione	Soluzione
Produttore–consumatore	Un buffer limitato condiviso	Semafori `empty`, `full`, `mutex` (vedi capitolo SO) oppure una coda bloccante
Lettori–scrittori	Molti lettori possono leggere insieme; uno scrittore vuole accesso esclusivo	Lock lettore/scrittore; attenzione alla **starvation** di scrittori o lettori
Cena dei filosofi	Cinque filosofi, cinque forchette; per mangiare ne servono due $\Rightarrow$ rischio di deadlock (attesa circolare)	Imporre un **ordine** sulle forchette; al massimo quattro filosofi a tavola; prendere le due forchette in modo atomico

12.5 Sostituzione delle Pagine

Nella memoria virtuale con paginazione su richiesta, quando un processo accede a una pagina non presente in RAM si verifica un **page fault**: il sistema operativo carica la pagina dal disco in un frame libero; se non ce ne sono, deve scegliere una pagina *vittima* da rimuovere. **Obiettivo**: minimizzare il numero di page fault.

12.5.1 Costo di un page fault

Con probabilità di fault p, tempo di accesso in memoria t_m e tempo di servizio del fault t_f ($\approx$ ms, perché richiede il disco), il **tempo medio di accesso** è

$$\mathrm{EAT} = (1 - p)\, t_m + p\, t_f.$$

Con $t_m = 200\,\text{ns}$, $t_f = 8\,\text{ms}$ e $p = 1/1000$: EAT $= 0{,}999 \cdot 200 + 0{,}001 \cdot 8\,000\,000 \approx 8200\,\text{ns}$: **un fault ogni mille accessi rallenta il sistema di circa 40 volte**. Per questo i page fault devono essere rarissimi.

12.5.2 Algoritmi

- **FIFO**: si rimuove la pagina caricata da più tempo. Semplice, ma può rimuovere pagine molto usate; soffre dell'**anomalia di Belady**.
- **OPT** (ottimo): si rimuove la pagina che sarà usata *più in là nel futuro*. Dà il minimo numero di fault, ma **non è realizzabile** (richiede di conoscere il futuro); serve come limite di confronto.
- **LRU** (*Least Recently Used*): si rimuove la pagina non usata da più tempo, assumendo la **località temporale**. Buona approssimazione di OPT, ma richiede hardware per tracciare l'uso (costoso).
- **Clock** (seconda chance): approssimazione di LRU con un bit di riferimento e un puntatore circolare; è molto usata in pratica.

12.5.3 Esempio svolto

Stringa di riferimenti: `7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1` (20 accessi) con **3 frame**.

Algoritmo	Page fault
FIFO	15
LRU	12
OPT	9

Metodo per risolverlo a mano: si scorre la stringa tenendo la lista dei frame; a ogni accesso, se la pagina è presente è un *hit*, altrimenti è un *fault* e, se i frame sono pieni, si applica la regola dell'algoritmo per scegliere la vittima (FIFO: la più vecchia; LRU: quella usata meno di recente; OPT: quella il cui prossimo uso è più lontano).

Anomalia di Belady. Con FIFO, *aumentare* i frame può *aumentare* i fault. Stringa `1 2 3 4 1 2 5 1 2 3 4 5`: FIFO ha 9 fault con 3 frame e **10** con 4 frame. LRU e OPT non soffrono di questa anomalia (sono *algoritmi a stack*): con LRU, 10 fault con 3 frame e 8 con 4.

12.5.4 Thrashing e working set

Se un processo non ha abbastanza frame per il proprio *working set* (l'insieme delle pagine usate nell'intervallo recente) genera page fault continui: il sistema passa quasi tutto il tempo a caricare pagine e quasi nessuno a lavorare (**thrashing**). Rimedi: ridurre il grado di multiprogrammazione, assegnare i frame in base al working set, politiche di sostituzione locali.

12.5.5 Dimensione della tabella delle pagine

Spazio di indirizzamento di 32 bit, pagine da 4 KiB (2^{12} byte): numero di pagine $= 2^{32}/2^{12} = 2^{20}$. Con 4 byte per voce la tabella occupa $2^{20} \cdot 4 = 4\,\text{MiB}$ **per processo**. Soluzioni: **tabelle a più livelli** (si allocano solo le parti usate), **tabella invertita**, **TLB** per accelerare la traduzione.

Parte II – Reti di Calcolatori

12.6 Il Concetto di Protocollo

Traccia 2026 (A2-INF-4): illustrare il concetto di protocollo nei sistemi distribuiti, il ruolo nel garantire interoperabilità, affidabilità e sincronizzazione; funzioni di un protocollo; modelli orientato e non orientato alla connessione; esempi dallo stack TCP/IP e impiego nei sistemi distribuiti e nelle applicazioni Internet.

12.6.1 Definizione

Un **protocollo di comunicazione** è l'insieme di **regole** che le entità comunicanti (*peer*) devono seguire per scambiarsi informazioni. Definisce:

- la **sintassi**: formato e struttura dei messaggi (campi, lunghezze, codifica);
- la **semantica**: il significato dei messaggi e le azioni da compiere;
- la **temporizzazione**: ordine degli scambi, velocità, timeout, sincronizzazione.

Un protocollo si descrive con il formato delle **PDU** (*Protocol Data Unit*: intestazione + dati) e con una **macchina a stati finiti** (stati, messaggi ricevuti, azioni).

12.6.2 Perché servono i protocolli

Due calcolatori diversi, di produttori e sistemi operativi diversi, collegati da un canale *non affidabile* e *senza orologio comune*, non possono comunicare senza regole concordate. Il protocollo risolve tre problemi:

- **Interoperabilità**: sistemi eterogenei si capiscono perché implementano lo *stesso standard* (RFC dell'IETF, standard IEEE). Un browser di un produttore parla con un server di un altro perché entrambi seguono HTTP.
- **Affidabilità**: la rete può perdere, duplicare, corrompere o riordinare i messaggi; il protocollo aggiunge rilevamento di errori, riscontri (*ACK*), ritrasmissioni e numerazione.
- **Sincronizzazione**: le entità devono concordare lo *stato* della comunicazione (la connessione è aperta? quale byte è stato ricevuto? quanto posso inviare?). Si ottiene con scambi di messaggi (handshake), numeri di sequenza e timer.

12.6.3 Funzioni tipiche di un protocollo

- **Indirizzamento e identificazione** (indirizzi IP, porte).
- **Incapsulamento e framing**: aggiunta di intestazioni; delimitazione dei messaggi.
- **Segmentazione e riassemblaggio**, **multiplazione** (più applicazioni sullo stesso canale tramite le porte).
- **Controllo degli errori**: checksum/CRC, ACK, timeout, ritrasmissione.
- **Controllo di flusso**: il ricevitore limita il ritmo del mittente (finestra).
- **Controllo di congestione**: il mittente adatta il ritmo allo stato della rete.
- **Instaurazione e chiusura** della connessione.
- **Instradamento** (routing).
- **Sicurezza**: autenticazione, riservatezza, integrità.

12.6.4 Servizio, protocollo e stratificazione

In un'architettura a **strati**, il **servizio** è ciò che uno strato offre a quello superiore (interfaccia *verticale*); il **protocollo** è l'insieme di regole tra due strati pari su sistemi diversi (comunicazione *orizzontale*). Ogni strato incapsula la PDU di quello superiore: `dati applicativi` → *segmento* (trasporto) → *pacchetto* (rete) → *frame* (collegamento).

Perché stratificare: **modularità** (si cambia uno strato, per esempio da Ethernet a Wi-Fi, senza toccare gli altri), **interoperabilità**, complessità suddivisa, standard per ogni livello. Il costo è un po' di overhead (intestazioni) e, a volte, di prestazioni.

12.7 Comunicazione Orientata e Non Orientata alla Connessione

	Orientata alla connessione	**Senza connessione**
Fasi	Instaurazione → trasferimento → chiusura	Si invia direttamente
Stato	Gli estremi **mantengono uno stato** (numeri di sequenza, finestre)	Nessuno stato condiviso; ogni messaggio è indipendente
Affidabilità	Può garantire consegna, ordine, assenza di duplicati	*Best effort*: nessuna garanzia
Overhead	Handshake iniziale (latenza di almeno 1 RTT), intestazioni più grandi, stato da mantenere	Minimo; nessuna latenza di apertura
Scalabilità	Lo stato per connessione limita il numero di client	Ottima: il server non ricorda i client
Controllo	Flusso e congestione	Nessuno (se serve, lo fa l'applicazione)
Esempi	**TCP**, TLS sopra TCP, ATM, MPLS	**UDP**, **IP**, DNS (su UDP), streaming, VoIP, giochi
Ambiti	Trasferimento di file, web, posta, accesso remoto, database: servono *tutti* i dati, nell'ordine corretto	Tempo reale (un pacchetto in ritardo è inutile), *query* brevi (DNS), broadcast/multicast, sensori IoT

Dove sta la scelta: dipende dal requisito. Se un dato perso va *recuperato* (un file) ⇒ connessione. Se un dato in ritardo è *peggio* di un dato perso (voce in diretta) o se il messaggio è breve e unico (richiesta DNS) ⇒ senza connessione. È anche possibile costruire l'affidabilità *sopra* UDP a livello applicativo (come fa QUIC).

12.8 Lo Stack TCP/IP e i Principali Protocolli

Lo stack TCP/IP ha quattro (o cinque) livelli. È il modello *effettivamente* usato da Internet; ISO/OSI è il modello di riferimento teorico.

TCP/IP	OSI	Protocolli ed esempi
Applicazione	Applicazione, Presentazione, Sessione	HTTP/HTTPS, DNS, DHCP, SMTP/IMAP, SSH, FTP, MQTT, gRPC, WebSocket
Trasporto	Trasporto	TCP, UDP, QUIC
Internet (rete)	Rete	IPv4, IPv6, ICMP; protocolli di routing (OSPF, BGP)
Accesso alla rete (collegamento + fisico)	Collegamento dati, Fisico	Ethernet, Wi-Fi (IEEE 802.11), ARP, PPP

12.8.1 Applicazione

- **HTTP/HTTPS** (porta 80/443): web e API REST; richiesta/risposta, *stateless*. HTTP/1.1 usa connessioni persistenti; **HTTP/2** multiplexa più richieste su una connessione TCP; **HTTP/3** usa QUIC su UDP.
- **DNS** (porta 53): traduce nomi in indirizzi IP.
- **DHCP** (UDP 67/68): assegna automaticamente indirizzo IP, maschera, gateway e DNS.
- **SMTP** (25/587): invio della posta; **IMAP/POP3**: lettura.
- **SSH** (22): accesso remoto cifrato; **FTP** (21): trasferimento file.
- **MQTT**: protocollo *publish/subscribe* leggero per l'IoT, con un *broker*.

12.8.2 Trasporto

- **TCP**: affidabile, orientato alla connessione, flusso di byte, controllo di flusso e congestione.
- **UDP**: datagrammi, intestazione di 8 byte, nessuna garanzia.
- **QUIC**: affidabilità, multiplexing e cifratura (TLS 1.3) *sopra UDP*, handshake in 0–1 RTT, senza il blocco di coda (*head-of-line blocking*) di TCP; base di HTTP/3.

12.8.3 Rete e collegamento

- **IP**: indirizzamento logico e instradamento dei pacchetti, *senza connessione* e *best effort*. IPv4: indirizzi a 32 bit; IPv6: 128 bit.
- **ICMP**: messaggi di controllo ed errore (`ping`, `traceroute`, "destination unreachable", "time exceeded").
- **ARP**: associa un indirizzo IP a un indirizzo MAC nella stessa rete locale, con una richiesta in *broadcast*.
- **Ethernet / Wi-Fi**: trasmissione di frame tra nodi adiacenti, con indirizzi MAC e controllo di errore (CRC).

12.9 TCP in Dettaglio

12.9.1 Intestazione e porte

L'intestazione TCP (almeno 20 byte) contiene: porta sorgente e destinazione (16 bit), **numero di sequenza** e **numero di riscontro** (32 bit), flag (`SYN`, `ACK`, `FIN`, `RST`), **finestra di ricezione** (16 bit), checksum. Una connessione è identificata dalla quaterna (IP sorgente, porta sorgente, IP destinazione, porta destinazione).

12.9.2 Apertura e chiusura

- **Handshake a tre vie**: il client invia `SYN` (con il proprio numero di sequenza iniziale), il server risponde `SYN-ACK`, il client conferma con `ACK`. **Perché tre messaggi**: ciascun lato deve *dichiarare* il proprio numero iniziale e *vedere confermato* l'altro; servono quindi quattro informazioni, e il secondo messaggio ne combina due.
- **Chiusura**: scambio di `FIN` e `ACK` in ciascuna direzione (la connessione è full-duplex e si chiude per direzione); `RST` chiude bruscamente.

12.9.3 Affidabilità

- **Numeri di sequenza e ACK cumulativi**: il riscontro indica il prossimo byte atteso.
- **Timeout di ritrasmissione (RTO)**: stimato dall'RTT misurato ($\text{RTO} = \text{SRTT} + 4 \cdot \text{RTTVAR}$).
- **Ritrasmissione rapida**: tre ACK duplicati fanno ritrasmettere il segmento senza aspettare il timeout.
- **Finestra scorrevole**: il mittente può avere più segmenti *in volo* senza riscontro (pipelining), per non sprecare la capacità del canale.

12.9.4 Controllo di flusso e di congestione

- **Controllo di flusso**: il ricevitore comunica la finestra `rwnd` (spazio libero nel buffer); il mittente non supera `rwnd`. Protegge il *ricevitore*.
- **Controllo di congestione**: il mittente mantiene una finestra di congestione `cwnd`; invia al più $\min(\texttt{cwnd}, \texttt{rwnd})$. Protegge la *rete*.
  - **Slow start**: `cwnd` parte da 1 MSS e *raddoppia* ogni RTT (cresce di 1 MSS per ogni ACK) fino alla soglia `ssthresh`.
  - **Congestion avoidance**: oltre la soglia cresce di circa 1 MSS per RTT (*additive increase*).
  - **Perdita**: con timeout, $\texttt{ssthresh} = \texttt{cwnd}/2$ e $\texttt{cwnd} = 1$ MSS (si riparte da slow start); con tre ACK duplicati (Reno), `cwnd` si dimezza (*multiplicative decrease*) e si prosegue in congestion avoidance.
  - Il comportamento complessivo è detto **AIMD** (*Additive Increase, Multiplicative Decrease*): converge a una ripartizione equa della capacità tra i flussi.

12.10 Indirizzamento, Routing e Servizi di Rete

12.10.1 IP, DHCP e NAT

- **Indirizzo IPv4**: 32 bit (rete + host); la maschera di sottorete separa le due parti (vedi *Subnetting*). **IPv6**: 128 bit, intestazione semplificata a lunghezza fissa (40 byte), nessuna frammentazione nei router, risolve l'esaurimento degli indirizzi.
- **Indirizzi privati** (non instradati in Internet), in particolare `10.0.0.0/8`, `172.16.0.0/12` e `192.168.0.0/16`.
- **DHCP**: il nuovo host trasmette in broadcast *Discover*; il server risponde *Offer*; l'host chiede *Request*; il server conferma *Ack* (sequenza **DORA**). L'indirizzo è *noleggiato* per un tempo limitato (*lease*).
- **NAT**: il router di bordo traduce gli indirizzi privati in uno (o pochi) indirizzo pubblico, usando le *porte* per distinguere le connessioni (*PAT*). **Perché**: risparmia indirizzi IPv4 e

nasconde la rete interna. **Costo**: rompe il principio end-to-end (un host interno non è raggiungibile dall'esterno senza configurazioni).

12.10.2 Routing e forwarding

- **Forwarding**: azione locale del router su ogni pacchetto: guarda l'indirizzo di destinazione, cerca nella tabella con la regola del **prefisso più lungo** (*longest prefix match*) e inoltra sull'interfaccia indicata.
- **Routing**: costruzione *globale* della tabella tramite algoritmi e protocolli.

	Distance vector	**Link state**	**Path vector**
Idea	Ogni router comunica ai vicini le distanze verso tutte le destinazioni	Ogni router conosce l'**intero grafo** e calcola i cammini minimi	Si annunciano i percorsi (sequenze di sistemi autonomi)
Algoritmo	Bellman–Ford	Dijkstra	Policy e selezione del percorso
Protocollo	RIP	**OSPF**	**BGP**
Ambito	Reti piccole	Interno a un'organizzazione (*intra-AS*)	**Tra** organizzazioni, Internet (*inter-AS*)
Problemi	Convergenza lenta, *count to infinity*	Più memoria e traffico iniziale	Complessità, rischio di errori di configurazione

Internet è diviso in **sistemi autonomi** (AS): all'interno si usa un protocollo come OSPF per ottimizzare le prestazioni; tra AS si usa BGP, dove contano le *politiche* (accordi commerciali) più dei costi.

12.10.3 DNS

Il **DNS** è un database distribuito e gerarchico che traduce nomi (`www.esempio.it`) in indirizzi IP.

- **Gerarchia**: radice (`.`) → TLD (`.it`, `.com`) → server autoritativi del dominio.
- **Risoluzione**: il client chiede al *resolver ricorsivo* (del provider o pubblico), che interroga in modo iterativo radice, TLD e server autoritativo e restituisce la risposta.
- **Caching** con **TTL**: le risposte sono memorizzate per ridurre carico e latenza.
- **Record**: `A` (IPv4), `AAAA` (IPv6), `CNAME` (alias), `MX` (posta), `NS` (server dei nomi), `TXT`.
- **Perché è distribuito**: una tabella centrale sarebbe un *singolo punto di guasto* e di carico; la gerarchia e il caching lo rendono scalabile e resiliente. Per le richieste brevi usa **UDP**.

12.11 Protocolli nei Sistemi Distribuiti e nelle Applicazioni Internet

Protocollo	Modello	Uso e motivazione
HTTP/REST	Richiesta/risposta, stateless	API web e mobile; semplice, diffuso, cacheabile, facile da scalare con load balancer
gRPC (HTTP/2)	RPC tipizzato, streaming	Comunicazione interna tra microservizi: efficiente, contratto esplicito (Protocol Buffers)
WebSocket	Canale bidirezionale persistente	Chat, notifiche, quotazioni in tempo reale: evita il polling
MQTT	Publish/subscribe con broker	IoT: leggero, funziona su reti poco affidabili, tre livelli di qualità del servizio
AMQP / Kafka	Code e log di messaggi	Disaccoppiare servizi, assorbire picchi, elaborazione di eventi in streaming
DNS	Richiesta/risposta su UDP	Scoperta di servizi per nome; bilanciamento semplice
SSH / TLS	Canale cifrato sopra TCP	Accesso sicuro e protezione di altri protocolli (HTTPS, IMAPS)

Nei sistemi distribuiti i protocolli devono gestire guasti e latenza: **timeout**, **ritentativi con attesa crescente**, operazioni **idempotenti**, **controllo di flusso** con code (*backpressure*); vedi il capitolo *Sistemi Distribuiti*.

12.12 Sicurezza di Rete: TLS e HTTPS

TLS aggiunge **riservatezza**, **integrità** e **autenticazione** sopra TCP; **HTTPS** è HTTP sopra TLS.

1. Il client propone i parametri di cifratura (*ClientHello*); il server sceglie e invia il proprio **certificato**.
2. Il client **verifica il certificato**: firma di una *autorità di certificazione* fidata, validità, nome del dominio.
3. I due lati concordano una **chiave di sessione** con uno scambio di chiavi (per esempio ECDHE), che garantisce la *forward secrecy*.
4. Il traffico è cifrato con **crittografia simmetrica** (per esempio AES-GCM), che è veloce, e autenticato.

Perché ibrida: la crittografia asimmetrica serve per autenticare e scambiare la chiave, ma è lenta; la simmetrica è veloce e protegge i dati. Altri strumenti: **firewall** (filtro dei pacchetti per indirizzi e porte), **VPN**, **IDS/IPS**.

12.13 Programmazione con i Socket

Un **socket** è il punto di accesso del programma alla rete. Il *tipo* di socket riflette la scelta tra connessione e assenza di connessione: `SOCK_STREAM` (TCP) e `SOCK_DGRAM` (UDP).

```
1 import socket
2
3 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:     # TCP
```

```
    srv.bind(("0.0.0.0", 8080))
    srv.listen()                                # passivo: attende connessioni
    conn, addr = srv.accept()                   # completa l'handshake a tre vie
    with conn:
        while True:
            dati = conn.recv(1024)              # flusso di byte, non messaggi
            if not dati:                        # il client ha chiuso (FIN)
                break
            conn.sendall(dati.upper())          # risponde: stesso canale
                affidabile
```

Listing 12.3: Server TCP: orientato alla connessione

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect(("127.0.0.1", 8080))              # SYN, SYN-ACK, ACK
    s.sendall(b"ciao")
    print(s.recv(1024))                         # b'CIAO'
```

Listing 12.4: Client TCP

```
import socket

# Server UDP: nessuna connessione, ogni datagramma e' indipendente
srv = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
srv.bind(("0.0.0.0", 8081))
# Client UDP
cli = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
cli.sendto(b"ciao", ("127.0.0.1", 8081))      # nessun handshake, nessuna
    garanzia
dati, mittente = srv.recvfrom(1024)           # un datagramma intero (con il
    mittente)
srv.sendto(dati.upper(), mittente)
print(cli.recvfrom(1024)[0])                  # b'CIAO'
```

Listing 12.5: Client e server UDP: senza connessione

Differenze da sottolineare

- TCP: `listen` e `accept` per il server, `connect` per il client; **flusso di byte** senza confini di messaggio (⇒ il programma deve delimitare i messaggi, per esempio con una lunghezza o con un carattere di fine riga).
- UDP: nessuna `connect` o `accept`; ogni `sendto`/`recvfrom` è un datagramma con i propri confini; la perdita non viene segnalata.
- Un server TCP reale serve molti client contemporaneamente con thread, un pool di thread o I/O asincrono (`select`/`epoll`); vedi la parte sui thread.

12.14 Scaletta per il Tema "Protocolli nei Sistemi Distribuiti"

1. Definire il protocollo (sintassi, semantica, temporizzazione) e il contesto: canale inaffidabile, sistemi eterogenei.
2. Il ruolo: **interoperabilità** (standard), **affidabilità** (ACK, ritrasmissioni), **sincronizzazione** (handshake, numeri di sequenza, timer).
3. Funzioni: indirizzamento, framing, controllo di errori, di flusso e di congestione, instaurazione, routing, sicurezza.
4. Stratificazione e incapsulamento; servizio contro protocollo; modelli OSI e TCP/IP.
5. Orientato alla connessione e non: tabella con vantaggi, svantaggi e ambiti.
6. Esempi dello stack: HTTP, DNS, TCP, UDP, IP, ARP; come si combinano in una richiesta web (DNS $\rightarrow$ TCP handshake $\rightarrow$ TLS $\rightarrow$ HTTP).
7. Impiego nei sistemi distribuiti: REST, gRPC, code di messaggi, MQTT; gestione dei guasti.

Cosa succede digitando un URL nel browser (domanda tipica, che riunisce tutto): (1) il browser cerca l'indirizzo con il **DNS** (cache locale, poi resolver); (2) se l'host è nella stessa sottorete usa **ARP**, altrimenti invia al **gateway**; (3) stabilisce una connessione **TCP** (handshake a tre vie); (4) per HTTPS esegue l'handshake **TLS**; (5) invia la richiesta **HTTP**; (6) il pacchetto attraversa i router (**forwarding** IP, eventualmente **NAT**); (7) il server risponde e il browser mostra la pagina.

Tracce Uscite

Seconda sessione 2018 – Informatica

Traccia: descrivere il modello ISO/OSI, esponendo nel dettaglio ogni livello e le sue caratteristiche; confrontarlo con il modello TCP/IP.

Dove trovare la risposta: capitolo *Reti di Calcolatori* (sette strati) e, per il confronto con TCP/IP, sezione *Lo Stack TCP/IP e i Principali Protocolli*, con la tabella di corrispondenza tra i livelli.

Prima sessione 2026 – A2-INF-4

Traccia: concetto di protocollo nei sistemi distribuiti; ruolo nel garantire interoperabilità, affidabilità e sincronizzazione; funzioni di un protocollo di comunicazione; modelli orientato e non orientato alla connessione con vantaggi, svantaggi e ambiti; esempi dallo stack TCP/IP e impiego nei sistemi distribuiti e nelle applicazioni Internet.

Dove trovare la risposta: sezioni *Il Concetto di Protocollo*, *Comunicazione Orientata e Non Orientata alla Connessione*, *Lo Stack TCP/IP* e *Protocolli nei Sistemi Distribuiti*, con la scaletta di risposta a fine capitolo.

Bologna (sessione non indicata nel file) – Ing. Informatica E04

Traccia: concetti di processo e thread in un sistema operativo moderno; differenze tra i due; modelli di multithreading (many-to-one, one-to-one, many-to-many); problematiche di concorrenza come le race condition.

Dove trovare la risposta: sezioni *Processi: Stati, PCB e Cambio di Contesto*, *Thread e Modelli di Multithreading* e *Race Condition*; per le definizioni di base vedi anche il capitolo *Sistemi Operativi*.

Prima sessione 2025 (Bologna) – Ing. Informatica E05

Traccia: processo di chiamata a funzione e ruolo dello stack.

Dove trovare la risposta: capitolo *Programmazione ad Oggetti e Algoritmi*, sezione *Chiamate di Funzione, Stack e Passaggio dei Parametri*.

Capitolo 13

Sicurezza Informatica e Crittografia

La sicurezza non è una funzionalità da aggiungere alla fine, ma una proprietà da progettare fin dall'inizio. Nelle tracce passate presenti in questo repository **non compaiono temi dedicati alla sicurezza**; il capitolo è però *preventivo*: la sicurezza è richiesta in modo trasversale in qualunque progetto (autenticazione degli utenti, protezione delle API, TLS, protezione dei dati personali) e un candidato di Ingegneria Informatica deve saperne parlare. Il capitolo si chiude con un esempio di messa in sicurezza del sistema di prenotazione del cinema del capitolo *Progettazione di Sistemi e API*.

13.1 Concetti Fondamentali

13.1.1 Obiettivi: la triade CIA e oltre

- **Confidenzialità** (*confidentiality*): solo i soggetti autorizzati accedono alle informazioni. Strumento: cifratura, controllo degli accessi.
- **Integrità**: i dati non sono modificati in modo non autorizzato o non rilevato. Strumenti: hash, MAC, firme digitali.
- **Disponibilità** (*availability*): il servizio è utilizzabile quando serve. Strumenti: ridondanza, backup, protezione da DoS.
- **Autenticazione**: verificare l'identità di chi comunica. **Autorizzazione**: stabilire che cosa può fare. **Non ripudio**: chi ha compiuto un'azione non può negarla (firma digitale). **Tracciabilità** (*accountability*): le azioni sono registrate.

13.1.2 Minacce, vulnerabilità, rischio

- **Minaccia**: evento o soggetto che può causare un danno (un attaccante, un malware, un guasto).
- **Vulnerabilità**: debolezza che può essere sfruttata (un errore di programmazione, una configurazione errata, un utente ingannabile).
- **Attacco**: azione che sfrutta una vulnerabilità. **Exploit**: strumento o tecnica che lo realizza.
- **Rischio** $\approx$ **probabilità** $\times$ **impatto**. Non si elimina: si **valuta** e si **tratta** (mitigare, trasferire con un'assicurazione, accettare, evitare).

13.1.3 Principi di progettazione sicura

- **Difesa in profondità** (*defense in depth*): più livelli di protezione indipendenti; se uno cade, gli altri reggono.
- **Privilegio minimo**: ogni utente e programma ha solo i permessi indispensabili. **Perché**: riduce il danno se un account o un componente è compromesso.
- **Default sicuri** e **fail-safe**: in caso di errore si nega l'accesso, non lo si concede.
- **Mediazione completa**: ogni accesso è controllato, ogni volta.
- **Principio di Kerckhoffs**: la sicurezza di un sistema crittografico deve dipendere *solo dalla segretezza della chiave*, non da quella dell'algoritmo. **Perché**: gli algoritmi pubblici sono studiati e attaccati da tutti; la "sicurezza per oscurità" crolla appena il segreto è scoperto.
- **Semplicità e riduzione della superficie di attacco**: meno codice, meno servizi esposti, meno vulnerabilità.
- **Mai fidarsi dell'input**: ogni dato esterno si valida.
- **Non inventare la propria crittografia**: usare algoritmi e librerie standard, verificati.

13.2 Crittografia Simmetrica

Mittente e destinatario condividono la **stessa chiave segreta** K: $c = E_K(m)$, $m = D_K(c)$.

13.2.1 Cifrari a blocchi e a flusso

- **Cifrari a blocchi**: operano su blocchi di dimensione fissa. **AES** (*Advanced Encryption Standard*): blocchi di 128 bit, chiavi da 128, 192 o 256 bit (10, 12, 14 *round* di sostituzioni e permutazioni). Lo standard attuale. **DES** (chiave a 56 bit) è obsoleto: esplorabile con la forza bruta; **3DES** è deprecato.
- **Cifrari a flusso**: generano un flusso di bit pseudocasuali sommato (XOR) al messaggio; per esempio **ChaCha20**, molto veloce senza hardware dedicato.
- **One-time pad**: XOR con una chiave casuale lunga quanto il messaggio e usata *una sola volta*; garantisce segretezza *perfetta* (Shannon), ma è impraticabile (distribuzione della chiave).
- **Confusione e diffusione** (Shannon): la confusione rende complessa la relazione tra chiave e testo cifrato; la diffusione fa dipendere ogni bit cifrato da molti bit del messaggio.

13.2.2 Modi di funzionamento

Modo	Come funziona	Note
ECB	Ogni blocco cifrato indipendentemente	**Insicuro**: blocchi uguali producono testi cifrati uguali; rivela la struttura dei dati (l'immagine cifrata "si vede")
CBC	Ogni blocco è messo in XOR con il precedente cifrato; serve un IV imprevedibile	Non fornisce integrità; vulnerabile ad attacchi *padding oracle* se mal usato
CTR	Trasforma il cifrario in cifrario a flusso con un contatore	Parallelizzabile; il *nonce* non va mai riusato con la stessa chiave
GCM	CTR più autenticazione (*AEAD*)	**Consigliato**: cifra e autentica insieme; riusare il nonce è catastrofico

Perché la cifratura da sola non basta: un attaccante può modificare un testo cifrato senza conoscere la chiave. Si usano quindi modi **AEAD** (*Authenticated Encryption with Associated Data*: AES-GCM, ChaCha20-Poly1305) che garantiscono riservatezza *e* integrità.

13.2.3 Il problema della distribuzione delle chiavi

La crittografia simmetrica è veloce, ma richiede che le parti *si siano già scambiate la chiave* in modo sicuro. Con n utenti che vogliono comunicare a coppie in modo riservato servono $n(n-1)/2$ chiavi (con 1000 utenti, quasi mezzo milione). Questo problema è stato superato dalla crittografia asimmetrica.

13.3 Crittografia Asimmetrica

Ogni soggetto ha una **coppia di chiavi**: una **pubblica** (distribuibile liberamente) e una **privata** (segreta). Ciò che è cifrato con una si decifra solo con l'altra.

- **Riservatezza**: si cifra con la chiave *pubblica del destinatario*; solo lui, con la privata, decifra.
- **Firma**: si firma con la chiave *privata del mittente*; chiunque, con la pubblica, verifica.

13.3.1 RSA

Si basa sulla difficoltà di **fattorizzare** il prodotto di due numeri primi grandi.

1. Si scelgono due primi grandi p e q; $n = p \cdot q$; $\varphi(n) = (p-1)(q-1)$.
2. Si sceglie e coprimo con $\varphi(n)$; si calcola d tale che $e \cdot d \equiv 1 \pmod{\varphi(n)}$.
3. **Chiave pubblica** (n, e); **chiave privata** (n, d).
4. **Cifratura**: $c = m^e \bmod n$. **Decifratura**: $m = c^d \bmod n$.

```
# RSA "giocattolo": SOLO a scopo didattico (numeri minuscoli, niente padding)
p, q = 61, 53
n = p * q                       # 3233: parte pubblica del modulo
phi = (p - 1) * (q - 1)         # 3120
e = 17                          # esponente pubblico, coprimo con phi
d = pow(e, -1, phi)             # esponente privato: e*d = 1 (mod phi)  ->
    2753

m = 65                          # messaggio (numero < n)
c = pow(m, e, n)                # cifratura con la chiave PUBBLICA  (n, e)
m2 = pow(c, d, n)               # decifratura con la chiave PRIVATA (n, d)
print(n, phi, d, c, m2)

# Firma: si "cifra" con la chiave privata, si verifica con la pubblica
firma = pow(m, d, n)
print(firma, pow(firma, e, n) == m)
```

Listing 13.1: RSA con numeri piccoli (solo didattico)

Con $p = 61$, $q = 53$: $n = 3233$, $\varphi = 3120$, $e = 17$, $d = 2753$; il messaggio 65 diventa 2790 e torna 65. **Attenzione**: questo è RSA "da manuale"; l'RSA reale usa chiavi di almeno **2048 bit** (3072

per la protezione di lungo periodo) e un **padding** (OAEP per la cifratura, PSS per la firma), altrimenti è insicuro.

13.3.2 Scambio di chiavi Diffie–Hellman

Permette a due parti di accordarsi su una chiave segreta *su un canale pubblico*, grazie alla difficoltà del **logaritmo discreto**.

```
# Diffie-Hellman con numeri piccoli (didattico)
p, g = 23, 5                    # parametri pubblici
a, b = 6, 15                    # segreti di Alice e Bob (mai trasmessi)
A = pow(g, a, p)                # Alice invia A = g^a mod p  -> 8
B = pow(g, b, p)                # Bob   invia B = g^b mod p  -> 19
print(A, B, pow(B, a, p), pow(A, b, p))     # stesso segreto condiviso: g^(ab)
    mod p = 2
```

Listing 13.2: Diffie-Hellman con numeri piccoli (didattico)

Un osservatore vede p, g, A, B ma non può calcolare facilmente g^{ab}. Usato da solo *non* autentica le parti: è vulnerabile all'attacco **man-in-the-middle** se non è accompagnato da firme o certificati. La variante effimera (DHE/ECDHE) garantisce la **forward secrecy**: la compromissione futura della chiave a lungo termine non permette di decifrare il traffico passato.

13.3.3 Crittografia a curve ellittiche (ECC)

Stessa idea con operazioni su curve ellittiche: a pari sicurezza servono chiavi *molto più corte* (una chiave ECC a 256 bit equivale circa a RSA a 3072 bit), con minor calcolo e banda. Esempi: ECDH, ECDSA, Ed25519.

13.3.4 Confronto e crittografia ibrida

	Simmetrica	**Asimmetrica**
Chiavi	Una, condivisa	Coppia pubblica/privata
Velocità	Molto alta	Da centinaia a migliaia di volte più lenta
Distribuzione delle chiavi	Problema (serve un canale sicuro)	Semplificata: si pubblica la chiave pubblica
Servizi	Riservatezza (e integrità con AEAD)	Scambio di chiavi, firma digitale, non ripudio
Esempi	AES, ChaCha20	RSA, ECC, Diffie–Hellman

Perché si usano insieme (**crittografia ibrida**, come in TLS e PGP): l'asimmetrica serve a *scambiare* una chiave di sessione e ad autenticare; i dati si cifrano poi con la simmetrica, veloce. Si ottiene la comodità della prima e la velocità della seconda.

Crittografia post-quantistica. Un computer quantistico sufficientemente potente romperebbe RSA, Diffie–Hellman ed ECC (algoritmo di Shor) e dimezzerebbe la sicurezza dei cifrari simmetrici (Grover: AES-256 resta adeguato). Il NIST ha standardizzato algoritmi *post-quantistici* (come ML-KEM e ML-DSA); è in corso la migrazione.

13.4 Funzioni Hash, MAC e Firma Digitale

13.4.1 Funzioni hash crittografiche

Una funzione hash H trasforma un messaggio di lunghezza arbitraria in un **digest** di lunghezza fissa (SHA-256: 256 bit). Proprietà:

- **Deterministica** e veloce da calcolare.
- **Resistenza alla preimmagine**: dato h è impraticabile trovare m con $H(m) = h$ (non è invertibile).
- **Resistenza alla seconda preimmagine**: dato m è impraticabile trovare $m' \neq m$ con lo stesso hash.
- **Resistenza alle collisioni**: è impraticabile trovare due messaggi qualsiasi con lo stesso hash. Per un hash di n bit la ricerca di una collisione richiede circa $2^{n/2}$ tentativi (*paradosso del compleanno*), quindi servono almeno 256 bit per una sicurezza di 128 bit.
- **Effetto valanga**: cambiare un bit del messaggio cambia circa metà dei bit del digest.

Algoritmi: **SHA-256/SHA-512** (famiglia SHA-2) e SHA-3 sono sicuri; **MD5** e **SHA-1** sono *rotti* (collisioni dimostrate) e non vanno più usati per scopi di sicurezza. **Usi**: verifica dell'integrità dei file, firme digitali (si firma l'hash, non il messaggio), memorizzazione delle password, blockchain, *deduplicazione*.

13.4.2 MAC e HMAC: integrità e autenticità con chiave condivisa

Un **MAC** (*Message Authentication Code*) è un tag calcolato da messaggio *e chiave segreta*; chi non conosce la chiave non può produrre un tag valido. **HMAC** costruisce un MAC a partire da una funzione hash. Un semplice hash *non* basta: chiunque può ricalcolarlo dopo aver modificato il messaggio.

```python
import hashlib, hmac, secrets

# 1) Hash: stesso ingresso -> stesso digest; una minima modifica -> digest completamente diverso
print(hashlib.sha256(b"abc").hexdigest())
print(hashlib.sha256(b"abd").hexdigest())

# 2) HMAC: integrita' + autenticita' con una chiave condivisa
chiave = secrets.token_bytes(32)
messaggio = b"importo=100;iban=IT60X0542811101000000123456"
tag = hmac.new(chiave, messaggio, hashlib.sha256).digest()

def verifica(chiave, messaggio, tag):
    atteso = hmac.new(chiave, messaggio, hashlib.sha256).digest()
    return hmac.compare_digest(atteso, tag)       # confronto a tempo costante

print(verifica(chiave, messaggio, tag))                                  # True
print(verifica(chiave, b"importo=900;iban=IT60X0542811101000000123456", tag))
    # False
```

Listing 13.3: Hash, effetto valanga e HMAC

13.4.3 Firma digitale

1. Il mittente calcola l'**hash** del documento.
2. Lo **firma** con la propria **chiave privata**.
3. Il destinatario **ricalcola l'hash** e verifica la firma con la **chiave pubblica** del mittente.

Garantisce **integrità**, **autenticità** e **non ripudio**: solo il possessore della chiave privata può aver firmato. **Differenza da MAC**: con il MAC la chiave è condivisa, quindi *entrambe* le parti potrebbero aver generato il tag, e non c'è non ripudio; con la firma solo il mittente possiede la chiave privata.

13.5 Infrastruttura a Chiave Pubblica (PKI) e Certificati

Come si sa che una chiave pubblica appartiene davvero a chi dice? Un **certificato digitale** (standard **X.509**) lega un'identità (nome di dominio, persona) a una chiave pubblica ed è **firmato** da una **Autorità di Certificazione (CA)**.

- **Catena di fiducia**: il certificato del server è firmato da una CA intermedia, che è firmata da una CA radice; le radici di cui ci si fida sono preinstallate nel sistema o nel browser.
- **Verifica**: firma valida, catena fino a una radice fidata, periodo di validità, nome corrispondente, certificato non revocato.
- **Revoca**: liste di revoca (CRL) o protocollo OCSP.
- **TLS**: usa i certificati per autenticare il server (e a volte il client) e per accordarsi sulla chiave di sessione (vedi capitolo *Sistemi Operativi e Reti: Approfondimenti*).

Contesto italiano ed europeo. La **firma digitale** italiana (qualificata) si basa su PKI; il regolamento europeo **eIDAS** attribuisce alla firma elettronica qualificata *lo stesso valore giuridico della firma autografa*. Altri strumenti: **SPID** e **CIE** (identità digitale), **PEC** (posta elettronica certificata). Per un ingegnere che firma progetti e relazioni la firma digitale è uno strumento professionale quotidiano.

13.6 Autenticazione e Autorizzazione

13.6.1 Fattori di autenticazione

Qualcosa che si sa (password), **che si ha** (telefono, chiave hardware, smart card), **che si è** (biometria). L'**autenticazione a più fattori (MFA)** ne combina almeno due, perché una password rubata non basta. Esempi: codici temporanei **TOTP** (RFC 6238, cambiano ogni 30 secondi), chiavi **FIDO2/WebAuthn** (resistenti al *phishing*).

13.6.2 Conservare le password

Non si memorizzano mai le password in chiaro, né con un semplice hash veloce. La regola è: **funzione di derivazione di chiave lenta** (*password hashing*) con **sale** casuale per utente.

- **Hash veloci** (SHA-256 semplice): un attaccante con una GPU prova miliardi di password al secondo (*brute force* e dizionari).
- **Sale** (*salt*): valore casuale per utente, memorizzato accanto all'hash. Perché: due utenti con la stessa password hanno hash diversi, e le tabelle pre-calcolate (*rainbow table*) diventano inutili.

- **Funzioni lente e configurabili**: **Argon2id** (consigliata), **scrypt**, **bcrypt**, **PBKDF2** con molte iterazioni. Il costo si regola in base all'hardware.
- **Confronto a tempo costante** per evitare attacchi basati sul tempo di risposta.

```python
import hashlib, hmac, secrets

ITERAZIONI = 600_000                                # costo configurabile: rallenta chi prova a indovinare

def hash_password(password: str) -> str:
    sale = secrets.token_bytes(16)                  # sale casuale, diverso per ogni utente
    dk = hashlib.pbkdf2_hmac("sha256", password.encode(), sale, ITERAZIONI)
    return f"{ITERAZIONI}${sale.hex()}${dk.hex()}"

def verifica_password(password: str, salvato: str) -> bool:
    it, sale_hex, dk_hex = salvato.split("$")
    dk = hashlib.pbkdf2_hmac("sha256", password.encode(), bytes.fromhex(sale_hex), int(it))
    return hmac.compare_digest(dk, bytes.fromhex(dk_hex))

h = hash_password("correct horse battery staple")
print(verifica_password("correct horse battery staple", h), verifica_password("password123", h))
print(hash_password("x") != hash_password("x"))     # stesso testo, sale diverso -> hash diversi
```

Listing 13.4: Salvataggio sicuro di una password (PBKDF2 con sale)

13.6.3 Sessioni, token e cookie

- **Sessione lato server**: un identificatore casuale nel cookie; lo stato è sul server. Facile da revocare.
- **Token (JWT)**: un token firmato `intestazione.contenuto.firma` che contiene l'identità e la scadenza; il server verifica la firma senza consultare uno stato (comodo con servizi senza stato). **Attenzioni**: scadenza breve, verifica sempre la firma e l'algoritmo atteso, nessun dato segreto nel contenuto (è solo codificato, non cifrato), difficoltà di revoca.
- **Cookie sicuri**: `HttpOnly` (non leggibile da JavaScript, limita il furto via XSS), `Secure` (solo HTTPS), `SameSite` (limita il CSRF).
- **OAuth 2.0 / OpenID Connect**: delega dell'autenticazione e dell'autorizzazione a un provider ("accedi con…"), senza condividere la password con l'applicazione.

13.6.4 Autorizzazione

- **RBAC** (basato sui ruoli: utente, gestore, amministratore) e **ABAC** (basato su attributi e contesto).
- **Controllo a ogni richiesta, lato server**: nascondere un pulsante nell'interfaccia non è una protezione.
- **Controllo della proprietà della risorsa**: un utente autenticato deve poter leggere *solo* le proprie prenotazioni (vedi *IDOR* più avanti).

13.7 Vulnerabilità delle Applicazioni

OWASP Top 10 (edizione 2021; verificare l'edizione più recente) elenca le categorie di rischio più frequenti nelle applicazioni web:

Categoria	Significato e contromisura
Broken Access Control	Controlli di autorizzazione mancanti o aggirabili (per esempio leggere i dati di un altro utente cambiando un identificativo). *Controllo lato server su ogni richiesta, privilegio minimo.*
Cryptographic Failures	Dati sensibili non cifrati, algoritmi deboli, chiavi gestite male. *TLS, cifratura a riposo, algoritmi standard.*
Injection	L'input è interpretato come comando (SQL, shell, LDAP). *Query parametriche, validazione.*
Insecure Design	Difetti di progetto, non di codice. *Threat modeling, progettazione sicura.*
Security Misconfiguration	Impostazioni di default, servizi non necessari, messaggi d'errore dettagliati. *Hardening, configurazione ripetibile.*
Vulnerable and Outdated Components	Librerie con vulnerabilità note. *Inventario delle dipendenze, aggiornamenti.*
Identification and Authentication Failures	Password deboli, sessioni non invalidate, assenza di MFA.
Software and Data Integrity Failures	Aggiornamenti o pipeline non verificati (*supply chain*). *Firme, controllo di integrità.*
Logging and Monitoring Failures	Attacchi non rilevati. *Log centralizzati, allarmi.*
Server-Side Request Forgery	Il server è indotto a fare richieste verso risorse interne. *Elenco di destinazioni consentite.*

13.7.1 SQL injection

Se l'input dell'utente è *concatenato* nella query, può cambiarne il significato. Con `nome = x' OR '1'='1` la condizione diventa sempre vera e la query restituisce tutti gli utenti. La causa è il **mescolamento di dati e codice**; la soluzione è tenerli separati con **query parametriche** (*prepared statements*): il database tratta il parametro sempre come un *dato*.

```python
import sqlite3

db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE utente (nome TEXT, password_hash TEXT)")
db.execute("INSERT INTO utente VALUES ('alice', 'h1'), ('bob', 'h2')")

def login_vulnerabile(nome):
    q = f"SELECT nome FROM utente WHERE nome = '{nome}'"     # concatenazione: PERICOLOSO
    return db.execute(q).fetchall()

def login_sicuro(nome):
    q = "SELECT nome FROM utente WHERE nome = ?"             # query parametrica
    return db.execute(q, (nome,)).fetchall()

attacco = "x' OR '1'='1"
print(login_vulnerabile(attacco))     # restituisce TUTTI gli utenti
```

```
print(login_sicuro(attacco))          # [] : l'input e' trattato come DATO,
    non come SQL
print(login_sicuro("alice"))
```

Listing 13.5: SQL injection e query parametrica

13.7.2 Altre vulnerabilità frequenti

- **XSS** (*Cross-Site Scripting*): lo script di un attaccante viene eseguito nel browser della vittima, perché l'applicazione include input non filtrato nelle pagine. *Contromisure*: **escape dell'output** secondo il contesto (HTML, attributo, JavaScript), **Content Security Policy**, cookie `HttpOnly`.
- **CSRF** (*Cross-Site Request Forgery*): un sito ostile induce il browser, già autenticato, a inviare una richiesta all'applicazione. *Contromisure*: **token anti-CSRF**, cookie `SameSite`, controllo dell'*origine*.
- **IDOR** (*Insecure Direct Object Reference*): `GET /prenotazioni/17` restituisce la prenotazione 17 anche se appartiene a un altro utente. *Contromisura*: verificare che l'oggetto appartenga all'utente autenticato.
- **Buffer overflow** (C/C++): scrivere oltre la fine di un array sovrascrive dati o l'indirizzo di ritorno nello stack (vedi *Record di attivazione*). *Contromisure*: controllo dei limiti, linguaggi con sicurezza della memoria (Rust, Java, Python), **stack canary**, **ASLR** (indirizzi casuali), **DEP/NX** (pagine dati non eseguibili).
- **Path traversal**: `../../etc/passwd` per uscire dalla cartella consentita. *Contromisura*: normalizzare e limitare i percorsi.
- **Deserializzazione non sicura** e **segreti nel codice** (chiavi API nel repository): usare gestori di segreti e variabili d'ambiente.

13.8 Sicurezza di Rete

Attacco	Descrizione	Contromisura
Intercettazione (*sniffing*)	Lettura del traffico non cifrato	Cifratura (TLS, VPN, Wi-Fi WPA3)
Man-in-the-middle	L'attaccante si pone tra le parti e legge o modifica	Certificati e autenticazione reciproca
Spoofing (IP, ARP, DNS)	Falsificazione di indirizzi o risposte	Filtri, ARP ispezionato, **DNSSEC**
Denial of Service (DoS/DDoS)	Esaurire le risorse con enormi quantità di richieste	Limitazione del traffico (*rate limiting*), CDN, ridondanza
Phishing e ingegneria sociale	Si inganna l'utente (non il sistema)	Formazione, MFA, filtri della posta
Malware (virus, worm, trojan, ransomware)	Software ostile	Aggiornamenti, antivirus/EDR, privilegi minimi, backup offline

13.8.1 Difese

- **Firewall**: filtra il traffico in base a indirizzi, porte e stato delle connessioni (*stateful*); a livello applicativo (*WAF*) ispeziona anche il contenuto.

- **Segmentazione** della rete e **DMZ**: i servizi esposti (web) stanno in una zona separata, i database in una zona interna non raggiungibile da Internet. **Perché**: limita i movimenti dell'attaccante (*lateral movement*) se un componente è compromesso.
- **IDS/IPS**: rilevano (e bloccano) attività sospette.
- **VPN**: tunnel cifrato tra reti o verso un utente remoto (IPsec, WireGuard, TLS).
- **Zero trust**: nessuna fiducia implicita nella rete interna; ogni accesso è autenticato e autorizzato.

13.9 Gestione del Rischio, Normativa e Ciclo di Vita

13.9.1 Valutare il rischio e modellare le minacce

Si identificano i **beni** da proteggere, le **minacce**, le **vulnerabilità**, e si stima probabilità e impatto per decidere le priorità. Il **threat modeling** con il metodo **STRIDE** classifica le minacce:

Minaccia	Proprietà violata	Contromisura tipica
Spoofing (falsa identità)	Autenticazione	Autenticazione forte, MFA, certificati
Tampering (manomissione)	Integrità	Firme, MAC, controllo degli accessi, vincoli nel database
Repudiation (negare un'azione)	Non ripudio	Log protetti, firme digitali
Information disclosure (divulgazione)	Riservatezza	Cifratura, controllo degli accessi
Denial of service	Disponibilità	Rate limiting, ridondanza, scalabilità
Elevation of privilege	Autorizzazione	Privilegio minimo, validazione, separazione dei ruoli

13.9.2 Standard e normativa

- **ISO/IEC 27001**: sistema di gestione della sicurezza delle informazioni (ciclo *plan–do–check–act*); **NIST Cybersecurity Framework**: identificare, proteggere, rilevare, rispondere, recuperare (in più: governare).
- **GDPR** (Regolamento UE 2016/679): liceità e minimizzazione dei dati; **privacy by design e by default** (art. 25); misure di sicurezza adeguate (art. 32, per esempio cifratura e pseudonimizzazione); valutazione d'impatto (DPIA); **notifica di una violazione dei dati al Garante entro 72 ore**; sanzioni fino a 20 milioni di euro o al 4% del fatturato.
- **Direttiva NIS2** (sicurezza di reti e sistemi per soggetti essenziali e importanti) e, in Italia, l'**Agenzia per la Cybersicurezza Nazionale (ACN)**.

13.9.3 Sicurezza nel ciclo di sviluppo (DevSecOps)

- **Spostare la sicurezza a sinistra** (*shift left*): requisiti di sicurezza e threat modeling nella progettazione, non dopo.
- **Analisi statica** (SAST) e **dinamica** (DAST), **scansione delle dipendenze** (SCA), **revisione del codice**, **test di penetrazione**, scansione dei segreti nel repository.
- **Aggiornamenti e gestione delle patch**; **backup** testati e copia offline; **piano di risposta agli incidenti**.

13.10 Esempio: Messa in Sicurezza del Sistema di Prenotazione

Si riprende il sistema del cinema (capitolo *Progettazione di Sistemi e API*). Per ogni componente si indica la minaccia e la misura, in ottica STRIDE.

Aspetto	Minaccia	Misura
Comunicazione Web ↔ backend	Intercettazione, man-in-the-middle	**HTTPS (TLS 1.2+)** ovunque; HSTS; certificati validi
Accesso degli utenti	Furto o indovinamento delle password; credenziali riusate	Password con **Argon2id** o PBKDF2 e sale; **MFA** opzionale; blocco o rallentamento dopo tentativi falliti; token con scadenza breve
Ruoli (utente, gestore)	Un utente usa le funzioni del gestore	**RBAC** verificato lato server sugli endpoint `/gestore/...`; privilegio minimo per l'account del database
Dati delle prenotazioni	**IDOR**: leggere o annullare le prenotazioni altrui	Controllo `prenotazione.id_utente = utente del token` su ogni richiesta; identificatori non indovinabili (UUID) come difesa aggiuntiva
Accesso al database	**SQL injection**	**Query parametriche** nel livello Repository; utente di database con permessi minimi; nessun SQL costruito per concatenazione
Input delle API	Dati malformati, valori fuori intervallo	**Validazione** (tipi, lunghezze, intervalli: la fila è una sola lettera, il numero è positivo) e rifiuto con 400/422
Prenotazione dei posti	**Accaparramento** automatico dei posti da parte di bot; DoS	**Rate limiting** per utente e per IP; limite al numero di posti per prenotazione; scadenza delle prenotazioni non confermate; **vincolo di unicità** nel database contro i doppioni
Errori e log	Divulgazione di dettagli interni; azioni non tracciabili	Messaggi d'errore generici al client; **log** con identificativo della richiesta e dell'utente, *senza* password o token; log protetti da modifiche
Dati personali (GDPR)	Violazione della privacy; perdita di dati	**Minimizzazione**: solo i dati necessari; cifratura a riposo di dati e backup; periodi di conservazione; procedura di notifica entro 72 ore
Modulo di Machine Learning	Esposizione dei dati degli spettatori; manipolazione del servizio di previsione	Addestrare su dati **aggregati e anonimizzati**; servizio non esposto a Internet, solo al backend; autenticazione tra servizi (*mTLS* o token); validazione dell'input
Dipendenze e segreti	Librerie vulnerabili; chiavi nel repository	Scansione delle dipendenze, aggiornamenti; **gestore di segreti**, mai le chiavi nel codice

```python
def leggi_prenotazione(db, utente, id_prenotazione):
    riga = db.una_riga(
        "SELECT id_prenotazione, id_utente, id_proiezione "
        "FROM prenotazione WHERE id_prenotazione = ?", (id_prenotazione,))
    if riga is None:
        raise NonTrovato()                          # 404
    if riga.id_utente != utente.id and utente.ruolo != "gestore":
        raise NonTrovato()                          # 404 (non rivela che
            esiste), oppure 403
    return riga
```

Listing 13.6: Controllo di autorizzazione contro l'IDOR (pseudocodice)

13.11 Scaletta per un Tema sulla Sicurezza

1. **Obiettivi**: riservatezza, integrità, disponibilità, autenticazione, non ripudio; minacce e rischio.
2. **Strumenti crittografici**: simmetrica (AES-GCM), asimmetrica (RSA, ECC, DH), hash, MAC, firma digitale; e *perché* si combinano (crittografia ibrida).
3. **Infrastruttura**: certificati, PKI, TLS.
4. **Autenticazione e autorizzazione**: password con hashing lento e sale, MFA, token, ruoli.
5. **Vulnerabilità tipiche** (injection, XSS, CSRF, IDOR, buffer overflow) e relative contromisure.
6. **Difesa in profondità**: firewall, segmentazione, monitoraggio; principio del privilegio minimo.
7. **Processo e normativa**: threat modeling, DevSecOps, GDPR, ISO 27001.
8. **Esempio concreto** nel sistema descritto dalla traccia, con una tabella minaccia–misura.

Collegamenti con le Tracce

Nelle tracce raccolte nella cartella `TraccePassate` (2016–2026) **non compare un tema dedicato alla sicurezza informatica**. Il capitolo è quindi preventivo. I punti in cui la sicurezza entra comunque in una traccia sono:

Prima sessione 2026 – A3-INF-4 (prova pratica: cinema multisala)

Il sistema ha utenti registrati, un'interfaccia Web, API e un database. Un'ottima risposta aggiunge una breve nota di sicurezza: autenticazione, controllo dei ruoli, protezione dagli accessi alle prenotazioni altrui, query parametriche, HTTPS. **Dove**: sezione *Esempio: Messa in Sicurezza del Sistema di Prenotazione*.

Prima sessione 2026 – A2-INF-4 (protocolli)

Tra i protocolli dello stack TCP/IP "impiegati nei moderni sistemi distribuiti e nelle applicazioni Internet" rientrano TLS e HTTPS. **Dove**: sezioni *Infrastruttura a Chiave Pubblica* e *Crittografia Asimmetrica*, e la sezione *Sicurezza di Rete: TLS e HTTPS* del capitolo *Sistemi Operativi e Reti: Approfondimenti*.

Seconda sessione 2024 – Informatica #2 (applicazioni distribuite enterprise)

Tra le "best practice" di cloud e architetture distribuite rientrano autenticazione, cifratura, segmentazione e gestione dei segreti. **Dove**: sezioni *Sicurezza di Rete* e *Sicurezza nel ciclo di sviluppo*.

Capitolo 14

Basi di Dati Avanzate: Algebra, SQL, Transazioni e Indici

Il capitolo *Basi di Dati* copre il modello ER, la progettazione logica, le forme normali fino alla terza e le basi di SQL. Qui si approfondiscono gli argomenti attesi per la Sezione A: **algebra relazionale**, **dipendenze funzionali e BCNF**, **SQL avanzato**, **transazioni**, **controllo della concorrenza**, **recupero dai guasti**, **indici** e **ottimizzazione delle query**. Gli esempi usano lo schema del cinema multisala del capitolo *Progettazione di Sistemi e API*, con un piccolo insieme di dati.

Dati di esempio. Film: Aurora (1), Zenit (2), Mare (3), Ombra (4, mai programmato). Sala 1 con 6 posti, Sala 2 con 4. Proiezioni: p1 e p2 di Aurora in Sala 1 (prezzi 8 e 10), p3 di Zenit in Sala 2 (9), p4 di Zenit in Sala 1 (10), p5 di Mare in Sala 2 (7). Biglietti venduti: p1 ne ha 4 su 6, p2 ne ha 2 su 6, p3 ne ha 4 su 4 (esaurita), p4 ne ha 1 su 6, p5 nessuno. Tutte le query e gli output riportati sono stati eseguiti su questi dati (con SQLite).

14.1 Algebra Relazionale

L'**algebra relazionale** è il linguaggio *procedurale* formale su cui si fonda SQL: ogni operatore prende relazioni e produce una relazione. È il linguaggio con cui il DBMS rappresenta e *ottimizza* internamente le query.

14.1.1 Richiami sul modello relazionale

- Una **relazione** è un insieme di **tuple** sugli stessi **attributi** (ciascuno con un dominio). Un **vincolo di chiave** garantisce l'unicità; la **chiave esterna** garantisce l'**integrità referenziale**.
- **NULL**: valore "sconosciuto/inesistente". SQL usa una **logica a tre valori** (vero, falso, *sconosciuto*): `NULL = NULL` non è vero, ma sconosciuto; si usa `IS NULL`.
- Differenza tra **insiemi** (algebra: nessun duplicato) e **multinsiemi** (SQL: i duplicati restano, salvo `DISTINCT`).

14.1.2 Operatori

Operatore	Simbolo	Equivalente in SQL
Selezione (filtra righe)	$\sigma_{\text{cond}}(R)$	`WHERE cond`
Proiezione (sceglie colonne)	$\pi_{A_1,\ldots,A_k}(R)$	`SELECT DISTINCT A1, ...`
Ridenominazione	$\rho_S(R)$	`AS`
Unione, intersezione, differenza	$R \cup S$, $R \cap S$, $R - S$	`UNION`, `INTERSECT`, `EXCEPT` (le relazioni devono avere gli stessi attributi)
Prodotto cartesiano	$R \times S$	`CROSS JOIN`
Join (theta/naturale)	$R \bowtie_\theta S$	`JOIN ... ON`
Join esterno (sinistro)	$R \bowtie_{\text{sx}} S$	`LEFT JOIN`
Divisione	$R \div S$	`NOT EXISTS` doppio
Raggruppamento e aggregazione	γ	`GROUP BY`, `SUM`, `COUNT`, ...

Esempio: i titoli dei film proiettati il 10 novembre 2026.

$$\pi_{\text{titolo}}\Big(\text{FILM} \bowtie_{\text{id_film}} \sigma_{\text{data=2026-11-10}}(\text{PROIEZIONE})\Big)$$

Operatori *primitivi*: σ, π, ρ, $\cup$, $-$, $\times$. Gli altri (intersezione, join, divisione) si ottengono da questi; per esempio $R \bowtie_\theta S = \sigma_\theta(R \times S)$.

14.1.3 Equivalenze usate per ottimizzare

- **Anticipare le selezioni**: $\sigma_p(R \bowtie S) = \sigma_p(R) \bowtie S$ se p riguarda solo attributi di R. **Perché**: si fa il join su meno righe. Esempio: se il join tra 10^6 biglietti e 10^4 proiezioni è seguito da un filtro che lascia una proiezione su cento, filtrare prima riduce di cento volte l'ingresso del join.
- **Anticipare le proiezioni**: scartare le colonne non necessarie il prima possibile.
- **Commutatività e associatività** del join: l'ordine dei join può cambiare; si sceglie quello più economico.
- $\sigma_{p \wedge q}(R) = \sigma_p(\sigma_q(R))$.

14.2 Dipendenze Funzionali e Forme Normali

14.2.1 Dipendenze funzionali

Una **dipendenza funzionale** $X \to Y$ vale in una relazione se due tuple con lo stesso valore di X hanno sempre lo stesso valore di Y (X *determina* Y). Esempio: `id_sala, fila, numero` $\to$ `tipo_poltrona`.

- **Assiomi di Armstrong** (corretti e completi): **riflessività** (se $Y \subseteq X$ allora $X \to Y$), **aumento** (se $X \to Y$ allora $XZ \to YZ$), **transitività** (se $X \to Y$ e $Y \to Z$ allora $X \to Z$). Derivate: unione, decomposizione, pseudotransitività.
- **Chiusura** X^+: insieme di tutti gli attributi determinati da X (si calcola aggiungendo ripetutamente i membri destri delle dipendenze i cui membri sinistri sono già inclusi).
- **Superchiave**: X con $X^+ =$ tutti gli attributi. **Chiave candidata**: superchiave *minimale*. **Attributo primo**: appartiene ad almeno una chiave candidata.

```python
from itertools import combinations

def chiusura(X, dipendenze):
    """Chiusura X+ di un insieme di attributi rispetto a un insieme di dipendenze funzionali."""
    risultato = set(X)
    cambiato = True
    while cambiato:
        cambiato = False
        for sx, dx in dipendenze:
            if set(sx) <= risultato and not set(dx) <= risultato:
                risultato |= set(dx)
                cambiato = True
    return risultato

def chiavi_candidate(attributi, dipendenze):
    """Insiemi minimali di attributi la cui chiusura contiene tutti gli attributi."""
    chiavi = []
    for k in range(1, len(attributi) + 1):
        for comb in combinations(sorted(attributi), k):
            if chiusura(comb, dipendenze) >= set(attributi):
                if not any(set(c) <= set(comb) for c in chiavi):   # minimalita'
                    chiavi.append(comb)
    return chiavi

def viola_bcnf(attributi, dipendenze):
    """Dipendenze non banali X -> Y in cui X non e' una superchiave."""
    return [(sx, dx) for sx, dx in dipendenze
            if not set(dx) <= set(sx) and chiusura(sx, dipendenze) < set(attributi)]

# R(Studente, Corso, Docente)  con  {Studente, Corso} -> Docente  e  Docente -> Corso
R = {"Studente", "Corso", "Docente"}
F = [(("Studente", "Corso"), ("Docente",)), (("Docente",), ("Corso",))]

print(sorted(chiusura({"Studente", "Corso"}, F)))
print(sorted(chiusura({"Docente"}, F)))
print(chiavi_candidate(R, F))
print(viola_bcnf(R, F))
```

Listing 14.1: Chiusura di un insieme di attributi, chiavi candidate e controllo BCNF

14.2.2 Forme normali, definizioni formali

- **2NF**: in 1NF e nessun attributo non primo dipende da una *parte* di una chiave candidata (niente dipendenze parziali).
- **3NF**: per ogni dipendenza non banale $X \to A$ vale: X è una superchiave **oppure** A è un attributo primo. (Niente dipendenze transitive di attributi non primi dalla chiave.)

- **BCNF** (Boyce–Codd): per ogni dipendenza non banale $X \to A$, X è una **superchiave**. È più restrittiva della 3NF. Ogni schema in BCNF è in 3NF, ma non viceversa.

14.2.3 Esempio: 3NF ma non BCNF

$R(\text{Studente}, \text{Corso}, \text{Docente})$ con $\{\text{Studente}, \text{Corso}\} \to \text{Docente}$ (ogni studente ha un solo docente per corso) e $\text{Docente} \to \text{Corso}$ (ogni docente insegna un solo corso).

- Chiavi candidate: {Studente, Corso} e {Studente, Docente} (verificate dal codice sopra). Tutti gli attributi sono *primi*, quindi R è in **3NF**.
- $\text{Docente} \to \text{Corso}$ ha come determinante `Docente`, che **non è superchiave**: R **non è in BCNF**. Conseguenza: la coppia (Docente, Corso) è ripetuta per ogni studente (ridondanza e anomalie di aggiornamento).
- **Decomposizione**: $R_1(\text{Docente}, \text{Corso})$ e $R_2(\text{Studente}, \text{Docente})$, entrambe in BCNF.
- **Senza perdita** (*lossless join*): l'attributo comune `Docente` è chiave di R_1, quindi il join ricostruisce esattamente R.
- **Perdita di dipendenze**: la dipendenza $\{\text{Studente}, \text{Corso}\} \to \text{Docente}$ non si può più verificare su una sola relazione (richiede un join).

Compromesso: la decomposizione in **3NF** è sempre senza perdita *e* conserva le dipendenze; la decomposizione in **BCNF** è sempre senza perdita ma *può* non conservare le dipendenze. Si sceglie la BCNF per eliminare ogni ridondanza, la 3NF se serve poter verificare i vincoli senza join.

14.2.4 Denormalizzazione

La normalizzazione riduce ridondanze e anomalie, ma richiede più join in lettura. Si **denormalizza** consapevolmente (per esempio memorizzando un totale già calcolato) quando le letture dominano e il join è troppo costoso, accettando di dover mantenere la coerenza a mano (con trigger o a livello applicativo). In un data warehouse è la norma (vedi più avanti). Regola: prima si normalizza, poi si denormalizza *dove le misure lo giustificano*.

14.3 SQL Avanzato

14.3.1 Join esterni e la trappola delle aggregazioni

Un `LEFT JOIN` mantiene le righe della tabella di sinistra anche senza corrispondenza, riempiendo con `NULL`. Ma una riga *con* NULL è comunque una riga: se si somma una colonna della tabella intermedia, il risultato è **sbagliato**.

```
-- ERRATO: per i film senza biglietti il LEFT JOIN genera comunque una riga,
-- e SUM(pr.prezzo) somma il prezzo della proiezione anche se non e' stato
   venduto nulla
SELECT f.titolo, COALESCE(SUM(pr.prezzo), 0) AS incasso
FROM film f
LEFT JOIN proiezione pr ON pr.id_film = f.id_film
LEFT JOIN biglietto b   ON b.id_proiezione = pr.id_proiezione
GROUP BY f.id_film, f.titolo;
```

Listing 14.2: Errore classico: LEFT JOIN seguito da SUM

Risultato: Aurora 52, Zenit 46, **Mare 7**, Ombra 0. Mare ha una proiezione (prezzo 7) ma *nessun biglietto venduto*: il valore corretto è 0.

```
-- CORRETTO: si somma il prezzo solo se esiste davvero un biglietto
SELECT f.titolo,
       SUM(CASE WHEN b.id_proiezione IS NOT NULL THEN pr.prezzo ELSE 0 END) AS
           incasso
FROM film f
LEFT JOIN proiezione pr ON pr.id_film = f.id_film
LEFT JOIN biglietto b   ON b.id_proiezione = pr.id_proiezione
GROUP BY f.id_film, f.titolo
ORDER BY incasso DESC, f.titolo;
```

Listing 14.3: Versione corretta

Risultato corretto: Aurora 52, Zenit 46, Mare 0, Ombra 0. **Regola**: dopo un join esterno, ricordare che la riga "vuota" esiste; contare `COUNT(colonna_della_tabella_a_destra)` (che ignora i NULL) oppure usare `CASE`.

14.3.2 Sottoquery

- **Scalare**: restituisce un solo valore (`WHERE prezzo > (SELECT AVG(prezzo) ...)`).
- **Con `IN`, con `EXISTS`**: verifica di appartenenza o di esistenza. `EXISTS` si ferma alla prima corrispondenza.
- **Correlata**: la sottoquery usa colonne della query esterna; concettualmente è eseguita per ogni riga (il DBMS spesso la riscrive come join).

```
-- Film mai programmati: anti-join con NOT EXISTS
SELECT f.titolo
FROM film f
WHERE NOT EXISTS (SELECT 1 FROM proiezione pr WHERE pr.id_film = f.id_film);
```

Listing 14.4: Anti-join con NOT EXISTS

Risultato: `Ombra`.

```
-- Proiezioni con piu' di meta' dei posti venduti
SELECT pr.id_proiezione,
       COUNT(b.fila) AS venduti,
       (SELECT COUNT(*) FROM posto p WHERE p.id_sala = pr.id_sala) AS capienza
FROM proiezione pr
LEFT JOIN biglietto b ON b.id_proiezione = pr.id_proiezione
GROUP BY pr.id_proiezione, pr.id_sala
HAVING COUNT(b.fila) * 2 > (SELECT COUNT(*) FROM posto p WHERE p.id_sala = pr.
    id_sala);
```

Listing 14.5: GROUP BY, HAVING e sottoquery: occupazione sopra il 50 per cento

Risultato: p1 (4 su 6) e p3 (4 su 4). `WHERE` filtra le *righe* prima del raggruppamento; `HAVING` filtra i *gruppi* dopo.

```sql
-- Sottoquery correlata: proiezioni con prezzo superiore alla media della
   propria sala
SELECT pr.id_proiezione, pr.id_sala, pr.prezzo
FROM proiezione pr
WHERE pr.prezzo > (SELECT AVG(p2.prezzo)
                   FROM proiezione p2
                   WHERE p2.id_sala = pr.id_sala);
```

Listing 14.6: Sottoquery correlata

Risultato: p2 (10 > media 9,33 della Sala 1), p3 (9 > media 8 della Sala 2), p4 (10).

```sql
-- Divisione relazionale: utenti che hanno prenotato TUTTE le proiezioni del
   film 1
-- ("non esiste una proiezione del film 1 per cui NON esiste una sua
   prenotazione")
SELECT u.nome
FROM utente u
WHERE NOT EXISTS (
  SELECT 1 FROM proiezione pr
  WHERE pr.id_film = 1
    AND NOT EXISTS (SELECT 1 FROM prenotazione pn
                    WHERE pn.id_utente = u.id_utente
                      AND pn.id_proiezione = pr.id_proiezione));
```

Listing 14.7: Divisione relazionale con doppio NOT EXISTS

Risultato: `Anna`, che ha prenotato sia p1 sia p2 (tutte le proiezioni del film 1). Il "*per ogni*" di SQL si esprime come "*non esiste un caso in cui non*".

```sql
-- Trappola dei NULL: se la sottoquery contiene un NULL, NOT IN non
   restituisce MAI righe
SELECT titolo FROM film
WHERE id_film NOT IN (SELECT id_film FROM (SELECT 1 AS id_film UNION ALL
   SELECT NULL));   -- 0 righe!

SELECT titolo FROM film f
WHERE NOT EXISTS (SELECT 1 FROM (SELECT 1 AS id_film UNION ALL SELECT NULL) t
                  WHERE t.id_film = f.id_film);
                                                          -- 3 film
```

Listing 14.8: La trappola di NOT IN con i valori NULL

Se la sottoquery contiene anche un solo `NULL`, `x NOT IN (...)` è *sconosciuto* per ogni x e non restituisce righe: nel primo caso 0 righe (sbagliato), nel secondo i 3 film corretti. **Per questo, per l'anti-join, si preferisce `NOT EXISTS`.**

14.3.3 CTE e query ricorsive

Una **CTE** (*Common Table Expression*, clausola `WITH`) dà un nome a una sottoquery e rende la query leggibile e componibile. Una CTE **ricorsiva** esegue un passo base e uno ricorsivo finché

non si arriva a un punto fisso: serve per gerarchie (organigrammi, categorie), grafi e sequenze.

```sql
-- CTE ricorsiva: elenco dei giorni da un'origine a un'altra (utile per
    trovare i giorni senza proiezioni)
WITH RECURSIVE giorni(g) AS (
  SELECT date('2026-11-09')
  UNION ALL
  SELECT date(g, '+1 day') FROM giorni WHERE g < date('2026-11-12')
)
SELECT g FROM giorni
WHERE NOT EXISTS (SELECT 1 FROM proiezione pr WHERE date(pr.data_ora) = g);
```

Listing 14.9: CTE ricorsiva: giorni senza proiezioni

Risultato: 9 e 12 novembre (i giorni dell'intervallo in cui non c'è alcuna proiezione).

14.3.4 Funzioni finestra

Calcolano un valore *per ogni riga* usando un insieme di righe correlate (la "finestra"), **senza** ridurre il numero di righe come fa `GROUP BY`.

```sql
-- CTE + funzioni finestra: classifica dei film per incasso
WITH incassi AS (
  SELECT f.titolo,
         SUM(CASE WHEN b.id_proiezione IS NOT NULL THEN pr.prezzo ELSE 0 END)
             AS incasso
  FROM film f
  LEFT JOIN proiezione pr ON pr.id_film = f.id_film
  LEFT JOIN biglietto b   ON b.id_proiezione = pr.id_proiezione
  GROUP BY f.id_film, f.titolo
)
SELECT titolo, incasso,
       RANK()       OVER (ORDER BY incasso DESC) AS rank_,
       DENSE_RANK() OVER (ORDER BY incasso DESC) AS dense_rank_,
       ROW_NUMBER() OVER (ORDER BY incasso DESC, titolo) AS riga
FROM incassi
ORDER BY incasso DESC, titolo;
```

Listing 14.10: Classifica con RANK, DENSE_RANK e ROW_NUMBER

titolo	incasso	RANK	DENSE_RANK	ROW_NUMBER
Aurora	52	1	1	1
Zenit	46	2	2	2
Mare	0	3	3	3
Ombra	0	3	3	4

A pari merito (Mare e Ombra) `RANK` dà la stessa posizione e *salta* le successive; `DENSE_RANK` non salta; `ROW_NUMBER` numera comunque in modo univoco.

```sql
-- Totale cumulato per giorno (funzione finestra con ORDER BY)
WITH giorno AS (
  SELECT date(pr.data_ora) AS g, SUM(pr.prezzo) AS incasso
```

```
  FROM biglietto b
  JOIN proiezione pr ON pr.id_proiezione = b.id_proiezione
  GROUP BY date(pr.data_ora)
)
SELECT g, incasso, SUM(incasso) OVER (ORDER BY g) AS cumulato
FROM giorno
ORDER BY g;
```

Listing 14.11: Totale cumulato con una funzione finestra

Risultato: 10 novembre: 88 (cumulato 88); 11 novembre: 10 (cumulato 98). `PARTITION BY` suddivide la finestra in gruppi (per esempio un cumulato per sala); `LAG`/`LEAD` accedono alla riga precedente o successiva.

14.3.5 Viste, vincoli e trigger

- **Vista**: query salvata con un nome, interrogabile come una tabella. **Perché**: nasconde la complessità (join ripetuti), restringe l'accesso (si espone una vista e non la tabella con dati sensibili) e fornisce *indipendenza logica* dallo schema. La **vista materializzata** memorizza il risultato (più veloce da leggere, ma da aggiornare).
- **Vincoli dichiarativi**: `PRIMARY KEY`, `UNIQUE`, `NOT NULL`, `CHECK`, `FOREIGN KEY` con azioni `ON DELETE CASCADE/RESTRICT/SET NULL`. **Perché preferirli**: sono garantiti dal DBMS *per ogni applicazione e ogni accesso*, anche concorrente, mentre un controllo nel codice si può aggirare o dimenticare.
- **Trigger**: codice eseguito automaticamente su `INSERT/UPDATE/DELETE`. Utile per audit, valori derivati, regole complesse; ma *nasconde* la logica, rende difficile il debug e può rallentare. Si usa con parsimonia.
- **Procedure memorizzate**: logica nel DBMS, vicino ai dati; riducono il traffico di rete ma legano l'applicazione a un prodotto.

```
CREATE VIEW v_occupazione AS
SELECT pr.id_proiezione, f.titolo,
       COUNT(b.fila) AS venduti,
       (SELECT COUNT(*) FROM posto p WHERE p.id_sala = pr.id_sala) AS capienza
FROM proiezione pr
JOIN film f ON f.id_film = pr.id_film
LEFT JOIN biglietto b ON b.id_proiezione = pr.id_proiezione
GROUP BY pr.id_proiezione, f.titolo, pr.id_sala;

-- uso: la vista si interroga come una tabella
SELECT titolo, venduti, capienza FROM v_occupazione WHERE venduti = capienza;
```

Listing 14.12: Vista sull'occupazione delle proiezioni

Risultato dell'ultima query: `Zenit`, 4 venduti su 4.

```
CREATE TABLE audit_biglietto (
  quando         TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  id_proiezione  INTEGER, fila CHAR(1), numero INTEGER, id_prenotazione
      INTEGER
);
```

```
5
6  CREATE TRIGGER trg_audit_biglietto
7  AFTER INSERT ON biglietto
8  FOR EACH ROW
9  BEGIN
10   INSERT INTO audit_biglietto (id_proiezione, fila, numero, id_prenotazione)
11   VALUES (NEW.id_proiezione, NEW.fila, NEW.numero, NEW.id_prenotazione);
12 END;
```

Listing 14.13: Trigger di audit sui biglietti (sintassi SQLite)

La sintassi dei trigger varia tra i sistemi (in PostgreSQL il corpo è una funzione richiamata dal trigger).

14.4 Transazioni

Una **transazione** è una sequenza di operazioni che il DBMS tratta come una **unità logica indivisibile**. Esempio: la prenotazione inserisce una riga in `prenotazione` e una riga per ogni posto in `biglietto`: se manca un pezzo, il database sarebbe incoerente.

14.4.1 Proprietà ACID, con il perché

Proprietà	Significato	Meccanismo tipico
Atomicity	Tutto o niente: se un passo fallisce, gli effetti dei precedenti sono annullati	Log di *undo*, rollback
Consistency	Dallo stato valido a un altro stato valido: i vincoli di integrità restano veri	Vincoli, trigger, correttezza della logica applicativa
Isolation	Ogni transazione si comporta come se fosse sola, anche se ne girano molte in contemporanea	Lock, MVCC
Durability	Dopo il commit gli effetti sopravvivono a guasti e riavvii	Log su disco scritto prima del commit (WAL), replicazione

```
1  BEGIN;
2    INSERT INTO prenotazione (id_prenotazione, id_utente, id_proiezione) VALUES
       (6, 3, 2);
3    INSERT INTO biglietto VALUES (2, 'B', 1, 6, 1);   -- posto libero: ok
4    INSERT INTO biglietto VALUES (2, 'A', 1, 6, 1);   -- posto gia' venduto:
       violazione di PK
5  ROLLBACK;     -- l'applicazione, ricevuto l'errore, annulla TUTTO: nessuna
     prenotazione parziale
```

Listing 14.14: Transazione: o tutto o niente

Il secondo biglietto viola la chiave primaria (il posto A1 della proiezione 2 è già venduto): il `ROLLBACK` annulla anche la prenotazione appena inserita. Dopo la prova, il database contiene ancora 5 prenotazioni e 11 biglietti, esattamente come prima.

14.4.2 Gestione

- `BEGIN`, `COMMIT`, `ROLLBACK`; `SAVEPOINT` per annullare solo una parte.

- In modalità **autocommit** ogni istruzione è una transazione a sé: per un'operazione composta serve una transazione esplicita.
- **Transazioni brevi**: tengono lock per meno tempo, riducono i conflitti. Non si aspetta mai l'input di un utente dentro una transazione.
- Dopo un errore di concorrenza (deadlock, conflitto di serializzazione) la transazione va **ritentata** dall'applicazione.

14.5 Controllo della Concorrenza

Il DBMS esegue molte transazioni insieme per usare bene CPU e dischi (mentre una attende il disco, un'altra lavora). Senza controllo, le operazioni interferiscono.

14.5.1 Anomalie

Anomalia	Descrizione
Lettura sporca (*dirty read*)	T_2 legge un dato modificato da T_1 che poi fa rollback: ha letto un valore mai esistito
Aggiornamento perso (*lost update*)	T_1 e T_2 leggono lo stesso valore, lo modificano e lo scrivono: l'ultima scrittura cancella l'altra (esempio con il contatore)
Lettura non ripetibile	T_1 legge due volte la stessa riga e ottiene valori diversi, perché T_2 l'ha modificata nel mezzo
Fantasma (*phantom*)	T_1 ripete una query con condizione e trova righe in più, inserite da T_2
Write skew	Due transazioni leggono dati sovrapposti, decidono in base a ciò che hanno letto e scrivono dati *diversi*; il risultato viola un vincolo che ciascuna da sola rispettava (per esempio "al massimo 2 prenotazioni attive per utente" controllato con un `SELECT` prima dell'`INSERT`)

14.5.2 Livelli di isolamento (SQL standard)

Livello	Lettura sporca	Non ripetibile	Fantasma
READ UNCOMMITTED	possibile	possibile	possibile
READ COMMITTED	**impedita**	possibile	possibile
REPEATABLE READ	impedita	**impedita**	possibile (secondo lo standard)
SERIALIZABLE	impedita	impedita	**impedita**

Livelli più alti ⇒ meno anomalie ma più attese, conflitti e tentativi. Impostazioni predefinite tipiche: `READ COMMITTED` in PostgreSQL, Oracle e SQL Server; `REPEATABLE READ` in MySQL (InnoDB). I sistemi con **snapshot isolation** (PostgreSQL in `REPEATABLE READ`) impediscono i fantasmi ma consentono ancora il *write skew*: solo `SERIALIZABLE` lo evita. Una scelta consapevole: l'isolamento più basso che garantisce la correttezza del caso d'uso.

14.5.3 Serializzabilità

Un **schedule** è l'ordine di esecuzione interleaved delle operazioni di più transazioni. È **seriale** se le transazioni non si sovrappongono; è **serializzabile** se equivale a uno schedule seriale. Il

criterio usato è la **serializzabilità per conflitto**.

- Due operazioni sono in **conflitto** se sono di transazioni diverse, sullo stesso dato e *almeno una* è una scrittura (lettura–scrittura, scrittura–lettura, scrittura–scrittura).
- **Grafo di precedenza**: un nodo per transazione; un arco $T_i \to T_j$ se un'operazione di T_i precede e confligge con una di T_j. Lo schedule è serializzabile per conflitto **se e solo se il grafo è aciclico**.

Esempio svolto (lost update). Schedule S: $R_1(X)\, R_2(X)\, W_1(X)\, W_2(X)$. Conflitti: $R_1(X)$ prima di $W_2(X)$ dà l'arco $T_1 \to T_2$; $R_2(X)$ prima di $W_1(X)$ dà l'arco $T_2 \to T_1$. Il grafo ha un **ciclo**: S **non è serializzabile**, ed è esattamente l'aggiornamento perso. Lo schedule $R_1(X)\, W_1(X)\, R_2(X)\, W_2(X)$ ha il solo arco $T_1 \to T_2$: è serializzabile (equivale a T_1 poi T_2).

14.5.4 Tecniche

- **Lock**: *condivisi* (S, per leggere: compatibili tra loro) ed *esclusivi* (X, per scrivere: incompatibili con tutto). Granularità: riga, pagina, tabella (lock di *intenzione* per coordinare i livelli).
- **Two-Phase Locking (2PL)**: ogni transazione ha una fase *crescente* (acquisisce lock) e una *calante* (li rilascia) e dopo aver rilasciato un lock non ne acquisisce più. **Garantisce la serializzabilità per conflitto.** Nella versione **rigorosa** (*strict*) i lock esclusivi si rilasciano solo al commit, evitando letture di dati non confermati e *aborti a cascata*.
- **Deadlock**: due transazioni attendono i lock l'una dell'altra. Si *rileva* con il grafo *wait-for* (un ciclo è un deadlock) e si interrompe una vittima, oppure si *previene* (*wait–die*, *wound–wait*) o con *timeout*. Per ridurli: acquisire le risorse **sempre nello stesso ordine**, transazioni brevi.
- **MVCC** (*Multi-Version Concurrency Control*): ogni scrittura crea una *nuova versione* del dato; ogni lettura vede uno *snapshot* coerente. **Perché**: *i lettori non bloccano gli scrittori e viceversa*. Usato da PostgreSQL, Oracle, MySQL InnoDB. Costo: pulizia delle versioni vecchie (*vacuum*).
- **Controllo ottimistico**: si lavora senza lock e si verifica alla fine che nessuno abbia modificato i dati letti; tipicamente con una colonna `versione` (`UPDATE ... WHERE versione = ?`; se nessuna riga è modificata, c'è stato un conflitto e si riprova). Adatto quando i conflitti sono rari.
- `SELECT ... FOR UPDATE`: blocca esplicitamente le righe lette fino al commit (controllo pessimistico).

Collegamento con il caso di studio: per la prenotazione dei posti la protezione più robusta è il **vincolo di unicità**, che non dipende dal livello di isolamento né dalla correttezza del codice; la transazione garantisce l'atomicità; i lock servono solo per scenari più complessi (*hold* dei posti).

14.6 Affidabilità e Ripristino dei Dati

14.6.1 Tipi di guasto

- **Della transazione**: errore logico, violazione di un vincolo, deadlock ⇒ rollback della singola transazione.
- **Di sistema** (crash, mancanza di corrente): il contenuto della memoria si perde, il disco resta.

- **Del supporto** (disco rotto): serve un **backup**.

14.6.2 Log e Write-Ahead Logging

Il DBMS tiene un **registro delle transazioni** (*log*) scritto in modo sequenziale su disco: per ogni modifica registra il *valore precedente* (*before image*, per l'**undo**) e il *valore nuovo* (*after image*, per il **redo**).

- **Regola WAL**: il record di log di una modifica va su disco *prima* della pagina di dati modificata, e il log va su disco *prima* del commit.
- **Perché**: scrivere in modo sequenziale il log è veloce; le pagine di dati (scritte a caso sul disco) si possono scrivere più tardi (*no-force*, *steal*), senza perdere la durabilità.

	Si scrive prima del commit	**Si scrive dopo il commit**
Pagine dati	**FORCE** (costoso: molte scritture casuali)	**NO-FORCE** (veloce; serve il *redo*)
Pagine non confermate	**NO-STEAL** (si tiene tutto in memoria)	**STEAL** (si possono scrivere su disco; serve l'*undo*)

I DBMS reali usano **STEAL + NO-FORCE** (più prestazioni), pagando con *undo* e *redo*.

14.6.3 Recupero dopo un crash

Si parte dall'ultimo **checkpoint** (un punto in cui si sa quali dati sono su disco, per non rileggere tutto il log). Poi: **analisi** (quali transazioni erano attive), **redo** (si rifanno le modifiche delle transazioni confermate non ancora su disco), **undo** (si annullano quelle delle transazioni non confermate). È lo schema dell'algoritmo **ARIES**.

14.6.4 Backup e alta disponibilità

Backup completi e incrementali, **point-in-time recovery** (ripristino a un istante, combinando backup e log), **replicazione** (copie su altri nodi: failover e distribuzione delle letture), con la scelta tra replica sincrona (nessuna perdita, più latenza) e asincrona (possibile perdita dell'ultimo tratto), vedi il capitolo *Sistemi Distribuiti*. Un backup non testato non è un backup.

14.7 Indici e Progettazione Fisica

14.7.1 Perché servono

Senza indice, per trovare le righe che soddisfano una condizione il DBMS deve **scandire l'intera tabella** (*full table scan*), con costo proporzionale alla dimensione. Un **indice** è una struttura dati ausiliaria che permette di trovare le righe senza scandire tutto, come l'indice analitico di un libro.

14.7.2 Strutture

- **B+ tree** (il più usato): albero *bilanciato* con alto *fan-out* (centinaia di figli per nodo); le foglie contengono le chiavi ordinate e sono collegate in lista. Supporta **uguaglianza**, **intervalli** (`BETWEEN`, `<`, `>`), **ordinamento** (`ORDER BY`) e **prefissi** (`LIKE 'ab%'`). La ricerca costa $O(\log_f N)$ accessi a pagina, dove f è il fan-out. Esempio: pagine da 8 KiB con voci

di 16 byte danno $f \approx 500$; con tre livelli si indicizzano $500^3 = 125$ milioni di righe: bastano **3–4 accessi a disco**.

- **Indice hash**: solo *uguaglianza*, in media $O(1)$; non supporta intervalli né ordinamento.
- **Altri**: indici *bitmap* (poche modalità distinte, analisi), *full-text*, *spaziali*.

14.7.3 Tipi di indice

- **Clustered** (primario): le righe della tabella sono *fisicamente ordinate* secondo la chiave dell'indice (al più uno per tabella; in InnoDB e SQL Server è la chiave primaria). **Non-clustered**: struttura separata con puntatori alle righe.
- **Indice composto** (su più colonne): vale la regola del **prefisso sinistro**. Un indice su (`id_proiezione`, `creata_il`) serve le ricerche su `id_proiezione` da sola o su entrambe le colonne, *non* quelle sulla sola seconda colonna. Ordine delle colonne: prima quelle con condizioni di *uguaglianza*, poi quella con *intervallo* o ordinamento.
- **Indice coprente** (*covering*): contiene tutte le colonne richieste dalla query, che si risolve *senza leggere la tabella*.
- **Indice univoco**: oltre che a velocizzare, impone un vincolo (la chiave primaria e `UNIQUE` lo creano).

14.7.4 Esperimento

Tabella `prenotazione` con **300 000 righe** (SQLite, la mia macchina; i valori variano, l'ordine di grandezza no).

```
-- "Le prenotazioni di un utente": senza indice, scansione dell'intera tabella
SELECT * FROM prenotazione WHERE id_utente = 777;

CREATE INDEX idx_prenotazione_utente ON prenotazione (id_utente);
-- indice composto: l'ordine delle colonne conta
CREATE INDEX idx_pren_proiezione_data ON prenotazione (id_proiezione,
    creata_il);

-- usa l'indice (prefisso sinistro):   WHERE id_proiezione = 10 AND creata_il
    = '2026-05-05'
-- usa l'indice (solo il prefisso):    WHERE id_proiezione = 10
-- NON usa l'indice:                   WHERE creata_il = '2026-05-05'      (
    manca il prefisso)
-- NON usa l'indice:                   WHERE id_utente + 0 = 777           (
    funzione sulla colonna)
-- NON usa l'indice:                   WHERE stato LIKE '%FERMATA'         (
    carattere jolly iniziale)
```

Listing 14.15: Indici semplici e composti; quando NON vengono usati

Query	Piano di esecuzione	Tempo
`WHERE id_utente = 777`, senza indice	`SCAN` (tutta la tabella)	5,5 ms
La stessa, con l'indice su `id_utente`	`SEARCH USING INDEX`	0,006 ms

Circa **mille volte** più veloce, e il divario cresce con la dimensione della tabella. Il piano si legge con `EXPLAIN` (`EXPLAIN ANALYZE` in PostgreSQL, che esegue e misura).

14.7.5 Costi e quando non indicizzare

- Ogni indice **rallenta** `INSERT`, `UPDATE`, `DELETE` (va aggiornato) e **occupa spazio**. Indicizzare le colonne usate in `WHERE`, `JOIN`, `ORDER BY`, non *tutte*.
- **Chiavi esterne**: alcuni DBMS (PostgreSQL) *non* creano automaticamente l'indice sulla colonna referenziante; senza, i join e le cancellazioni a cascata sono lenti.
- **Selettività**: un indice su una colonna con pochissimi valori distinti (per esempio `stato` con due valori) serve a poco; l'ottimizzatore preferisce comunque la scansione se la query restituisce una grossa frazione della tabella.
- L'indice **non viene usato** se si applica una **funzione alla colonna** (`WHERE id_utente + 0 = 777`, `WHERE YEAR(data) = 2026`), con un **carattere jolly iniziale** (`LIKE '%x'`), con una **conversione di tipo implicita**, o senza il prefisso sinistro di un indice composto: nell'esperimento ciascuno di questi casi dà `SCAN`.

14.7.6 Indici per il sistema del cinema

- La chiave primaria di `biglietto` (`id_proiezione`, `fila`, `numero`) serve già la query dei posti occupati di una proiezione e il vincolo anti-doppioni.
- `prenotazione(id_utente)`: "le mie prenotazioni".
- `proiezione(data_ora)` o `(id_film, data_ora)`: la programmazione per giorno o per film.
- `biglietto(id_prenotazione)`: i biglietti di una prenotazione (e le chiavi esterne).

14.7.7 Altri strumenti di progettazione fisica

Partizionamento orizzontale (per intervallo di date, per hash) per tabelle enormi: le query e la manutenzione toccano solo le partizioni utili; **sharding** su più nodi; **viste materializzate** e tabelle riassuntive; **compressione**; **memorizzazione per colonne** per l'analisi.

14.8 Ottimizzazione delle Query

14.8.1 Fasi di elaborazione

1. **Analisi** sintattica e semantica; **traduzione** in algebra relazionale (piano logico).
2. **Riscrittura logica**: anticipare selezioni e proiezioni, eliminare ridondanze, trasformare sottoquery in join.
3. **Scelta del piano fisico** basata sui **costi**: per ogni operatore si sceglie l'algoritmo (scansione o uso di un indice; tipo di join), usando **statistiche** (numero di righe, valori distinti, istogrammi).
4. **Esecuzione**.

Il costo si misura soprattutto in **accessi a disco** e in CPU. Le statistiche vanno tenute *aggiornate* (`ANALYZE`); statistiche vecchie causano piani sbagliati.

14.8.2 Algoritmi di join

Algoritmo	Idea	Quando conviene
Nested loop	Per ogni riga di R cerca le corrispondenze in S; $O(\|R\| \cdot \|S\|)$ senza indice	Una delle due tabelle è piccola, oppure c'è un **indice** sulla colonna di join di S ($O(\|R\| \log \|S\|)$)
Hash join	Costruisce una tabella hash sulla relazione più piccola, poi scandisce l'altra; $O(\|R\| + \|S\|)$	Join di uguaglianza su grandi insiemi non ordinati; richiede memoria
Sort-merge join	Ordina entrambe sulla colonna di join e le fonde	Dati già ordinati o intervalli; join di non-uguaglianza

L'**ordine dei join** conta molto: conviene unire prima le relazioni che producono meno righe intermedie.

14.8.3 Buone pratiche

- **Leggere solo ciò che serve**: indicare le colonne (non `SELECT *`), filtrare nel database, non nell'applicazione.
- **Indici adatti** alle query più frequenti e **predicati "sargable"** (condizioni che permettono l'uso dell'indice: nessuna funzione sulla colonna).
- **Operazioni a blocchi**: inserimenti in batch, non una riga per volta.
- **Evitare il problema N+1** (sotto).
- **Paginazione keyset** invece di `OFFSET` grandi.
- **Misurare**: leggere il piano con `EXPLAIN`, non indovinare.

```sql
-- Paginazione con OFFSET: il database scarta le prime 100000 righe a ogni richiesta
SELECT id_prenotazione, creata_il FROM prenotazione
ORDER BY creata_il, id_prenotazione LIMIT 20 OFFSET 100000;

-- Paginazione "keyset": si riparte dall'ultima riga vista (usa l'indice, costo costante)
SELECT id_prenotazione, creata_il FROM prenotazione
WHERE (creata_il, id_prenotazione) > ('2026-05-05', 4711)
ORDER BY creata_il, id_prenotazione LIMIT 20;
```

Listing 14.16: Paginazione: OFFSET contro keyset

Con `OFFSET 100000` il database legge e scarta centomila righe a ogni pagina (costo crescente con la pagina); con la *keyset* riparte dall'ultima riga vista e usa l'indice (costo costante). Il confronto di tuple `(a, b) > (x, y)` è supportato da PostgreSQL e MySQL; per altri sistemi si scrive con `OR`.

14.9 Accesso ai Dati dalle Applicazioni

- **Pool di connessioni**: aprire una connessione costa (rete, autenticazione, risorse sul server); si riusano connessioni già aperte. **Perché**: il numero di connessioni del DBMS è limitato e ogni richiesta web non deve pagare l'apertura.

- **Query parametriche** (*prepared statements*): sicure contro la SQL injection (vedi capitolo *Sicurezza*) e con piani riusabili.
- **Transazioni a livello di servizio**: la transazione racchiude un'operazione di business (nel capitolo *Progettazione di Sistemi e API*, `creaPrenotazione`).
- **ORM** (*Object–Relational Mapping*): mappa le classi sulle tabelle. **Pro**: produttività, meno SQL ripetitivo, portabilità. **Contro**: nasconde le query (rischio di query inefficienti), il *problema N+1*, divario tra modello a oggetti e relazionale (*impedance mismatch*).

14.9.1 Il problema N+1

Si carica un elenco con **1 query**, poi per ognuno degli N elementi se ne fa un'altra: $N + 1$ **query**. Con 1000 film sono 1001 viaggi al database. È un difetto tipico degli ORM con il caricamento "pigro" (*lazy loading*).

```
import sqlite3

db = sqlite3.connect("cinema.db")
film = db.execute("SELECT id_film, titolo FROM film").fetchall()      # 1
    query

# N+1: una query in piu' per ogni film
for id_film, titolo in film:
    n = db.execute("SELECT COUNT(*) FROM proiezione WHERE id_film = ?",
                   (id_film,)).fetchone()[0]                          # N
                      query

# Una sola query con JOIN e raggruppamento
righe = db.execute("""
    SELECT f.titolo, COUNT(pr.id_proiezione)
    FROM film f LEFT JOIN proiezione pr ON pr.id_film = f.id_film
    GROUP BY f.id_film, f.titolo""").fetchall()                       # 1
        query
```

Listing 14.17: N+1 query contro una sola query con JOIN

Contando le istruzioni inviate al database (4 film): **5 query** con il ciclo, **1** con il join. Soluzioni: `JOIN`, caricamento anticipato (*eager loading*, `JOIN FETCH`), oppure una seconda query con `IN (...)` per tutti gli identificativi insieme.

14.10 Data Warehouse e Analisi OLAP

I sistemi **OLTP** (transazionali: prenotazioni, ordini) gestiscono molte piccole operazioni concorrenti con dati normalizzati. I sistemi **OLAP** (analisi) eseguono poche interrogazioni pesanti su grandi dati storici. Il **data warehouse** è un archivio separato, integrato e storico per l'analisi, alimentato con processi **ETL** (*Extract, Transform, Load*) o **ELT**; vedi anche la sezione sui Big Data del capitolo *Sistemi Distribuiti*.

- **Schema a stella**: una **tabella dei fatti** al centro (per esempio *biglietti venduti*: misure come numero di biglietti e incasso, più le chiavi delle dimensioni) circondata da **tabelle delle dimensioni** (*tempo*, *film*, *sala*, *cliente*). Dimensioni **denormalizzate**: meno join, query più rapide. **Schema a fiocco di neve**: dimensioni normalizzate.

- **Operazioni OLAP**: *roll-up* (aggregare a un livello più alto: da giorno a mese), *drill-down* (l'opposto), *slice* e *dice* (selezionare una o più dimensioni), *pivot*. In SQL: `GROUP BY`, `ROLLUP`, `CUBE`, funzioni finestra.
- **Perché separare** OLTP e OLAP: le query analitiche pesanti rallenterebbero le transazioni; i dati operativi sono normalizzati e senza storico; l'analisi richiede dati integrati da più fonti.
- **Archivi a colonne**: memorizzare per colonna accelera le aggregazioni su poche colonne e comprime meglio.

14.11 Scalette di Risposta

Tema: "consistenza dei dati nei sistemi SQL e NoSQL" (vedi anche il capitolo *Sistemi Distribuiti*).

1. Transazioni e **ACID**: che cosa garantiscono e con quali meccanismi (log, lock, MVCC).
2. **Isolamento**: anomalie, livelli, serializzabilità, il compromesso con le prestazioni.
3. Scalabilità dei DBMS relazionali: verticale, replicazione, partizionamento; costo delle transazioni distribuite (two-phase commit).
4. NoSQL: BASE, consistenza eventuale, teorema CAP; esempi di scelta.
5. Conclusione: **persistenza poliglotta**.

Tema: "prestazioni di una base di dati".

1. Modello logico adeguato: normalizzazione e, se serve, denormalizzazione.
2. **Indici**: B+ tree, composti, copranti, costi; query "sargable".
3. Ottimizzatore: piano logico e fisico, statistiche, algoritmi di join, `EXPLAIN`.
4. Livello applicativo: pool di connessioni, N+1, paginazione, batch.
5. Architettura: cache, replicazione per le letture, partizionamento, separazione OLTP/OLAP.
6. **Misurare prima di ottimizzare**.

Checklist per un esercizio di basi di dati (ER, relazionale, SQL).

- ER con **cardinalità** e **identificatori**, vincoli non esprimibili elencati a parte.
- Schema relazionale con **chiavi primarie ed esterne**; verifica delle forme normali (almeno 3NF, meglio BCNF).
- `CREATE TABLE` con **vincoli dichiarativi** (`NOT NULL`, `CHECK`, `UNIQUE`, chiavi esterne).
- Query: attenzione ai **join esterni**, ai **NULL**, a `HAVING` contro `WHERE`, ai duplicati (`DISTINCT`); l'*"per ogni"* si scrive con doppio `NOT EXISTS`.
- Transazioni dove un'operazione tocca più tabelle.
- Indici sulle colonne usate nei join e nei filtri più frequenti.

Tracce Uscite

Prima sessione 2024 – Informatica #1

Traccia: consistenza del dato in un sistema distribuito; confronto tra soluzioni dei database SQL e NoSQL, con esempi e casi in cui un'azienda preferirebbe l'uno o l'altro; teorema CAP.

Dove trovare la risposta: sezioni *Transazioni*, *Controllo della Concorrenza* (ACID, isolamento) di questo capitolo e sezioni *Il Teorema CAP* e *Database SQL e NoSQL* del capitolo *Sistemi Distribuiti*; scaletta a fine capitolo.

Seconda sessione 2024 – Informatica #3

Traccia: database per i Big Data; modelli relazionale e non relazionale; scalabilità, flessibilità, velocità; elaborazione batch e streaming; teorema CAP.

Dove trovare la risposta: sezione *Data Warehouse e Analisi OLAP*, indici e partizionamento (*Progettazione Fisica*) e capitolo *Sistemi Distribuiti*.

Prima sessione 2026 – A3-INF-4, Parte 1 (punti 1–3)

Traccia: modello ER del cinema, modello relazionale con chiavi e vincoli, SQL di creazione delle tabelle e query dei posti disponibili.

Dove trovare la risposta: capitolo *Progettazione di Sistemi e API*, sezione del tema svolto. In questo capitolo trovi query di verifica sullo stesso schema, la trappola dei join esterni, gli indici da creare, le transazioni e le forme normali.

Esercizio tipo (non una traccia ufficiale)

Per esercitarsi sulla normalizzazione: dato lo schema $R(\text{Studente}, \text{Corso}, \text{Docente})$ con $\{\text{Studente}, \text{Corso}\} \to \text{Docente}$ e $\text{Docente} \to \text{Corso}$, trovare le chiavi candidate, stabilire se è in 3NF e in BCNF, decomporlo e discutere la perdita di dipendenze. Svolto nella sezione *Dipendenze Funzionali e Forme Normali*.

Domande teoriche già presenti nel repository

Le domande su ER, normalizzazione e SQL con soluzione sono in `domande_ed_esercizi_bd.tex`.

Capitolo 15

Architettura del Calcolatore

L'architettura del calcolatore descrive *come è fatta e come funziona* una CPU con la sua memoria: quali unità la compongono, come eseguono le istruzioni, come la **pipeline** ne aumenta la velocità e come la **gerarchia di memoria** nasconde la lentezza della RAM. Nella Sezione A è uscita come tema di Elettronica (prima sessione 2024: datapath RISC e pipeline; prima sessione 2018: struttura interna di un microcontrollore). Il capitolo termina con la risposta svolta al tema 2024.

Gli esempi usano **RISC-V** (architettura RISC aperta, la più semplice da studiare); i concetti valgono anche per MIPS e ARM. I numeri dei diagrammi di pipeline sono stati verificati con un piccolo simulatore.

15.1 Il Modello di von Neumann

15.1.1 Componenti

- **CPU**: **unità di controllo** (decodifica le istruzioni e genera i segnali di controllo), **ALU** (operazioni aritmetiche e logiche), **registri** (memoria velocissima dentro la CPU), tra cui il **Program Counter** (PC, indirizzo della prossima istruzione) e il registro delle istruzioni.
- **Memoria**: contiene *sia le istruzioni sia i dati* ("programma memorizzato", *stored program*).
- **Ingresso/uscita** (I/O): periferiche, collegate tramite controller.
- **Bus**: collegamenti condivisi (indirizzi, dati, controllo).

15.1.2 Il ciclo di istruzione

La CPU ripete all'infinito: **fetch** (legge l'istruzione all'indirizzo del PC), **decode** (decodifica e legge i registri), **execute** (calcola), **memory** (accede ai dati, se serve), **write back** (scrive il risultato) e aggiorna il PC (di norma $\text{PC} \leftarrow \text{PC} + 4$; un salto lo sostituisce con l'indirizzo di destinazione).

15.1.3 Perché "programma memorizzato"

Se il programma è in memoria come i dati, cambiare programma significa solo caricarne un altro: lo stesso hardware esegue qualunque software (un compilatore produce dati che sono programmi). È la base di ogni calcolatore moderno.

15.1.4 Von Neumann e Harvard

- **Von Neumann**: una sola memoria e un solo bus per istruzioni e dati ⇒ il **collo di bottiglia di von Neumann**: CPU veloce e memoria lenta, e non si può leggere un'istruzione e un dato nello stesso ciclo.
- **Harvard**: memorie e bus **separati** per istruzioni e dati: si accede a entrambe nello stesso ciclo. È la scelta tipica dei **microcontrollori** e, *dentro la CPU*, delle cache di primo livello (cache istruzioni e cache dati separate): le CPU moderne sono "von Neumann" verso la RAM e "Harvard" verso la cache.

15.2 Richiami di Reti Logiche

Il datapath e la pipeline sono costruiti con due tipi di circuiti digitali.

- **Reti combinatorie**: l'uscita dipende *solo* dagli ingressi attuali (porte logiche, **multiplexer** che sceglie un ingresso, **decoder**, **addizionatore**, **ALU**). Hanno un ritardo di propagazione.
- **Reti sequenziali**: l'uscita dipende anche dallo **stato**, memorizzato in **flip-flop** e **registri** aggiornati sul fronte del **clock**. Un **registro** memorizza una parola; un **register file** ne raccoglie molti con porte di lettura e scrittura. Una **macchina a stati finiti** (FSM) è la base dell'unità di controllo.
- **Perché il clock**: sincronizza il sistema. Il periodo del clock deve essere **maggiore del cammino combinatorio più lungo** tra due registri (più i tempi di setup), altrimenti il dato non fa in tempo ad arrivare. La frequenza è l'inverso del periodo.
- **Sommatore**: il *ripple-carry* propaga il riporto bit a bit (ritardo proporzionale al numero di bit); il *carry-lookahead* calcola i riporti in parallelo, più veloce e più costoso.

15.3 Instruction Set Architecture (ISA)

L'**ISA** è il *contratto* tra software e hardware: insieme delle istruzioni, registri, modi di indirizzamento, formato in memoria. Il compilatore produce istruzioni per un'ISA; molte implementazioni hardware diverse la realizzano (stessa ISA, microarchitetture diverse).

15.3.1 RISC e CISC

	RISC (RISC-V, MIPS, ARM)	**CISC** (x86)
Istruzioni	Poche, semplici, tutte della **stessa lunghezza** (32 bit)	Molte, complesse, **lunghezza variabile**
Accesso alla memoria	**Load/store**: solo `lw` e `sw` accedono alla memoria; le altre operano sui registri	Molte istruzioni operano direttamente su operandi in memoria
Registri	Molti (32)	Meno (storicamente)
Decodifica	Semplice e veloce	Complessa (spesso tradotta in micro-operazioni RISC-like)
Vantaggio	Hardware semplice, **pipeline efficiente**, bassa potenza	Codice più compatto, compatibilità storica
Esempi	Smartphone, microcontrollori, server (ARM), RISC-V	PC e server x86

Perché RISC: istruzioni uniformi e semplici rendono facile dividere l'esecuzione in stadi di uguale durata, cioè fare la pipeline (vedi più avanti). Oggi la differenza si è attenuata: i processori x86 traducono internamente le istruzioni in micro-operazioni simili a quelle RISC.

15.3.2 RISC-V: registri e formati

- **32 registri** a 32 bit (`x0`...`x31`); `x0` vale **sempre zero**. Convenzioni: `sp` (stack pointer), `ra` (indirizzo di ritorno), `a0`–`a7` (argomenti e valore di ritorno), `t0`–`t6` (temporanei), `s0`–`s11` (da preservare). Le chiamate di funzione usano lo stack con il record di attivazione (vedi i capitoli *Sistemi Operativi* e *Programmazione*).
- **Formato R** (registro–registro): `funct7 | rs2 | rs1 | funct3 | rd | opcode`. Esempio `add x1, x2, x3`.
- **Formato I** (immediato, `lw`): `imm[11:0] | rs1 | funct3 | rd | opcode`. **Formato S** (`sw`): immediato diviso in due parti. **Formato B** (salti condizionati): offset di salto.

Esempi di codifica a 32 bit (verificati):

Istruzione	Significato	Codice (esadecimale)
`add x1, x2, x3`	$x_1 \leftarrow x_2 + x_3$	`003100B3`
`sub x5, x6, x7`	$x_5 \leftarrow x_6 - x_7$	`407302B3`
`addi x5, x0, 10`	$x_5 \leftarrow 10$	`00A00293`
`lw x6, 8(x7)`	$x_6 \leftarrow \mathrm{Mem}[x_7 + 8]$	`0083A303`
`sw x6, 8(x7)`	$\mathrm{Mem}[x_7 + 8] \leftarrow x_6$	`0063A423`
`beq x1, x2, +16`	se $x_1 = x_2$: $\mathrm{PC} \leftarrow \mathrm{PC} + 16$	`00208863`

15.3.3 Modi di indirizzamento

Registro (operandi nei registri), **immediato** (costante nell'istruzione), **base + spostamento** (`lw x6, 8(x7)`), **relativo al PC** (salti), **pseudo-diretto** (jump). **Perché pochi**: ogni modo complica decodifica e pipeline.

15.3.4 Esempio: da C ad assembly

Somma degli elementi di un vettore di 4 interi (`x10` = indirizzo base).

```
        addi x5, x0, 0        # somma = 0
        addi x6, x0, 0        # i = 0
        addi x7, x0, 4        # n = 4
loop:   slli x8, x6, 2        # offset = i * 4  (ogni intero e' 4 byte)
        add  x8, x8, x10      # indirizzo di a[i]
        lw   x9, 0(x8)        # x9 = a[i]
        add  x5, x5, x9       # somma += a[i]
        addi x6, x6, 1        # i++
        bne  x6, x7, loop     # se i != n, ripeti
```

Listing 15.1: Somma di un vettore in assembly RISC-V (eseguita su un simulatore)

Con $a = [1, 2, 3, 4]$ il programma esegue **27 istruzioni** (3 di inizializzazione e 4 iterazioni da 6) e lascia $x_5 = 10$.

15.4 Il Datapath

Il **datapath** è l'insieme di unità funzionali e collegamenti che eseguono le istruzioni; l'**unità di controllo** stabilisce, per ogni istruzione, quali unità si usano e come.

15.4.1 Unità funzionali

Unità	Funzione
Program Counter (PC)	Registro con l'indirizzo dell'istruzione da eseguire
Sommatore PC + 4	Calcola l'indirizzo dell'istruzione successiva (le istruzioni sono lunghe 4 byte)
Memoria istruzioni	Dato l'indirizzo (PC), restituisce l'istruzione (di sola lettura)
Register file	Banco di registri con **due porte di lettura** (`rs1`, `rs2`) e **una di scrittura** (`rd`, abilitata da `RegWrite`)
Generatore di immediati	Estende con il segno la costante contenuta nell'istruzione
ALU	Esegue somma, sottrazione, AND, OR, confronti; calcola anche l'*indirizzo* di `lw`/`sw` (registro base + spostamento) e confronta nei salti condizionati
Memoria dati	Lettura (`lw`) e scrittura (`sw`) dei dati
Sommatore dei salti	Calcola l'indirizzo di destinazione PC + offset
Multiplexer	Scelgono tra più sorgenti (secondo operando dell'ALU: registro o immediato; dato da scrivere nel registro: ALU o memoria; prossimo PC: PC + 4 o destinazione del salto)
Unità di controllo	Legge l'*opcode* e genera i segnali di controllo

15.4.2 Segnali di controllo

Istruzione	RegWrite	ALUSrc	MemRead	MemWrite	MemtoReg	Branch
Tipo R (`add`)	1	registro	0	0	ALU	0
`lw`	1	immediato	1	0	memoria	0
`sw`	0	immediato	0	1	–	0
`beq`	0	registro	0	0	–	1

15.4.3 Quali unità usa ciascuna istruzione

- **Tipo R** (`add x1,x2,x3`): memoria istruzioni → register file (legge x_2, x_3) → ALU (somma) → register file (scrive x_1). *Non usa la memoria dati.*
- **`lw`**: legge x_{base} → ALU (base + spostamento) → **memoria dati** (lettura) → scrive il registro destinazione.
- **`sw`**: legge base e dato → ALU (indirizzo) → **memoria dati** (scrittura). *Non scrive nei registri.*
- **`beq`**: legge due registri → ALU (confronto) → se uguali, il sommatore dei salti sostituisce il PC.

15.4.4 Datapath a ciclo singolo e perché non basta

Nel datapath **a ciclo singolo** ogni istruzione si esegue in *un solo ciclo di clock*. Il periodo deve però bastare per l'istruzione *più lenta* (`lw`, che usa tutte le unità): con ritardi tipici (accesso a

memoria istruzioni 200 ps, lettura registri 100 ps, ALU 200 ps, memoria dati 200 ps, scrittura registri 100 ps) il periodo è di 800 ps, anche per istruzioni che sarebbero state più rapide. Inoltre ogni unità è usata solo in una piccola frazione del ciclo: **si spreca hardware**. La **pipeline** risolve entrambi i problemi.

15.5 La Pipeline

15.5.1 L'idea

Si divide l'esecuzione in **stadi** e, *mentre un'istruzione è in un certo stadio, la successiva occupa lo stadio precedente*: come in una catena di montaggio (o una lavanderia con lavatrice, asciugatrice e stiro che lavorano insieme su carichi diversi). Ogni unità lavora *in ogni ciclo*, su un'istruzione diversa.

15.5.2 I cinque stadi (RISC classico)

Stadio	Che cosa avviene	Unità funzionali coinvolte
IF – *Instruction Fetch*	Si legge l'istruzione all'indirizzo del PC e si calcola PC + 4	PC, memoria istruzioni, sommatore +4
ID – *Instruction Decode*	Si decodifica l'istruzione, si leggono i registri sorgente, si estende l'immediato, si generano i segnali di controllo	Unità di controllo, **register file** (lettura), generatore di immediati, unità di rilevamento degli hazard
EX – *Execute*	Si esegue l'operazione, si calcola l'indirizzo di memoria o di salto, si confrontano i registri	**ALU**, sommatore dei salti, multiplexer degli operandi, unità di forwarding
MEM – *Memory*	Si legge o si scrive la memoria dati (solo `lw`/`sw`); nei salti si decide il PC	**Memoria dati**
WB – *Write Back*	Si scrive il risultato (ALU o memoria) nel registro destinazione	**Register file** (scrittura), multiplexer

15.5.3 Registri di pipeline

Tra gli stadi si inseriscono **registri di pipeline** (`IF/ID`, `ID/EX`, `EX/MEM`, `MEM/WB`). **Perché**: a ogni ciclo ogni stadio lavora su un'istruzione diversa, quindi i dati e i segnali di controllo di ciascuna istruzione devono essere *conservati* mentre procede da uno stadio al successivo; senza registri, il risultato di un'istruzione sarebbe sovrascritto da quella che la segue.

15.5.4 Diagramma temporale

Cinque istruzioni indipendenti (una colonna per ciclo di clock):

	1	2	3	4	5	6	7	8	9
istruzione 1	IF	ID	EX	MEM	WB				
istruzione 2		IF	ID	EX	MEM	WB			
istruzione 3			IF	ID	EX	MEM	WB		
istruzione 4				IF	ID	EX	MEM	WB	
istruzione 5					IF	ID	EX	MEM	WB

Dal ciclo 5 la pipeline è *piena*: ogni ciclo termina un'istruzione. Con k stadi, n istruzioni richiedono $k + (n - 1)$ cicli.

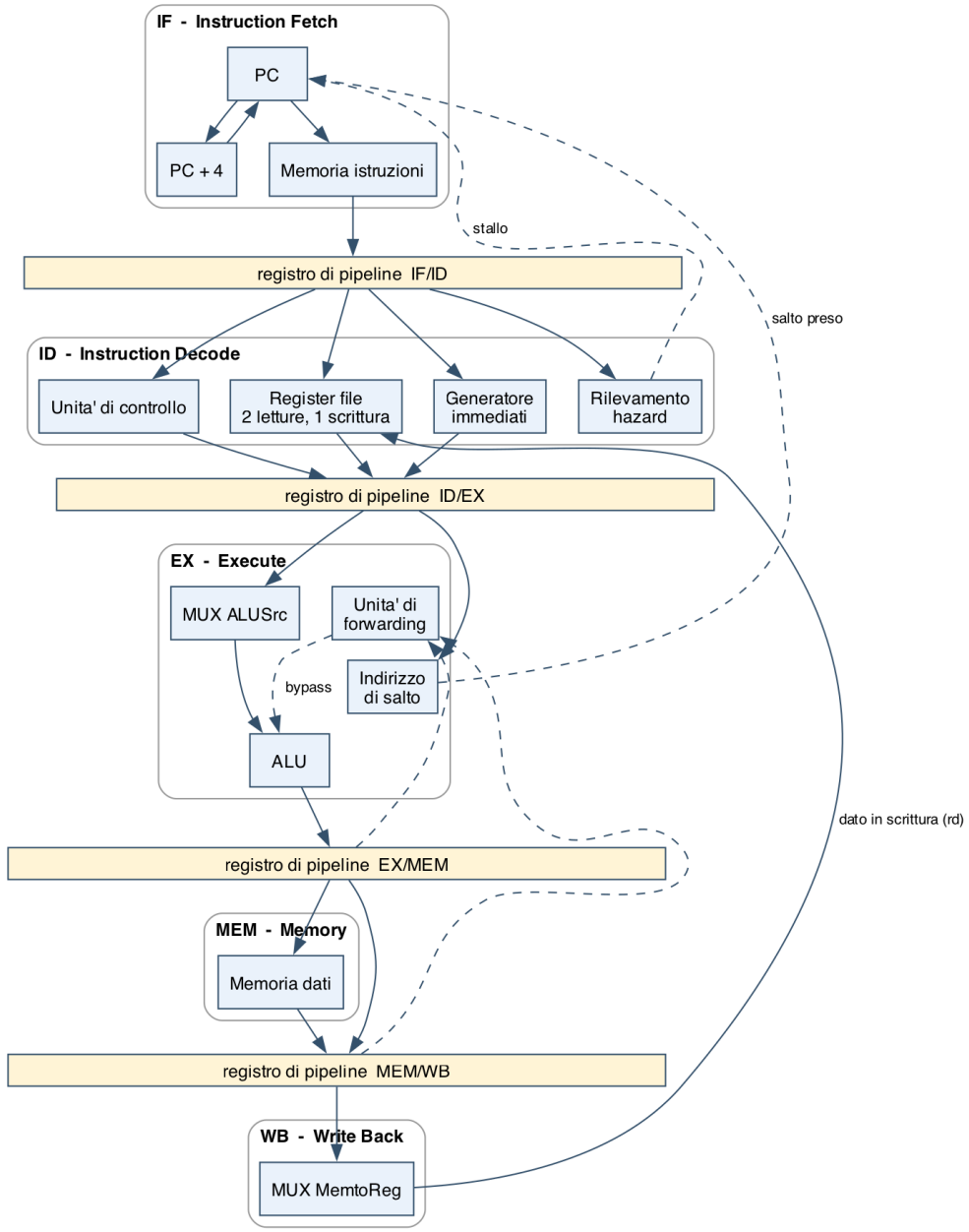


Figura 15.1: Datapath a pipeline a cinque stadi: i registri di pipeline separano gli stadi; le linee tratteggiate sono l'unità di forwarding, il rilevamento degli hazard e il salto preso.

15.5.5 Vantaggi, con i numeri

Con gli stessi ritardi di prima (200, 100, 200, 200, 100 ps):

- **Ciclo singolo**: periodo = 800 ps; 1000 istruzioni $\Rightarrow 1000 \cdot 800 = 800\,000$ ps.
- **Pipeline**: il periodo è il ritardo dello stadio *più lento*, 200 ps; 1000 istruzioni $\Rightarrow (1000 + 4) \cdot 200 = 200\,800$ ps.
- **Accelerazione** $= 800\,000/200\,800 \approx \mathbf{3{,}98}$ (non 5: gli stadi non sono bilanciati; con stadi uguali l'accelerazione ideale tende al numero di stadi).
- **Throughput** (istruzioni al secondo): aumenta, idealmente a una istruzione per ciclo (CPI $\to 1$) con un clock più veloce.
- **Latenza** di una singola istruzione: *non migliora*, anzi peggiora: $5 \cdot 200 = 1000$ ps contro 800 ps. La pipeline migliora il *numero di istruzioni completate per unità di tempo*, non il

tempo di una singola.
- **Utilizzo dell'hardware**: ogni unità lavora ad ogni ciclo.

15.6 Hazard: i Problemi della Pipeline

Un **hazard** è una situazione in cui l'istruzione successiva *non può* entrare nel ciclo seguente. Se non gestito, produce risultati sbagliati o richiede di fermare (*stallo*) la pipeline.

15.6.1 Hazard strutturali

Due istruzioni vogliono usare la *stessa risorsa* nello stesso ciclo. Esempio: con *una sola memoria* per istruzioni e dati, `lw` nello stadio MEM e il fetch di un'istruzione successiva nello stadio IF vorrebbero la memoria insieme. **Soluzione**: duplicare la risorsa (memoria istruzioni e memoria dati separate, o cache separate: architettura Harvard); register file con scrittura nella prima metà del ciclo e lettura nella seconda.

15.6.2 Hazard sui dati

Un'istruzione usa un risultato di una precedente *non ancora scritto* nel registro (dipendenza **RAW**, *read after write*):

```
add x1, x2, x3     # scrive x1 nello stadio WB (ciclo 5)
sub x4, x1, x5     # legge x1 nello stadio ID (ciclo 3): valore ANCORA VECCHIO
```

Listing 15.2: Dipendenza RAW

Soluzioni

- **Stallo** (*bolla*): si inseriscono cicli vuoti finché il dato è pronto. Senza altre misure sono **2 cicli** se l'istruzione dipendente segue subito, **1 ciclo** se c'è un'istruzione nel mezzo.
- **Forwarding** (*bypass*): il risultato dell'ALU è *inoltrato* direttamente dai registri `EX/MEM` o `MEM/WB` all'ingresso dell'ALU, senza aspettare la scrittura nel register file. **Perché funziona**: il dato è già calcolato alla fine dello stadio EX, solo non ancora "ufficialmente" scritto. Un multiplexer in ingresso all'ALU e un'**unità di forwarding** che confronta i numeri di registro lo realizzano. Elimina quasi tutti gli stalli sulle istruzioni aritmetiche.
- **Il caso *load-use***: dopo un `lw` il dato è disponibile solo alla fine dello stadio **MEM**, troppo tardi per essere usato nello stadio EX dell'istruzione successiva: anche con il forwarding serve **1 stallo**, inserito dall'**unità di rilevamento degli hazard** (blocca PC e IF/ID e inserisce una bolla).

	1	2	3	4	5	6	7
`lw x1, 0(x2)`	IF	ID	EX	MEM	WB		
`add x3, x1, x4`		IF	ID	**stallo**	EX	MEM	WB

Sequenza	Cicli totali	Stalli senza forwarding	Stalli con forwarding
`add x1,...` poi `sub x4,x1,...`	6 con forwarding, 8 senza	2	**0**
`lw x1,...` poi `add x3,x1,...`	7 con forwarding, 8 senza	2	**1**
`add x1,...`, un'istruzione indipendente, `sub x4,x1,...`	7 con forwarding, 8 senza	1	**0**

Riordino delle istruzioni da parte del compilatore. Il compilatore può mettere istruzioni indipendenti nello spazio tra un `lw` e il suo uso (*instruction scheduling*):

```
lw  x1, 0(x2)
lw  x3, 4(x2)
add x4, x1, x3      # usa x3, caricato dall'istruzione precedente: STALLO
sw  x4, 8(x2)
lw  x5, 12(x2)
add x6, x1, x5      # usa x5, caricato dall'istruzione precedente: STALLO
sw  x6, 16(x2)
```

Listing 15.3: Codice originale: 2 stalli, 13 cicli (con forwarding)

```
lw  x1, 0(x2)
lw  x3, 4(x2)
lw  x5, 12(x2)      # anticipata: riempie lo spazio dopo lw x3
add x4, x1, x3
sw  x4, 8(x2)
add x6, x1, x5
sw  x6, 16(x2)
```

Listing 15.4: Codice riordinato: 0 stalli, 11 cicli

Le due sequenze producono gli stessi risultati; il riordino è lecito perché `lw x5` non dipende dalle istruzioni che supera.

Altre dipendenze. **WAR** (*write after read*) e **WAW** (*write after write*) non creano problemi in una pipeline con esecuzione *in ordine*; contano nell'esecuzione fuori ordine, dove si risolvono con la **rinomina dei registri**.

15.6.3 Hazard di controllo

I salti condizionati (`beq`, `bne`) cambiano il PC, ma l'esito si conosce solo dopo lo stadio EX: nel frattempo la pipeline ha già caricato (e iniziato a eseguire) le istruzioni *successive*, che potrebbero essere quelle sbagliate.

- **Stallo** fino alla decisione: sempre costoso.

- **Predizione "non preso"**: si prosegue come se il salto non avvenisse; se invece è preso, si fa il **flush** (le istruzioni sbagliate sono trasformate in bolle). Costo: 2 cicli persi per ogni salto preso (salto risolto in EX).
- **Anticipare la decisione** (confronto nello stadio ID): meno cicli persi, con hardware in più.
- **Predizione dinamica**: una tabella ricorda il comportamento passato di ogni salto. Un **contatore a 2 bit** (*saturating counter*) cambia predizione solo dopo *due* errori consecutivi, ed è molto accurato nei cicli. Un **Branch Target Buffer** memorizza anche l'indirizzo di destinazione.
- **Delay slot** (MIPS): l'istruzione dopo il salto si esegue comunque; il compilatore la riempie con un'istruzione utile.

15.6.4 Effetto sulle prestazioni: il ciclo di somma

Per il codice assembly del vettore con 4 elementi (27 istruzioni eseguite):

Configurazione	Stalli	Cicli persi per i salti	Cicli totali	CPI
Con forwarding	4 (un load-use per iterazione)	6 (3 salti presi × 2)	41	1,52
Senza forwarding	36	6	73	2,70

Il CPI ideale è 1 (con la pipeline piena i cicli totali sono $27 + 4 = 31$); gli hazard lo portano a 1,52. Senza forwarding quasi dimezzerebbe le prestazioni.

15.7 Oltre la Pipeline Semplice

- **Pipeline più profonde**: più stadi e clock più alto, ma hazard più costosi.
- **Superscalare**: più istruzioni per ciclo, con più unità di esecuzione (CPI inferiore a 1).
- **Esecuzione fuori ordine** e **rinomina dei registri**: le istruzioni indipendenti non aspettano quelle bloccate; un *reorder buffer* garantisce che gli effetti appaiano in ordine.
- **Esecuzione speculativa**: si prosegue lungo il percorso predetto, annullando se sbagliato. Se mal progettata può far trapelare dati (vulnerabilità tipo *Spectre*: vedi il capitolo *Sicurezza*).
- **Parallelismo dei dati**: istruzioni **SIMD** (una istruzione opera su più dati), GPU.
- **Multithreading**, **multicore**: più flussi di esecuzione in parallelo.
- **Limiti**: il **power wall** (la potenza dissipata impedisce di alzare ancora il clock) ha spostato l'evoluzione dal clock al *parallelismo* (core multipli), e quindi alla programmazione concorrente (vedi i capitoli sui thread).

15.8 Prestazioni

15.8.1 Equazione fondamentale

$$T_{\text{CPU}} = N_{\text{istr}} \times \text{CPI} \times T_{\text{clock}} = \frac{N_{\text{istr}} \times \text{CPI}}{f_{\text{clock}}}$$

- N_{istr}: numero di istruzioni (dipende da programma, compilatore e ISA).
- CPI: cicli per istruzione (dipende dalla microarchitettura: pipeline, hazard, cache).
- f_{clock}: frequenza (dipende da tecnologia e microarchitettura).

Esempio: $2 \cdot 10^9$ istruzioni, CPI = 1,5, $f = 3$ GHz $\Rightarrow T = 2 \cdot 10^9 \cdot 1{,}5/(3 \cdot 10^9) = 1{,}0$ s. Le tre grandezze *interagiscono*: la pipeline abbassa il CPI e alza la frequenza; RISC aumenta N_{istr} ma riduce CPI e periodo. Il **MIPS** (milioni di istruzioni al secondo) è una misura *ingannevole* (istruzioni diverse fanno lavori diversi): si preferisce il tempo di esecuzione su **benchmark** realistici.

15.8.2 Legge di Amdahl

Se una frazione p del tempo si accelera di un fattore s:

$$\text{Speedup} = \frac{1}{(1-p) + p/s}$$

Esempio: $p = 80\%$ parallelizzabile su 8 core: $1/(0{,}2 + 0{,}8/8) = 3{,}33$. Anche con infiniti core il limite è $1/(1-p) = 5$. **Perché è importante**: la parte non migliorabile limita il guadagno totale; conviene ottimizzare ciò che pesa di più.

15.8.3 Potenza e consumo

- **Potenza dinamica** (commutazione): $P_{\text{din}} = \alpha\, C\, V^2 f$, con α fattore di attività, C capacità, V tensione, f frequenza. Dipende dal **quadrato della tensione**: ridurre V è la leva più efficace.
- **Potenza statica**: correnti di *leakage* anche a circuito fermo; cresce con la miniaturizzazione.
- **Tecniche**: *clock gating* (si spegne il clock dove non serve), *DVFS* (si variano tensione e frequenza in base al carico), *power gating*, modalità di sospensione. Un'ottima architettura è quella più efficiente in *energia per istruzione*, non solo veloce.

Collegamento con il capitolo *Elettronica*: in un inverter CMOS la potenza dinamica deriva dalla carica e scarica delle capacità a ogni commutazione, la statica dalle correnti di dispersione.

15.9 Gerarchia di Memoria e Cache

15.9.1 Il problema

La CPU è molto più veloce della memoria principale (DRAM): un accesso alla RAM costa centinaia di cicli. Non esiste una memoria al tempo stesso veloce, grande ed economica: si usa una **gerarchia**, dove i livelli vicini alla CPU sono piccoli e veloci e quelli lontani grandi e lenti. Valori indicativi (ordini di grandezza, variabili da una generazione all'altra):

Livello	**Dimensione**	**Tempo di accesso**
Registri	decine di parole	< 1 ns
Cache L1	32–64 KiB	≈ 1 ns
Cache L2	256 KiB–2 MiB	≈ 3–5 ns
Cache L3	MiB–decine di MiB	≈ 10–20 ns
Memoria principale (DRAM)	GiB	≈ 60–100 ns
SSD	centinaia di GB–TB	≈ 50–100 μs
Disco magnetico	TB	$\approx$ ms

15.9.2 Il principio di località

La gerarchia funziona perché i programmi hanno **località**:

- **Temporale**: un dato usato di recente sarà probabilmente riusato (variabili di un ciclo).
- **Spaziale**: dopo un indirizzo, si accedono presto quelli vicini (scansione di un array, istruzioni sequenziali).

Per questo la cache memorizza dati *recenti* e li carica a **blocchi** (*linee*, per esempio 64 byte) anziché a singola parola.

15.9.3 Organizzazione di una cache

- **Hit**: il dato è in cache. **Miss**: non c'è, va letto dal livello inferiore (**penalità di miss**).
- **Mappatura diretta**: ogni blocco di memoria può stare in *una sola* linea della cache (indice = numero di blocco modulo numero di linee). Semplice e veloce, ma soggetta a *conflitti*.
- **Completamente associativa**: un blocco può stare in qualsiasi linea; pochi conflitti, ma confronto parallelo su tutte le linee (costoso).
- **Set-associativa a n vie**: il blocco va in un *insieme* e può occupare una qualsiasi delle n linee dell'insieme. È il compromesso usato (2, 4, 8, 16 vie).

Un indirizzo si divide in: **offset** (byte nel blocco), **indice** (quale insieme), **tag** (identifica il blocco: confrontato per decidere hit o miss). Ogni linea ha anche un bit di **validità**.

Esempio: cache da 32 KiB, blocchi da 64 byte, indirizzi a 32 bit. Numero di blocchi $= 32768/64 = 512$. Offset $= \log_2 64 = 6$ bit.

Organizzazione	Insiemi	Indice	Tag
Mappatura diretta	512	9 bit	$32 - 9 - 6 = 17$ bit
4 vie	128	7 bit	$32 - 7 - 6 = 19$ bit
Completamente associativa	1	0 bit	$32 - 0 - 6 = 26$ bit

Più associatività: meno indice, più tag (più confronti).

Esempio: simulazione di una cache a mappatura diretta con 4 linee da 16 byte. Accessi (indirizzi in byte): $0, 4, 16, 32, 0, 64, 16, 0$. Blocco = indirizzo/16; linea = blocco mod 4.

Indirizzo	Blocco	Linea	Esito	Motivo
0	0	0	miss	primo accesso (*compulsory*)
4	0	0	**hit**	stesso blocco: località spaziale
16	1	1	miss	primo accesso
32	2	2	miss	primo accesso
0	0	0	**hit**	località temporale
64	4	0	miss	sostituisce il blocco 0 nella linea 0
16	1	1	**hit**	ancora presente
0	0	0	miss	**conflitto**: il blocco 0 era stato espulso dal 4

3 hit e 5 miss su 8 accessi. L'ultimo miss è dovuto a un *conflitto*: due blocchi (0 e 4) contendono la stessa linea; con una cache a 2 vie non sarebbe avvenuto.

15.9.4 Tipi di miss (le "3C")

Compulsory (primo accesso al blocco), **Capacity** (la cache è troppo piccola per il working set), **Conflict** (più blocchi nella stessa linea o insieme). **Rimedi**: blocchi più grandi (compulsory), cache più grande (capacity), maggiore associatività (conflict).

15.9.5 Politiche

- **Sostituzione**: quale blocco espellere in un insieme pieno: **LRU** (il meno recentemente usato), **FIFO**, casuale.
- **Scrittura**: **write-through** (si scrive sempre anche nel livello inferiore: semplice, più traffico) o **write-back** (si scrive solo in cache e si marca la linea *dirty*; si scrive in memoria all'espulsione: meno traffico, più complessa). In caso di miss in scrittura: **write-allocate** (si carica il blocco) o **no-write-allocate**.

15.9.6 Tempo medio di accesso

$$\text{AMAT} = \text{tempo di hit} + \text{tasso di miss} \times \text{penalità di miss}$$

Esempio: hit 1 ns, miss 5%, penalità 100 ns $\Rightarrow$ AMAT $= 1 + 0{,}05 \cdot 100 = 6$ ns. Con una **cache L2** (hit 10 ns, il 20% degli accessi che mancano la L1 manca anche la L2): AMAT $= 1 + 0{,}05\,(10 + 0{,}2 \cdot 100) = 2{,}5$ ns. **Perché più livelli**: una cache sola, o piccola e veloce (molti miss) o grande e lenta; i livelli combinano i due vantaggi.

15.9.7 Effetto sul codice: percorrere una matrice

In C le matrici sono memorizzate *per righe*: `a[i][j]` e `a[i][j+1]` sono contigui. Scorrere per righe sfrutta la località spaziale (un blocco da 64 byte contiene 16 interi); scorrere per colonne fa un salto di una riga a ogni accesso e provoca molti miss.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define N 4096
int main(void) {
    int *a = malloc((size_t)N * N * sizeof(int));
    for (size_t i = 0; i < (size_t)N * N; i++) a[i] = 1;
    long somma = 0;
    clock_t t0 = clock();
    for (int i = 0; i < N; i++)          // per righe: indirizzi consecutivi
        for (int j = 0; j < N; j++)
            somma += a[(size_t)i * N + j];
    double t_righe = (double)(clock() - t0) / CLOCKS_PER_SEC;
    t0 = clock();
    for (int j = 0; j < N; j++)          // per colonne: salti di N*4 byte
        for (int i = 0; i < N; i++)
            somma += a[(size_t)i * N + j];
    double t_colonne = (double)(clock() - t0) / CLOCKS_PER_SEC;
    printf("somma=%ld righe=%.3fs colonne=%.3fs rapporto=%.1f\n", somma,
        t_righe, t_colonne, t_colonne / t_righe);
    free(a);
    return 0;
}
```

Listing 15.5: Stessa somma, due ordini di scansione

Matrice 4096×4096 di interi (64 MiB), compilata con `-O1`: la scansione per colonne è risultata da circa **4 a 11 volte più lenta** di quella per righe nelle prove eseguite (i valori variano con la macchina e le prove). Il numero di operazioni è *identico*: cambia solo l'ordine degli accessi. **Lezione**: la struttura di un algoritmo e l'ordine degli accessi alla memoria contano quanto il numero di operazioni.

15.9.8 Cache, multicore e memoria virtuale

- **Coerenza della cache**: nei sistemi multicore ogni core ha la sua cache; una scrittura su un dato condiviso deve essere visibile agli altri. I protocolli di coerenza (**MESI**: Modified, Exclusive, Shared, Invalid) invalidano o aggiornano le copie. Costo: traffico; **false sharing** (due thread scrivono variabili diverse nella stessa linea: la linea "rimbalza" tra le cache e le prestazioni crollano).
- **Memoria virtuale**: la CPU genera indirizzi virtuali, tradotti in fisici dalla **tabella delle pagine**; il **TLB** è una piccola cache delle traduzioni recenti (vedi il capitolo *Sistemi Operativi*).

15.10 Ingresso/Uscita

- **Controller**: ogni periferica ha un controller con registri di stato, controllo e dati. **Memory-mapped I/O**: i registri della periferica sono ai normali indirizzi di memoria (`lw`/`sw` li leggono e scrivono); in alternativa, istruzioni dedicate (*port-mapped*).
- **Polling**: la CPU legge ripetutamente lo stato della periferica. Semplice, ma spreca tempo di CPU.
- **Interrupt**: la periferica avvisa la CPU solo quando serve; la CPU esegue la *routine di servizio* e riprende (vedi il capitolo *Sistemi Operativi*). Efficiente se gli eventi sono rari.
- **DMA** (*Direct Memory Access*): un controller trasferisce blocchi di dati tra periferica e memoria *senza coinvolgere la CPU*, che viene avvisata con un interrupt alla fine. Essenziale per dischi, rete, audio/video.

15.11 Microcontrollori

Traccia 2018 (seconda prova, Elettronica): descrivere la struttura interna di un microcontrollore e fornire esempi di sistemi in cui è utilizzato.

15.11.1 Che cos'è

Un **microcontrollore** (MCU) è un **sistema completo su un solo chip**: CPU, memorie e periferiche. È progettato per *controllare* un dispositivo (sistema **embedded**) con basso costo, bassa potenza e comportamento prevedibile. Un **microprocessore** (CPU di un PC) contiene invece solo la CPU e richiede memoria e periferiche esterne.

15.11.2 Struttura interna

- **Core** (CPU): ARM Cortex-M, RISC-V, AVR, PIC; a 8, 16 o 32 bit; frequenza da pochi MHz a centinaia di MHz. Spesso con architettura **Harvard**.

- **Memoria**: **Flash** (non volatile, contiene il programma), **SRAM** (dati e stack, volatile), **EEPROM** (parametri di configurazione).
- **Clock e gestione dell'energia**: oscillatore interno o a quarzo, PLL, modalità di sospensione (*sleep*) a bassissimo consumo.
- **Periferiche digitali**: **GPIO** (ingressi/uscite generici), **timer/contatori** (temporizzazioni, uscite **PWM** per motori e LED), **watchdog** (riavvia il sistema se il programma si blocca).
- **Periferiche analogiche**: **ADC** (convertitore analogico–digitale per i sensori) e a volte **DAC** e comparatori.
- **Interfacce di comunicazione**: **UART**, **SPI**, **I²C**, **CAN**, USB, Ethernet, Wi-Fi e Bluetooth.
- **Controller degli interrupt**: gestisce le richieste delle periferiche.
- **Bus interno** che collega core, memorie e periferiche.

15.11.3 Esempio: stazione meteo

Un sensore di temperatura analogico è collegato a un ingresso dell'ADC; un *timer* genera un interrupt ogni secondo; la routine di servizio avvia la conversione; il valore è inviato via **UART** a un modulo radio; tra una misura e l'altra il microcontrollore va in *sleep* (consumo di microampere, alimentazione a batteria per anni).

```
volatile uint16_t campione;          // modificato dall'interrupt

void timer_isr(void) {               // chiamata dal timer ogni secondo
    campione = adc_leggi(CANALE_TEMP);
    flag_pronto = 1;
}

int main(void) {
    clock_init(); adc_init(); uart_init(9600); timer_init(1000 /* ms */);
    while (1) {
        if (flag_pronto) {
            flag_pronto = 0;
            uart_invia(campione);    // invia il dato
        }
        dormi();                     // sleep fino al prossimo interrupt
    }
}
```

Listing 15.6: Schema in C di un'applicazione embedded (pseudocodice, non eseguito)

15.11.4 Esempi di impiego

Elettrodomestici, automotive (centraline), **IoT** (sensori in rete), dispositivi medici (pacemaker, glucometri), automazione industriale, robotica, droni, periferiche di PC (tastiere, mouse).

15.11.5 Vincoli tipici

Tempo reale (risposta entro una scadenza garantita), memoria molto limitata (KiB), basso consumo, affidabilità; spesso si programma in C, talvolta senza sistema operativo (*bare metal*) o

con un RTOS.

15.12 Rappresentazione dei Numeri

Gli interi in complemento a due sono nel capitolo *Sistemi Numerici*. Qui si aggiungono i numeri in virgola mobile e l'ordine dei byte.

15.12.1 IEEE 754 (precisione singola, 32 bit)

Segno (1 bit), **esponente** polarizzato (8 bit, *bias* 127), **mantissa** (23 bit, con un 1 iniziale implicito). Valore: $(-1)^s \times 1{,}m \times 2^{e-127}$.

Esempio: $-6{,}25 = -110{,}01_2 = -1{,}1001_2 \times 2^2$. Segno = 1; esponente = $2 + 127 = 129 = 10000001_2$; mantissa = 1001 0000 ... . Risultato: `1 10000001 10010000000000000000000` = `C0C80000` (verificato).

- **Casi speciali**: ± 0, $\pm\infty$ (esponente tutto 1, mantissa 0), **NaN** (esponente tutto 1, mantissa $\neq 0$), numeri *denormalizzati* (esponente 0).
- **Doppia precisione**: 64 bit (1 + 11 + 52), bias 1023. Precisione relativa $2^{-23} \approx 1{,}2 \cdot 10^{-7}$ (singola) e $2^{-52} \approx 2{,}2 \cdot 10^{-16}$ (doppia).
- **Errori di rappresentazione**: 0,1 non è esprimibile esattamente in binario (in singola precisione vale `3DCCCCCD`, cioè 0,100000001490 ...); in doppia 0,1+0,2 = 0,30000000000000004 $\neq$ 0,3. **Regole**: non confrontare numeri in virgola mobile con `==` (usare una tolleranza); per i valori monetari usare interi (centesimi) o tipi decimali; sommare numeri di ordini di grandezza molto diversi perde precisione.

15.12.2 Endianness

L'ordine dei byte di una parola in memoria: **little-endian** (il byte meno significativo all'indirizzo più basso; x86, RISC-V e ARM tipicamente) o **big-endian** (il più significativo per primo; protocolli di rete: "ordine di rete"). Conta quando si scambiano dati binari tra sistemi diversi.

15.13 Tema Svolto: Datapath RISC e Pipeline (2024)

Traccia (prima sessione 2024, Elettronica #2): la Figura #1 mostra il datapath di una semplice CPU e le memorie esterne con le loro interconnessioni, in un'architettura RISC. (1) Descrivere le principali unità funzionali (ALU, registri, PC...). (2) Descrivere gli stadi che portano all'esecuzione di un'istruzione e, per ogni stadio, le unità coinvolte. (3) Descrivere il funzionamento della pipeline e i suoi vantaggi. (4) Descrivere alcune problematiche dovute alla pipeline e come gestirle.

Che cosa mostra la figura. È il datapath *a ciclo singolo semplificato*: il **PC** che alimenta la **memoria istruzioni** e un primo **sommatore** (con ingresso costante 4); un secondo **sommatore** per l'indirizzo di salto; il blocco **Registri** con tre ingressi di numero di registro (due da leggere, uno da scrivere) e un ingresso dati; l'**ALU**; la **memoria dati** con ingresso indirizzo e ingresso/uscita dato. Mancano nel disegno, ma sono necessari, i *multiplexer* (scelta degli operandi e del prossimo PC), il *generatore di immediati* e l'*unità di controllo*.

Punto 1: unità funzionali

- **PC**: registro con l'indirizzo dell'istruzione corrente; è aggiornato a ogni ciclo.
- **Sommatore (+4)**: calcola PC + 4, indirizzo dell'istruzione successiva. **Secondo sommatore**: calcola l'indirizzo di destinazione dei salti (PC + offset).
- **Memoria istruzioni**: dato l'indirizzo, restituisce l'istruzione a 32 bit.
- **Register file**: legge due registri e ne scrive uno; contiene i dati in elaborazione.
- **ALU**: esegue le operazioni aritmetico–logiche, calcola gli indirizzi di `lw`/`sw` e confronta nei salti.
- **Memoria dati**: letta da `lw`, scritta da `sw`.
- (Non disegnati) **multiplexer**, **generatore di immediati**, **unità di controllo**.

Punto 2: stadi di esecuzione

IF (memoria istruzioni, PC, sommatore +4), **ID** (unità di controllo, lettura dei registri, immediato), **EX** (ALU, sommatore di salto), **MEM** (memoria dati; solo `lw`/`sw`), **WB** (scrittura nel register file). Vedi la tabella degli stadi con le unità coinvolte e la figura della pipeline.

Punto 3: pipeline e vantaggi

- **Principio**: si sovrappone l'esecuzione di più istruzioni, ciascuna in uno stadio diverso, con **registri di pipeline** tra gli stadi che conservano dati e segnali di controllo.
- **Vantaggi**: periodo di clock ridotto al ritardo dello stadio più lento (nell'esempio 200 ps contro 800 ps); **throughput** fino a un'istruzione per ciclo; ogni unità lavora a ogni ciclo. Con 5 stadi e 1000 istruzioni l'accelerazione è $\approx 3{,}98$ (ideale 5).
- **Nota**: la latenza di una singola istruzione *non* diminuisce (1000 ps contro 800 ps); migliora il numero di istruzioni completate al secondo. Perché l'ISA RISC si presta: istruzioni semplici e di uguale lunghezza, accessi alla memoria solo con `lw`/`sw`.

Punto 4: problemi e soluzioni

Hazard	Causa	Gestione
Strutturale	Due istruzioni usano la stessa risorsa nello stesso ciclo (una sola memoria)	Duplicare le risorse: memoria istruzioni e dati separate (o cache separate); scrittura dei registri nella prima metà del ciclo
Dati (RAW)	Un'istruzione usa un risultato non ancora scritto	**Forwarding**; **stallo** di 1 ciclo nel caso `lw` seguito dall'uso; riordino delle istruzioni da parte del compilatore
Controllo	Un salto cambia il PC dopo che sono già state caricate istruzioni successive	Predizione (statica o dinamica a 2 bit), *flush* se errata, anticipo della decisione, *delay slot*

Esempio numerico: nel ciclo di somma, con il forwarding ma con i salti che costano 2 cicli, il CPI sale da 1 a 1,52; senza forwarding a 2,70.

15.14 Scalette di Risposta

Tema: "pipeline di una CPU".

1. Ciclo di istruzione e datapath a ciclo singolo: unità e limite del periodo (istruzione più lenta).
2. Pipeline: stadi, registri di pipeline, diagramma temporale, throughput e latenza, accelerazione con numeri.
3. Hazard strutturali, dati e di controllo, con le soluzioni (forwarding, stalli, predizione).
4. Perché RISC si presta alla pipeline.
5. Limiti e oltre: superscalare, fuori ordine, speculazione, power wall.

Tema: "gerarchia di memoria".

1. Divario CPU–memoria; livelli, dimensioni e tempi.
2. Località temporale e spaziale.
3. Organizzazione della cache (diretta, set-associativa), indirizzo (tag, indice, offset), politiche.
4. AMAT con un esempio; tipi di miss.
5. Esempio di codice dipendente dalla località; coerenza nei sistemi multicore.

Tema: "microcontrollore".

1. Definizione e confronto con il microprocessore.
2. Struttura: core, memorie (Flash, SRAM), periferiche digitali e analogiche, interfacce, interrupt, clock, watchdog.
3. Esempio applicativo e schema di programma (interrupt, sleep).
4. Vincoli: tempo reale, consumo, costo.

Tracce Uscite

Prima sessione 2024 – Elettronica #2

Traccia: Figura #1 con il datapath di una semplice CPU RISC e le memorie esterne. (1) Unità funzionali (ALU, registri, PC…); (2) stadi di esecuzione di un'istruzione e unità coinvolte in ciascuno; (3) funzionamento e vantaggi della pipeline; (4) problematiche dovute alla pipeline e come gestirle.

Dove trovare la risposta: sezione *Tema Svolto: Datapath RISC e Pipeline (2024)*, che risponde ai quattro punti con riferimento alla figura; approfondimenti nelle sezioni *Il Datapath*, *La Pipeline* e *Hazard*.

Prima sessione 2018 (seconda prova) – Elettronica

Traccia: struttura interna di un microcontrollore e alcuni esempi di sistemi elettronici in cui viene utilizzato.

Dove trovare la risposta: sezione *Microcontrollori*, con la scaletta di risposta.

Prima sessione 2025 (Bologna) – Ing. elettronica E03

Traccia: consumo di potenza nei circuiti digitali CMOS; potenza statica e dinamica, cause della dissipazione, strategie di riduzione.

Dove trovare la risposta: sezione *Potenza e consumo* (formula $P = \alpha C V^2 f$, leakage, clock gating, DVFS); per il transistore e l'inverter CMOS vedi il capitolo *Elettronica*.

Prima sessione 2025 (Bologna) – Ing. Informatica E05

Traccia: chiamata a funzione e ruolo dello stack.

Dove trovare la risposta: capitolo *Programmazione ad Oggetti e Algoritmi* e, per l'implementazione a livello di registri (`sp`, `ra`, argomenti), la sezione *RISC-V: registri e formati* di questo capitolo.

Il progetto

Appunti per l'Esame di Stato è un progetto editoriale open-source pensato per la preparazione all'Esame di Stato in Ingegneria dell'Informazione (Sezione B). Il documento è gratuito e cresce grazie ai contributi della community.

Partecipa

Correzioni, esempi, nuovi esercizi e suggerimenti sono benvenuti: invia una Pull Request o apri una Issue nel repository.

Repository GitHub

`github.com/Pablo-gitub/esame-di-stato-ingegneria-informazione-appunti-pdf`

Licenza

Suggerita: CC BY-NC 4.0. Vedi la sezione licenza nel repository per i dettagli.